Introduction to Scientific Computing

Aditya Dendukuri

UC Santa Barbara

Contents

Preface **5**

1 Fundamentals **6**

1.1 Mathematical Notation and Proof Techniques . . . 6
1.1.1 Logic and Set Notation . . . 6
1.1.2 Proof Techniques . . . 6
1.1.3 Notation Conventions . . . 7
1.2 Vectors and Matrices . . . 8
1.3 Matrix Arithmetic . . . 12
1.3.1 Addition and Subtraction . . . 12
1.3.2 Dot Product . . . 14
1.3.3 Matrix Inverse . . . 16
1.4 Introduction to Norms . . . 18
1.5 Computational Complexity . . . 20
1.5.1 Big-O Notation . . . 20
1.5.2 Flop Counting . . . 21
1.6 Floating Point Arithmetic . . . 22
1.6.1 Floating Point Representation . . . 22
1.6.2 IEEE 754 Special Values . . . 24
1.6.3 Floating Point Arithmetic Pitfalls . . . 25
1.7 Quantifying Approximation Error . . . 26
1.7.1 Absolute and Relative Errors . . . 26
1.7.2 Forward and Backward Error Analysis . . . 28
1.8 Python and NumPy Primer . . . 30
1.8.1 Arrays and Basic Operations . . . 30
1.8.2 Linear Algebra Operations . . . 31
1.8.3 Useful Idioms . . . 31

2 Direct Methods for Linear Systems **33**
2.1 Linearity . . . 33
2.1.1 Linear Systems . . . 33
2.1.2 Rank . . . 36
2.1.3 Subspaces and the Four Fundamental Subspaces . . . 37
2.2 LU Factorization . . . 39
2.2.1 LU Factorization and Gaussian Elimination . . . 40
2.2.2 Pivoting and LUP Decomposition . . . 41
2.2.3 Solving Linear Systems using LU Factorization . . . 42
2.3 Symmetric Positive Definite (SPD) Matrices . . . 44
2.3.1 Cholesky Decomposition . . . 46

3 Orthogonality and Least Squares **49**
3.1 Orthogonal Matrices . . . 49
3.1.1 QR Decomposition . . . 52
3.1.2 Householder Reflectors . . . 53
3.1.3 Gram-Schmidt . . . 54
3.2 Vector and Matrix Norms . . . 59
3.2.1 Vector Norms . . . 59
3.2.2 Matrix Norms . . . 61
3.3 Stability and Condition Number . . . 64
3.3.1 Why Not Invert Directly? . . . 64
3.3.2 Numerical Stability . . . 65
3.3.3 Condition Number . . . 66
3.3.4 Conditioning vs. Stability . . . 68
3.3.5 Stability of Matrix Factorizations . . . 68
3.4 Overdetermined Systems and Least Squares . . . 70
3.4.1 Overdetermined Systems . . . 70
3.4.2 Normal Equations . . . 70
3.4.3 QR-Based Least Squares . . . 71
3.5 Chebyshev Polynomials and Polynomial Approximation . . . 73
3.5.1 Why the Monomial Basis Fails . . . 73
3.5.2 Chebyshev Polynomials . . . 75
3.5.3 Orthogonality and Chebyshev Nodes . . . 76
3.5.4 The Minimax Property . . . 77
3.5.5 Chebyshev Series and Approximation . . . 78

4 Eigenvalues and the SVD **80**
4.1 Eigenvalues and Eigenvectors . . . 80
4.1.1 Power Iteration and Inverse Iteration . . . 86
4.1.2 Invariant Subspaces . . . 87
4.1.3 Schur Decomposition . . . 90
4.2 Singular Value Decomposition and the Four Fundamental Subspaces . . . 93
4.2.1 The Four Fundamental Subspaces . . . 93
4.2.2 Rank Truncation and the Eckart-Young-Mirsky Theorem . . . 94

5 Iterative Methods for Linear Systems **96**
5.1 Iterative Methods for Linear Systems . . . 96
5.2 Jacobi Iteration . . . 98
5.2.1 Jacobi Iteration . . . 98
5.2.2 Convergence of Jacobi Iteration . . . 100
5.2.3 Stopping Criteria . . . 101
5.2.4 Gauss-Seidel Iteration . . . 103
5.2.5 Successive Over-Relaxation (SOR) . . . 105
5.3 Conjugate Gradient Method . . . 106
5.3.1 From Linear Systems to Optimization . . . 107
5.3.2 **A**-Conjugate Directions . . . 107
5.3.3 The CG Algorithm and Convergence . . . 108
5.3.4 Preconditioning . . . 110
5.4 GMRES . . . 112
5.4.1 Krylov Subspaces and the Arnoldi Process . . . 112
5.4.2 The GMRES Algorithm . . . 113

6 Applications **114**
6.1 Principal Component Analysis via SVD . . . 114
6.1.1 Principal Components as Variance-Maximizing Directions . . . 115
6.1.2 The SVD-PCA Connection . . . 116
6.1.3 Dimensionality Reduction . . . 116
6.1.4 Statistical Interpretation . . . 117
6.2 Spectral Graph Theory and PageRank . . . 119
6.2.1 Graph Matrices . . . 119
6.2.2 The Graph Laplacian . . . 120
6.2.3 Courant-Fischer and Graph Cuts . . . 123
6.2.4 Random Walks and Spectral Gap . . . 125
6.2.5 PageRank . . . 126
6.3 Linear Operators and the Poisson Equation . . . 129
6.3.1 From Differential Operators to Matrices . . . 129
6.3.2 The Discrete Laplacian . . . 130
6.3.3 Operator Norms and Boundedness . . . 132
6.3.4 Point Spectrum and Continuous Spectrum . . . 133
6.4 Fourier Analysis as Change of Basis . . . 135
6.4.1 The Fourier Basis . . . 135
6.4.2 The Discrete Fourier Transform . . . 137
6.4.3 Circulant Matrices and the Convolution Theorem . . . 138
6.4.4 The Fast Fourier Transform . . . 140
6.5 Markov Chains and Stochastic Matrices . . . 141
6.5.1 Stochastic Matrices as Linear Operators . . . 141
6.5.2 Stationary Distributions . . . 143
6.5.3 Spectral Gap and Mixing Time . . . 144
6.5.4 Ergodic Theory and Time Averages . . . 146
6.5.5 Computing Stationary Distributions . . . 149

6.6 Koopman Operator Theory, DMD, and SINDy 151
6.6.1 The Koopman Operator 151
6.6.2 Dynamic Mode Decomposition 153
6.6.3 SINDy: Sparse Identification of Nonlinear Dynamics 154

Preface

Every time I encountered a concept I could not explain from scratch, I wrote it down. Richard Feynman held that the surest test of understanding is whether you can teach something in plain terms. These notes exist because that test kept exposing gaps. They started as a study guide for CS 111 at UC Santa Barbara and grew as topics came up in research: eigenvalue algorithms, iterative solvers, spectral graph methods, Koopman theory. The result is a reference guide written by a graduate student for himself, not a textbook. Treat it accordingly.

For a rigorous treatment of this material, use the lecture notes of Professor David Bindel (Cornell, CS 6210: Matrix Computations), publicly available on his course website.

Computers were built to do numerical computation. The word "computer" originally meant a person whose job was to evaluate functions by hand. The U.S. Army employed hundreds of them during the Second World War to compute artillery firing tables, a task so large it directly motivated the construction of ENIAC (1945). The machine's first classified application was thermonuclear weapon simulation, directed by John von Neumann, who also contributed early work on numerical stability. In 1950, Jule Charney used ENIAC to run the first machine weather forecast — Lewis Fry Richardson had estimated the same computation would require 64,000 human computers working in parallel. The Apollo guidance computer navigated to the Moon in real time with 4 kilobytes of memory. The Cooley-Tukey FFT (1965) reduced a core signal processing computation from $O(n^2)$ to $O(n \log n)$. The demand for numerical computation at scale was the original motivation for programmable machines.

Almost every numerical algorithm reduces a nonlinear problem to a sequence of linear ones, because linear systems are the only class of problems we understand completely: their solution structure, their sensitivity to perturbations, their reliable algorithms. Newton's method, finite element discretization, PCA, gradient-based optimization — each is, at its core, a repeated linearization. Linear algebra is the substrate of scientific computing.

Much of the perspective here came from **CS 210A: Matrix Analysis**, taught by Professor Shivkumar Chandrasekaran in the ECE department at UC Santa Barbara. If you have the chance to take that course, take it.

Aditya Dendukuri
UC Santa Barbara, 2025

1 Fundamentals

1.1 Mathematical Notation and Proof Techniques

These notes assume familiarity with calculus and basic proof writing, but the conventions below establish a shared vocabulary used throughout. Skimming this subsection now will prevent confusion when proofs appear later.

1.1.1 Logic and Set Notation

Def 1.1 **(Logical Quantifiers)** The *universal quantifier* $\forall$ means "for all"; the *existential quantifier* $\exists$ means "there exists". For example, "$\forall \varepsilon > 0,\ \exists \delta > 0$ such that..." reads as "for every positive ε there is a positive δ such that...". The negation of $\forall x,\ P(x)$ is $\exists x,\ \neg P(x)$, and vice versa.

Def 1.2 **(Implication and Equivalence)** $P \Rightarrow Q$ ("P implies Q") means: whenever P is true, Q is also true. $P \Leftrightarrow Q$ ("P if and only if Q", abbreviated *iff*) means $P \Rightarrow Q$ and $Q \Rightarrow P$. The *contrapositive* of $P \Rightarrow Q$ is $\neg Q \Rightarrow \neg P$, which is logically equivalent and often easier to prove.

Def 1.3 **(Set Notation)** A *set* is a collection of distinct objects. Common notation:

- $x \in S$: x is an element of S; $\quad x \notin S$: x is not.
- $S \subseteq T$: S is a subset of T (every element of S is in T).
- $S \cup T$: union; $\quad S \cap T$: intersection; $\quad S \setminus T$: set difference.
- $\{x \in S : P(x)\}$: the set of elements in S satisfying property P.

Common number sets used throughout these notes: $\mathbb{Z}$ (integers), $\mathbb{R}$ (reals), $\mathbb{C}$ (complex numbers), $\mathbb{R}^n$ (real n-vectors), $\mathbb{R}^{m\times n}$ (real $m \times n$ matrices).

Ex 1.1

1. Write the statement "every invertible matrix has a nonzero determinant" using quantifiers and implication.
2. State the contrapositive of: "if $\mathbf{A}$ is invertible, then $\mathbf{A}\boldsymbol{x} = \mathbf{0}$ implies $\boldsymbol{x} = \mathbf{0}$."
3. Let $S = \{1, 2, 3, 4\}$ and $T = \{3, 4, 5, 6\}$. Compute $S \cup T$, $S \cap T$, and $S \setminus T$.

1.1.2 Proof Techniques

Nearly every claim in these notes is accompanied by a proof. There are three main strategies used here; recognizing which applies is the first step in writing your own proofs.

Def 1.4 **(Direct Proof)** A *direct proof* of $P \Rightarrow Q$ assumes P is true and derives Q through a chain of valid logical steps. Most proofs in these notes are direct.

Def 1.5 **(Proof by Contradiction)** To prove statement P: assume $\neg P$ and derive a logical contradiction (R and $\neg R$ simultaneously, for some statement R). This shows $\neg P$ is impossible, so P must hold.

Def 1.6 **(Proof by Induction)** To prove $P(n)$ holds for all integers $n \geq n_0$:

1. *Base case*: verify $P(n_0)$ directly.
2. *Inductive step*: assume $P(k)$ (the *inductive hypothesis*) and prove $P(k+1)$.

Mathematical induction appears whenever an algorithm or formula is defined recursively.

Ex 1.2

1. Use a direct proof to show: if $\mathbf{A}$ and $\mathbf{B}$ are symmetric ($\mathbf{A}^T = \mathbf{A}$, $\mathbf{B}^T = \mathbf{B}$), then $\mathbf{A} + \mathbf{B}$ is symmetric.
2. Use proof by contradiction to show: if $\mathbf{A}\boldsymbol{x} = \mathbf{0}$ implies $\boldsymbol{x} = \mathbf{0}$, then $\mathbf{A}$ has no zero eigenvalue.
3. By induction, prove $\displaystyle\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$.

1.1.3 Notation Conventions

The following conventions are used consistently throughout these notes.

Note 1.1 **Scalars, Vectors, and Matrices.** Scalars are lowercase italic: a, b, c, x. Vectors are lowercase bold: $\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{a}$. Matrices are uppercase bold: $\mathbf{A}, \mathbf{B}, \mathbf{Q}$. The i-th entry of $\boldsymbol{x}$ is x_i; the (i, j) entry of $\mathbf{A}$ is a_{ij} or $(\mathbf{A})_{ij}$. Indexing is 1-based throughout (unlike Python).

Note 1.2 **Transpose and Inverse.** $\mathbf{A}^T$ is the transpose of $\mathbf{A}$; $\mathbf{A}^{-1}$ is the matrix inverse (exists only for square, invertible $\mathbf{A}$). The notation $\mathbf{A}^{-T}$ means $(\mathbf{A}^{-1})^T = (\mathbf{A}^T)^{-1}$. $\mathbf{A}^k$ denotes $\mathbf{A}$ multiplied by itself k times; $\mathbf{A}^0 = \mathbf{I}$.

Note 1.3 **Sums and Products.**

$$\sum_{i=1}^{n} a_i = a_1 + a_2 + \cdots + a_n, \qquad \prod_{i=1}^{n} a_i = a_1 a_2 \cdots a_n.$$

When the index set is clear from context we write $\sum_i a_i$.

Note 1.4 **Functions.** $f : \mathbb{R}^n \to \mathbb{R}^m$ means f maps n-vectors to m-vectors. $f \circ g$ denotes function composition: $(f \circ g)(\boldsymbol{x}) = f(g(\boldsymbol{x}))$. A function $f : \mathbb{R}^{m \times n} \to \mathbb{R}$ maps matrices to scalars (e.g., a norm or the determinant).

1.2 Vectors and Matrices

Scientific computing is built on two fundamental data structures: vectors and matrices. Before introducing these formally, we fix notation for the standard number systems that provide the raw material for all numerical computation.

Standard Number Systems. Throughout these notes, $\mathbb{R}$ denotes the set of all real numbers, $\mathbb{Z}$ the set of all integers, and $\mathbb{C}$ the set of all complex numbers. Our primary focus is real-valued computation; complex numbers arise naturally in eigenvalue analysis and signal processing.

Def 1.7 **(Scalar)** An element of a number system ($\mathbb{R}$, $\mathbb{Z}$, or $\mathbb{C}$) is called a **scalar**. Scalars represent individual numerical quantities and serve as the atomic building blocks from which vectors and matrices are constructed.

Ex 1.3 Refer to Def 1.7.

1. Classify each of the following as a real scalar, an integer, a complex (non-real) scalar, or *not* a scalar: 3.14; -7; $2+3i$; $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$.
2. In Python, evaluate `type(3.14)`, `type(3)`, and `type(3+2j)`. Which NumPy `dtype` would you use to store each?

Def 1.8 **(Vectors)** A **vector** is a one-dimensional ordered array of scalars. Vectors can represent points, directions, or more abstract quantities within a vector space, and are fundamental in encoding data and linear relationships.

Note 1.5 Vectors may be written as row vectors or column vectors. Although both forms carry the same information, the distinction matters when applying linear transformations and computing inner products.

Example

$$\begin{pmatrix} 1 & 2 & 3 \end{pmatrix}, \quad \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \quad \begin{pmatrix} 1+2i \\ 7i \\ 3 \end{pmatrix}, \ldots$$

Ex 1.4 Refer to Def 1.8.

1. Write a column vector $\boldsymbol{x} \in \mathbb{R}^4$ whose i-th entry equals 2^i for $i = 1, 2, 3, 4$.
2. In NumPy, construct this vector with `np.array([...])` and confirm `x.shape == (4,)`. Why does NumPy report shape `(4,)` rather than `(4,1)`?
3. Can a vector in $\mathbb{R}^3$ be added to a vector in $\mathbb{R}^4$? Justify in one sentence.

Just as scalars belong to number systems like $\mathbb{R}$ or $\mathbb{C}$, vectors inhabit **vector spaces**. While these notes treat vector spaces informally, subsequent chapters provide more rigorous definitions and properties.

Def 1.9 **($\mathbb{R}^n$)** The set of all real vectors of size n. When we write $\boldsymbol{x} \in \mathbb{R}^n$, it means $\boldsymbol{x}$ has n real entries. Vectors in $\mathbb{R}^n$ take one of the following forms:

$$\boldsymbol{x} \in \mathbb{R}^{n\times 1} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \quad \text{(column vector, default convention)}$$

$$\boldsymbol{x} \in \mathbb{R}^{1\times n} = \begin{pmatrix} x_1 & x_2 & \cdots & x_n \end{pmatrix} \quad \text{(row vector)}$$

Note 1.6 These notes focus on finite-dimensional spaces. Infinite-dimensional vector spaces, central to functional analysis, arise in differential equations and quantum mechanics but are beyond our scope here.

Ex 1.5 Refer to Def 1.9.

1. Let $\boldsymbol{x} = (1, -2, 3)^T \in \mathbb{R}^3$. Identify the value of entry x_2. What is the dimension of $\mathbb{R}^3$?
2. In NumPy, create a column vector (shape `(3,1)`) from the list `[1, -2, 3]` in two ways: using `np.reshape` and using `[:, np.newaxis]`.
3. What does `x.T` return when `x.shape == (3,)`? How does the result differ when `x.shape == (3,1)`?

Def 1.10 **(Matrices)** A **matrix** is a two-dimensional array of scalars. A matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ is written as

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}.$$

Matrices are used to represent linear transformations, systems of linear equations, and more complex data structures.

Example Two-dimensional matrices include:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \in \mathbb{R}^{2\times 3}, \quad \begin{pmatrix} 1 & 2 \\ 4 & 5 \\ 6 & 7 \end{pmatrix} \in \mathbb{R}^{3\times 2}.$$

Note 1.7 Matrices can be interpreted as vectors whose entries are themselves vectors:

$$\mathbf{A} = \begin{pmatrix} \text{---} & \boldsymbol{r}_1 & \text{---} \\ & \vdots & \\ \text{---} & \boldsymbol{r}_n & \text{---} \end{pmatrix} = \begin{pmatrix} | & & | \\ \boldsymbol{c}_1 & \cdots & \boldsymbol{c}_n \\ | & & | \end{pmatrix},$$

where $r_i = \mathbf{A}[i,:]$ is the i-th row and $c_i = \mathbf{A}[:,i]$ is the i-th column. This viewpoint is useful when studying block matrices and matrix factorizations.

Ex 1.6 Refer to Def 1.10.

1. State the dimensions (m,n) of $\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$. What is the shape of its second column as a vector?
2. In NumPy (0-based indexing), write the expression to access entry a_{12} (first row, second column in 1-based notation). Then extract the second column of $\mathbf{A}$ and verify its shape is `(2,)`.
3. True or False: a vector in $\mathbb{R}^n$ is a special case of a matrix in $\mathbb{R}^{n\times 1}$. How does NumPy handle this distinction in practice?

Def 1.11 **(Matrix Transpose)** The **transpose** of $\mathbf{A} \in \mathbb{R}^{m\times n}$, denoted $\mathbf{A}^T$, is the $n \times m$ matrix obtained by interchanging rows and columns:

$$(\mathbf{A}^T)_{ij} = a_{ji}, \quad 1 \le i \le n,\ 1 \le j \le m.$$

For a column vector $\boldsymbol{x} \in \mathbb{R}^n$, the transpose $\boldsymbol{x}^T$ is its corresponding row vector and vice versa.

Def 1.12 **(Symmetric and Skew-Symmetric Matrices)** A square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ is:

- **symmetric** if $\mathbf{A}^T = \mathbf{A}$, equivalently $a_{ij} = a_{ji}$ for all i, j.
- **skew-symmetric** if $\mathbf{A}^T = -\mathbf{A}$, equivalently $a_{ij} = -a_{ji}$ for all i, j.

Note 1.8 Every square matrix $\mathbf{A}$ decomposes uniquely into a symmetric part and a skew-symmetric part:

$$\mathbf{A} = \underbrace{\tfrac{1}{2}(\mathbf{A} + \mathbf{A}^T)}_{\text{symmetric}} + \underbrace{\tfrac{1}{2}(\mathbf{A} - \mathbf{A}^T)}_{\text{skew-symmetric}}.$$

Symmetric matrices arise constantly in scientific computing: covariance matrices in statistics, graph Laplacians in network problems, and Hessians of quadratic objectives in optimization are all symmetric.

Thm 1.1 **(Properties of the Transpose)** Let $\mathbf{A} \in \mathbb{R}^{m\times n}$, $\mathbf{B} \in \mathbb{R}^{n\times p}$, and $\alpha \in \mathbb{R}$. Then:

1. $(\mathbf{A}^T)^T = \mathbf{A}$
2. $(\mathbf{A}+\mathbf{B})^T = \mathbf{A}^T + \mathbf{B}^T \quad$ (for conformable $\mathbf{A}, \mathbf{B}$)
3. $(\alpha\mathbf{A})^T = \alpha\mathbf{A}^T$
4. $(\mathbf{AB})^T = \mathbf{B}^T\mathbf{A}^T \quad$ **(Reversal Law)**

Proof *Proof.* Properties (1)–(3) follow directly from the definition $(\mathbf{A}^T)_{ij} = a_{ji}$ applied entry-wise. For example, for (2):

$$(\mathbf{A}+\mathbf{B})^T_{ij} = (\mathbf{A}+\mathbf{B})_{ji} = a_{ji} + b_{ji} = (\mathbf{A}^T)_{ij} + (\mathbf{B}^T)_{ij} = (\mathbf{A}^T + \mathbf{B}^T)_{ij}.$$

For (4), let $\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{m\times p}$. By definition of transpose and matrix product:

$$(\mathbf{C}^T)_{ij} = c_{ji} = \sum_{k=1}^{n} a_{jk}\, b_{ki}.$$

Expanding $\mathbf{B}^T\mathbf{A}^T$ using $(\mathbf{B}^T)_{ik} = b_{ki}$ and $(\mathbf{A}^T)_{kj} = a_{jk}$:

$$(\mathbf{B}^T\mathbf{A}^T)_{ij} = \sum_{k=1}^{n} (\mathbf{B}^T)_{ik}\,(\mathbf{A}^T)_{kj} = \sum_{k=1}^{n} b_{ki}\, a_{jk}.$$

Both sums are identical, so $(\mathbf{AB})^T = \mathbf{B}^T\mathbf{A}^T$. □

Cor 1.2 **(Reversal Law for Multiple Factors)** By induction on Thm 1.1(4):

$$(\mathbf{A}_1\mathbf{A}_2\cdots\mathbf{A}_k)^T = \mathbf{A}_k^T\cdots\mathbf{A}_2^T\mathbf{A}_1^T.$$

Ex 1.7 Refer to Def 1.11 and Thm 1.1. Let $\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$.

1. State the dimensions of $\mathbf{A}$, $\mathbf{A}^T$, $\mathbf{AA}^T$, and $\mathbf{A}^T\mathbf{A}$.
2. Compute $\mathbf{A}^T$ explicitly. Then compute $\mathbf{AA}^T$ and verify it is symmetric using Def 1.12.
3. In NumPy (0-based indexing), write the expression to retrieve element a_{12} (first row, second column in 1-based notation). Then write a one-liner for $\mathbf{AA}^T$.
4. Is $\mathbf{AA}^T = \mathbf{A}^T\mathbf{A}$ in general? What can you say about each product individually?

Cor 1.3 **(Outer Product is Symmetric of Rank 1)** For any nonzero $\boldsymbol{x} \in \mathbb{R}^n$, the outer product $\boldsymbol{x}\boldsymbol{x}^T \in \mathbb{R}^{n\times n}$ is symmetric and has rank 1. The inner product $\boldsymbol{x}^T\boldsymbol{x} \in \mathbb{R}$ satisfies $\boldsymbol{x}^T\boldsymbol{x} \geq 0$ with equality iff $\boldsymbol{x} = \mathbf{0}$.

Ex 1.8 In this exercise, we will prove Cor 1.3. Refer to Def 1.11 and Thm 1.1.
Let $\boldsymbol{x} = (x_1, \ldots, x_n)^T \in \mathbb{R}^n$ be a nonzero column vector.

1. The **outer product** is $\boldsymbol{x}\boldsymbol{x}^T \in \mathbb{R}^{n\times n}$. Write out entry (i,j) explicitly.
2. Use the Reversal Law (Thm 1.1(4)) to show $\boldsymbol{x}\boldsymbol{x}^T$ is symmetric.
3. Show that $\boldsymbol{x}\boldsymbol{x}^T$ has rank 1 by arguing about the column space. (*Hint: every column of $\boldsymbol{x}\boldsymbol{x}^T$ is a scalar multiple of $\boldsymbol{x}$.*)
4. Express $\boldsymbol{x}^T\boldsymbol{x}$ as a sum of squares. Conclude $\boldsymbol{x}^T\boldsymbol{x} \geq 0$ with equality iff $\boldsymbol{x} = \mathbf{0}$, and identify the geometric quantity $\sqrt{\boldsymbol{x}^T\boldsymbol{x}}$.

1.3 Matrix Arithmetic

We are all familiar with scalar arithmetic: addition, subtraction, multiplication, and properties such as associativity and distributivity. In the following sections we extend these operations to vectors and matrices, building the algebraic machinery used throughout scientific computing.

1.3.1 Addition and Subtraction

Matrix addition generalizes scalar addition by operating entry-wise. The key constraint is dimensional compatibility: two matrices can be added only when they share the same shape.

Def 1.13 **(Matrix Sum)** Let $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m\times n}$. Their sum is defined as

$$\mathbf{A}+\mathbf{B} = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{pmatrix} + \begin{pmatrix} b_{11} & \cdots & b_{1n} \\ \vdots & \ddots & \vdots \\ b_{n1} & \cdots & b_{nn} \end{pmatrix} = \begin{pmatrix} (a+b)_{11} & \cdots & (a+b)_{1n} \\ \vdots & \ddots & \vdots \\ (a+b)_{n1} & \cdots & (a+b)_{nn} \end{pmatrix},$$

where each entry satisfies $(a+b)_{ij} = a_{ij} + b_{ij}$ for $1 \leq i \leq m$ and $1 \leq j \leq n$.

Note 1.9 For $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{B} \in \mathbb{R}^{p\times q}$, the sum $\mathbf{A}+\mathbf{B}$ is defined if and only if $m = p$ and $n = q$.

Ex 1.9 Refer to Def 1.13.

1. Let $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$. Compute $\mathbf{A}+\mathbf{B}$ and $2\mathbf{A}-\mathbf{B}$ by hand.
2. In NumPy, verify your answers using `np.array`. Then check whether `np.array([1,2,3])` + `np.array([1,2])` raises an error and explain why.

Commutativity and Associativity of Matrix Addition

Let

$$\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \quad \text{and} \quad \mathbf{B} = \begin{pmatrix} e & f \\ g & h \end{pmatrix}.$$

Then,

$$\mathbf{A}+\mathbf{B} = \begin{pmatrix} a+e & b+f \\ c+g & d+h \end{pmatrix} \quad \text{and} \quad \mathbf{B}+\mathbf{A} = \begin{pmatrix} e+a & f+b \\ g+c & h+d \end{pmatrix}.$$

Since addition of real numbers is commutative, $\mathbf{A}+\mathbf{B}=\mathbf{B}+\mathbf{A}$. Furthermore, for any third matrix $\mathbf{C}=\begin{pmatrix} i & j \\ k & l \end{pmatrix}$,

$$(\mathbf{A}+\mathbf{B})+\mathbf{C}=\begin{pmatrix} (a+e)+i & (b+f)+j \\ (c+g)+k & (d+h)+l \end{pmatrix},$$

and

$$\mathbf{A}+(\mathbf{B}+\mathbf{C})=\begin{pmatrix} a+(e+i) & b+(f+j) \\ c+(g+k) & d+(h+l) \end{pmatrix}.$$

By associativity of real addition, these are identical. Hence matrix addition is both commutative and associative.

Thm 1.4 **($\mathbb{R}^{m\times n}$ as a Vector Space)** The set $\mathbb{R}^{m\times n}$ equipped with matrix addition and scalar multiplication forms a real vector space of dimension mn.

Proof *Proof.* Each axiom follows from the corresponding property of $\mathbb{R}$ applied entry-wise. For **commutativity**:

$$(\mathbf{A}+\mathbf{B})_{ij}=a_{ij}+b_{ij}=b_{ij}+a_{ij}=(\mathbf{B}+\mathbf{A})_{ij}.$$

The **additive identity** is $\mathbf{0}$ since $(\mathbf{A}+\mathbf{0})_{ij}=a_{ij}$. The **additive inverse** of $\mathbf{A}$ is $-\mathbf{A}$ (negate every entry), satisfying $(\mathbf{A}+(-\mathbf{A}))_{ij}=0$.
For the **dimension**: the mn matrices $\mathbf{E}^{(ij)}$ with a 1 in position (i,j) and zeros elsewhere are linearly independent, and every $\mathbf{A}\in\mathbb{R}^{m\times n}$ decomposes as $\mathbf{A}=\sum_{i,j} a_{ij}\,\mathbf{E}^{(ij)}$, so they form a basis of size mn. □

Def 1.14 **(Scalar Multiplication)** Let $\mathbf{A}\in\mathbb{R}^{m\times n}$ and $c\in\mathbb{R}$. Then scalar multiplication is defined as

$$c\mathbf{A}=c\begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{pmatrix}=\begin{pmatrix} ca_{11} & \cdots & ca_{1n} \\ \vdots & \ddots & \vdots \\ ca_{n1} & \cdots & ca_{nn} \end{pmatrix},$$

where each entry satisfies $(ca)_{ij}=c\cdot a_{ij}$.

Distributivity of Scalar Multiplication over Matrix Addition

Let $\alpha\in\mathbb{R}$ and

$$\mathbf{A}=\begin{pmatrix} a & b \\ c & d \end{pmatrix},\quad \mathbf{B}=\begin{pmatrix} e & f \\ g & h \end{pmatrix}.$$

Then,

$$\alpha(\mathbf{A}+\mathbf{B})=\alpha\begin{pmatrix} a+e & b+f \\ c+g & d+h \end{pmatrix}=\begin{pmatrix} \alpha(a+e) & \alpha(b+f) \\ \alpha(c+g) & \alpha(d+h) \end{pmatrix}.$$

Since scalar multiplication distributes over addition for real numbers, this equals

$$\begin{pmatrix} \alpha a+\alpha e & \alpha b+\alpha f \\ \alpha c+\alpha g & \alpha d+\alpha h \end{pmatrix}=\alpha\mathbf{A}+\alpha\mathbf{B}.$$

Ex 1.10 Refer to Def 1.14 and Thm 1.4.

1. Compute $3\mathbf{A} - 2\mathbf{B}$ for $\mathbf{A} = \begin{pmatrix} 1 & 0 \\ -1 & 2 \end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} 2 & 1 \\ 0 & -1 \end{pmatrix}$.

2. The standard basis for $\mathbb{R}^{2\times 2}$ consists of four matrices $\mathbf{E}^{(ij)}$. Write them out explicitly and verify that $\mathbf{A} = \sum_{i,j} a_{ij}\mathbf{E}^{(ij)}$ holds for your $\mathbf{A}$ above.

1.3.2 Dot Product

Before defining the general matrix product, we introduce the dot product of two vectors. The matrix product will then be understood entry-wise as a collection of dot products.

Def 1.15 **(Dot Product)** For vectors $\boldsymbol{a}, \boldsymbol{b} \in \mathbb{R}^n$, the **dot product** (or inner product) is defined as

$$\boldsymbol{a} \cdot \boldsymbol{b} = \boldsymbol{a}^T \boldsymbol{b} = \begin{pmatrix} a_1 & \cdots & a_n \end{pmatrix} \begin{pmatrix} b_1 \\ \vdots \\ b_n \end{pmatrix} = \sum_{i=1}^{n} a_i b_i. \tag{1}$$

Ex 1.11 Refer to Def 1.15.

1. Compute $\boldsymbol{a} \cdot \boldsymbol{b}$ for $\boldsymbol{a} = (1, 2, 3)^T$ and $\boldsymbol{b} = (-1, 0, 2)^T$ by hand.

2. In NumPy, evaluate this using `np.dot(a, b)` and also with `a @ b`. Confirm both give the same result.

3. Show that $\boldsymbol{a} \cdot \boldsymbol{a} \geq 0$ with equality iff $\boldsymbol{a} = \mathbf{0}$. What geometric quantity does $\sqrt{\boldsymbol{a} \cdot \boldsymbol{a}}$ represent?

Def 1.16 **(Matrix Product)** Let $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{B} \in \mathbb{R}^{n\times p}$. Their product $\mathbf{AB}$ is an $m \times p$ matrix defined by

$$(\mathbf{AB})_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj},$$

or equivalently, if $\boldsymbol{a}_i$ denotes the i-th row of $\mathbf{A}$ and $\boldsymbol{b}_j$ the j-th column of $\mathbf{B}$:

$$\mathbf{AB} = \begin{pmatrix} \boldsymbol{a}_1 \cdot \boldsymbol{b}_1 & \cdots & \boldsymbol{a}_1 \cdot \boldsymbol{b}_p \\ \vdots & \ddots & \vdots \\ \boldsymbol{a}_m \cdot \boldsymbol{b}_1 & \cdots & \boldsymbol{a}_m \cdot \boldsymbol{b}_p \end{pmatrix}.$$

Note 1.10 Matrix multiplication is defined only when the number of columns of $\mathbf{A}$ equals the number of rows of $\mathbf{B}$. It is associative and distributive over addition, but in general not commutative: $\mathbf{AB} \neq \mathbf{BA}$ for most matrices.

Lem 1.5 **(Computational Cost of Matrix Multiplication)** Computing $\mathbf{AB}$ where $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\mathbf{B} \in \mathbb{R}^{n\times p}$ requires exactly mnp multiplications and $m(n-1)p$ additions using the

standard algorithm, giving $O(mnp)$ floating-point operations (flops). For two square $n \times n$ matrices this reduces to $O(n^3)$ flops.

Note 1.11 The $O(n^3)$ cost of naive matrix multiplication is a central bottleneck in scientific computing. Strassen's algorithm reduces this to $O(n^{\log_2 7}) \approx O(n^{2.807})$, but is rarely used in practice due to numerical stability concerns and cache-unfriendly memory access. Modern BLAS/LAPACK implementations approach peak hardware throughput via blocking strategies, but the asymptotic cost remains $O(n^3)$. For many applications, avoiding explicit matrix-matrix products entirely (e.g., exploiting sparsity or factored structure) is more impactful than algorithmic improvements.

Compatibility for Matrix Multiplication

Let $\mathbf{A}$ be a 3×4 matrix and $\mathbf{B}$ a 4×2 matrix. Since the number of columns of $\mathbf{A}$ (4) equals the number of rows of $\mathbf{B}$ (4), the product $\mathbf{AB}$ is defined and results in a 3×2 matrix. Conversely, if $\mathbf{B}$ were a 5×2 matrix, the product would be undefined.

Distributive Property of Matrix Multiplication

Let

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}, \quad \mathbf{C} = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix}.$$

Then,

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{A} \begin{pmatrix} b_{11} + c_{11} & b_{12} + c_{12} \\ b_{21} + c_{21} & b_{22} + c_{22} \end{pmatrix},$$

and by computing the entries using Def 1.16, one can verify that

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}.$$

Non-Commutativity of Matrix Multiplication

Consider

$$\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} \quad \text{and} \quad \mathbf{B} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Then,

$$\mathbf{AB} = \begin{pmatrix} 0 & 1 \\ 2 & 0 \end{pmatrix} \quad \text{and} \quad \mathbf{BA} = \begin{pmatrix} 0 & 2 \\ 1 & 0 \end{pmatrix}.$$

Since $\mathbf{AB} \neq \mathbf{BA}$, matrix multiplication is not commutative.

Ex 1.12 Refer to Def 1.16 and Lem 1.5. Let $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} 1 & -1 \\ 2 & 4 \end{pmatrix}$.

1. Compute $\mathbf{AB}$ and $\mathbf{BA}$ by hand. Confirm $\mathbf{AB} \neq \mathbf{BA}$.
2. Find a nonzero matrix $\mathbf{C} \neq \mathbf{I}$ that commutes with $\mathbf{A}$ (i.e., $\mathbf{AC} = \mathbf{CA}$). *Hint: try* $\mathbf{C} = \alpha\mathbf{A} + \beta\mathbf{I}$.

3. Verify associativity: compute $(\mathbf{AB})\boldsymbol{c}$ and $\mathbf{A}(\mathbf{B}\boldsymbol{c})$ for $\boldsymbol{c} = (1, 0)^T$ and confirm they are equal.

4. Using Lem 1.5, compare the flop count of $(\mathbf{AB})\boldsymbol{c}$ versus $\mathbf{A}(\mathbf{B}\boldsymbol{c})$ for general $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n\times n}$ and $\boldsymbol{c} \in \mathbb{R}^n$. Which order is cheaper and by what factor?

Many properties of scalar arithmetic extend naturally to vectors and matrices. In fact, matrices can be viewed as elements of a higher-dimensional vector space (see Thm 1.4), and matrix multiplication can be interpreted as the composition of linear transformations. While the notations for vectors and matrices are similar, their roles differ:

- Vectors typically represent points or directions in space.
- Matrices often represent linear maps or transformations between vector spaces.

This conceptual distinction becomes crucial when discussing eigenvalues and linear transformations.

Cor 1.6 **(Parenthesization Reduces Cost from $O(n^3)$ to $O(n^2)$)** For $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n\times n}$ and $\boldsymbol{x} \in \mathbb{R}^n$, computing $\mathbf{A}(\mathbf{B}\boldsymbol{x})$ via two matrix-vector products costs $O(n^2)$ flops, whereas forming $(\mathbf{AB})\boldsymbol{x}$ via a matrix-matrix product costs $O(n^3)$ flops.

Ex 1.13 In this exercise, we will prove Cor 1.6. Refer to Lem 1.5.
Suppose $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n\times n}$ and $\boldsymbol{x} \in \mathbb{R}^n$.

1. *Approach 1:* Compute $\mathbf{M} = \mathbf{AB}$ first (cost?), then $\mathbf{M}\boldsymbol{x}$ (cost?). Write the total flop count.

2. *Approach 2:* Compute $\boldsymbol{y} = \mathbf{B}\boldsymbol{x}$ first (cost?), then $\mathbf{A}\boldsymbol{y}$ (cost?). Write the total flop count.

3. Conclude the asymptotic improvement factor as $n \to \infty$. This proves Cor 1.6.

4. Write NumPy code for both approaches using `@`. Does NumPy automatically choose the optimal evaluation order for a chain like `A @ B @ x`?

1.3.3 Matrix Inverse

We now turn to a fundamental question: when can we "divide" by a matrix? The matrix inverse plays the role of reciprocal in scalar arithmetic, but it exists only under a specific condition, and computing it explicitly is almost always the wrong approach in practice.

Def 1.17 **(Matrix Inverse)** For a square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, the **inverse** of $\mathbf{A}$, denoted $\mathbf{A}^{-1}$, is the unique matrix satisfying

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I},$$

where $\mathbf{I}$ is the $n \times n$ identity matrix. A matrix that has an inverse is called *invertible* or *nonsingular*.

Note 1.12 A square matrix $\mathbf{A}$ is invertible if and only if $\det(\mathbf{A}) \neq 0$, which also implies that $\mathbf{A}$ has full rank.

Uniqueness of the Inverse

Suppose $\mathbf{A}$ is invertible and both $\mathbf{B}$ and $\mathbf{C}$ satisfy the conditions of Def 1.17. Then,

$$\mathbf{B} = \mathbf{BI} = \mathbf{B}(\mathbf{AC}) = (\mathbf{BA})\mathbf{C} = \mathbf{IC} = \mathbf{C}.$$

Thus the inverse of $\mathbf{A}$ is unique.

Thm 1.7 **(Properties of the Matrix Inverse)** Let $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n\times n}$ be invertible and $\alpha \in \mathbb{R} \setminus \{0\}$. Then:

1. $(\mathbf{A}^{-1})^{-1} = \mathbf{A}$
2. $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$ **(Reversal Law)**
3. $(\alpha\mathbf{A})^{-1} = \frac{1}{\alpha}\mathbf{A}^{-1}$
4. $(\mathbf{A}^T)^{-1} = (\mathbf{A}^{-1})^T$

Proof *Proof.* **(1):** $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$ and $\mathbf{AA}^{-1} = \mathbf{I}$ show $\mathbf{A}$ is both a left and right inverse of $\mathbf{A}^{-1}$, so $(\mathbf{A}^{-1})^{-1} = \mathbf{A}$.
(2): Check the right-inverse condition:

$$(\mathbf{AB})(\mathbf{B}^{-1}\mathbf{A}^{-1}) = \mathbf{A}(\mathbf{BB}^{-1})\mathbf{A}^{-1} = \mathbf{AIA}^{-1} = \mathbf{AA}^{-1} = \mathbf{I}.$$

Left-inverse: $(\mathbf{B}^{-1}\mathbf{A}^{-1})(\mathbf{AB}) = \mathbf{B}^{-1}(\mathbf{A}^{-1}\mathbf{A})\mathbf{B} = \mathbf{I}$. By uniqueness of the inverse, $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$.
(3): $(\alpha\mathbf{A})\left(\frac{1}{\alpha}\mathbf{A}^{-1}\right) = \frac{\alpha}{\alpha}\mathbf{AA}^{-1} = \mathbf{I}$.
(4): Transpose both sides of $\mathbf{AA}^{-1} = \mathbf{I}$ using the Reversal Law (see the transpose analogue): $(\mathbf{A}^{-1})^T\mathbf{A}^T = \mathbf{I}$, so $(\mathbf{A}^T)^{-1} = (\mathbf{A}^{-1})^T$. □

Note 1.13 Property (2) is critical in scientific computing. To solve $(\mathbf{AB})\boldsymbol{x} = \boldsymbol{c}$, we should *not* compute $(\mathbf{AB})^{-1}$ explicitly (large constant factor, numerical error amplification). Instead, use the Reversal Law: first solve $\mathbf{A}\boldsymbol{y} = \boldsymbol{c}$ for $\boldsymbol{y}$ via LU factorization, then solve $\mathbf{B}\boldsymbol{x} = \boldsymbol{y}$. The function `np.linalg.solve` does exactly this.

Ex 1.14 Refer to Def 1.17 and Thm 1.7.

1. For $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 5 & 3 \end{pmatrix}$, compute $\mathbf{A}^{-1}$ by hand using the 2×2 formula $\mathbf{A}^{-1} = \frac{1}{\det \mathbf{A}}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$. Verify $\mathbf{AA}^{-1} = \mathbf{I}$.
2. Given $\mathbf{B} = \begin{pmatrix} 1 & 0 \\ -1 & 1 \end{pmatrix}$, use the Reversal Law (Thm 1.7(2)) to compute $(\mathbf{AB})^{-1}$ without inverting a new 2×2 system.

3. Explain why `np.linalg.solve(A, b)` is preferred over `np.linalg.inv(A) @ b` for solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$. Address both numerical accuracy and computational cost.

Def 1.18 **(Involutory Matrix)** A square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ is called **involutory** if $\mathbf{A}^2 = \mathbf{I}$.

Ex 1.15 In this exercise, we will prove that a matrix is involutory (Def 1.18) if and only if it is its own inverse. Refer to Def 1.17 and Thm 1.7.

1. Suppose $\mathbf{A}^2 = \mathbf{I}$. Multiply both sides of $\mathbf{AA} = \mathbf{I}$ on the right by $\mathbf{A}^{-1}$ to deduce $\mathbf{A}^{-1} = \mathbf{A}$.
2. Conversely, suppose $\mathbf{A}^{-1} = \mathbf{A}$. Show $\mathbf{A}^2 = \mathbf{I}$ directly from Def 1.17.
3. Give a concrete 2×2 example of an involutory matrix other than $\pm\mathbf{I}$. Verify it satisfies $\mathbf{A}^2 = \mathbf{I}$.

Ex 1.16 **(Identifying Invertibility)** For each matrix below, determine without computing the inverse whether it is invertible. Give a one-sentence justification.

$$(a)\ \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix} \qquad (b)\ \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \qquad (c)\ \begin{pmatrix} 3 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & -1 \end{pmatrix} \qquad (d)\ \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 2 & 1 \end{pmatrix}$$

1.4 Introduction to Norms

A *norm* is a function that assigns a non-negative scalar "size" to a vector or matrix. Norms are the fundamental tool for measuring errors, bounding perturbations, and defining convergence throughout scientific computing. This subsection introduces the key vector norm definitions and the families most used in practice. We will study norms more rigorously in Section 3.2, where we develop matrix norms, operator inequalities, Cauchy-Schwarz, Hölder's inequality, and the condition number.

Def 1.19 **(Vector Norm)** A function $\|\cdot\| : \mathbb{R}^n \to \mathbb{R}$ is a *vector norm* if for all $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$ and $c \in \mathbb{R}$:

1. $\|\boldsymbol{x}\| \geq 0$ and $\|\boldsymbol{x}\| = 0 \Leftrightarrow \boldsymbol{x} = \mathbf{0}$ (positive definiteness)
2. $\|c\boldsymbol{x}\| = |c|\|\boldsymbol{x}\|$ (absolute homogeneity)
3. $\|\boldsymbol{x} + \boldsymbol{y}\| \leq \|\boldsymbol{x}\| + \|\boldsymbol{y}\|$ (triangle inequality)

Def 1.20 **(ℓ^p Norms)** For $\boldsymbol{x} \in \mathbb{R}^n$ and $p \geq 1$, the ℓ^p *norm* is

$$\|\boldsymbol{x}\|_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p}.$$

The three most common cases are:

- $\|\boldsymbol{x}\|_1 = \sum_{i=1}^{n} |x_i|$ (*Manhattan norm* — sum of absolute values)
- $\|\boldsymbol{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2} = \sqrt{\boldsymbol{x}^T\boldsymbol{x}}$ (*Euclidean norm*)
- $\|\boldsymbol{x}\|_\infty = \max_{1\le i\le n} |x_i|$ (*max norm*, the limit as $p \to \infty$)

Example For $\boldsymbol{x} = (3, -4, 0)^T$:

$$\|\boldsymbol{x}\|_1 = 3 + 4 + 0 = 7, \qquad \|\boldsymbol{x}\|_2 = \sqrt{9 + 16 + 0} = 5, \qquad \|\boldsymbol{x}\|_\infty = 4.$$

The unit balls $\{\boldsymbol{x} : \|\boldsymbol{x}\|_p \le 1\}$ in $\mathbb{R}^2$ are: a diamond ($p = 1$), a circle ($p = 2$), and a square ($p = \infty$). As p increases, the unit ball grows toward the ℓ^∞ square.

Note 1.14 Throughout these notes, $\|\cdot\|$ without a subscript denotes the Euclidean norm $\|\cdot\|_2$ unless stated otherwise. Errors and residuals are almost always measured in $\|\cdot\|_2$. In NumPy: `np.linalg.norm(x)` computes $\|\boldsymbol{x}\|_2$; `np.linalg.norm(x, ord=p)` computes $\|\boldsymbol{x}\|_p$.

Thm 1.8 **(All Norms are Equivalent on $\mathbb{R}^n$)** For any two norms $\|\cdot\|_\alpha$ and $\|\cdot\|_\beta$ on $\mathbb{R}^n$, there exist constants $c_1, c_2 > 0$ such that

$$c_1\|\boldsymbol{x}\|_\beta \le \|\boldsymbol{x}\|_\alpha \le c_2\|\boldsymbol{x}\|_\beta \quad \text{for all } \boldsymbol{x} \in \mathbb{R}^n.$$

In particular: $\|\boldsymbol{x}\|_2 \le \|\boldsymbol{x}\|_1 \le \sqrt{n}\,\|\boldsymbol{x}\|_2$ and $\|\boldsymbol{x}\|_\infty \le \|\boldsymbol{x}\|_2 \le \sqrt{n}\,\|\boldsymbol{x}\|_\infty$.

Proof *Proof.* Since $\mathbb{R}^n$ is finite-dimensional, the unit sphere $\{\boldsymbol{x} : \|\boldsymbol{x}\|_\beta = 1\}$ is compact. Any norm is a continuous function, so it attains its minimum and maximum on a compact set — these extremal values are c_1 and $1/c_2$. The explicit inequalities follow from Cauchy-Schwarz: $\|\boldsymbol{x}\|_1 = \sum |x_i| \cdot 1 \le \|\boldsymbol{x}\|_2\,\|\mathbf{1}\|_2 = \sqrt{n}\|\boldsymbol{x}\|_2$. □

Ex 1.17 Refer to Defs 1.19–1.20 and Thm 1.8.

1. For $\boldsymbol{x} = (-1, 2, -3, 4)^T$, compute $\|\boldsymbol{x}\|_1$, $\|\boldsymbol{x}\|_2$, $\|\boldsymbol{x}\|_\infty$ by hand. Verify all three norm-equivalence inequalities hold.
2. Verify the triangle inequality $\|\boldsymbol{x} + \boldsymbol{y}\|_2 \le \|\boldsymbol{x}\|_2 + \|\boldsymbol{y}\|_2$ for $\boldsymbol{x} = (1, 2)^T$, $\boldsymbol{y} = (-3, 1)^T$ by computing both sides.
3. Using NumPy, confirm your hand calculations with `np.linalg.norm(x, 1)`, `np.linalg.norm(x` and `np.linalg.norm(x, np.inf)`.
4. Show that $\|\boldsymbol{x}\|_\infty = \lim_{p\to\infty} \|\boldsymbol{x}\|_p$. (*Hint: let $M = \max_i |x_i|$; factor M out of $\|\boldsymbol{x}\|_p$ and take the limit.*)

1.5 Computational Complexity

Every algorithm in scientific computing carries a cost: the number of arithmetic operations it performs. Understanding that cost determines whether an algorithm is practical at scale and lets us compare competing methods objectively. The flop counts derived here will be referenced throughout the course whenever we compare algorithms.

1.5.1 Big-O Notation

Def 1.21 **(Big-O Notation)** We write $f(n) = O(g(n))$ (read "f is big-O of g") if there exist constants $C > 0$ and $n_0 \geq 1$ such that

$$|f(n)| \leq C\,|g(n)| \quad \text{for all } n \geq n_0.$$

Informally, f grows no faster than g asymptotically. Related notations:

- $f = \Omega(g)$: f grows at least as fast as g (i.e., $g = O(f)$).
- $f = \Theta(g)$: f and g grow at the same rate ($f = O(g)$ and $g = O(f)$).
- $f = o(g)$: f grows strictly slower than g, i.e., $f(n)/g(n) \to 0$.

Example Common growth rates, ordered from fastest to slowest for large n:

$$O(n!) \gg O(2^n) \gg O(n^3) \gg O(n^2) \gg O(n \log n) \gg O(n) \gg O(\log n) \gg O(1).$$

For $n = 1000$: an $O(n^2)$ algorithm performs $\sim 10^6$ operations; an $O(n^3)$ algorithm performs $\sim 10^9$ — three orders of magnitude slower. The distinction between $O(n^2)$ and $O(n^3)$ is the difference between milliseconds and seconds.

Thm 1.9 **(Properties of Big-O)** Let f, g, h be positive functions of n. Then:

1. (Transitivity) If $f = O(g)$ and $g = O(h)$, then $f = O(h)$.
2. (Sum rule) $O(f) + O(g) = O(\max(f, g))$. In particular, $O(n^3) + O(n^2) = O(n^3)$.
3. (Product rule) $O(f) \cdot O(g) = O(fg)$.
4. (Constant factors) $O(cf) = O(f)$ for any constant $c > 0$.

Proof *Proof.* All four follow directly from the definition. For the sum rule: if $|f_1| \leq C_1 g$ and $|f_2| \leq C_2 h$ for large n, then $|f_1 + f_2| \leq C_1 g + C_2 h \leq (C_1 + C_2)\max(g, h)$. For the product rule: $|f_1 f_2| \leq C_1 C_2 \cdot gh$. □

Ex 1.18 Refer to Def 1.21 and Thm 1.9.

1. Show that $3n^2 + 100n + 5 = O(n^2)$ by finding explicit constants C and n_0.

2. Is $n \log n = O(n^2)$? Is $n^2 = O(n \log n)$? Justify both.

3. Rank the following by asymptotic cost for large n: (a) $5n^3+n$, (b) $100n^2$, (c) $n^2 \log n$, (d) $2^{10}n^2$.

4. An algorithm has cost $T(n) = 2T(n/2) + n$. Use the master theorem (or unroll the recurrence) to show $T(n) = O(n \log n)$.

1.5.2 Flop Counting

Def 1.22 **(Floating-Point Operation (flop))** A *floating-point operation* (*flop*) is a single arithmetic operation (addition, subtraction, multiplication, or division) on floating-point numbers. Flop counts give a hardware-independent measure of algorithmic work. Modern CPUs execute billions of flops per second (GFlop/s); GPUs achieve TFlop/s.

Thm 1.10 **(Basic Operation Costs)** For vectors $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$ and matrices $\mathbf{A} \in \mathbb{R}^{m\times n}$, $\mathbf{B} \in \mathbb{R}^{n\times p}$:

- **Dot product $\boldsymbol{x}^T\boldsymbol{y}$**: n multiplications + $(n-1)$ additions $= 2n-1 \approx 2n$ flops.
- **Matrix-vector product $\mathbf{A}\boldsymbol{x}$**: m dot products of length $n = 2mn - m \approx 2mn$ flops.
- **Matrix-matrix product $\mathbf{AB}$**: p matrix-vector products $= 2mnp$ flops; for square $n \times n$ matrices, $2n^3$ flops.
- **Triangular solve $\mathbf{L}\boldsymbol{x} = \boldsymbol{b}$** (lower triangular): n^2 flops.

Proof *Proof.* Each row i of $\mathbf{A}\boldsymbol{x}$ is the dot product $\boldsymbol{a}_i^T\boldsymbol{x}$, costing $2n-1$ flops. Summing over m rows gives $m(2n-1) = 2mn - m$ flops. For $\mathbf{AB}$, column j of the product is $\mathbf{A}\boldsymbol{b}_j$, a matrix-vector product costing $2mn - m$ flops; summing over p columns gives $p(2mn - m) \approx 2mnp$. For the triangular solve, substituting entry k of $\boldsymbol{x}$ costs $2k-1$ flops; summing $k = 1, \ldots, n$ gives $\sum_{k=1}^{n}(2k-1) = n^2$. □

Note 1.15 These baseline costs appear throughout the course. For reference: LU factorization costs $\frac{2}{3}n^3$ flops (Section 2.2); Cholesky costs $\frac{1}{3}n^3$ — half the price for symmetric positive definite systems; the QR factorization via Householder reflections costs $\frac{4}{3}n^3$ flops (Lem 3.9).

Ex 1.19 Refer to Def 1.22 and Thm 1.10.

1. Verify the exact flop count for $\mathbf{A}\boldsymbol{x}$ ($\mathbf{A} \in \mathbb{R}^{m\times n}$) by counting each multiply-add in each row.

2. A computer performs 10^9 flops/second. Estimate wall-clock times for:

   - $\mathbf{A}\boldsymbol{x}$ with $\mathbf{A} \in \mathbb{R}^{1000\times 1000}$.
   - $\mathbf{AB}$ with $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{1000\times 1000}$.
   - $\mathbf{AB}$ with $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{10000\times 10000}$.

3. In Python, time `np.dot(A, B)` for $n \in \{100, 500, 1000, 2000\}$ with random $n \times n$ matrices. Plot wall-clock time vs. n on a log-log scale and estimate the empirical exponent. Does it match $O(n^3)$?

4. Explain why the empirical exponent from the previous part may be less than 3. (*Hint: BLAS libraries exploit cache hierarchy and vectorized instructions.*)

1.6 Floating Point Arithmetic

Real numbers cannot be represented exactly on digital computers. This subsection covers how numbers are stored in the IEEE 754 floating point format, the precision limits of that representation, and the special values and arithmetic pitfalls that arise in practice. Understanding these fundamentals is a prerequisite for analyzing any numerical algorithm.

1.6.1 Floating Point Representation

Real numbers are stored in computers using a finite-bit format analogous to scientific notation. The key parameters are the number of bits for the sign, exponent, and significand.

Def 1.23 **(Floating Point Representation)** A normalized floating point number has the form

$$x = (-1)^s \times (1.f) \times 2^e,$$

where s is the sign bit, f is the fractional significand (mantissa), e is the exponent, and the leading 1 is implicit (hidden bit). The IEEE 754 standard defines two primary formats:

- **Single precision (32-bit)**: 1 sign, 8 exponent, 23 fraction bits.
- **Double precision (64-bit)**: 1 sign, 11 exponent, 52 fraction bits.

The exponent field uses a biased representation with bias $= 2^{k-1} - 1$ ($k =$ exponent bits).

Example **16-bit floating point.** With 1 sign, 5 exponent (bias = 15), and 10 fraction bits, represent -3.25:

1. Sign = 1 (negative).
2. $3.25_{10} = 11.01_2$.
3. Normalize: $1.101_2 \times 2^1$.
4. Biased exponent: $1 + 15 = 16 = 10000_2$.
5. Fraction bits: 1010000000.
6. Final: 1 10000 1010000000.

Ex 1.20 Refer to Def 1.23.

1. Encode the number +5.5 in the 16-bit format above (1 sign, 5 exponent, 10 fraction bits, bias = 15). Show each step.

2. How many distinct normalized numbers can be represented with 5 exponent bits and 10 fraction bits (excluding special values)? *Hint:* $2 \times (2^5 - 2) \times 2^{10}$.

Def 1.24 **(Machine Epsilon) Machine epsilon** $\varepsilon_{\text{mach}}$ is the difference between 1 and the next larger representable floating point number. For a system with p significand bits (including the hidden bit):

$$\varepsilon_{\text{mach}} = 2^{1-p}.$$

For any normalized floating point number x, the gap to the next representable number is approximately $\varepsilon_{\text{mach}} \cdot |x|$.

Example For IEEE double precision: $p = 52 + 1 = 53$ bits, so $\varepsilon_{\text{mach}} = 2^{-52} \approx 2.22 \times 10^{-16}$. In Python: `import sys; sys.float_info.epsilon`.

Ex 1.21 Refer to Def 1.24.

1. In Python, evaluate `1.0 + 1e-16` and `1.0 + 2e-16`. Explain both results using $\varepsilon_{\text{mach}}$.

2. Write a function `compute_eps()` that finds machine epsilon empirically (without `sys.float_info.epsilon` or `np.finfo`). *Hint: start from 1.0, halve repeatedly until* $1 + \varepsilon = 1$.

Def 1.25 **(Unit Roundoff)** The **unit roundoff** is $u = \varepsilon_{\text{mach}}/2 = 2^{-p}$. It is the maximum relative error when rounding a real number x to its nearest floating point representation $\text{fl}(x)$:

$$\frac{|\text{fl}(x) - x|}{|x|} \leq u.$$

Note 1.16 Machine epsilon is the smallest number for which $1 + \varepsilon_{\text{mach}} > 1$ in floating point. Unit roundoff is the maximum relative rounding error for any number. In IEEE round-to-nearest mode, $u = \varepsilon_{\text{mach}}/2$.

Ex 1.22 Refer to Def 1.25.

1. Confirm that $(1.0 + u) - 1.0 = u$ but $(1.0 + u/2) - 1.0 = 0$ in double precision. Explain in terms of the spacing between adjacent floats near 1.

2. How many significant decimal digits does an IEEE double carry? *Hint:* $u = 2^{-52} \approx 2.22 \times 10^{-16}$*; compute* $-\log_{10}(u)$.

Thm 1.11 **(Fundamental Theorem of Floating Point Arithmetic)** For any real x in the normalized range, the nearest floating point number satisfies

$$\mathrm{fl}(x) = x(1+\delta), \quad |\delta| \leq u.$$

Furthermore, for any floating point a, b and operation $\circ \in \{+, -, \times, \div\}$, IEEE 754 guarantees:

$$\mathrm{fl}(a \circ b) = (a \circ b)(1+\delta), \quad |\delta| \leq u.$$

Proof *Proof.* The real line near x is partitioned into intervals each containing exactly one floating point number (the left endpoint). Rounding to nearest halves the worst-case error, giving $|\delta| \leq u$. For the second statement, IEEE 754 mandates that each basic operation is computed as if in infinite precision and then rounded, ensuring the same $|\delta| \leq u$. □

Note 1.17 The IEEE 754 guarantee that each individual operation incurs at most unit roundoff is the foundation of all backward error analyses. After n sequential operations, the accumulated error grows as $O(nu)$, which is acceptable for double precision when $n \ll 10^{16}$.

Ex 1.23 Refer to Thm 1.11.

1. Suppose $a = 1.0$ and $b = -0.9999999999999999$ in double precision. What is $\mathrm{fl}(a+b)$? Is the relative error bounded by u?
2. A computation performs 10^6 floating point additions. Using the $O(nu)$ bound, estimate the worst-case accumulated relative error in double precision.

1.6.2 IEEE 754 Special Values

Beyond normalized numbers, IEEE 754 defines special bit patterns to handle results outside the representable range. Each has a distinct behavior that must be understood when writing numerical code.

Def 1.26 **(Signed Zero)** Both $+0$ and -0 are represented and compare as equal, but behave differently in arithmetic: $1/(+0) = +\infty$ and $1/(-0) = -\infty$.

Ex 1.24 Refer to Def 1.26.

1. In Python, evaluate `1.0/0.0` and `1.0/(-0.0)`. Are the results $+\infty$ and $-\infty$ respectively, or does Python raise an error?
2. Evaluate `(+0.0) == (-0.0)`. What does this confirm about signed zero?

Def 1.27 **(Subnormal Numbers)** Numbers smaller than the smallest normalized value $2^{e_{\min}}$ are represented with a leading zero bit, allowing **gradual underflow** at the cost of reduced precision. The smallest positive double-precision subnormal is $2^{-1074} \approx 4.94 \times 10^{-324}$.

Note 1.18 Subnormal numbers fill the gap between 0 and the smallest normalized number (2^{-1022} in double precision), but with fewer significant bits. A subnormal near 2^{-1073} has only 1 bit of precision, making its relative rounding error much larger than that of a normalized number.

Ex 1.25 Refer to Def 1.27.

1. What is the smallest positive subnormal number in IEEE single precision (1 sign, 8 exponent, 23 fraction bits)?
2. In Python, evaluate `5e-324` (the smallest positive double). What is `5e-324 / 2`? Why?

Def 1.28 **(Infinities)** $+\infty$ and $-\infty$ arise from overflow or division of a nonzero number by zero (e.g., `1.0/0.0`). They propagate correctly through arithmetic: $x + \infty = \infty$ for any finite x, and $\infty \cdot 0 = \text{NaN}$.

Ex 1.26 Refer to Def 1.28.

1. In Python, evaluate `float('inf') + 1`, `float('inf') - float('inf')`, and `0.0 * float('inf')`. Classify each result and explain using IEEE 754 rules.
2. Is `float('inf') > 1e308` true? What does this imply about comparisons involving infinity?

Def 1.29 **(NaN (Not a Number))** A **NaN** is produced by undefined operations: $0/0$, $\infty - \infty$, $\sqrt{-1}$. Any arithmetic involving a NaN returns a NaN, and `nan != nan` is `True` in Python (NaN is unequal to everything, including itself).

Ex 1.27 Refer to Def 1.29.

1. Explain why `float('nan') == float('nan')` returns `False`. What does this imply for checking whether a computation produced a NaN?
2. In Python, evaluate `float('nan') > 0` and `float('nan') < 0`. What are the results, and why?

1.6.3 Floating Point Arithmetic Pitfalls

Floating point arithmetic differs from real arithmetic in several fundamental ways. The following properties, while counterintuitive, are direct consequences of finite precision and must be kept in mind when designing algorithms.

Note 1.19 **Non-Associativity.** $(a \oplus b) \oplus c \neq a \oplus (b \oplus c)$ in general. Each addition rounds its result, and changing the order changes which intermediate values are rounded.

Ex 1.28

1. Using 4-significant-digit decimal arithmetic, compute $(10000. + 0.001) - 10000.$ and $10000. + (0.001 - 10000.)$. Which order is accurate? Which demonstrates *absorption*?

2. In Python, verify with `a=1e10; b=1.0; c=-1e10`: does `(a+b)+c == a+(b+c)`? Explain.

Note 1.20 **Non-Distributivity.** $a \otimes (b \oplus c) \neq (a \otimes b) \oplus (a \otimes c)$ in general, because $b \oplus c$ is rounded before multiplication, while $a \otimes b$ and $a \otimes c$ are rounded separately before addition.

Ex 1.29

1. In Python, compute `a*(b+c)` vs `a*b + a*c` for `a = 1e15`, `b = 1.0`, `c = -1.0`. Explain the discrepancy using the definition above.

2. True or False: if `a == b` in floating point, then `a - b == 0.0` always. What about the converse?

Note 1.21 **Absorption.** When $|a| \gg |b|$, the sum $\mathrm{fl}(a+b)$ may equal a exactly, because b falls below the unit roundoff relative to a. The smaller operand contributes nothing.

Ex 1.30

1. In Python, evaluate `1e16 + 1.0 - 1e16`. What do you get and why?

2. Give an example where absorption causes a running sum of n small numbers to yield zero when added to a large number, even though the true sum is nonzero.

1.7 Quantifying Approximation Error

Floating point arithmetic introduces errors at every operation. This subsection develops the vocabulary and tools for measuring those errors and reasoning about how they propagate through a computation. Absolute and relative error quantify the gap between computed and true values; catastrophic cancellation identifies a common failure mode; and forward and backward analysis provide complementary frameworks for bounding the total error in an algorithm.

1.7.1 Absolute and Relative Errors

Every floating point operation introduces a small rounding error. Understanding how these errors arise and propagate determines whether an algorithm produces trustworthy results.

Def 1.30 **(Absolute and Relative Error)** When a real number x is approximated by $\hat{x}$, the **absolute error** is $|\hat{x} - x|$, and the **relative error** is

$$\varepsilon = \frac{|\hat{x} - x|}{|x|},$$

so that $\hat{x} = x(1+\varepsilon)$.

Example Approximating $\pi \approx 3.14159\ldots$:

Approximation	Absolute Error	Relative Error
3.14	$0.00159\ldots$	0.051%
3.1	$0.0416\ldots$	1.32%
3	$0.1416\ldots$	4.51%

Relative error is usually more meaningful: a tiny absolute error can still represent a large relative error when the true value is small.

Ex 1.31 Refer to Def 1.30.

1. Approximate $\sqrt{2} \approx 1.414$. Compute the absolute and relative errors.
2. For two computations with relative errors ε_1 and ε_2, what is the relative error in their *product*? (*Hint: use* $\hat{x}\hat{y} = xy(1+\varepsilon_1)(1+\varepsilon_2) \approx xy(1+\varepsilon_1+\varepsilon_2)$ *for small errors.*)

Def 1.31 **(Catastrophic Cancellation) Catastrophic cancellation** occurs when subtracting two nearly equal numbers. If $\hat{x} = x(1+\varepsilon_x)$ and $\hat{y} = y(1+\varepsilon_y)$ with $x \approx y > 0$, the relative error in $\hat{x} - \hat{y}$ is

$$\frac{(\hat{x}-\hat{y})-(x-y)}{x-y} = \frac{x\varepsilon_x - y\varepsilon_y}{x-y},$$

which is unbounded as $x \to y^+$ — a total loss of significant digits.

Thm 1.12 **(Cancellation Error Bound)** Let $\hat{x} = x(1+\varepsilon_x)$, $\hat{y} = y(1+\varepsilon_y)$ with $|\varepsilon_x|, |\varepsilon_y| \le u$ and $x > y > 0$. Then

$$\left|\frac{(\hat{x}-\hat{y})-(x-y)}{x-y}\right| \le u \cdot \frac{x+y}{x-y}.$$

The amplification factor $(x+y)/(x-y) \to \infty$ as $x \to y^+$.

Proof *Proof.* The error is $x\varepsilon_x - y\varepsilon_y$. Taking absolute value and dividing by $x-y$:

$$\frac{|x\varepsilon_x - y\varepsilon_y|}{x-y} \le \frac{x|\varepsilon_x| + y|\varepsilon_y|}{x-y} \le u \cdot \frac{x+y}{x-y}.$$

□

Example Computing $f(x) = \sqrt{x+1} - \sqrt{x}$ for $x = 10^8$ in double precision: $\sqrt{10^8+1} \approx 10000.00000005$ and $\sqrt{10^8} = 10000$, so direct subtraction gives approximately 0 due to cancellation. Rationalizing avoids this:

$$f(x) = \frac{1}{\sqrt{x+1}+\sqrt{x}} \approx \frac{1}{20000} = 5 \times 10^{-5}.$$

Mathematical reformulation eliminates the cancellation without increasing cost.

Ex 1.32 **(Cancellation and Reformulation)** Refer to Def 1.31 and Thm 1.12. For each expression, identify whether catastrophic cancellation can occur and derive a stable equivalent form.

1. $\dfrac{1-\cos x}{x^2}$ for small x. *Hint: use* $1-\cos x = 2\sin^2(x/2)$.
2. $\sqrt{x+1}-\sqrt{x}$ for large x (rationalize the numerator).
3. The smaller root of $x^2 - 10^8 x + 1 = 0$ via the quadratic formula. What goes wrong and how do you fix it?
4. $e^x - 1$ for small x. Why does NumPy provide `np.expm1(x)`? Verify numerically that `np.expm1(1e-15)` is more accurate than `np.exp(1e-15) - 1`.

1.7.2 Forward and Backward Error Analysis

Error analysis is the systematic study of how errors enter and propagate through a computation. Forward and backward analysis are complementary frameworks; condition number analysis measures the inherent difficulty of the problem itself, independent of the algorithm.

Def 1.32 **(Forward Error Analysis)** **Forward error analysis** tracks how input errors and per-operation rounding errors propagate *forward* through the algorithm to bound the output error. Straightforward but can produce pessimistic overestimates.

Ex 1.33 Refer to Def 1.32.

1. For the computation $\hat{z} = \mathrm{fl}(\mathrm{fl}(a+b)+c)$ with a, b, c exact, use the model $\mathrm{fl}(x \circ y) = (x \circ y)(1+\delta)$, $|\delta| \le u$, to derive a forward error bound on $|\hat{z} - (a+b+c)|$.

Def 1.33 **(Backward Error Analysis)** **Backward error analysis** asks: what is the smallest perturbation to the input that would make the computed output exact? If this backward error is $O(u\|\mathbf{A}\|)$, the algorithm solved a nearby problem exactly — the best one can hope for in finite precision.

Ex 1.34 Refer to Def 1.33.

1. For solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, explain the difference between the forward error $\|\hat{\boldsymbol{x}} - \boldsymbol{x}\|/\|\boldsymbol{x}\|$ and the backward error $\|\boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}\|/(\|\mathbf{A}\|\|\hat{\boldsymbol{x}}\|)$. Which is easier to compute after the fact, and why?
2. Why is backward error analysis generally preferred over forward error analysis for evaluating numerical algorithms?

Def 1.34 **(Interval Arithmetic)** In **interval arithmetic**, each number is replaced by an enclosing interval $[a, b]$, and every operation maps intervals to intervals in a way that always contains

the true result. This produces certified error bounds but can suffer from interval explosion for long computations.

Ex 1.35 Refer to Def 1.34.

1. Compute $[1.0, 1.1] + [2.0, 2.5]$ and $[1.0, 1.1] \times [2.0, 2.5]$ using interval arithmetic rules. Verify the result contains the true values $1.05 + 2.25$ and 1.05×2.25.

Def 1.35 **(Condition Number Analysis)** The **condition number** of a problem measures its intrinsic sensitivity to small perturbations in the input, independently of the algorithm. A problem with condition number κ can lose up to $\log_{10} \kappa$ digits of accuracy even with a perfect algorithm. An algorithm is **backward stable** if its backward error is $O(u)$.

Ex 1.36 Refer to Def 1.35.

1. An algorithm has backward error 10^{-15} applied to a problem with condition number $\kappa(\mathbf{A}) = 10^8$. What is the worst-case forward error? Is this acceptable for double-precision computation?

2. In Python, construct an ill-conditioned matrix using `np.diag([1, 1e-10])`, solve with `np.linalg.solve`, and compare `np.linalg.cond(A)` with the observed forward error.

Ex 1.37 In this exercise, we will prove the backward error formula for linear systems. Refer to Def 1.33 and Thm 1.11.
Let $\hat{\boldsymbol{x}}$ be the computed solution to $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ obtained by LU with partial pivoting. Define the residual $\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}$.

1. Define $\mathbf{E} = -\boldsymbol{r}\hat{\boldsymbol{x}}^T / \|\hat{\boldsymbol{x}}\|^2$. Show by direct substitution that $(\mathbf{A} + \mathbf{E})\hat{\boldsymbol{x}} = \boldsymbol{b}$, confirming $\hat{\boldsymbol{x}}$ is the exact solution to a nearby system.

2. Show that the relative backward error satisfies $\|\mathbf{E}\|_2 / \|\mathbf{A}\|_2 = \|\boldsymbol{r}\|_2 / (\|\mathbf{A}\|_2 \|\hat{\boldsymbol{x}}\|_2)$. (*Hint: for a rank-1 matrix,* $\|\boldsymbol{r}\boldsymbol{v}^T\|_2 = \|\boldsymbol{r}\|_2 \|\boldsymbol{v}\|_2$.)

3. For a well-conditioned system, a small backward error implies a small forward error. Write the bound
$$\frac{\|\hat{\boldsymbol{x}} - \boldsymbol{x}\|_2}{\|\boldsymbol{x}\|_2} \leq \kappa(\mathbf{A}) \cdot \frac{\|\mathbf{E}\|_2}{\|\mathbf{A}\|_2},$$
where $\kappa(\mathbf{A}) = \|\mathbf{A}\|_2 \|\mathbf{A}^{-1}\|_2$. If $\kappa(\mathbf{A}) = 10^{10}$ and the backward error is 10^{-16}, what is the worst-case forward error?

4. In Python, construct an ill-conditioned matrix using `np.diag([1, 1e-10])`, solve with `np.linalg.solve`, and compare `np.linalg.cond(A)` with the observed forward error.

Floating point arithmetic is not real arithmetic. Its inherent limitations shape the design of every numerical algorithm. By understanding representation, error propagation, and special cases, we can build algorithms that produce reliable results. Mathematical reformulations, careful ordering of operations, and backward error analysis are often more effective than simply increasing precision.

1.8 Python and NumPy Primer

Every exercise in these notes uses Python with NumPy. This subsection is a condensed reference — not a full tutorial. If you are new to NumPy, work through the exercises here before proceeding; if you are experienced, use this as a quick lookup.

1.8.1 Arrays and Basic Operations

Def 1.36 **(NumPy Arrays)** The core object in NumPy is the `ndarray`. Key creation patterns:

- `np.array([1.0, 2.0, 3.0])` — 1-D array (vector) from a Python list.
- `np.array([[1, 2], [3, 4]])` — 2-D array (matrix).
- `np.zeros((m, n))`, `np.ones((m, n))`, `np.eye(n)` — special matrices.
- `np.random.randn(m, n)` — $m \times n$ matrix with i.i.d. $\mathcal{N}(0, 1)$ entries.
- `np.arange(start, stop, step)`, `np.linspace(a, b, n)` — ranges.

Arrays are 0-indexed: `A[i, j]` accesses row i, column j; `A[i, :]` is row i; `A[:, j]` is column j.

Note 1.22 **Shape and dtype.** `A.shape` returns a tuple `(m, n)`; `x.shape` for a 1-D array returns `(n,)`, *not* `(n, 1)`. Use `x.reshape(-1, 1)` or `x[:, None]` to convert to a column vector. The default floating-point dtype is `float64` (IEEE double precision, $\varepsilon_{\text{mach}} \approx 2.2 \times 10^{-16}$).

Ex 1.38 Refer to Def 1.36.

1. Create a 4×4 identity matrix. Print its `shape` and `dtype`.
2. Create a 3×3 random matrix `A`. Extract its second row and third column as separate arrays.
3. Verify that `np.zeros((3,))` and `np.zeros((3, 1))` have different shapes. What error occurs if you pass a shape-`(3,)` array where a column vector is expected in a matrix-vector multiply?

1.8.2 Linear Algebra Operations

Def 1.37 **(Key `np.linalg` Functions)** The `numpy.linalg` module provides the core linear algebra routines used throughout this course:

Function	Computes
`A @ B` (or `np.dot(A, B)`)	Matrix product $\mathbf{AB}$
`np.linalg.solve(A, b)`	Solution $\boldsymbol{x}$ to $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$
`np.linalg.inv(A)`	Matrix inverse $\mathbf{A}^{-1}$ (avoid; use `solve`)
`np.linalg.det(A)`	Determinant $\det(\mathbf{A})$
`np.linalg.norm(x)`	Euclidean norm $\|\boldsymbol{x}\|_2$
`np.linalg.norm(A, 'fro')`	Frobenius norm $\|\mathbf{A}\|_F$
`np.linalg.eig(A)`	Eigenvalues and eigenvectors of $\mathbf{A}$
`np.linalg.eigh(A)`	Eigenvalues of symmetric $\mathbf{A}$ (more stable)
`np.linalg.svd(A)`	Full SVD: returns $\mathbf{U}$, $\mathbf{\Sigma}$, $\mathbf{V}^T$
`np.linalg.qr(A)`	QR factorization: returns $\mathbf{Q}$, $\mathbf{R}$
`np.linalg.cond(A)`	Condition number $\kappa_2(\mathbf{A}) = \sigma_{\max}/\sigma_{\min}$

Note 1.23 **Use `solve`, not `inv`.** `np.linalg.solve(A, b)` computes the solution to $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ via LU factorization in $O(n^3)$ flops. The alternative `np.linalg.inv(A) @ b` also costs $O(n^3)$ but performs an extra matrix-vector multiply and accumulates more rounding error. Computing $\mathbf{A}^{-1}$ explicitly is almost never necessary or desirable in numerical computing.

Ex 1.39 Refer to Def 1.37.

1. Solve $\begin{pmatrix} 2 & 1 \\ 5 & 7 \end{pmatrix} \boldsymbol{x} = \begin{pmatrix} 11 \\ 13 \end{pmatrix}$ using `np.linalg.solve`. Verify your solution by computing the residual $\mathbf{A}\boldsymbol{x} - \boldsymbol{b}$.

2. For a random 500×500 matrix, time `np.linalg.solve(A, b)` vs. `np.linalg.inv(A) @ b` over 10 trials. Which is faster, and by how much?

3. Use `np.linalg.eig` on a 3×3 symmetric matrix. Do any eigenvalues have nonzero imaginary parts? Repeat with `np.linalg.eigh` and compare the results.

1.8.3 Useful Idioms

Note 1.24 **Element-wise vs. matrix operations.** `*` between two arrays is *element-wise* multiplication (Hadamard product), not matrix multiplication. Use `@` for matrix products. Similarly, `A**2` squares every entry; `np.linalg.matrix_power(A, 2)` computes $\mathbf{A}^2 = \mathbf{AA}$.

Note 1.25 **Broadcasting.** NumPy broadcasts operations across dimensions when shapes are compatible. `A + 1` adds 1 to every entry of $\mathbf{A}$; `A * v` for a length-n vector `v` scales column j of an $m \times n$ matrix by `v[j]`. Broadcasting avoids explicit loops and is critical for performance.

Note 1.26 **Views vs. copies.** `B = A` does *not* copy the array — `B` is a view (alias) of the same data. Modifying `B` modifies `A`. Use `B = A.copy()` to create an independent copy. Slices (`A[i, :]`) are also views.

Ex 1.40

1. Let `A = np.array([[1,2],[3,4]])`. Compute `A * A` and `A @ A`. Which gives $\mathbf{A}^2 = \mathbf{AA}$?

2. Demonstrate the view pitfall: set `B = A`, then `B[0, 0] = 99`. What is `A[0, 0]`? Repeat with `B = A.copy()`.

3. For a diagonal scaling $\mathbf{AD}$ where $\mathbf{D} = \text{diag}(\mathbf{d})$, compute the product using broadcasting (`A * d`) instead of `A @ np.diag(d)`. Verify the two agree. Why is the broadcasting version preferred for large n?

4. In Python, construct the matrix $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ and compute $\mathbf{A}^k$ for $k = 1, 2, 5, 10$ using `np.linalg.matrix_power`. Describe the pattern in the $(1, 2)$ entry.

2 Direct Methods for Linear Systems

2.1 Linearity

Linear algebra is built around the idea of linearity: a property that lets us decompose complex problems into independent parts and recombine their solutions. Before studying specific algorithms, we establish the core vocabulary — linear systems, rank, and subspaces — that underpins everything in this course.

A mapping L is **linear** if it satisfies two properties for all inputs $\boldsymbol{x}, \boldsymbol{y}$ and scalars α, β:

- **Additivity**: $L(\boldsymbol{x} + \boldsymbol{y}) = L(\boldsymbol{x}) + L(\boldsymbol{y})$
- **Homogeneity**: $L(\alpha \boldsymbol{x}) = \alpha L(\boldsymbol{x})$

Together these give the **superposition principle**: $L(\alpha \boldsymbol{x} + \beta \boldsymbol{y}) = \alpha L(\boldsymbol{x}) + \beta L(\boldsymbol{y})$.

2.1.1 Linear Systems

A linear system packages finitely many linear equations into a single matrix equation. The central question is when such a system has solutions and how many.

Def 2.1 **(Linear System)** A **system of linear equations** in matrix-vector form is

$$\mathbf{A}\boldsymbol{x} = \boldsymbol{b},$$

where $\mathbf{A} \in \mathbb{R}^{m \times n}$ is the coefficient matrix, $\boldsymbol{x} \in \mathbb{R}^n$ the unknown vector, and $\boldsymbol{b} \in \mathbb{R}^m$ the right-hand side:

$$\mathbf{A} = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{pmatrix}, \quad \boldsymbol{x} = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} b_1 \\ \vdots \\ b_m \end{pmatrix}.$$

Ex 2.1 Refer to Def 2.1.

1. Write the system $\{x + 2y = 3,\ 3x - y = 1\}$ in matrix-vector form $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$.
2. In NumPy, solve it using `np.linalg.solve(A, b)` and verify by computing the residual `np.linalg.norm(b - A @ x)`.

Thm 2.1 **(Matrix-Vector Multiplication is Linear)** The map $T : \mathbb{R}^n \to \mathbb{R}^m$ defined by $T(\boldsymbol{x}) = \mathbf{A}\boldsymbol{x}$ is a linear transformation: for all $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$ and $\alpha, \beta \in \mathbb{R}$,

$$T(\alpha \boldsymbol{x} + \beta \boldsymbol{y}) = \alpha T(\boldsymbol{x}) + \beta T(\boldsymbol{y}).$$

Proof *Proof.* By distributivity: $\mathbf{A}(\boldsymbol{x}+\boldsymbol{y}) = \mathbf{A}\boldsymbol{x} + \mathbf{A}\boldsymbol{y} = T(\boldsymbol{x}) + T(\boldsymbol{y})$. By scalar compatibility: $\mathbf{A}(\alpha\boldsymbol{x}) = \alpha(\mathbf{A}\boldsymbol{x}) = \alpha T(\boldsymbol{x})$. Combining gives $T(\alpha\boldsymbol{x}+\beta\boldsymbol{y}) = \alpha T(\boldsymbol{x}) + \beta T(\boldsymbol{y})$. □

Cor 2.2 **(Superposition Principle)** If $\mathbf{A}\boldsymbol{x}_1 = \boldsymbol{b}_1$ and $\mathbf{A}\boldsymbol{x}_2 = \boldsymbol{b}_2$, then for any $\alpha, \beta \in \mathbb{R}$,

$$\mathbf{A}(\alpha\boldsymbol{x}_1 + \beta\boldsymbol{x}_2) = \alpha\boldsymbol{b}_1 + \beta\boldsymbol{b}_2.$$

Ex 2.2 In this exercise, we will prove Cor 2.2. Refer to Thm 2.1.

1. Suppose $\boldsymbol{x}_1$ solves $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}_1$ and $\boldsymbol{x}_2$ solves $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}_2$. Apply T to $\alpha\boldsymbol{x}_1 + \beta\boldsymbol{x}_2$ and use linearity to conclude $\mathbf{A}(\alpha\boldsymbol{x}_1 + \beta\boldsymbol{x}_2) = \alpha\boldsymbol{b}_1 + \beta\boldsymbol{b}_2$.
2. Use this result to solve $\mathbf{A}\boldsymbol{x} = 3\boldsymbol{b}_1 - 2\boldsymbol{b}_2$ without any new computation.
3. Why is Cor 2.2 computationally valuable when solving $\mathbf{AX} = \mathbf{B}$ for a matrix right-hand side $\mathbf{B}$? How does `np.linalg.solve(A, B)` exploit this?

Thm 2.3 **(Existence and Uniqueness)** For a square system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ with $\mathbf{A} \in \mathbb{R}^{n\times n}$, the system has a unique solution if and only if $\mathbf{A}$ is invertible ($\det(\mathbf{A}) \neq 0$). If $\mathbf{A}$ is singular, the system either has no solution or infinitely many.

Thm 2.4 **(Equivalences for Invertibility)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$. The following are equivalent:

1. $\mathbf{A}$ is invertible.
2. $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ has a unique solution for every $\boldsymbol{b} \in \mathbb{R}^n$.
3. $\mathbf{A}\boldsymbol{x} = \mathbf{0}$ has only the trivial solution $\boldsymbol{x} = \mathbf{0}$.
4. $\operatorname{rank}(\mathbf{A}) = n$.
5. The columns of $\mathbf{A}$ are linearly independent.

Proof *Proof.* (1) $\Rightarrow$ (2): Set $\boldsymbol{x} = \mathbf{A}^{-1}\boldsymbol{b}$. If $\mathbf{A}\boldsymbol{x}_1 = \mathbf{A}\boldsymbol{x}_2$, then $\mathbf{A}(\boldsymbol{x}_1 - \boldsymbol{x}_2) = \mathbf{0}$, so $\boldsymbol{x}_1 = \boldsymbol{x}_2$.
(2) $\Rightarrow$ (3): Apply (2) with $\boldsymbol{b} = \mathbf{0}$; the unique solution is $\boldsymbol{x} = \mathbf{0}$.
(3) $\Rightarrow$ (4): The null space is $\{\mathbf{0}\}$, so $\operatorname{nullity}(\mathbf{A}) = 0$. By the Rank-Nullity Theorem, $\operatorname{rank}(\mathbf{A}) = n$.
(4) $\Rightarrow$ (1): A square matrix with rank n has n linearly independent columns spanning $\mathbb{R}^n$, which is exactly the invertibility condition. □

Ex 2.3 Refer to Thm 2.4.

1. For each matrix, determine invertibility and give a one-sentence justification:

$$\mathbf{A} = \begin{pmatrix} 2 & -1 \\ 4 & -2 \end{pmatrix}, \qquad \mathbf{B} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix}.$$

2. For $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 6 \end{pmatrix}$ and $\boldsymbol{b} = \begin{pmatrix} 1 \\ 4 \end{pmatrix}$: is the system consistent, and if so, how many solutions? Verify with `np.linalg.matrix_rank`.

Def 2.2 **(Linear Combination)** A **linear combination** of vectors $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k\}$ is any vector of the form

$$\alpha_1 \boldsymbol{x}_1 + \alpha_2 \boldsymbol{x}_2 + \cdots + \alpha_k \boldsymbol{x}_k, \quad \alpha_i \in \mathbb{R}.$$

Note 2.1 Matrix-vector multiplication $\mathbf{A}\boldsymbol{x}$ is a linear combination of the columns of $\mathbf{A}$ with coefficients given by the entries of $\boldsymbol{x}$:

$$\mathbf{A}\boldsymbol{x} = x_1 \boldsymbol{a}_1 + x_2 \boldsymbol{a}_2 + \cdots + x_n \boldsymbol{a}_n,$$

where $\boldsymbol{a}_i$ is the i-th column of $\mathbf{A}$. The system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ asks: is $\boldsymbol{b}$ a linear combination of the columns of $\mathbf{A}$?

Ex 2.4 Refer to Def 2.2.

1. Express $\boldsymbol{b} = (7, -1)^T$ as a linear combination of $\boldsymbol{v}_1 = (1, 1)^T$ and $\boldsymbol{v}_2 = (2, -1)^T$ by solving a 2×2 system for the coefficients.
2. For $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$, write $\mathbf{A}\boldsymbol{x}$ explicitly as a linear combination of the columns of $\mathbf{A}$.

Def 2.3 **(Linear Dependence and Independence)** A set of vectors $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k\}$ is **linearly dependent** if there exist scalars $\alpha_1, \ldots, \alpha_k$, not all zero, such that

$$\alpha_1 \boldsymbol{x}_1 + \cdots + \alpha_k \boldsymbol{x}_k = \mathbf{0}.$$

If the only solution is $\alpha_1 = \cdots = \alpha_k = 0$, the vectors are **linearly independent**.

Note 2.2 Linear dependence means redundancy: some vector can be written as a combination of the others. If the columns of $\mathbf{A}$ are linearly dependent, then $\mathbf{A}\boldsymbol{x} = \mathbf{0}$ has a nontrivial solution, so $\mathbf{A}$ is not invertible (Thm 2.4(3)).

Ex 2.5 Refer to Def 2.3.

1. Determine whether $\{(1, 2, 3)^T, (4, 5, 6)^T, (7, 8, 9)^T\}$ is linearly dependent or independent. (*Hint: stack as columns, find the rank.*)
2. If $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m < n$, must the columns be linearly dependent? Justify.

Def 2.4 **(Basis)** A **basis** for a vector space $\mathcal{V}$ is a linearly independent spanning set. Every vector in $\mathcal{V}$ can be uniquely written as a linear combination of the basis vectors. The number of basis vectors is the **dimension** of $\mathcal{V}$.

Ex 2.6 Refer to Def 2.4 and Def 2.3.

1. Show that $\boldsymbol{e}_1 = (1,0,0)^T$, $\boldsymbol{e}_2 = (0,1,0)^T$, $\boldsymbol{e}_3 = (0,0,1)^T$ form a basis for $\mathbb{R}^3$.
2. Can a set of 4 vectors form a basis for $\mathbb{R}^3$? Why or why not?

2.1.2 Rank

Rank measures how many truly independent directions a matrix can produce. It determines whether a linear system has solutions and, if so, how many.

Def 2.5 **(Rank)** The **rank** of $\mathbf{A} \in \mathbb{R}^{m\times n}$ is the maximum number of linearly independent rows (equivalently, columns) of $\mathbf{A}$. It equals the dimension of the column space and satisfies $\operatorname{rank}(\mathbf{A}) \le \min(m,n)$.

Note 2.3 Rank measures the effective information content of $\mathbf{A}$: the number of independent directions it can produce. A matrix with low rank relative to its size has redundant rows or columns. In NumPy, `np.linalg.matrix_rank(A)` computes rank via SVD. In many applications such as data compression and machine learning, low-rank structure is actively exploited to reduce dimensionality.

Def 2.6 **(Full Rank)** A matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ is **full rank** if $\operatorname{rank}(\mathbf{A}) = \min(m,n)$. For a square matrix, full rank is equivalent to invertibility.

Example Consider

$$\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 1 & 1 \end{pmatrix}.$$

Row 2 is twice row 1. Row-reducing:

$$\xrightarrow{R_2-2R_1,\ R_3-R_1} \begin{pmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \\ 0 & -1 & -2 \end{pmatrix} \xrightarrow{\text{swap } R_2,R_3} \begin{pmatrix} 1 & 2 & 3 \\ 0 & -1 & -2 \\ 0 & 0 & 0 \end{pmatrix}.$$

Two pivot rows, so $\operatorname{rank}(\mathbf{A}) = 2$.

Ex 2.7 Refer to Def 2.5 and Def 2.6.

1. Determine the rank by row reduction:

$$\mathbf{A} = \begin{pmatrix} 1 & 3 \\ 2 & 6 \end{pmatrix}, \qquad \mathbf{B} = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & -1 \\ 2 & -1 & 5 \end{pmatrix}.$$

2. For a 5×3 matrix $\mathbf{A}$ with $\operatorname{rank}(\mathbf{A}) = 3$, what can you say about solutions to $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$?

Thm 2.5 **(Rank-Nullity Theorem)** For any $\mathbf{A} \in \mathbb{R}^{m\times n}$,

$$\operatorname{rank}(\mathbf{A}) + \operatorname{nullity}(\mathbf{A}) = n,$$

where $\text{nullity}(\mathbf{A}) = \dim(\ker(\mathbf{A}))$ and $\ker(\mathbf{A}) = \{\boldsymbol{x} \in \mathbb{R}^n : \mathbf{A}\boldsymbol{x} = \mathbf{0}\}$.

Note 2.4 For a square $n \times n$ system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$:

- $\text{rank}(\mathbf{A}) = n$: the null space is $\{\mathbf{0}\}$, so $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ has a unique solution.
- $\text{rank}(\mathbf{A}) = r < n$: if a solution exists, there are infinitely many; the solution set is an affine subspace of dimension $n - r$.

Ex 2.8 Refer to Thm 2.5. Let

$$\mathbf{A} = \begin{pmatrix} 1 & 2 & -1 \\ 2 & 5 & 0 \\ -1 & 1 & 6 \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} 1 \\ 4 \\ 3 \end{pmatrix}.$$

1. Determine $\text{rank}(\mathbf{A})$ by row-reducing $[\mathbf{A} \mid \boldsymbol{b}]$.
2. How many solutions does this system have? Justify using Thm 2.5.
3. Find the solution(s).
4. In NumPy, solve using `np.linalg.solve(A, b)` and check the residual `np.linalg.norm(b - A @ x)`. What residual do you expect for a consistent system?

Ex 2.9 **(Linear Independence and the Null Space)** Refer to Def 2.3 and Thm 2.5. Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m < n$.

1. Argue that the columns of $\mathbf{A}$ must be linearly dependent. (*Hint: rank*$(\mathbf{A}) \leq m < n$, *so nullity* > 0.)
2. What does this imply about solutions to $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$?
3. Give a 2×3 example where $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ has infinitely many solutions. Find the general solution as a particular solution plus the null space.
4. Can a wide system ever have no solution? Explain.

2.1.3 Subspaces and the Four Fundamental Subspaces

Every matrix naturally defines four subspaces that completely characterize the solution structure of $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$. These subspaces reappear in least squares, SVD, and eigenvalue problems throughout the notes.

Def 2.7 **(Subspace)** A nonempty subset $\mathcal{S} \subseteq \mathbb{R}^n$ is a **subspace** if it is closed under linear combinations:

1. $\mathbf{0} \in \mathcal{S}$,
2. $\boldsymbol{u}, \boldsymbol{v} \in \mathcal{S} \Rightarrow \boldsymbol{u} + \boldsymbol{v} \in \mathcal{S}$,

3. $\boldsymbol{u} \in \mathcal{S},\ \alpha \in \mathbb{R} \Rightarrow \alpha \boldsymbol{u} \in \mathcal{S}$.

Note 2.5 Every subspace must contain $\mathbf{0}$ (set $\alpha = 0$ in condition 3). Geometrically, subspaces are flat objects passing through the origin. Lines and planes through the origin in $\mathbb{R}^3$ are subspaces; those not passing through the origin are affine subspaces.

Ex 2.10 Refer to Def 2.7.

1. Determine whether each set is a subspace of $\mathbb{R}^2$:

$$\mathcal{S}_1 = \{(x,\, 2x)^T : x \in \mathbb{R}\}, \qquad \mathcal{S}_2 = \{(x,\, y)^T : x + y = 1\}.$$

2. Show that $\ker(\mathbf{A}) = \{\boldsymbol{x} : \mathbf{A}\boldsymbol{x} = \mathbf{0}\}$ is always a subspace of $\mathbb{R}^n$.

Def 2.8 **(The Four Fundamental Subspaces)** For $\mathbf{A} \in \mathbb{R}^{m\times n}$ of rank r, four subspaces arise naturally:

1. **Column space**: $\mathcal{R}(\mathbf{A}) \subseteq \mathbb{R}^m, \quad \dim = r$.
2. **Null space**: $\mathcal{N}(\mathbf{A}) \subseteq \mathbb{R}^n, \quad \dim = n - r$.
3. **Row space**: $\mathcal{R}(\mathbf{A}^T) \subseteq \mathbb{R}^n, \quad \dim = r$.
4. **Left null space**: $\mathcal{N}(\mathbf{A}^T) \subseteq \mathbb{R}^m, \quad \dim = m - r$.

The dimensions follow from Thm 2.5 applied to $\mathbf{A}$ and $\mathbf{A}^T$.

Ex 2.11 Refer to Def 2.8 and Thm 2.5. Let $\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{pmatrix}$.

1. State the dimension of each of the four fundamental subspaces.
2. Find explicit bases for $\mathcal{R}(\mathbf{A})$ and $\mathcal{N}(\mathbf{A})$.
3. Verify the dimensions sum correctly as required by Thm 2.5.

Thm 2.6 **(Fundamental Theorem of Linear Algebra)** The four fundamental subspaces of $\mathbf{A} \in \mathbb{R}^{m\times n}$ satisfy:

$$\mathcal{R}(\mathbf{A}^T) \perp \mathcal{N}(\mathbf{A}) \ \text{ in } \mathbb{R}^n, \qquad \mathcal{R}(\mathbf{A}) \perp \mathcal{N}(\mathbf{A}^T) \ \text{ in } \mathbb{R}^m.$$

Moreover, these are orthogonal complements: $\mathcal{R}(\mathbf{A}^T) \oplus \mathcal{N}(\mathbf{A}) = \mathbb{R}^n$ and $\mathcal{R}(\mathbf{A}) \oplus \mathcal{N}(\mathbf{A}^T) = \mathbb{R}^m$.

Proof *Proof.* If $\boldsymbol{v} \in \mathcal{R}(\mathbf{A}^T)$, then $\boldsymbol{v} = \mathbf{A}^T\boldsymbol{y}$ for some $\boldsymbol{y}$. For any $\boldsymbol{x} \in \mathcal{N}(\mathbf{A})$:

$$\boldsymbol{v}^T\boldsymbol{x} = (\mathbf{A}^T\boldsymbol{y})^T\boldsymbol{x} = \boldsymbol{y}^T(\mathbf{A}\boldsymbol{x}) = \boldsymbol{y}^T\mathbf{0} = 0.$$

Complementarity follows from Thm 2.5: dimensions sum to n and the intersection is $\{\mathbf{0}\}$. The second relation follows by applying the first to $\mathbf{A}^T$. $\square$

Note 2.6 The orthogonal complement picture has immediate payoffs for $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$:

- **Solvability**: $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ has a solution iff $\boldsymbol{b} \in \mathcal{R}(\mathbf{A})$, equivalently $\boldsymbol{b} \perp \mathcal{N}(\mathbf{A}^T)$.
- **Solution structure**: If $\hat{\boldsymbol{x}}$ is any particular solution, the full set is $\hat{\boldsymbol{x}} + \mathcal{N}(\mathbf{A})$.
- **Uniqueness**: Unique iff $\mathcal{N}(\mathbf{A}) = \{\mathbf{0}\}$, i.e., full column rank.

Example Let $\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{pmatrix}$, $\operatorname{rank}(\mathbf{A}) = 1$.

- $\mathcal{R}(\mathbf{A}) = \operatorname{span}\{(1,2)^T\}$; $\quad \mathcal{N}(\mathbf{A}^T) = \operatorname{span}\{(2,-1)^T\}$. Check: $(1,2)\cdot(2,-1) = 0$. ✓
- $\mathcal{R}(\mathbf{A}^T) = \operatorname{span}\{(1,2,3)^T\}$; $\quad \mathcal{N}(\mathbf{A})$ is a plane spanned by $(-2,1,0)^T$ and $(-3,0,1)^T$.

$\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ has a solution only when $\boldsymbol{b} = \alpha(1,2)^T$.

These four subspaces reappear in every major topic of these notes. In least squares, the residual $\boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}$ lies in $\mathcal{N}(\mathbf{A}^T)$. In SVD, the right singular vectors split into bases for $\mathcal{R}(\mathbf{A}^T)$ and $\mathcal{N}(\mathbf{A})$. In eigenvalue problems, the concept generalizes to invariant subspaces.

Def 2.9 **(Linear Span)** The **span** of $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k\}$ is the set of all their linear combinations:

$$\operatorname{span}\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k\} = \{\alpha_1\boldsymbol{x}_1 + \cdots + \alpha_k\boldsymbol{x}_k : \alpha_i \in \mathbb{R}\}.$$

This is the smallest subspace of $\mathbb{R}^n$ containing the given vectors.

Example For $\boldsymbol{v}_1 = (1,2)^T$ and $\boldsymbol{v}_2 = (3,4)^T$: assume $\alpha(1,2) + \beta(3,4) = (0,0)$. Then $\alpha + 3\beta = 0$ and $2\alpha + 4\beta = 0$, forcing $\beta = 0$ then $\alpha = 0$. So $\{\boldsymbol{v}_1, \boldsymbol{v}_2\}$ is independent and $\operatorname{span}\{\boldsymbol{v}_1, \boldsymbol{v}_2\} = \mathbb{R}^2$.

Ex 2.12 Refer to Def 2.9 and Def 2.7.

1. What is $\operatorname{span}\{(1,0,0)^T, (0,1,0)^T\}$ in $\mathbb{R}^3$? What is its dimension?
2. For $\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{pmatrix}$, verify that $\mathcal{R}(\mathbf{A}) = \operatorname{span}\{\text{columns of } \mathbf{A}\}$ and find its dimension.
3. In NumPy, confirm that adding the column $(6,12)^T$ to $\mathbf{A}$ does not increase `np.linalg.matrix_ra`

2.2 LU Factorization

The LU factorization expresses a square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ as

$$\mathbf{A} = \mathbf{L}\mathbf{U},$$

where $\mathbf{L}$ is unit lower triangular (ones on the diagonal) and $\mathbf{U}$ is upper triangular. Solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ then reduces to two triangular solves: forward substitution for $\mathbf{L}\boldsymbol{y} = \boldsymbol{b}$, then backward substitution for $\mathbf{U}\boldsymbol{x} = \boldsymbol{y}$. Each triangular solve costs $O(n^2)$, far cheaper than the $O(n^3)$ factorization that is paid only once.

2.2.1 LU Factorization and Gaussian Elimination

Gaussian elimination is the standard algorithm for computing the LU factorization. Understanding the elimination procedure is essential for analyzing both the cost and the conditions under which the factorization exists.

Def 2.10 **(LU Factorization)** An **LU factorization** of $\mathbf{A} \in \mathbb{R}^{n\times n}$ is a decomposition $\mathbf{A} = \mathbf{LU}$ with

$$\mathbf{L} = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ \ell_{21} & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \ell_{n1} & \ell_{n2} & \cdots & 1 \end{pmatrix} \quad \text{and} \quad \mathbf{U} = \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & u_{nn} \end{pmatrix}.$$

Ex 2.13 Refer to Def 2.10.

1. How many free parameters does an $n \times n$ unit lower triangular matrix $\mathbf{L}$ have? How many does $\mathbf{U}$ have? Together, can they encode n^2 free parameters (as required to represent a general $\mathbf{A}$)?

2. If $\mathbf{A} = \mathbf{LU}$ and $\mathbf{A}$ is invertible, show that $\mathbf{L}$ and $\mathbf{U}$ are each individually invertible. (*Hint:* $\det(\mathbf{A}) = \det(\mathbf{L})\det(\mathbf{U})$ *and diagonal entries of* $\mathbf{L}$ *are all 1.*)

Def 2.11 **(Gaussian Elimination)** To compute $\mathbf{A} = \mathbf{LU}$, Gaussian elimination eliminates entries below the pivot in each column. At step k, for each row $i > k$, compute the multiplier

$$\ell_{ik} = \frac{a_{ik}^{(k)}}{a_{kk}^{(k)}},$$

and update:

$$a_{ij}^{(k+1)} = a_{ij}^{(k)} - \ell_{ik}\, a_{kj}^{(k)}, \quad j = k+1, \ldots, n.$$

After $n-1$ steps, the updated matrix is $\mathbf{U}$ and the multipliers ℓ_{ik} form $\mathbf{L}$.

Thm 2.7 **(LU Factorization Existence and Uniqueness)** $\mathbf{A} \in \mathbb{R}^{n\times n}$ has a unique LU factorization (without pivoting) if and only if all leading principal submatrices $\mathbf{A}_k \in \mathbb{R}^{k\times k}$ are nonsingular for $k = 1, \ldots, n-1$.

Proof *Proof.* At step k, the pivot equals $\det(\mathbf{A}_k)/\det(\mathbf{A}_{k-1})$ (by induction). This is nonzero iff $\det(\mathbf{A}_k) \neq 0$. If all $n-1$ pivots are nonzero, elimination completes and each multiplier $\ell_{ik} = a_{ik}/a_{kk}$ and updated entry is uniquely determined, giving a unique $\mathbf{L}, \mathbf{U}$. $\square$

Thm 2.8 **(LU Factorization Cost)** Computing the LU factorization of $\mathbf{A} \in \mathbb{R}^{n\times n}$ by Gaussian elimination costs

$$\frac{2}{3}n^3 + O(n^2) \text{ flops.}$$

Proof *Proof.* At step k, we compute $n-k$ multipliers and update an $(n-k)\times(n-k)$ submatrix, costing $2(n-k)^2$ flops. Summing:

$$\sum_{k=1}^{n-1} 2(n-k)^2 = 2\sum_{j=1}^{n-1} j^2 = 2\cdot\frac{(n-1)n(2n-1)}{6} = \frac{2n^3}{3} + O(n^2).$$

□

Note 2.7 The $\frac{2}{3}n^3$ factorization cost is paid once. Each subsequent triangular solve costs only $O(n^2)$. When solving k systems with the same $\mathbf{A}$, the total cost is $\frac{2}{3}n^3 + 2kn^2$, versus $k\cdot\frac{2}{3}n^3$ if factorizing each time.

Example Consider

$$\mathbf{A} = \begin{pmatrix} 2 & 3 & 1 \\ 4 & 7 & 3 \\ -2 & -3 & 1 \end{pmatrix}.$$

Step 1: Take row 1 of $\mathbf{U}$: $u_{11} = 2$, $u_{12} = 3$, $u_{13} = 1$. Multipliers: $\ell_{21} = 4/2 = 2$, $\ell_{31} = -2/2 = -1$. After elimination: $u_{22} = 7-2\cdot 3 = 1$, $u_{23} = 3-2\cdot 1 = 1$; $\tilde{a}_{32} = -3+3 = 0$, $\tilde{a}_{33} = 1+1 = 2$. *Step 2*: $\ell_{32} = 0/1 = 0$, $u_{33} = 2$.

$$\mathbf{L} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix}, \quad \mathbf{U} = \begin{pmatrix} 2 & 3 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{pmatrix}.$$

Verify: $\mathbf{LU} = \mathbf{A}$. ✓

Ex 2.14 Refer to Def 2.11 and Thm 2.7. Compute the LU factorization (without pivoting) of

$$\mathbf{A} = \begin{pmatrix} 1 & 2 & 1 \\ 3 & 8 & 1 \\ 0 & 4 & 1 \end{pmatrix}.$$

1. Carry out elimination: compute multipliers ℓ_{21}, ℓ_{31} at step 1, update the submatrix, then compute ℓ_{32} at step 2.
2. Write $\mathbf{L}$ and $\mathbf{U}$ explicitly and verify $\mathbf{LU} = \mathbf{A}$.
3. Use the factorization to solve $\mathbf{A}\boldsymbol{x} = (5, 15, 4)^T$ via forward then backward substitution.
4. In Python, verify with `from scipy.linalg import lu` and `lu(A)`. Why does SciPy always return a permutation $\mathbf{P}$ even if no row swaps are needed?

2.2.2 Pivoting and LUP Decomposition

When a pivot is zero (or dangerously small), Gaussian elimination breaks down. Pivoting resolves this by reordering rows before each elimination step.

Def 2.12 **(LUP Decomposition)** When zero or near-zero pivots arise, row interchanges are recorded by a permutation matrix $\mathbf{P}$, giving the **LUP decomposition**:

$$\mathbf{PA} = \mathbf{LU},$$

where $\mathbf{L}$ is unit lower triangular and $\mathbf{U}$ is upper triangular. **Partial pivoting** selects the largest-magnitude entry in the current column as the pivot; this minimizes division by small numbers and limits error growth.

Ex 2.15 Refer to Def 2.12.

1. Give a concrete 2×2 matrix where Gaussian elimination without pivoting fails (division by zero) but LUP succeeds. Write out $\mathbf{P}$, $\mathbf{L}$, $\mathbf{U}$.
2. A permutation matrix satisfies $\mathbf{P}^T\mathbf{P} = \mathbf{I}$. Why does this make $\mathbf{P}^{-1}$ trivial to compute?

Why partial pivoting? Partial pivoting selects the largest available pivot (in absolute value) by interchanging rows. This limits the *growth factor* $\rho = \max |u_{ij}| / \max |a_{ij}|$, which controls how much rounding errors can amplify during elimination. Complete pivoting (swapping rows and columns) further bounds ρ but is rarely used in practice due to its extra cost and bookkeeping.

Example For

$$\mathbf{A} = \begin{pmatrix} 0 & 2 & 1 \\ 2 & 1 & 0 \\ 1 & 0 & 2 \end{pmatrix},$$

column 1 has a zero pivot. The largest entry in column 1 is 2 (row 2), so swap rows 1 and 2:

$$\mathbf{P} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad \mathbf{PA} = \begin{pmatrix} 2 & 1 & 0 \\ 0 & 2 & 1 \\ 1 & 0 & 2 \end{pmatrix}.$$

Elimination on $\mathbf{PA}$ now proceeds without issue: multipliers $\ell_{21} = 0$, $\ell_{31} = 0.5$; then $\ell_{32} = -0.25$; yielding

$$\mathbf{L} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0.5 & -0.25 & 1 \end{pmatrix}, \quad \mathbf{U} = \begin{pmatrix} 2 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 2.25 \end{pmatrix}.$$

2.2.3 Solving Linear Systems using LU Factorization

Once $\mathbf{A} = \mathbf{LU}$ (or $\mathbf{PA} = \mathbf{LU}$), solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ costs only $O(n^2)$ via two triangular solves. Crucially, the same factorization can be reused for multiple right-hand sides.

Def 2.13 **(LU Solve)** Given the LU factorization $\mathbf{A} = \mathbf{LU}$, solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ in two steps:

1. **Forward substitution**: solve $\mathbf{L}\boldsymbol{y} = \boldsymbol{b}$ for $\boldsymbol{y}$:

$$y_i = b_i - \sum_{j=1}^{i-1} \ell_{ij}\, y_j, \quad i = 1, \ldots, n.$$

2. **Backward substitution**: solve $\mathbf{U}\boldsymbol{x} = \boldsymbol{y}$ for $\boldsymbol{x}$:

$$x_i = \frac{1}{u_{ii}} \left(y_i - \sum_{j=i+1}^{n} u_{ij}\, x_j \right), \quad i = n, \ldots, 1.$$

Note 2.8 Each triangular solve requires $O(n^2)$ operations: forward substitution costs $\sum_{i=1}^{n}(i-1) = \frac{n(n-1)}{2}$ multiply-adds, and backward substitution the same. Together they are far cheaper than the $O(n^3)$ factorization.

Thm 2.9 **(Total Cost of LU-Based Solve)** Solving k linear systems $\mathbf{A}\boldsymbol{x}_i = \boldsymbol{b}_i$ with the same $\mathbf{A} \in \mathbb{R}^{n\times n}$ via LU factorization costs

$$\frac{2}{3}n^3 + 2kn^2 + O(n)$$

flops, versus $k \cdot \left(\frac{2}{3}n^3 + O(n^2)\right)$ if $\mathbf{A}$ is re-factorized each time.

Cor 2.10 **(Cost of Each Triangular Solve)** Forward substitution (solving $\mathbf{L}\boldsymbol{y} = \boldsymbol{b}$) and backward substitution (solving $\mathbf{U}\boldsymbol{x} = \boldsymbol{y}$) each cost exactly $n^2 - n$ arithmetic operations.

Ex 2.16 In this exercise, we will prove Cor 2.10. Refer to Def 2.13 and Thm 2.8.

1. At step i of forward substitution, how many multiplications and additions are needed to compute y_i? Sum over $i = 1, \ldots, n$ and show the total is $n^2 - n$.

2. Conclude the asymptotic cost is $O(n^2)$. Compare this to the $O(n^3)$ cost of the factorization (Thm 2.8) for large n.

3. Suppose $\mathbf{A}$ is 1000×1000 and you need to solve for 500 right-hand sides. How many flops does the LU-then-solve approach save compared to calling `np.linalg.solve` 500 times (assuming each call re-factorizes)?

4. In Python, use `scipy.linalg.lu_factor(A)` once and `scipy.linalg.lu_solve` for each right-hand side. Write code for 3 right-hand sides and verify each residual is near machine precision.

Ex 2.17 **(Stability of Gaussian Elimination)** Refer to Def 2.12 and Thm 2.8.

1. The **growth factor** of LU with partial pivoting is $\rho = \max_{i,j,k} |u_{ij}^{(k)}| / \max_{i,j} |a_{ij}|$. The theoretical bound is $\rho \le 2^{n-1}$. What is this for $n = 50$?

2. In practice ρ is almost always $O(n)$ or smaller. Does this make LU with partial pivoting backward stable in theory or just in practice?

3. Complete pivoting (interchanging both rows and columns) guarantees $\rho = O(n^{2/3})$ but is rarely used. What is the computational reason?

2.3 Symmetric Positive Definite (SPD) Matrices

Symmetric positive definite (SPD) matrices arise throughout scientific computing — in optimization (Hessians of convex functions), statistics (covariance matrices), and finite element methods (stiffness matrices). Their defining property — that the quadratic form $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x}$ is strictly positive for any nonzero $\boldsymbol{x}$ — guarantees invertibility and enables more efficient algorithms than are possible for general matrices.

Def 2.14 **(Positive Definite Matrix)** A symmetric matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ is *positive definite* (PD) if

$$\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} > 0 \quad \text{for every nonzero } \boldsymbol{x} \in \mathbb{R}^n.$$

A matrix that is both symmetric and positive definite is called *symmetric positive definite* (SPD).

Ex 2.18 Refer to Def 2.14.

1. Is $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ positive definite? Compute $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x}$ for a general $\boldsymbol{x} = (x_1, x_2)^T$ and determine when it is positive.

2. Is $\mathbf{B} = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$ positive definite? Find a nonzero $\boldsymbol{x}$ with $\boldsymbol{x}^T\mathbf{B}\boldsymbol{x} \le 0$.

3. A diagonal matrix $\mathbf{D} = \operatorname{diag}(d_1, \ldots, d_n)$ is positive definite if and only if all $d_i > 0$. Prove this directly from Def 2.14.

Thm 2.11 **(SPD Matrices Induce a Metric)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be an SPD matrix. The function

$$d_{\mathbf{A}}(\boldsymbol{x}, \boldsymbol{y}) = \sqrt{(\boldsymbol{x}-\boldsymbol{y})^T\mathbf{A}(\boldsymbol{x}-\boldsymbol{y})}$$

defines a metric on $\mathbb{R}^n$, satisfying non-negativity, identity of indiscernibles, symmetry, and the triangle inequality.

Proof *Proof.* Non-negativity and identity of indiscernibles follow from the positive-definite property: $(\boldsymbol{x}-\boldsymbol{y})^T\mathbf{A}(\boldsymbol{x}-\boldsymbol{y}) \ge 0$ with equality iff $\boldsymbol{x}-\boldsymbol{y} = \mathbf{0}$. Symmetry holds since $(\boldsymbol{x}-\boldsymbol{y})^T\mathbf{A}(\boldsymbol{x}-\boldsymbol{y}) = (\boldsymbol{y}-\boldsymbol{x})^T\mathbf{A}(\boldsymbol{y}-\boldsymbol{x})$. The triangle inequality follows from the Cauchy-Schwarz inequality applied to the inner product $\langle \boldsymbol{u}, \boldsymbol{v}\rangle_{\mathbf{A}} = \boldsymbol{u}^T\mathbf{A}\boldsymbol{v}$. □

Note 2.9 When $\mathbf{A} = \mathbf{I}$, $d_{\mathbf{A}}$ reduces to the standard Euclidean distance. The general $\mathbf{A}$-metric stretches space according to the eigenvectors and eigenvalues of $\mathbf{A}$: directions with large eigenvalues are "expensive" to move in.

Ex 2.19 Refer to Thm 2.11.

1. For $\mathbf{A} = \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}$, compute $d_{\mathbf{A}}\big((1,0)^T, (0,0)^T\big)$ and $d_{\mathbf{A}}\big((0,1)^T, (0,0)^T\big)$. Compare to the Euclidean distances. In what direction is the $\mathbf{A}$-metric "larger"?

2. The set $\{\boldsymbol{x} : d_{\mathbf{A}}(\boldsymbol{x}, \mathbf{0}) = 1\}$ is an ellipse. What are its axes and their lengths in terms of the eigenvalues of $\mathbf{A}$?

3. Where does this metric appear in machine learning? (*Hint: Mahalanobis distance.*)

Note 2.10 An SPD matrix $\mathbf{A}$ induces an $\mathbf{A}$-norm $\|\boldsymbol{x}\|_{\mathbf{A}} = \sqrt{\boldsymbol{x}^T\mathbf{A}\boldsymbol{x}}$, strictly positive for all nonzero $\boldsymbol{x}$. The set of vectors with constant $\mathbf{A}$-norm forms an ellipsoid whose axes align with the eigenvectors of $\mathbf{A}$ with half-lengths $1/\sqrt{\lambda_i}$. This geometry is central to iterative solvers such as the conjugate gradient method.

Def 2.15 **(A-Inner Product)** For an SPD matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, the function

$$\langle \boldsymbol{x}, \boldsymbol{y}\rangle_{\mathbf{A}} = \boldsymbol{x}^T\mathbf{A}\boldsymbol{y}$$

defines a valid inner product on $\mathbb{R}^n$, inducing the norm $\|\boldsymbol{x}\|_{\mathbf{A}} = \sqrt{\boldsymbol{x}^T\mathbf{A}\boldsymbol{x}}$. This is the natural inner product for problems where $\mathbf{A}$ encodes the geometry (e.g., solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ via conjugate gradients).

Ex 2.20 Refer to Def 2.15.

1. Verify that $\langle \boldsymbol{x}, \boldsymbol{y}\rangle_{\mathbf{A}} = \boldsymbol{x}^T\mathbf{A}\boldsymbol{y}$ satisfies the three inner product axioms: linearity in the first argument, symmetry, and positive definiteness.

2. For $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$, $\boldsymbol{x} = (1,0)^T$, $\boldsymbol{y} = (0,1)^T$, compute $\langle \boldsymbol{x}, \boldsymbol{y}\rangle_{\mathbf{A}}$, $\|\boldsymbol{x}\|_{\mathbf{A}}$, and $\|\boldsymbol{y}\|_{\mathbf{A}}$.

3. Are $\boldsymbol{x}$ and $\boldsymbol{y}$ orthogonal in the $\mathbf{A}$-inner product? In the standard inner product? Explain why these can differ.

Thm 2.12 **(SPD Characterization via Eigenvalues)** A symmetric matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ is positive definite if and only if all its eigenvalues are strictly positive. (By the Spectral Theorem, such a matrix is orthogonally diagonalizable with real eigenvalues.)

Proof *Proof.* By the Spectral Theorem, $\mathbf{A} = \mathbf{Q}\mathbf{D}\mathbf{Q}^T$ where $\mathbf{Q}$ is orthogonal and $\mathbf{D} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$. For any nonzero $\boldsymbol{x}$, let $\boldsymbol{y} = \mathbf{Q}^T\boldsymbol{x} \neq \mathbf{0}$. Then:

$$\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} = \boldsymbol{y}^T\mathbf{D}\boldsymbol{y} = \sum_{i=1}^{n} \lambda_i y_i^2.$$

($\Rightarrow$): If all $\lambda_i > 0$ and $\boldsymbol{y} \neq \mathbf{0}$, then at least one $y_i \neq 0$, so the sum is > 0.
($\Leftarrow$): If some $\lambda_k \leq 0$, choose $\boldsymbol{x} = \mathbf{Q}\mathbf{e}_k$, giving $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} = \lambda_k \leq 0$. $\square$

Cor 2.13 **(SPD Matrices are Invertible)** Every SPD matrix is invertible, since all eigenvalues are positive and thus $\det(\mathbf{A}) = \prod_i \lambda_i > 0$.

Ex 2.21 Refer to Thm 2.12 and Cor 2.13.

1. Use Thm 2.12 to determine which are SPD:

$$\mathbf{A}_1 = \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{A}_2 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad \mathbf{A}_3 = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

2. If $\mathbf{A}$ is SPD and $\mathbf{B} = \mathbf{A} + c\mathbf{I}$ for $c > 0$, show $\mathbf{B}$ is also SPD. How do the eigenvalues of $\mathbf{B}$ relate to those of $\mathbf{A}$?

3. In NumPy, compute `np.linalg.eigvalsh(A)` for each matrix above and verify which are SPD.

From a computational perspective, the condition $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} > 0$ for all nonzero $\boldsymbol{x}$ guarantees invertibility and precludes zero or negative pivots. When solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ where $\mathbf{A}$ is SPD, the Cholesky decomposition applies, and the solution is guaranteed to exist and be unique.

Ex 2.22 **(SPD Matrices in Practice)** SPD matrices appear across scientific computing. For each application below, verify the SPD property concretely.

1. **(Covariance matrix.)** Generate a random data matrix `X = np.random.randn(100, 4)` (100 observations, 4 features). Compute the sample covariance `C = X.T @ X / 99`. Verify `C` is SPD using `np.linalg.eigvalsh(C)` and `np.linalg.cholesky(C)`.

2. **(Hessian of a convex function.)** The function $f(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} - \boldsymbol{b}^T\boldsymbol{x}$ has Hessian $\nabla^2 f = \mathbf{A}$. For $\mathbf{A} = \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix}$ and $\boldsymbol{b} = (1, 2)^T$, verify $\mathbf{A}$ is SPD, then find the minimizer $\boldsymbol{x}^* = \mathbf{A}^{-1}\boldsymbol{b}$ by solving the linear system `np.linalg.solve(A, b)`.

3. **(Gram matrix.)** For any matrix $\mathbf{X} \in \mathbb{R}^{m\times n}$ with $m \geq n$ and full column rank, show analytically that $\mathbf{G} = \mathbf{X}^T\mathbf{X}$ is SPD. (*Hint: compute* $\boldsymbol{v}^T\mathbf{G}\boldsymbol{v}$ *and use the fact that* $\mathbf{X}\boldsymbol{v} \neq \mathbf{0}$ *when* $\boldsymbol{v} \neq \mathbf{0}$ *and* $\mathbf{X}$ *has full column rank.*)

4. **(Efficiency of Cholesky vs. LU.)** For $n = 1000$, time `np.linalg.cholesky(A)` vs. `scipy.linalg.lu(A)` on a random SPD matrix (construct one via `A = X.T @ X + n*np.eye(n)` for `X = np.random.randn(n, n)`). Is the observed speedup close to the theoretical factor of 2?

2.3.1 Cholesky Decomposition

The Cholesky decomposition is the SPD analogue of LU factorization: it expresses $\mathbf{A} = \mathbf{L}\mathbf{L}^T$ where $\mathbf{L}$ is lower triangular with positive diagonal. Because $\mathbf{A}$ is symmetric and positive definite, no pivoting is required and the factorization costs only half as many flops as LU.

Def 2.16 **(Cholesky Decomposition)** For an SPD matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, the *Cholesky decomposition* expresses $\mathbf{A}$ as

$$\mathbf{A} = \mathbf{L}\mathbf{L}^T,$$

where $\mathbf{L}$ is a unique lower triangular matrix with strictly positive diagonal entries.

SPD matrices, which satisfy $\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} > 0$ for all nonzero $\boldsymbol{x}$, are prevalent in optimization, statistics, and finite element methods. The Cholesky factorization is both computationally efficient and numerically stable since it avoids the need for pivoting.

Ex 2.23 Refer to Def 2.16.

1. Verify by direct multiplication that for $\mathbf{L} = \begin{pmatrix} 2 & 0 \\ 1 & \sqrt{2} \end{pmatrix}$, we have $\mathbf{L}\mathbf{L}^T = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$. Is the result SPD?
2. Why does Cholesky not need a permutation matrix $\mathbf{P}$ (unlike LU)? (*Hint: use Thm 2.12.*)
3. If $\mathbf{A} = \mathbf{L}\mathbf{L}^T$, show that $\det(\mathbf{A}) = \prod_{i=1}^n l_{ii}^2$. How does this give a fast way to compute $\det(\mathbf{A})$ once $\mathbf{L}$ is known?

Cholesky Algorithm. For each entry l_{ij} of $\mathbf{L}$ where $1 \le i, j \le n$:

$$l_{ij} = \begin{cases} 0, & i < j \\ \sqrt{a_{ii} - \sum_{k=1}^{i-1} l_{ik}^2}, & i = j \\ \dfrac{1}{l_{jj}}\left(a_{ij} - \sum_{k=1}^{j-1} l_{ik}l_{jk}\right), & i > j \end{cases}$$

The algorithm proceeds row by row, using previously computed entries of $\mathbf{L}$ to determine subsequent ones.

Derivation of the Cholesky Algorithm. Expanding $\mathbf{A} = \mathbf{L}\mathbf{L}^T$ entry by entry:
Diagonal entry (i, i): From $a_{ii} = \sum_{k=1}^i l_{ik}^2$, solve for l_{ii}:

$$l_{ii} = \sqrt{a_{ii} - \sum_{k=1}^{i-1} l_{ik}^2}.$$

Below-diagonal entry (i, j) with $i > j$: From $a_{ij} = \sum_{k=1}^j l_{ik}l_{jk}$ (since $l_{jk} = 0$ for $k > j$), solve for l_{ij}:

$$l_{ij} = \frac{1}{l_{jj}}\left(a_{ij} - \sum_{k=1}^{j-1} l_{ik}l_{jk}\right).$$

Above-diagonal entry (i, j) with $i < j$: $l_{ij} = 0$ by definition (lower triangular).

Example Compute the Cholesky decomposition of $\mathbf{A} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$.

Step 1: $l_{11} = \sqrt{4} = 2$; $l_{21} = 2/2 = 1$.
Step 2: $l_{22} = \sqrt{3 - 1^2} = \sqrt{2}$.

$$\mathbf{L} = \begin{pmatrix} 2 & 0 \\ 1 & \sqrt{2} \end{pmatrix}, \qquad \mathbf{L}\mathbf{L}^T = \begin{pmatrix} 2 & 0 \\ 1 & \sqrt{2} \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 0 & \sqrt{2} \end{pmatrix} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix} = \mathbf{A}. \checkmark$$

Note 2.11 The uniqueness of the Cholesky decomposition follows from the positivity of all leading principal minors of an SPD matrix: each $\det(\mathbf{A}_k) > 0$, so the algorithm never encounters a zero or negative value under a square root, and each l_{ii} is uniquely determined as a positive number.

Thm 2.14 **(Cholesky Existence and Uniqueness)** A matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ admits a Cholesky factorization $\mathbf{A} = \mathbf{L}\mathbf{L}^T$ (with $\mathbf{L}$ lower triangular and strictly positive diagonal) if and only if $\mathbf{A}$ is SPD. The factorization is unique.

Proof *Proof.* ($\Rightarrow$): If $\mathbf{A} = \mathbf{L}\mathbf{L}^T$, then $\mathbf{A}^T = \mathbf{L}\mathbf{L}^T = \mathbf{A}$ (symmetric). For any nonzero $\boldsymbol{x}$:

$$\boldsymbol{x}^T \mathbf{A} \boldsymbol{x} = \boldsymbol{x}^T \mathbf{L}\mathbf{L}^T \boldsymbol{x} = \|\mathbf{L}^T \boldsymbol{x}\|_2^2 \geq 0.$$

Equality holds iff $\mathbf{L}^T \boldsymbol{x} = \mathbf{0}$. Since $\mathbf{L}$ has strictly positive diagonal, $\mathbf{L}$ is invertible, so $\boldsymbol{x} = \mathbf{0}$. Hence $\mathbf{A}$ is SPD.
($\Leftarrow$): If $\mathbf{A}$ is SPD, all leading principal minors are positive (each submatrix is also SPD), so the algorithm never encounters zero or negative under a square root.
Uniqueness: If $\mathbf{L}_1\mathbf{L}_1^T = \mathbf{L}_2\mathbf{L}_2^T$, set $\mathbf{M} = \mathbf{L}_1^{-1}\mathbf{L}_2$. Then $\mathbf{M}\mathbf{M}^T = \mathbf{I}$, so $\mathbf{M}$ is lower triangular and orthogonal, hence $\mathbf{M} = \mathbf{I}$ and $\mathbf{L}_1 = \mathbf{L}_2$. □

Thm 2.15 **(Cholesky Factorization Cost)** The Cholesky factorization of an SPD matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ costs $\frac{1}{3}n^3 + O(n^2)$ flops — exactly half the cost of LU factorization — and requires storing only $\frac{n(n+1)}{2}$ entries.

Proof *Proof.* At step j, we compute l_{jj} (1 square root) and $n - j$ below-diagonal entries l_{ij} for $i > j$, each requiring j multiplications and 1 division. The dominant cost is:

$$\sum_{j=1}^{n} (n - j) \cdot j \approx \int_0^n (n - t)\, t\, dt = \frac{n^3}{6} + \frac{n^3}{6} = \frac{n^3}{3}.$$

□

Ex 2.24 Refer to Def 2.16 and Thm 2.14.

1. Which of the following matrices are SPD? For each, either prove it or find a nonzero $\boldsymbol{x}$ with $\boldsymbol{x}^T \mathbf{A} \boldsymbol{x} \leq 0$.

$$\mathbf{A}_1 = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \quad \mathbf{A}_2 = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}, \quad \mathbf{A}_3 = \begin{pmatrix} 4 & 2 & 0 \\ 2 & 3 & 1 \\ 0 & 1 & 1 \end{pmatrix}.$$

2. Compute the Cholesky factorization of $\mathbf{A}_1$ and $\mathbf{A}_3$ by hand. Verify $\mathbf{L}\mathbf{L}^T = \mathbf{A}$.

3. Use the Cholesky factorization of $\mathbf{A}_1$ to solve $\mathbf{A}_1\boldsymbol{x} = (3, 4)^T$ via forward then backward substitution.

4. In NumPy, `np.linalg.cholesky(A)` raises `LinAlgError` if $\mathbf{A}$ is not SPD. Write code to attempt Cholesky on each matrix above and handle the exception. What does a `LinAlgError` tell you about the matrix?

Ex 2.25 **(Condition Number of SPD Matrices)** For an SPD matrix with eigenvalues $\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$, the 2-norm condition number is $\kappa_2(\mathbf{A}) = \lambda_n/\lambda_1$.

1. Prove this formula using Thm 2.12. (*Hint:* $\|\mathbf{A}\|_2 = \max_i |\lambda_i|$ *for symmetric* $\mathbf{A}$*; and* $\mathbf{A}^{-1}$ *has eigenvalues* $1/\lambda_i$*.*)

2. For $\mathbf{A} = \begin{pmatrix} 10 & 0 \\ 0 & 0.01 \end{pmatrix}$, what is $\kappa_2(\mathbf{A})$? What does this imply for solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$?

3. The Cholesky factor $\mathbf{L}$ satisfies $\kappa_2(\mathbf{L}) = \sqrt{\kappa_2(\mathbf{A})}$. Why is this numerically significant? (*Think: solving* $\mathbf{L}\boldsymbol{y} = \boldsymbol{c}$ *and* $\mathbf{L}^T\boldsymbol{x} = \boldsymbol{y}$ *each introduces what error?*)

4. In NumPy, compute `np.linalg.cond(A)` for an SPD matrix and verify it matches $\lambda_{\max}/\lambda_{\min}$ from `np.linalg.eigvalsh(A)`.

3 Orthogonality and Least Squares

3.1 Orthogonal Matrices

Orthogonal matrices represent the class of linear transformations that preserve lengths and angles — rotations and reflections. They are numerically ideal: multiplying by an orthogonal matrix introduces no error amplification ($\kappa_2(\mathbf{Q}) = 1$). This section develops orthogonal matrices and their role in the QR factorization, which underpins least-squares solvers and the QR eigenvalue algorithm.

Def 3.1 **(Orthogonal Vectors)** Two vectors $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$ are called *orthogonal* if their inner product is zero:

$$\boldsymbol{x}^T\boldsymbol{y} = 0.$$

Geometrically, orthogonal vectors are perpendicular. A set of vectors $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_k\}$ is *mutually orthogonal* if every pair of distinct vectors is orthogonal:

$$\boldsymbol{v}_i^T\boldsymbol{v}_j = 0 \quad \text{for all } i \neq j.$$

Ex 3.1 Refer to Def 3.1.

1. Are $(1, 2)^T$ and $(4, -2)^T$ orthogonal? Are $(1, 1)^T$ and $(1, -1)^T$ orthogonal?

2. If $\boldsymbol{x} \perp \boldsymbol{y}$ and $\boldsymbol{x} \perp \boldsymbol{z}$, show that $\boldsymbol{x} \perp (\alpha \boldsymbol{y} + \beta \boldsymbol{z})$ for any scalars α, β.

3. In $\mathbb{R}^3$, find a nonzero vector orthogonal to both $(1, 0, 1)^T$ and $(0, 1, 1)^T$. (*Hint: solve a homogeneous system or use the cross product.*)

Def 3.2 **(Orthonormal Basis)** A set of vectors $\{\boldsymbol{q}_1, \boldsymbol{q}_2, \ldots, \boldsymbol{q}_n\}$ in $\mathbb{R}^n$ is an *orthonormal basis* if the vectors are mutually orthogonal and each has unit length:

$$\boldsymbol{q}_i^T \boldsymbol{q}_j = \delta_{ij} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j. \end{cases}$$

Any n such vectors in $\mathbb{R}^n$ are automatically linearly independent and span $\mathbb{R}^n$. If we arrange them as columns of $\mathbf{Q} = [\boldsymbol{q}_1, \ldots, \boldsymbol{q}_n]$, then $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$.

Note 3.1 Any vector $\boldsymbol{v} \in \mathbb{R}^n$ can be expressed uniquely as a linear combination of an orthonormal basis $\{\boldsymbol{q}_1, \ldots, \boldsymbol{q}_n\}$:

$$\boldsymbol{v} = \sum_{i=1}^{n} (\boldsymbol{q}_i^T \boldsymbol{v})\, \boldsymbol{q}_i.$$

The coefficients $\boldsymbol{q}_i^T \boldsymbol{v}$ are simple inner products — no linear system needs to be solved. This is the key computational advantage of orthonormal bases.

Ex 3.2 Refer to Def 3.2.

1. Verify that $\boldsymbol{q}_1 = (1/\sqrt{2}, 1/\sqrt{2})^T$ and $\boldsymbol{q}_2 = (1/\sqrt{2}, -1/\sqrt{2})^T$ form an orthonormal basis for $\mathbb{R}^2$.

2. Express $\boldsymbol{v} = (3, -1)^T$ as $\sum_i (\boldsymbol{q}_i^T \boldsymbol{v}) \boldsymbol{q}_i$ using the basis above. Verify the result equals $\boldsymbol{v}$.

3. A set of n mutually orthogonal nonzero vectors in $\mathbb{R}^n$ is linearly independent. Prove this. (*Hint: assume* $\sum c_i \boldsymbol{v}_i = \mathbf{0}$ *and take the inner product with each* $\boldsymbol{v}_j$.)

Def 3.3 **(Orthogonal Matrix)** A square matrix $\mathbf{Q} \in \mathbb{R}^{n \times n}$ is *orthogonal* if its columns form an orthonormal set:

$$\mathbf{Q}^T\mathbf{Q} = \mathbf{Q}\mathbf{Q}^T = \mathbf{I}_n.$$

Equivalently, $\mathbf{Q}^T = \mathbf{Q}^{-1}$, meaning the transpose is its own inverse.

Orthogonal matrices represent rotations, reflections, or combinations of these in $\mathbb{R}^n$. They preserve vector lengths and angles between vectors, making them particularly valuable in scientific computing where maintaining geometric properties is important.

Ex 3.3 Refer to Def 3.3.

1. Verify that $\mathbf{Q} = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$ is orthogonal for any θ. What geometric operation does multiplication by $\mathbf{Q}$ perform?

2. Is $\mathbf{M} = \begin{pmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & 1/\sqrt{2} \end{pmatrix}$ orthogonal? Why or why not?

3. If $\mathbf{Q}$ is orthogonal, show that $\mathbf{Q}^T$ is also orthogonal.

Thm 3.1 **(Orthogonal Matrices Preserve Norms and Inner Products)** Let $\mathbf{Q} \in \mathbb{R}^{n\times n}$ be an orthogonal matrix. For all $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$:

1. $(\mathbf{Q}\boldsymbol{x})^T(\mathbf{Q}\boldsymbol{y}) = \boldsymbol{x}^T\boldsymbol{y}$ (inner-product-preserving)
2. $\|\mathbf{Q}\boldsymbol{x}\|_2 = \|\boldsymbol{x}\|_2$ (isometry)
3. $\kappa_2(\mathbf{Q}) = 1$ (optimally conditioned)

Proof *Proof.* (1): $(\mathbf{Q}\boldsymbol{x})^T(\mathbf{Q}\boldsymbol{y}) = \boldsymbol{x}^T\mathbf{Q}^T\mathbf{Q}\boldsymbol{y} = \boldsymbol{x}^T\mathbf{I}\boldsymbol{y} = \boldsymbol{x}^T\boldsymbol{y}$.
(2): Set $\boldsymbol{x} = \boldsymbol{y}$ in (1): $\|\mathbf{Q}\boldsymbol{x}\|_2^2 = \boldsymbol{x}^T\boldsymbol{x} = \|\boldsymbol{x}\|_2^2$.
(3): By (2), $\|\mathbf{Q}\|_2 = \max_{\|\boldsymbol{x}\|=1} \|\mathbf{Q}\boldsymbol{x}\|_2 = 1$. Since $\mathbf{Q}^{-1} = \mathbf{Q}^T$ is also orthogonal, $\|\mathbf{Q}^{-1}\|_2 = 1$. Thus $\kappa_2(\mathbf{Q}) = \|\mathbf{Q}\|_2\|\mathbf{Q}^{-1}\|_2 = 1$. □

Note 3.2 The condition number of 1 is the best possible. Multiplying by an orthogonal matrix introduces no amplification of errors, which is why orthogonal transformations (Householder reflections, Givens rotations) are preferred over Gaussian elimination when stability is paramount.

Prop 3.2 **(Rows of an Orthogonal Matrix are Orthonormal)** If $\mathbf{Q} \in \mathbb{R}^{n\times n}$ satisfies $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$, then $\mathbf{Q}\mathbf{Q}^T = \mathbf{I}$ as well, so the rows of $\mathbf{Q}$ also form an orthonormal basis for $\mathbb{R}^n$.

Proof *Proof.* $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$ implies $\mathbf{Q}$ is invertible with $\mathbf{Q}^{-1} = \mathbf{Q}^T$. Left-multiplying by $\mathbf{Q}$ and right-multiplying by $\mathbf{Q}^T$ gives $\mathbf{Q}\mathbf{Q}^T = \mathbf{I}$. □

Prop 3.3 **(Determinant of an Orthogonal Matrix)** For any orthogonal $\mathbf{Q}$, $\det(\mathbf{Q}) = \pm 1$. When $\det(\mathbf{Q}) = +1$, $\mathbf{Q}$ is a rotation; when $\det(\mathbf{Q}) = -1$, it includes a reflection.

Proof *Proof.* From $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$: $\det(\mathbf{Q}^T)\det(\mathbf{Q}) = 1$, and $\det(\mathbf{Q}^T) = \det(\mathbf{Q})$, so $\det(\mathbf{Q})^2 = 1$. □

Prop 3.4 **(Product of Orthogonal Matrices is Orthogonal)** If $\mathbf{Q}_1, \mathbf{Q}_2 \in \mathbb{R}^{n\times n}$ are orthogonal, then $\mathbf{Q}_1\mathbf{Q}_2$ is also orthogonal.

Proof *Proof.* $(\mathbf{Q}_1\mathbf{Q}_2)^T(\mathbf{Q}_1\mathbf{Q}_2) = \mathbf{Q}_2^T(\mathbf{Q}_1^T\mathbf{Q}_1)\mathbf{Q}_2 = \mathbf{Q}_2^T\mathbf{I}\mathbf{Q}_2 = \mathbf{I}$. □

Ex 3.4 Refer to Props 3.2–3.4.

1. For $\mathbf{Q} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ (rotation by 90°), compute $\det(\mathbf{Q})$ and verify the rows are orthonormal.
2. For the reflection $\mathbf{Q} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$, verify $\det(\mathbf{Q}) = -1$ and $\mathbf{Q}\mathbf{Q}^T = \mathbf{I}$.

3. Is $\mathbf{Q}_1 + \mathbf{Q}_2$ necessarily orthogonal when $\mathbf{Q}_1$ and $\mathbf{Q}_2$ are? Give a counterexample.

3.1.1 QR Decomposition

The QR decomposition factors a matrix into an orthogonal factor $\mathbf{Q}$ and an upper triangular factor $\mathbf{R}$, combining the stability benefits of orthogonal transformations with the efficiency of triangular solves. It is the factorization of choice for least-squares problems and forms the basis of the QR eigenvalue algorithm.

Def 3.4 **(QR Decomposition)** For a matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ with $m \geq n$, the *QR decomposition* factors $\mathbf{A}$ as

$$\mathbf{A} = \mathbf{QR},$$

where $\mathbf{Q} \in \mathbb{R}^{m\times m}$ is orthogonal ($\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$) and $\mathbf{R} \in \mathbb{R}^{m\times n}$ is upper triangular with zeros below the main diagonal.

The QR factorization is particularly valuable in scientific computing because it transforms problems involving arbitrary matrices into problems with triangular matrices, which are much easier to solve. The orthogonality of $\mathbf{Q}$ ensures numerical stability in many applications, as orthogonal transformations preserve Euclidean norms and do not amplify errors.

Ex 3.5 Refer to Def 3.4.

1. If $\mathbf{A} = \mathbf{QR}$, explain how to solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ using $\mathbf{Q}$ and $\mathbf{R}$ separately. What is the cost of each step?
2. For $\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}$, write a QR factorization by inspection. (*Hint:* $\mathbf{A}$ *already has orthonormal columns.*)
3. Why does the QR decomposition always exist for any $\mathbf{A} \in \mathbb{R}^{m\times n}$ with $m \geq n$? (*Hint: consider Gram-Schmidt applied to the columns of* $\mathbf{A}$.)

Prop 3.5 **(QR and the Column Space)** If $\mathbf{A} \in \mathbb{R}^{m\times n}$ ($m \geq n$) has full column rank, the first n columns of $\mathbf{Q}$ (from the full QR factorization $\mathbf{A} = \mathbf{QR}$) form an orthonormal basis for $\mathrm{col}(\mathbf{A})$.

Prop 3.6 **(Diagonal Entries of R)** In the QR factorization of a full-column-rank matrix $\mathbf{A}$, the diagonal entries of $\mathbf{R}$ satisfy $|r_{ii}| > 0$ for each i. In particular, $\mathbf{R}$ is invertible.

Ex 3.6 Refer to Props 3.5–3.6.

1. If $r_{11} = 0$ in the QR factorization of $\mathbf{A}$, what does this imply about the first column of $\mathbf{A}$?

2. For a 3×2 full-rank matrix $\mathbf{A}$, identify which columns of the full $\mathbf{Q} \in \mathbb{R}^{3\times 3}$ span $\text{col}(\mathbf{A})$ and which span $\text{col}(\mathbf{A})^{\perp}$.

3. In NumPy, compute the full and thin QR of a 4×3 matrix and verify: (a) the first 3 columns of $\mathbf{Q}$ span $\text{col}(\mathbf{A})$, (b) the 4th column of $\mathbf{Q}$ is orthogonal to each column of $\mathbf{A}$.

3.1.2 Householder Reflectors

A key method for computing the QR factorization is via *Householder reflectors*: orthogonal matrices that reflect a vector across a hyperplane and can zero out all entries below a chosen position in a column. Applying n such reflectors in sequence reduces $\mathbf{A}$ to upper triangular form.

Def 3.5 **(Householder Reflector)** Given a nonzero vector $\boldsymbol{v} \in \mathbb{R}^m$, the *Householder reflector* is the matrix

$$\mathbf{H} = \mathbf{I} - \frac{2\boldsymbol{v}\boldsymbol{v}^T}{\boldsymbol{v}^T\boldsymbol{v}}.$$

Geometrically, $\mathbf{H}\boldsymbol{x}$ reflects $\boldsymbol{x}$ across the hyperplane $\boldsymbol{v}^{\perp} = \{\boldsymbol{w} : \boldsymbol{v}^T\boldsymbol{w} = 0\}$.

Thm 3.7 **(Properties of Householder Reflectors)** For any nonzero $\boldsymbol{v} \in \mathbb{R}^m$, the Householder matrix $\mathbf{H} = \mathbf{I} - 2\boldsymbol{v}\boldsymbol{v}^T/\|\boldsymbol{v}\|_2^2$ satisfies:

1. $\mathbf{H}^T = \mathbf{H}$ (symmetric)
2. $\mathbf{H}^T\mathbf{H} = \mathbf{I}$ (orthogonal)
3. $\mathbf{H}^2 = \mathbf{I}$, so $\mathbf{H}^{-1} = \mathbf{H}$ (involution)
4. $\det(\mathbf{H}) = -1$ (reflection, not rotation)

Proof *Proof.* Let $\tau = \boldsymbol{v}^T\boldsymbol{v} = \|\boldsymbol{v}\|_2^2 > 0$.
(1): $\mathbf{H}^T = (\mathbf{I} - 2\boldsymbol{v}\boldsymbol{v}^T/\tau)^T = \mathbf{I} - 2\boldsymbol{v}\boldsymbol{v}^T/\tau = \mathbf{H}$.
(2): $\mathbf{H}^T\mathbf{H} = \mathbf{H}^2 = (\mathbf{I} - 2\boldsymbol{v}\boldsymbol{v}^T/\tau)^2 = \mathbf{I} - 4\boldsymbol{v}\boldsymbol{v}^T/\tau + 4\boldsymbol{v}(\boldsymbol{v}^T\boldsymbol{v})\boldsymbol{v}^T/\tau^2 = \mathbf{I} - 4\boldsymbol{v}\boldsymbol{v}^T/\tau + 4\boldsymbol{v}\boldsymbol{v}^T/\tau = \mathbf{I}$.
(3): Follows from (1) and (2): $\mathbf{H}^2 = \mathbf{H}^T\mathbf{H} = \mathbf{I}$.
(4): $\mathbf{H}$ has eigenvalue -1 (for any $\boldsymbol{x} \parallel \boldsymbol{v}$: $\mathbf{H}\boldsymbol{v} = \boldsymbol{v} - 2\boldsymbol{v} = -\boldsymbol{v}$) with multiplicity 1, and eigenvalue $+1$ (for $\boldsymbol{x} \perp \boldsymbol{v}$) with multiplicity $m - 1$, so $\det(\mathbf{H}) = (-1)^1 \cdot 1^{m-1} = -1$. $\square$

Thm 3.8 **(Householder Zeroing)** For any $\boldsymbol{x} \in \mathbb{R}^m$ with $\boldsymbol{x} \neq \pm\|\boldsymbol{x}\|_2\boldsymbol{e}_1$, there exists a Householder vector $\boldsymbol{v}$ such that

$$\mathbf{H}\boldsymbol{x} = \pm\|\boldsymbol{x}\|_2\boldsymbol{e}_1,$$

where $\boldsymbol{e}_1 = (1, 0, \ldots, 0)^T$. The choice $\boldsymbol{v} = \boldsymbol{x} + \|\boldsymbol{x}\|_2\boldsymbol{e}_1$ (taking the $+$ sign) avoids cancellation and is numerically preferred.

Proof *Proof.* We want $\mathbf{H}\boldsymbol{x} = \alpha \boldsymbol{e}_1$ for some scalar α. Since $\mathbf{H}$ is orthogonal, $|\alpha| = \|\boldsymbol{x}\|_2$. Setting $\boldsymbol{v} = \boldsymbol{x} - \alpha \boldsymbol{e}_1$:

$$\mathbf{H}\boldsymbol{x} = \boldsymbol{x} - \frac{2\boldsymbol{v}\boldsymbol{v}^T\boldsymbol{x}}{\boldsymbol{v}^T\boldsymbol{v}}.$$

We compute $\boldsymbol{v}^T\boldsymbol{x} = \|\boldsymbol{x}\|_2^2 - \alpha x_1$ and $\boldsymbol{v}^T\boldsymbol{v} = 2(\|\boldsymbol{x}\|_2^2 - \alpha x_1) = 2\boldsymbol{v}^T\boldsymbol{x}$, so $\mathbf{H}\boldsymbol{x} = \boldsymbol{x} - \boldsymbol{v} = \alpha \boldsymbol{e}_1$. Choosing $\alpha = -\text{sign}(x_1)\|\boldsymbol{x}\|_2$ avoids cancellation when x_1 is large. □

Note 3.3 The key use of Householder reflectors in QR factorization is iterative zeroing: at step k, choose $\mathbf{H}_k$ to zero all entries below the (k, k) position in column k of the current matrix. After n such reflections, the result is upper triangular. Since each $\mathbf{H}_k$ is orthogonal, the product $\mathbf{Q} = \mathbf{H}_1\mathbf{H}_2 \cdots \mathbf{H}_n$ is orthogonal, giving $\mathbf{A} = \mathbf{QR}$. Each reflector is applied as $\mathbf{H}\boldsymbol{x} = \boldsymbol{x} - 2\boldsymbol{v}(\boldsymbol{v}^T\boldsymbol{x})/\|\boldsymbol{v}\|^2$ — a rank-1 update costing only $O(m)$, not $O(m^2)$.

Ex 3.7 Refer to Def 3.5 and Thm 3.8.

1. Let $\boldsymbol{x} = (3, 4)^T \in \mathbb{R}^2$. Find the Householder vector $\boldsymbol{v}$ such that $\mathbf{H}\boldsymbol{x} = -5\boldsymbol{e}_1$. Write $\mathbf{H}$ explicitly and verify $\mathbf{H}\boldsymbol{x} = (-5, 0)^T$.
2. For $\boldsymbol{x} = (1, 2, 2)^T$, find $\boldsymbol{v}$ and $\mathbf{H}$ such that $\mathbf{H}\boldsymbol{x} = -3\boldsymbol{e}_1$. Verify $\mathbf{H}$ is symmetric and $\mathbf{H}^2 = \mathbf{I}$.
3. In NumPy, apply two successive Householder reflectors to triangularize $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 2 & 3 \\ 2 & 4 \end{pmatrix}$ and recover $\mathbf{Q}$ and $\mathbf{R}$. Compare with `np.linalg.qr(A)`.

Lem 3.9 **(QR Factorization Cost)** The QR factorization of $\mathbf{A} \in \mathbb{R}^{m \times n}$ (via Householder reflections) costs approximately $2mn^2 - \frac{2}{3}n^3$ flops.

Ex 3.8 Refer to Lem 3.9.

1. For $\mathbf{A} \in \mathbb{R}^{n \times n}$ (square), simplify the QR cost formula. Compare it to LU factorization cost ($\frac{2}{3}n^3$). Which is cheaper and by what factor?
2. Why does a zero diagonal entry in $\mathbf{R}$ imply $\mathbf{A}$ is rank-deficient? (*Hint:* $\mathbf{A} = \mathbf{QR}$ *and* $\mathbf{Q}$ *is invertible.*)
3. In NumPy, `np.linalg.qr(A)` returns the thin QR by default (`mode='reduced'`). What are the shapes of $\mathbf{Q}$ and $\mathbf{R}$ for a 5×3 matrix? When would you want the full QR (`mode='complete'`)?

3.1.3 Gram-Schmidt

The Gram-Schmidt process is a constructive algorithm for building an orthonormal basis from any set of linearly independent vectors. It directly yields the QR factorization: the orthonormal vectors form the columns of $\mathbf{Q}$ and the projection coefficients populate $\mathbf{R}$. Two variants exist — classical and modified — with different numerical stability properties.

Def 3.6 **(Orthogonal Complement)** Let $S \subseteq \mathbb{R}^n$ be a subspace. Its *orthogonal complement* is

$$S^\perp = \{\boldsymbol{w} \in \mathbb{R}^n : \boldsymbol{w}^T\boldsymbol{v} = 0 \text{ for all } \boldsymbol{v} \in S\}.$$

$S^\perp$ is itself a subspace of $\mathbb{R}^n$.

Prop 3.10 **(Dimension of the Orthogonal Complement)** If $S \subseteq \mathbb{R}^n$ has dimension k, then $\dim(S^\perp) = n - k$ and $S \cap S^\perp = \{\mathbf{0}\}$.

Prop 3.11 **(Orthogonal Decomposition)** Every vector $\boldsymbol{x} \in \mathbb{R}^n$ decomposes uniquely as $\boldsymbol{x} = \boldsymbol{x}_S + \boldsymbol{x}_{S^\perp}$ where $\boldsymbol{x}_S \in S$ and $\boldsymbol{x}_{S^\perp} \in S^\perp$. The component $\boldsymbol{x}_S$ is the orthogonal projection of $\boldsymbol{x}$ onto S.

Prop 3.12 **(Null Space as an Orthogonal Complement)** For $\mathbf{A} \in \mathbb{R}^{m\times n}$ with columns spanning $S = \text{col}(\mathbf{A})$, the orthogonal complement satisfies $S^\perp = \ker(\mathbf{A}^T)$, i.e., $S^\perp = \{\boldsymbol{w} : \mathbf{A}^T\boldsymbol{w} = \mathbf{0}\}$.

Ex 3.9 Refer to Def 3.6 and Props 3.10–3.12.

1. Let $S = \text{span}\{(1,1,0)^T, (0,1,1)^T\}$ in $\mathbb{R}^3$. Find a basis for $S^\perp$ by solving $\mathbf{A}^T\boldsymbol{w} = \mathbf{0}$ where $\mathbf{A}$ has the spanning vectors as columns.
2. Verify that your answer to (1) satisfies $\dim(S) + \dim(S^\perp) = 3$.
3. In NumPy, use the QR factorization of $\mathbf{A}$ to extract an orthonormal basis for $\text{col}(\mathbf{A})$ and for $\text{col}(\mathbf{A})^\perp = \ker(\mathbf{A}^T)$.

Def 3.7 **(Gram-Schmidt Process)** The *Gram-Schmidt process* converts a set of linearly independent vectors $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n\}$ into an orthonormal basis for the same subspace, by sequentially removing each vector's projection onto the span of previously orthogonalized vectors.

Classical Gram-Schmidt Algorithm. Given linearly independent vectors $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$, compute an orthogonal set $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_n\}$ by:

$$\boldsymbol{u}_1 = \boldsymbol{v}_1, \tag{2}$$

$$\boldsymbol{u}_k = \boldsymbol{v}_k - \sum_{j=1}^{k-1} \frac{\boldsymbol{u}_j^T\boldsymbol{v}_k}{\boldsymbol{u}_j^T\boldsymbol{u}_j}\boldsymbol{u}_j, \quad k = 2, \ldots, n. \tag{3}$$

Normalize each vector to obtain an orthonormal basis: $\boldsymbol{q}_i = \boldsymbol{u}_i/\|\boldsymbol{u}_i\|_2$.

Note 3.4 The Gram-Schmidt process can be interpreted geometrically:

- $\boldsymbol{u}_1$ is simply the first vector $\boldsymbol{v}_1$.
- $\boldsymbol{u}_2$ is $\boldsymbol{v}_2$ minus its projection onto $\boldsymbol{u}_1$.

- $\boldsymbol{u}_3$ is $\boldsymbol{v}_3$ minus its projections onto both $\boldsymbol{u}_1$ and $\boldsymbol{u}_2$.

Each $\boldsymbol{u}_k$ is the component of $\boldsymbol{v}_k$ that is orthogonal to the subspace spanned by $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_{k-1}\}$.

Example Gram-Schmidt ProcessLet $\boldsymbol{v}_1 = (1,1,1)^T$ and $\boldsymbol{v}_2 = (0,1,2)^T$ in $\mathbb{R}^3$.
Step 1: $\boldsymbol{u}_1 = \boldsymbol{v}_1 = (1,1,1)^T$.
Step 2:

$$\boldsymbol{u}_2 = \boldsymbol{v}_2 - \frac{\boldsymbol{u}_1^T \boldsymbol{v}_2}{\boldsymbol{u}_1^T \boldsymbol{u}_1}\boldsymbol{u}_1 = (0,1,2)^T - \frac{3}{3}(1,1,1)^T = (-1,0,1)^T. \tag{4}$$

Normalizing:

$$\boldsymbol{q}_1 = \tfrac{1}{\sqrt{3}}(1,1,1)^T, \qquad \boldsymbol{q}_2 = \tfrac{1}{\sqrt{2}}(-1,0,1)^T. \tag{5}$$

Ex 3.10 Refer to Def 3.7.

1. Apply Gram-Schmidt to $\boldsymbol{v}_1 = (1,0,1)^T$, $\boldsymbol{v}_2 = (1,1,0)^T$, $\boldsymbol{v}_3 = (0,1,1)^T$. Verify each pair of output vectors is orthogonal.
2. Show that the Gram-Schmidt process fails (produces $\boldsymbol{u}_k = \mathbf{0}$) if and only if the input vectors are linearly dependent.
3. In NumPy, implement the classical Gram-Schmidt algorithm for a 4×3 matrix $\mathbf{A}$ and compare your output $\mathbf{Q}$ with `np.linalg.qr(A)[0]`.

The Gram-Schmidt process provides a constructive proof of the existence of an orthonormal basis for any subspace of $\mathbb{R}^n$. In practice, however, the classical algorithm can suffer from numerical instability due to accumulated rounding errors, especially for large sets of nearly dependent vectors. The modified Gram-Schmidt algorithm, which applies orthogonalization against each previous vector sequentially (using the partially orthogonalized vector at each step), offers better numerical stability and is preferred in computational applications.

Modified Gram-Schmidt Algorithm. Instead of computing all projections from the original vector $\boldsymbol{v}_k$, modified Gram-Schmidt re-uses the *partially orthogonalized* vector at each step:

$$\boldsymbol{v}_k^{(1)} = \boldsymbol{v}_k, \tag{6}$$

$$\boldsymbol{v}_k^{(j+1)} = \boldsymbol{v}_k^{(j)} - \frac{\boldsymbol{u}_j^T \boldsymbol{v}_k^{(j)}}{\boldsymbol{u}_j^T \boldsymbol{u}_j}\boldsymbol{u}_j, \quad j = 1, \ldots, k-1, \tag{7}$$

$$\boldsymbol{u}_k = \boldsymbol{v}_k^{(k)}. \tag{8}$$

Normalize as before: $\boldsymbol{q}_i = \boldsymbol{u}_i / \|\boldsymbol{u}_i\|_2$.

In exact arithmetic, classical and modified Gram-Schmidt produce identical results. The difference is numerical: modified Gram-Schmidt limits error accumulation by immediately removing components already seen.

Example Gram-Schmidt and QR DecompositionConsider the matrix $\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}$ whose columns are $\boldsymbol{v}_1 = (1,1,1)^T$ and $\boldsymbol{v}_2 = (0,1,2)^T$ from the previous example.
From the Gram-Schmidt process, $\boldsymbol{q}_1 = \frac{1}{\sqrt{3}}(1,1,1)^T$ and $\boldsymbol{q}_2 = \frac{1}{\sqrt{2}}(-1,0,1)^T$. The $\mathbf{R}$ matrix entries are:

$$r_{11} = \boldsymbol{q}_1^T \boldsymbol{v}_1 = \sqrt{3}, \quad r_{12} = \boldsymbol{q}_1^T \boldsymbol{v}_2 = \sqrt{3}, \quad r_{22} = \boldsymbol{q}_2^T \boldsymbol{u}_2 = \sqrt{2}. \tag{9}$$

Therefore:

$$\mathbf{R} = \begin{bmatrix} \sqrt{3} & \sqrt{3} \\ 0 & \sqrt{2} \end{bmatrix}, \qquad \mathbf{QR} = \begin{bmatrix} \frac{1}{\sqrt{3}} & -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{3}} & 0 \\ \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} \sqrt{3} & \sqrt{3} \\ 0 & \sqrt{2} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \mathbf{A}. \checkmark$$

Thm 3.13 **(Classical vs. Modified Gram-Schmidt Numerical Stability)** Both CGS and MGS produce the QR factorization in exact arithmetic, but differ numerically. Let $\hat{\mathbf{Q}}$ denote the computed factor. Then:

- **Classical (CGS):** $\|\hat{\mathbf{Q}}^T\hat{\mathbf{Q}} - \mathbf{I}\|_2 = O(\varepsilon_{\text{mach}}\, \kappa(\mathbf{A})^2)$
- **Modified (MGS):** $\|\hat{\mathbf{Q}}^T\hat{\mathbf{Q}} - \mathbf{I}\|_2 = O(\varepsilon_{\text{mach}}\, \kappa(\mathbf{A}))$

Both algorithms cost $O(mn^2)$ flops for $\mathbf{A} \in \mathbb{R}^{m\times n}$.

Note 3.5 The key difference: CGS computes all projections from the *original* vector $\boldsymbol{v}_k$, so rounding errors in early projections contaminate all subsequent ones. MGS re-uses the *partially orthogonalized* vector at each step, immediately removing components already seen, which limits error accumulation. For maximum stability (at the same $O(mn^2)$ cost), Householder QR achieves $O(\varepsilon_{\text{mach}})$ loss of orthogonality regardless of $\kappa(\mathbf{A})$; it is what `np.linalg.qr` actually uses.

Note 3.6 The Gram-Schmidt process is closely related to the QR decomposition. Arranging the original vectors as columns of $\mathbf{A}$ and the orthonormal vectors as columns of $\mathbf{Q}$, we have $\mathbf{A} = \mathbf{QR}$ where $\mathbf{R}$ is upper triangular with entries:

$$r_{ij} = \begin{cases} \boldsymbol{q}_i^T \boldsymbol{v}_j & \text{if } i \le j \\ 0 & \text{if } i > j. \end{cases}$$

The elements of $\mathbf{R}$ are precisely the projection coefficients computed during the Gram-Schmidt process.

Ex 3.11 Let $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 0 & 1 \end{pmatrix}$.

1. Apply classical Gram-Schmidt by hand to the columns of $\mathbf{A}$ to find an orthonormal basis $\{\boldsymbol{q}_1, \boldsymbol{q}_2\}$ for $\text{col}(\mathbf{A})$.
2. Write out $\mathbf{Q} \in \mathbb{R}^{3\times 2}$ and $\mathbf{R} \in \mathbb{R}^{2\times 2}$ and verify $\mathbf{A} = \mathbf{QR}$.
3. Apply modified Gram-Schmidt to the same columns. Do you get the same $\mathbf{Q}, \mathbf{R}$ (in exact arithmetic)?
4. In NumPy, compute `np.linalg.qr(A)` and compare. Verify `Q.T @ Q` is close to $\mathbf{I}$.

Ex 3.12 **(Orthogonal Projection)** Let $\mathbf{Q} \in \mathbb{R}^{m\times n}$ (with $m > n$) have orthonormal columns (thin QR). The orthogonal projector onto $\text{col}(\mathbf{Q})$ is $\mathbf{P} = \mathbf{QQ}^T$.

1. Show that $\mathbf{P}^2 = \mathbf{P}$ (idempotent) and $\mathbf{P}^T = \mathbf{P}$ (symmetric).
2. Show that $\mathbf{P}\boldsymbol{x}$ is the closest vector in $\text{col}(\mathbf{Q})$ to $\boldsymbol{x}$, i.e., $\|\boldsymbol{x} - \mathbf{P}\boldsymbol{x}\|_2 \leq \|\boldsymbol{x} - \boldsymbol{y}\|_2$ for all $\boldsymbol{y} \in \text{col}(\mathbf{Q})$.
3. If $\mathbf{A} = \mathbf{QR}$ (full QR with $\mathbf{Q} \in \mathbb{R}^{m\times m}$ orthogonal), what is the projection onto $\text{col}(\mathbf{A})$ in terms of the thin $\mathbf{Q}_1$ (first n columns of $\mathbf{Q}$)?
4. Write NumPy code to project a vector `x = np.random.randn(5)` onto the column space of a random 5×3 matrix `A`. Verify the residual `x - P @ x` is orthogonal to each column of `A`.

Ex 3.13 **(Stability Experiment)** Generate a nearly linearly dependent set of vectors:

```
V = np.column_stack([np.ones(50), np.linspace(0,1,50)**k for k in
                                    range(1,6)])
```

This is a Vandermonde-like matrix with $\kappa(\mathbf{V}) \gg 1$.

1. Implement classical Gram-Schmidt and compute $\|\hat{\mathbf{Q}}^T\hat{\mathbf{Q}} - \mathbf{I}\|_F$.
2. Implement modified Gram-Schmidt and compute the same quantity. Compare.
3. Use `np.linalg.qr(V)` (Householder) and compute the same quantity. What do you observe?
4. Compute $\kappa(\mathbf{V})$ via `np.linalg.cond(V)`. Does the stability gap between CGS and MGS scale as Thm 3.13 predicts?

Note 3.7 The Gram-Schmidt process and QR factorization are foundational tools in numerical linear algebra, underlying algorithms for solving linear systems, least-squares problems, eigenvalue computation (the QR algorithm), and orthogonal basis construction in iterative solvers.

Ex 3.14 For each application below, identify which property of QR (orthogonality of $\mathbf{Q}$, triangularity of $\mathbf{R}$, or both) is the key ingredient and write one sentence explaining why.

1. Solving a square linear system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ via $\mathbf{A} = \mathbf{QR}$.
2. Computing the least-squares solution to an overdetermined system $\mathbf{A}\boldsymbol{x} \approx \boldsymbol{b}$.
3. One step of the QR eigenvalue algorithm: $\mathbf{A}_k = \mathbf{Q}_k\mathbf{R}_k$, $\mathbf{A}_{k+1} = \mathbf{R}_k\mathbf{Q}_k$.
4. Constructing a coordinate system aligned with a given set of data directions.

3.2 Vector and Matrix Norms

3.2.1 Vector Norms

Before defining norms on matrices, we establish the vector norm foundations on which they rest. A vector norm assigns a non-negative scalar "length" to every vector, and the key families each capture a different geometric intuition.

Def 3.8 **(Vector Norm)** A function $\|\cdot\| : \mathbb{R}^n \to \mathbb{R}$ is a *vector norm* if for all $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$ and $c \in \mathbb{R}$:

1. $\|\boldsymbol{x}\| \geq 0$ and $\|\boldsymbol{x}\| = 0 \Leftrightarrow \boldsymbol{x} = \mathbf{0}$ (positive definiteness)
2. $\|c\boldsymbol{x}\| = |c|\|\boldsymbol{x}\|$ (absolute homogeneity)
3. $\|\boldsymbol{x} + \boldsymbol{y}\| \leq \|\boldsymbol{x}\| + \|\boldsymbol{y}\|$ (triangle inequality)

Def 3.9 **(ℓ^p Norms)** For $\boldsymbol{x} \in \mathbb{R}^n$ and $p \geq 1$, the ℓ^p *norm* is

$$\|\boldsymbol{x}\|_p = \left(\sum_{i=1}^{n} |x_i|^p\right)^{1/p}.$$

The three most common cases are:

- $\|\boldsymbol{x}\|_1 = \sum_{i=1}^{n} |x_i|$ (*Manhattan norm* — sum of absolute values)
- $\|\boldsymbol{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2} = \sqrt{\boldsymbol{x}^T\boldsymbol{x}}$ (*Euclidean norm*)
- $\|\boldsymbol{x}\|_\infty = \max_{1\leq i\leq n} |x_i|$ (*max norm* — limit as $p \to \infty$)

Thm 3.14 **(Cauchy-Schwarz Inequality)** For any $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$:

$$|\boldsymbol{x}^T\boldsymbol{y}| \leq \|\boldsymbol{x}\|_2 \, \|\boldsymbol{y}\|_2,$$

with equality if and only if $\boldsymbol{x}$ and $\boldsymbol{y}$ are linearly dependent.

Proof *Proof.* If either vector is zero the inequality holds trivially. Otherwise, for any $t \in \mathbb{R}$:

$$0 \leq \|\boldsymbol{x} - t\boldsymbol{y}\|_2^2 = \|\boldsymbol{x}\|_2^2 - 2t\,\boldsymbol{x}^T\boldsymbol{y} + t^2\|\boldsymbol{y}\|_2^2.$$

This is a quadratic in t that is always non-negative, so its discriminant must satisfy $4(\boldsymbol{x}^T\boldsymbol{y})^2 - 4\|\boldsymbol{x}\|_2^2\|\boldsymbol{y}\|_2^2 \leq 0$, giving $|\boldsymbol{x}^T\boldsymbol{y}| \leq \|\boldsymbol{x}\|_2\|\boldsymbol{y}\|_2$. Equality holds iff $\boldsymbol{x} = t\boldsymbol{y}$ for the optimal $t = \boldsymbol{x}^T\boldsymbol{y}/\|\boldsymbol{y}\|_2^2$. □

Lem 3.15 **(Young's Inequality)** For $a, b \geq 0$ and conjugate exponents $p, q \geq 1$ with $\frac{1}{p} + \frac{1}{q} = 1$,

$$ab \leq \frac{a^p}{p} + \frac{b^q}{q}.$$

Equality holds if and only if $a^p = b^q$.

Proof *Proof.* If $a = 0$ or $b = 0$ the inequality is immediate. Otherwise, the function $f(t) = t/p - t^{1/p}/p + 1/q$ satisfies $f(1) = 0$ and $f'(t) = (1 - t^{1/p-1})/p$, so f has a global minimum at $t = 1$ with $f \geq 0$. Setting $t = a^p/b^q$ and multiplying through by b^q gives the result. Alternatively: the weighted AM-GM inequality $\alpha u + \beta v \geq u^\alpha v^\beta$ (for $\alpha + \beta = 1$, $u, v > 0$) applied with $\alpha = 1/p$, $\beta = 1/q$, $u = a^p$, $v = b^q$ gives $a^p/p + b^q/q \geq (a^p)^{1/p}(b^q)^{1/q} = ab$. □

Thm 3.16 **(Hölder's Inequality)** For $p, q \geq 1$ with $\frac{1}{p} + \frac{1}{q} = 1$ (conjugate exponents), and any $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$:

$$|\boldsymbol{x}^T\boldsymbol{y}| \leq \|\boldsymbol{x}\|_p \, \|\boldsymbol{y}\|_q.$$

Cauchy-Schwarz is the special case $p = q = 2$.

Proof *Proof.* If $\boldsymbol{x} = \boldsymbol{0}$ or $\boldsymbol{y} = \boldsymbol{0}$ the result is immediate. Otherwise, set $a_i = |x_i|/\|\boldsymbol{x}\|_p$ and $b_i = |y_i|/\|\boldsymbol{y}\|_q$. By Young's inequality applied to each i:

$$a_i b_i \leq \frac{a_i^p}{p} + \frac{b_i^q}{q}.$$

Summing over i and using $\sum a_i^p = 1$, $\sum b_i^q = 1$ gives $\sum a_i b_i \leq 1/p + 1/q = 1$. Multiplying by $\|\boldsymbol{x}\|_p\|\boldsymbol{y}\|_q$ yields $|\boldsymbol{x}^T\boldsymbol{y}| \leq \sum |x_i||y_i| \leq \|\boldsymbol{x}\|_p\|\boldsymbol{y}\|_q$. □

Thm 3.17 **(Equivalence of Vector Norms)** All norms on $\mathbb{R}^n$ are equivalent: for any two norms $\|\cdot\|_\alpha$ and $\|\cdot\|_\beta$, there exist constants $c_1, c_2 > 0$ such that

$$c_1\|\boldsymbol{x}\|_\beta \leq \|\boldsymbol{x}\|_\alpha \leq c_2\|\boldsymbol{x}\|_\beta \quad \text{for all } \boldsymbol{x} \in \mathbb{R}^n.$$

In particular, for $\boldsymbol{x} \in \mathbb{R}^n$:

$$\|\boldsymbol{x}\|_2 \leq \|\boldsymbol{x}\|_1 \leq \sqrt{n}\,\|\boldsymbol{x}\|_2, \qquad \|\boldsymbol{x}\|_\infty \leq \|\boldsymbol{x}\|_2 \leq \sqrt{n}\,\|\boldsymbol{x}\|_\infty.$$

Proof *Proof.* Existence of c_1, c_2 follows from the compactness of the unit sphere $\{\boldsymbol{x} : \|\boldsymbol{x}\|_\beta = 1\}$ in $\mathbb{R}^n$: any norm is a continuous function, so it attains its bounds on a compact set. The explicit inequalities follow from Cauchy-Schwarz: $\|\boldsymbol{x}\|_1 = \sum |x_i| \cdot 1 \leq \|\boldsymbol{x}\|_2\|\boldsymbol{1}\|_2 = \sqrt{n}\|\boldsymbol{x}\|_2$, and $\|\boldsymbol{x}\|_2^2 = \sum x_i^2 \leq \|\boldsymbol{x}\|_\infty \sum |x_i| = \|\boldsymbol{x}\|_\infty\|\boldsymbol{x}\|_1 \leq n\|\boldsymbol{x}\|_\infty^2$. □

Note 3.8 Norm equivalence means convergence in one norm implies convergence in all norms. In practice, the choice of norm affects constants but not qualitative behavior. The ℓ^2 norm is

preferred for geometric intuition; the ℓ^∞ norm is often used in error analysis; the ℓ^1 norm appears in sparse recovery and optimization.

Ex 3.15

1. For $\boldsymbol{x} = (3, -4, 0)^T$, compute $\|\boldsymbol{x}\|_1$, $\|\boldsymbol{x}\|_2$, $\|\boldsymbol{x}\|_\infty$. Verify all three equivalence inequalities hold.
2. Prove the triangle inequality for $\|\cdot\|_2$ using Cauchy-Schwarz: expand $\|\boldsymbol{x}+\boldsymbol{y}\|_2^2 = (\boldsymbol{x}+\boldsymbol{y})^T(\boldsymbol{x}+\boldsymbol{y})$.
3. In NumPy, `np.linalg.norm(x, ord=p)` computes ℓ^p norms. For a random $\boldsymbol{x} \in \mathbb{R}^{10}$, verify numerically that $\|\boldsymbol{x}\|_\infty \le \|\boldsymbol{x}\|_2 \le \sqrt{10}\|\boldsymbol{x}\|_\infty$.
4. Show that $\|\boldsymbol{x}\|_\infty = \lim_{p\to\infty} \|\boldsymbol{x}\|_p$. (*Hint: factor out the maximum entry.*)

3.2.2 Matrix Norms

Matrix norms assign a scalar "size" to a matrix. They are fundamental in numerical analysis: they measure approximation errors in low-rank truncation, bound how much a matrix amplifies vectors, and define the condition number. Every norm inequality we use in convergence analysis ultimately rests on a handful of key theorems proved here.

Def 3.10 **(Matrix Norm)** A function $\|\cdot\| : \mathbb{R}^{m\times n} \to \mathbb{R}$ is a *matrix norm* if for all $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m\times n}$ and all $c \in \mathbb{R}$: (i) $\|\mathbf{A}\| \ge 0$ and $\|\mathbf{A}\| = 0 \Leftrightarrow \mathbf{A} = \mathbf{0}$ (non-negativity and definiteness); (ii) $\|c\mathbf{A}\| = |c|\|\mathbf{A}\|$ (absolute homogeneity); (iii) $\|\mathbf{A}+\mathbf{B}\| \le \|\mathbf{A}\| + \|\mathbf{B}\|$ (triangle inequality). A square-matrix norm is additionally *submultiplicative* if $\|\mathbf{AB}\| \le \|\mathbf{A}\|\|\mathbf{B}\|$.

Def 3.11 **(Induced (Operator) Norm)** Given vector norms $\|\cdot\|_u$ on $\mathbb{R}^m$ and $\|\cdot\|_v$ on $\mathbb{R}^n$, the *induced* (or *operator*) norm of $\mathbf{A} \in \mathbb{R}^{m\times n}$ is

$$\|\mathbf{A}\|_{u,v} = \max_{\boldsymbol{x}\neq\mathbf{0}} \frac{\|\mathbf{A}\boldsymbol{x}\|_u}{\|\boldsymbol{x}\|_v} = \max_{\|\boldsymbol{x}\|_v=1} \|\mathbf{A}\boldsymbol{x}\|_u.$$

It measures the maximum stretching factor applied by $\mathbf{A}$.

Thm 3.18 **(Induced Norms are Submultiplicative)** Every induced norm satisfies $\|\mathbf{AB}\| \le \|\mathbf{A}\|\|\mathbf{B}\|$.

Proof *Proof.* For any $\boldsymbol{x} \neq \mathbf{0}$,

$$\|\mathbf{AB}\boldsymbol{x}\| \le \|\mathbf{A}\|\,\|\mathbf{B}\boldsymbol{x}\| \le \|\mathbf{A}\|\,\|\mathbf{B}\|\,\|\boldsymbol{x}\|.$$

Dividing by $\|\boldsymbol{x}\|$ and taking the supremum over all $\boldsymbol{x} \neq \mathbf{0}$ gives $\|\mathbf{AB}\| \le \|\mathbf{A}\|\|\mathbf{B}\|$. □

Def 3.12 **(L_1 Norm (Maximum Absolute Column Sum))** The matrix L_1 norm, induced by the vector ℓ^1 norm, is

$$\|\mathbf{A}\|_1 = \max_{1\le j\le n} \sum_{i=1}^{m} |a_{ij}|.$$

It equals the maximum absolute column sum of $\mathbf{A}$.

Def 3.13 **(L_∞-Norm (Maximum Absolute Row Sum))** The matrix L_∞ norm, induced by the vector ℓ^∞ norm, is

$$\|\mathbf{A}\|_\infty = \max_{1\le i\le m} \sum_{j=1}^{n} |a_{ij}|.$$

It equals the maximum absolute row sum of $\mathbf{A}$.

Def 3.14 **(Spectral Norm)** The matrix L_2 norm (spectral norm), induced by the vector ℓ^2 norm, is

$$\|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A}) = \sqrt{\lambda_{\max}(\mathbf{A}^T\mathbf{A})}.$$

It equals the largest singular value and measures the maximum Euclidean stretching factor of $\mathbf{A}$.

Thm 3.19 **(Spectral Norm via Singular Values)** $\|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A})$.

Proof *Proof.* Write the SVD as $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$. For $\|\boldsymbol{x}\|_2 = 1$, let $\boldsymbol{y} = \mathbf{V}^T\boldsymbol{x}$ (which also satisfies $\|\boldsymbol{y}\|_2 = 1$, since $\mathbf{V}$ is orthogonal). Then

$$\|\mathbf{A}\boldsymbol{x}\|_2 = \|\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T\boldsymbol{x}\|_2 = \|\mathbf{\Sigma}\boldsymbol{y}\|_2 \le \sigma_1\|\boldsymbol{y}\|_2 = \sigma_1,$$

with equality when $\boldsymbol{y} = \mathbf{e}_1$, i.e., $\boldsymbol{x} = \boldsymbol{v}_1$ (the first right singular vector). Hence $\|\mathbf{A}\|_2 = \sigma_1 = \sigma_{\max}(\mathbf{A})$. □

Def 3.15 **(Frobenius Norm)** The *Frobenius norm* treats $\mathbf{A}$ as a vector in $\mathbb{R}^{mn}$:

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n} a_{ij}^2}.$$

Thm 3.20 **(Frobenius Norm via Trace and Singular Values)** $\|\mathbf{A}\|_F = \sqrt{\mathrm{Tr}(\mathbf{A}^T\mathbf{A})} = \sqrt{\mathrm{Tr}(\mathbf{A}\mathbf{A}^T)} = \sqrt{\sum_k \sigma_k^2}$.

Proof *Proof.* By definition, $\|\mathbf{A}\|_F^2 = \sum_{i,j} a_{ij}^2 = \sum_j (\mathbf{A}^T\mathbf{A})_{jj} = \mathrm{Tr}(\mathbf{A}^T\mathbf{A})$. The matrix $\mathbf{A}^T\mathbf{A}$ is symmetric positive semidefinite with eigenvalues $\sigma_1^2 \ge \cdots \ge \sigma_n^2 \ge 0$ (the squared singular values of $\mathbf{A}$), so $\mathrm{Tr}(\mathbf{A}^T\mathbf{A}) = \sum_k \sigma_k^2$. The identity $\mathrm{Tr}(\mathbf{A}^T\mathbf{A}) = \mathrm{Tr}(\mathbf{A}\mathbf{A}^T)$ follows since both equal $\sum_{i,j} a_{ij}^2$. □

Note 3.9 The Frobenius norm is submultiplicative and is consistent with the vector ℓ^2 norm: $\|\mathbf{A}\boldsymbol{x}\|_2 \le \|\mathbf{A}\|_F\|\boldsymbol{x}\|_2$. It is *not* induced by any vector norm, but satisfies $\|\mathbf{A}\|_2 \le \|\mathbf{A}\|_F \le \sqrt{r}\,\|\mathbf{A}\|_2$ where $r = \mathrm{rank}(\mathbf{A})$.

Def 3.16 **(Max Norm)** The *max norm* is $\|\mathbf{A}\|_{\max} = \max_{i,j} |a_{ij}|$. It is a valid matrix norm but is *not* submultiplicative in general ($\|\mathbf{A}\mathbf{B}\|_{\max}$ can exceed $\|\mathbf{A}\|_{\max}\|\mathbf{B}\|_{\max}$) and is not an induced norm.

Thm 3.21 **(Norm Equivalence in Finite Dimensions)** All matrix norms on $\mathbb{R}^{m\times n}$ are equivalent: for any two norms $\|\cdot\|_\alpha$ and $\|\cdot\|_\beta$, there exist constants $c_1, c_2 > 0$ such that $c_1\|\mathbf{A}\|_\beta \leq \|\mathbf{A}\|_\alpha \leq c_2\|\mathbf{A}\|_\beta$ for all $\mathbf{A}$.

Proof *Proof.* Since $\mathbb{R}^{m\times n}$ is a finite-dimensional vector space (of dimension mn), any norm on it is equivalent to the standard Euclidean norm on $\mathbb{R}^{mn}$ (which equals the Frobenius norm). Specifically, the unit ball of any norm is compact, so the continuous function $\mathbf{A} \mapsto \|\mathbf{A}\|_\alpha/\|\mathbf{A}\|_\beta$ attains its supremum and infimum on the unit sphere $\{\mathbf{A} : \|\mathbf{A}\|_\beta = 1\}$, yielding the constants c_1 and c_2. □

Prop 3.22 **(Consistency with Vector Norms)** An induced norm $\|\mathbf{A}\|$ satisfies $\|\mathbf{A}\boldsymbol{x}\|_v \leq \|\mathbf{A}\|\|\boldsymbol{x}\|_v$ for all $\boldsymbol{x}$ by definition. The Frobenius norm also satisfies $\|\mathbf{A}\boldsymbol{x}\|_2 \leq \|\mathbf{A}\|_F\|\boldsymbol{x}\|_2$.

Proof *Proof.* For the Frobenius norm, use Cauchy-Schwarz: $(\mathbf{A}\boldsymbol{x})_i = \sum_j a_{ij}x_j$, so $|(\mathbf{A}\boldsymbol{x})_i|^2 \leq \sum_j a_{ij}^2 \cdot \|\boldsymbol{x}\|_2^2$. Summing over i gives $\|\mathbf{A}\boldsymbol{x}\|_2^2 \leq \|\mathbf{A}\|_F^2\|\boldsymbol{x}\|_2^2$. □

Note 3.10 Matrix norms appear in the condition number $\kappa(\mathbf{A}) = \|\mathbf{A}\|\|\mathbf{A}^{-1}\|$. For the 2-norm this gives $\kappa_2(\mathbf{A}) = \sigma_{\max}/\sigma_{\min}$. For the Frobenius norm, $\|\mathbf{A}\|_F$ prominently features in the Eckart-Young-Mirsky theorem for optimal low-rank approximation.

Example Let $\mathbf{A} = \begin{pmatrix} 1 & -2 \\ -3 & 4 \end{pmatrix}$.

- $\|\mathbf{A}\|_1 = \max(1+3,\, 2+4) = 6$ (maximum column sum).
- $\|\mathbf{A}\|_\infty = \max(1+2,\, 3+4) = 7$ (maximum row sum).
- $\|\mathbf{A}\|_F = \sqrt{1+4+9+16} = \sqrt{30} \approx 5.48$.
- $\|\mathbf{A}\|_2 = \sigma_{\max}$: eigenvalues of $\mathbf{A}^T\mathbf{A}$ satisfy $\lambda^2 - 30\lambda + 4 = 0$, giving $\lambda_{\max} = 15 + \sqrt{221}$, so $\|\mathbf{A}\|_2 \approx 5.46$.

Note $\|\mathbf{A}\|_2 \leq \|\mathbf{A}\|_F$ as guaranteed by the theorem.

Ex 3.16

1. Prove that $\|\mathbf{A}\|_2 \leq \|\mathbf{A}\|_F \leq \sqrt{r}\,\|\mathbf{A}\|_2$ where $r = \operatorname{rank}(\mathbf{A})$. *Hint: use the singular value representation of both norms.*
2. For $n \times n$ matrices, prove $\|\mathbf{A}\|_2 \leq \sqrt{n}\|\mathbf{A}\|_\infty$ and $\|\mathbf{A}\|_\infty \leq \sqrt{n}\|\mathbf{A}\|_2$.
3. Using NumPy, verify these inequalities for 1000 random 5×5 matrices drawn from `np.random.randn`. Plot histograms of the ratios $\|\mathbf{A}\|_F/\|\mathbf{A}\|_2$ and $\|\mathbf{A}\|_\infty/\|\mathbf{A}\|_2$.

Ex 3.17 **(Max norm is not submultiplicative)** Find a pair of 2×2 matrices $\mathbf{A}, \mathbf{B}$ such that $\|\mathbf{AB}\|_{\max} > \|\mathbf{A}\|_{\max}\|\mathbf{B}\|_{\max}$. Conclude that the max norm cannot be an induced norm.

3.3 Stability and Condition Number

In an idealized mathematical setting, the solution to $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is simply $\boldsymbol{x} = \mathbf{A}^{-1}\boldsymbol{b}$. In practice, two separate issues corrupt the computed answer: the *inherent sensitivity* of the problem to perturbations in the data, measured by the condition number, and the *algorithm's ability to avoid amplifying errors* beyond this inherent sensitivity, measured by numerical stability. Understanding both is essential for reliable scientific computing. This section develops the theory behind each and shows how they interact in the matrix factorizations studied so far.

3.3.1 Why Not Invert Directly?

The formula $\boldsymbol{x} = \mathbf{A}^{-1}\boldsymbol{b}$ is mathematically correct but computationally inadvisable. Computing the full inverse $\mathbf{A}^{-1}$ is unnecessary for solving a single system, wastes memory, and is less numerically stable than factorization-based approaches. The arguments below quantify exactly what is lost.

Note 3.11 **Cost of explicit inversion.** Computing $\mathbf{A}^{-1}$ requires solving n systems $\mathbf{A}\mathbf{x}_i = \boldsymbol{e}_i$. After the LU factorization (cost $\frac{2}{3}n^3$), each solve costs $2n^2$, giving a total of $\frac{2}{3}n^3 + 2n^3 \approx \frac{8}{3}n^3$ flops. Compare this to solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ for a single $\boldsymbol{b}$: factorize once at $\frac{2}{3}n^3$, then one triangular solve at $2n^2$. Explicitly forming $\mathbf{A}^{-1}$ costs four times as much as necessary.

Note 3.12 **Stability of explicit inversion.** When $\mathbf{A}$ is nearly singular, explicitly forming $\mathbf{A}^{-1}$ amplifies round-off errors severely. A single erroneous column of $\mathbf{A}^{-1}$ propagates into the entire product $\mathbf{A}^{-1}\boldsymbol{b}$, whereas solving via a stable factorization confines errors to individual triangular solve steps. The practical rule is: never explicitly form $\mathbf{A}^{-1}$. Use `np.linalg.solve(A, b)` for a single system, or `scipy.linalg.lu_factor` and `scipy.linalg.lu_sol` for multiple right-hand sides.

Note 3.13 **Factorize once, solve many.** Given the LU factorization $\mathbf{A} = \mathbf{PLU}$ computed once at cost $\frac{2}{3}n^3$, each new right-hand side $\boldsymbol{b}$ requires only two triangular solves at cost $2n^2$. For k right-hand sides the total is $\frac{2}{3}n^3 + 2kn^2$, far better than k separate $O(n^3)$ solves or storing and applying $\mathbf{A}^{-1}$.

Ex 3.18

1. For $n = 500$, estimate the flop counts for (a) computing $\mathbf{A}^{-1}$ and then applying it to 10 right-hand sides, versus (b) computing the LU factorization once and solving 10 triangular systems. Which is faster, and by what factor?

2. In Python, time `np.linalg.inv(A) @ b` vs. `np.linalg.solve(A, b)` for a random 500×500 matrix over 20 solves. Which is faster? Does the timing match your flop count estimate?

3. Construct a nearly singular matrix `A = np.diag([1, 1, 1e-12])` and a right-hand

side `b = [1, 1, 1]`. Solve via `np.linalg.inv(A) @ b` and `np.linalg.solve(A, b)`. Compare the residuals $\|\mathbf{A}\hat{\boldsymbol{x}} - \boldsymbol{b}\|$.

3.3.2 Numerical Stability

Even after choosing a factorization-based approach, there are good and bad ways to implement the factorization. The concept of backward stability gives a precise criterion for what "good" means: a backward stable algorithm is one whose computed output is the exact output for a slightly perturbed input. This is the best one can hope for in finite-precision arithmetic.

Def 3.17 **(Backward Stability)** An algorithm for computing $f(\boldsymbol{x})$ is *backward stable* if for every input $\boldsymbol{x}$, the computed output $\hat{f}(\boldsymbol{x})$ satisfies

$$\hat{f}(\boldsymbol{x}) = f(\boldsymbol{x} + \delta\boldsymbol{x})$$

for some perturbation $\|\delta\boldsymbol{x}\|/\|\boldsymbol{x}\| = O(\varepsilon_{\text{mach}})$. Applied to $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$: an algorithm is backward stable if the computed solution $\hat{\boldsymbol{x}}$ satisfies $(\mathbf{A} + \delta\mathbf{A})\hat{\boldsymbol{x}} = \boldsymbol{b} + \delta\boldsymbol{b}$ with $\|\delta\mathbf{A}\|/\|\mathbf{A}\| = O(\varepsilon_{\text{mach}})$ and $\|\delta\boldsymbol{b}\|/\|\boldsymbol{b}\| = O(\varepsilon_{\text{mach}})$.

Note 3.14 Backward stability is a property of the *algorithm*, not the problem. Gaussian elimination without pivoting can be unstable; with partial pivoting it is backward stable for virtually all practical matrices.

Ex 3.19 Refer to Def 3.17.

1. Explain in your own words the difference between a backward stable algorithm and one that simply produces a small residual $\|\mathbf{A}\hat{\boldsymbol{x}} - \boldsymbol{b}\|$. Are the two equivalent? (*Hint: think about the relationship between backward error and residual.*)

2. Consider computing e^x for large negative x directly versus via $1/e^{-x}$. Which is backward stable? (*Hint: evaluate both at $x = -30$ in Python and compare to the true value.*)

Prop 3.23 **(Gaussian Elimination with Partial Pivoting is Backward Stable)** Gaussian elimination with partial pivoting (GEPP) produces a computed solution $\hat{\boldsymbol{x}}$ satisfying $(\mathbf{A} + \delta\mathbf{A})\hat{\boldsymbol{x}} = \boldsymbol{b}$ with

$$\|\delta\mathbf{A}\|_\infty \leq g_n \cdot \varepsilon_{\text{mach}} \cdot \|\mathbf{A}\|_\infty,$$

where g_n is the *growth factor*. In the worst case $g_n \leq 2^{n-1}$, but for random matrices in practice $g_n = O(n^{2/3})$.

Proof *Proof.* At each elimination step, partial pivoting selects the largest entry in the current column as pivot, ensuring all multipliers satisfy $|m_{ij}| \leq 1$. This prevents exponential amplification of round-off at each stage. A detailed rounding error analysis over the $O(n^3)$

scalar operations shows the accumulated backward error satisfies the stated bound. For a complete proof see Higham, *Accuracy and Stability of Numerical Algorithms*, Ch. 9. $\square$

Ex 3.20 Refer to Prop 3.23.

1. The growth factor for a random $n \times n$ matrix is typically much smaller than 2^{n-1}. In Python, construct 100 random 50×50 matrices and compute the LU factorization via `scipy.linalg.lu`. For each, compute the growth factor $g_n = \|\mathbf{U}\|_\infty / \|\mathbf{A}\|_\infty$. Plot a histogram. Does g_n stay small in practice?

2. Construct the $n \times n$ Kahan matrix (a known worst-case for GEPP) and verify the growth factor can be large. Use `scipy.linalg.khatri_rao` or build it manually.

3.3.3 Condition Number

Backward stability tells us about the algorithm; the condition number tells us about the problem. Even a perfectly backward stable algorithm cannot produce an accurate solution when the problem itself is ill-conditioned: small perturbations in the input lead to large changes in the output. The condition number quantifies exactly how bad this amplification can be.

Def 3.18 **(Condition Number)** The *condition number* of an invertible matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ with respect to a norm $\|\cdot\|$ is

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \, \|\mathbf{A}^{-1}\|.$$

For the 2-norm, $\kappa_2(\mathbf{A}) = \sigma_{\max}/\sigma_{\min}$ (ratio of largest to smallest singular values). For a symmetric matrix, $\kappa_2(\mathbf{A}) = |\lambda_{\max}|/|\lambda_{\min}|$.

Ex 3.21 Refer to Def 3.18.

1. Compute $\kappa_2(\mathbf{A})$ for $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$ by hand using the singular values. Verify with `np.linalg.cond(A)`.

2. For $\mathbf{A} = \operatorname{diag}(1, 10^{-3}, 10^{-6})$, compute $\kappa_2(\mathbf{A})$, $\kappa_1(\mathbf{A})$, and $\kappa_\infty(\mathbf{A})$. How do they compare?

3. The Hilbert matrix $H_{ij} = 1/(i + j - 1)$ is notoriously ill-conditioned. Compute $\kappa_2(H_n)$ for $n = 3, 5, 8, 10$ using `scipy.linalg.hilbert`. How fast does the condition number grow with n?

Thm 3.24 **(Perturbation Bound for Linear Systems)** Let $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ with $\mathbf{A}$ invertible and $\boldsymbol{b} \neq \mathbf{0}$. If $\hat{\boldsymbol{x}}$ satisfies $(\mathbf{A} + \delta\mathbf{A})\hat{\boldsymbol{x}} = \boldsymbol{b} + \delta\boldsymbol{b}$ and $\|\mathbf{A}^{-1}\| \, \|\delta\mathbf{A}\| < 1$, then

$$\frac{\|\hat{\boldsymbol{x}} - \boldsymbol{x}\|}{\|\boldsymbol{x}\|} \leq \frac{\kappa(\mathbf{A})}{1 - \kappa(\mathbf{A}) \, \|\delta\mathbf{A}\|/\|\mathbf{A}\|} \left(\frac{\|\delta\mathbf{A}\|}{\|\mathbf{A}\|} + \frac{\|\delta\boldsymbol{b}\|}{\|\boldsymbol{b}\|} \right).$$

Proof *Proof.* **Step 1.** Subtract $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ from $(\mathbf{A} + \delta\mathbf{A})\hat{\boldsymbol{x}} = \boldsymbol{b} + \delta\boldsymbol{b}$:

$$(\mathbf{A} + \delta\mathbf{A})(\hat{\boldsymbol{x}} - \boldsymbol{x}) = \delta\boldsymbol{b} - \delta\mathbf{A}\,\boldsymbol{x}.$$

Step 2. Since $\|\mathbf{A}^{-1}\delta\mathbf{A}\| \le \|\mathbf{A}^{-1}\|\,\|\delta\mathbf{A}\| < 1$, the Neumann series gives $(\mathbf{I} + \mathbf{A}^{-1}\delta\mathbf{A})^{-1} = \sum_{k=0}^{\infty}(-\mathbf{A}^{-1}\delta\mathbf{A})^k$, so $\mathbf{A} + \delta\mathbf{A}$ is invertible and

$$\|(\mathbf{A} + \delta\mathbf{A})^{-1}\| \le \frac{\|\mathbf{A}^{-1}\|}{1 - \|\mathbf{A}^{-1}\|\,\|\delta\mathbf{A}\|}.$$

Step 3. Applying this bound to the error from Step 1:

$$\|\hat{\boldsymbol{x}} - \boldsymbol{x}\| \le \frac{\|\mathbf{A}^{-1}\|}{1 - \|\mathbf{A}^{-1}\|\,\|\delta\mathbf{A}\|}\big(\|\delta\boldsymbol{b}\| + \|\delta\mathbf{A}\|\,\|\boldsymbol{x}\|\big).$$

Divide by $\|\boldsymbol{x}\|$ and use $\|\boldsymbol{b}\| \le \|\mathbf{A}\|\,\|\boldsymbol{x}\|$ to convert $\|\boldsymbol{x}\|$ terms to $\|\boldsymbol{b}\|$ terms. This yields the stated bound. □

Cor 3.25 **(Perturbation in the Right-Hand Side Only)** If only $\boldsymbol{b}$ is perturbed ($\delta\mathbf{A} = \mathbf{0}$), then

$$\frac{\|\hat{\boldsymbol{x}} - \boldsymbol{x}\|}{\|\boldsymbol{x}\|} \le \kappa(\mathbf{A})\,\frac{\|\delta\boldsymbol{b}\|}{\|\boldsymbol{b}\|}.$$

Proof *Proof.* Set $\delta\mathbf{A} = \mathbf{0}$ in Thm 3.24. Then $\hat{\boldsymbol{x}} - \boldsymbol{x} = \mathbf{A}^{-1}\delta\boldsymbol{b}$, so $\|\hat{\boldsymbol{x}} - \boldsymbol{x}\| \le \|\mathbf{A}^{-1}\|\,\|\delta\boldsymbol{b}\|$. Dividing by $\|\boldsymbol{x}\| \ge \|\boldsymbol{b}\|/\|\mathbf{A}\|$ gives $\kappa(\mathbf{A})\,\|\delta\boldsymbol{b}\|/\|\boldsymbol{b}\|$. □

Note 3.15 The condition number is the worst-case amplification factor: a relative perturbation of size ε in the data can produce a relative error up to $\kappa(\mathbf{A}) \cdot \varepsilon$ in the solution. In double precision $\varepsilon_{\text{mach}} \approx 10^{-16}$, so we can expect at most $\lfloor 16 - \log_{10}\kappa(\mathbf{A})\rfloor$ correct decimal digits in the computed solution.

Example Let $\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 0 & 10^{-6} \end{pmatrix}$, $\boldsymbol{b} = (1,1)^T$. The exact solution is $\boldsymbol{x}^* = (1, 10^6)^T$ and $\kappa_2(\mathbf{A}) = 10^6$. Perturbing $\boldsymbol{b}$ to $\tilde{\boldsymbol{b}} = (1 + 10^{-4}, 1)^T$ has $\|\delta\boldsymbol{b}\|/\|\boldsymbol{b}\| \approx 10^{-4}$, and the bound predicts relative error up to $10^6 \cdot 10^{-4} = 100$. The actual perturbed solution is $(1 + 10^{-4}, 10^6)^T$, giving relative error $\approx 10^{-10}$: much smaller than the bound because the perturbation was in the well-conditioned direction. The bound is worst-case.

Ex 3.22 Refer to Thm 3.24 and Cor 3.25.

1. For $\mathbf{A}_\epsilon = \begin{pmatrix} 1 & 0 \\ 0 & \epsilon \end{pmatrix}$ and $\boldsymbol{b} = (1,1)^T$, compute $\kappa_2(\mathbf{A}_\epsilon)$ for $\epsilon = 10^{-4}, 10^{-8}, 10^{-12}$. Perturb $\boldsymbol{b}$ to $\tilde{\boldsymbol{b}} = (1 + 10^{-4}, 1)^T$, solve both systems, and compare the actual relative error to the bound $\kappa_2(\mathbf{A}_\epsilon) \cdot \|\delta\boldsymbol{b}\|/\|\boldsymbol{b}\|$.

2. At which ϵ does `np.linalg.solve` give an unreliable answer in double precision? Explain in terms of the number of correct digits formula.

3. Show that for a symmetric matrix $\mathbf{A}$ with eigenvalues $\lambda_1, \ldots, \lambda_n$, $\kappa_2(\mathbf{A}) = |\lambda_{\max}|/|\lambda_{\min}|$. (*Hint: use the Spectral Theorem and the formula* $\kappa_2 = \sigma_{\max}/\sigma_{\min}$.)

3.3.4 Conditioning vs. Stability

Conditioning and stability are often conflated but are fundamentally different. Conditioning is a property of the mathematical problem; stability is a property of the algorithm. The two interact through the forward error bound: for a backward stable algorithm applied to a problem with condition number κ, the forward error is at most $O(\kappa \cdot \varepsilon_{\text{mach}})$. Neither property alone is sufficient: good conditioning with an unstable algorithm, or bad conditioning with a stable one, both lead to inaccurate results.

Def 3.19 **(Conditioning)** *Conditioning* is an intrinsic property of the mathematical problem. It measures the sensitivity of the exact solution to perturbations in the data. A matrix is *ill-conditioned* if $\kappa(\mathbf{A}) \gg 1$ and *well-conditioned* if $\kappa(\mathbf{A})$ is modest. Conditioning cannot be improved by choosing a better algorithm; it can only be improved by reformulating the problem or preconditioning.

Def 3.20 **(Numerical Stability)** *Numerical stability* is a property of the algorithm. It measures how well the algorithm avoids introducing errors beyond those forced by the problem's conditioning. A backward stable algorithm introduces no more error than is intrinsic to the problem.

Note 3.16 A well-conditioned problem solved by an unstable algorithm may yield inaccurate results. An ill-conditioned problem solved by a stable algorithm is still inherently sensitive: no algorithm can overcome poor conditioning. The best strategy is to use a numerically stable algorithm, and if $\kappa(\mathbf{A})$ is large, reformulate or precondition the problem.

Ex 3.23 Refer to Defs 3.19 and 3.20.

1. Give an example of a well-conditioned problem solved inaccurately due to an unstable algorithm. (*Hint: evaluate* $e^x - 1$ *for small* x *in Python directly vs. via* `np.expm1(x)`.)
2. The Hilbert matrix H_n is ill-conditioned. Compute $\kappa_2(H_{12})$ and estimate how many correct digits to expect from double-precision arithmetic. Verify by solving $H_{12}\boldsymbol{x} = H_{12} \cdot \mathbf{1}$ and checking the error.
3. Can you make an ill-conditioned problem well-conditioned by scaling? Construct a diagonal scaling matrix $\mathbf{D} = \text{diag}(d_1, \ldots, d_n)$ such that $\kappa_2(\mathbf{DAD})$ is minimized, and apply it to a 3×3 example.

3.3.5 Stability of Matrix Factorizations

The three main factorizations studied in this course have different stability properties, and the right choice depends on the structure of the problem.

Prop 3.26 **(Stability of Cholesky Factorization)** For symmetric positive definite $\mathbf{A}$, Cholesky factorization $\mathbf{A} = \mathbf{LL}^T$ is backward stable without pivoting. All diagonal entries ℓ_{kk} remain positive and bounded by $\sqrt{\|\mathbf{A}\|_2}$, so no element growth occurs.

Proof *Proof.* For an SPD matrix, all principal submatrices are SPD, so each diagonal entry encountered during Cholesky is positive and the square root is well-defined. Since no cancellation or sign change can occur, the factorization is inherently stable. A rounding error analysis shows the backward error satisfies $\|\mathbf{A} + \delta\mathbf{A} - \hat{\mathbf{L}}\hat{\mathbf{L}}^T\|_F \leq c_n \varepsilon_{\text{mach}} \|\mathbf{A}\|_F$ for a modest constant c_n. □

Prop 3.27 **(Stability of QR via Householder Reflections)** QR decomposition using Householder reflections is backward stable with no growth factor. Each Householder reflector $\mathbf{H} = \mathbf{I} - 2\boldsymbol{v}\boldsymbol{v}^T/\|\boldsymbol{v}\|_2^2$ is orthogonal with $\kappa_2(\mathbf{H}) = 1$, so orthogonal transformations cannot amplify errors.

Proof *Proof.* Each Householder reflection is orthogonal, so $\|\mathbf{H}\|_2 = \|\mathbf{H}^{-1}\|_2 = 1$. Round-off errors at each step are not amplified by subsequent orthogonal transformations. A detailed analysis shows the overall backward error satisfies $\|(\hat{\mathbf{Q}} + \delta\mathbf{Q})(\hat{\mathbf{R}} + \delta\mathbf{R}) - \mathbf{A}\|_F = O(\varepsilon_{\text{mach}}\|\mathbf{A}\|_F)$. □

Note 3.17 **Hierarchy of stability.** SVD $\geq$ QR (Householder) $\geq$ Cholesky (SPD only) $\approx$ LU with partial pivoting $\gg$ LU without pivoting. SVD is the most stable but most expensive. For general systems, GEPP is standard; for SPD systems, Cholesky is preferred; for least squares or ill-conditioned problems, QR is recommended.

Def 3.21 **(Iterative Refinement)** After computing $\hat{\boldsymbol{x}}$ via a factorization, *iterative refinement* improves accuracy by three steps: (1) compute the residual $\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}$; (2) solve $\mathbf{A}\boldsymbol{\delta} = \boldsymbol{r}$ using the same factorization; (3) update $\hat{\boldsymbol{x}} \leftarrow \hat{\boldsymbol{x}} + \boldsymbol{\delta}$. Each cycle recovers roughly $\lfloor \log_{10}(1/\kappa(\mathbf{A})) \rfloor$ additional correct digits, up to the precision limit.

Ex 3.24 Refer to Def 3.21.

1. Implement one step of iterative refinement for $H_{10}\boldsymbol{x} = H_{10} \cdot \mathbf{1}$ (Hilbert matrix, $n = 10$). Compare the error before and after refinement. How many digits did you recover?

2. Apply two steps of refinement. Does a second step help? Is there a limit to how many digits can be recovered?

Ex 3.25 **(Comparing factorization stability.)** Build the $n \times n$ Hilbert matrix via `scipy.linalg.hilbert(n`

1. For $n = 10$: compute $\kappa_2(H)$ and predict how many correct digits to expect. Solve $H\boldsymbol{x} = H \cdot \mathbf{1}$ via `np.linalg.solve` (LU) and `np.linalg.lstsq` (QR-based). Compare the errors $\|\hat{\boldsymbol{x}} - \mathbf{1}\|_2$.

2. Use `mpmath` with `mp.dps = 50` to solve the same system in high precision. Does the error decrease significantly compared to double precision? What does this tell you about conditioning vs. precision?

3. Apply one step of iterative refinement to the LU solution. How much improvement do you observe?

Ex 3.26 **(Backward error analysis in practice.)** For $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ with a random 10×10 matrix scaled so $\kappa(\mathbf{A}) \approx 10^4$:

1. Compute $\hat{\boldsymbol{x}}$ via `np.linalg.solve`. Compute the backward error $\|\mathbf{A}\hat{\boldsymbol{x}} - \boldsymbol{b}\|_2/\|\boldsymbol{b}\|_2$. Is it consistent with $O(\varepsilon_{\text{mach}})$?
2. Compute the forward error $\|\hat{\boldsymbol{x}} - \boldsymbol{x}^*\|_2/\|\boldsymbol{x}^*\|_2$ (using the true solution). Is the ratio (forward error)/(backward error) consistent with $\kappa(\mathbf{A})$?
3. Repeat with $\kappa \approx 10^{12}$ and interpret the results.

3.4 Overdetermined Systems and Least Squares

So far we have focused on square invertible systems $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$. In practice, data fitting and regression produce overdetermined systems ($m > n$ equations, n unknowns) that typically have no exact solution. The method of least squares provides the principled approach: find the $\boldsymbol{x}$ minimizing $\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2$. The geometry, optimality conditions, and numerically stable algorithms for this problem form the core of this section.

3.4.1 Overdetermined Systems

Def 3.22 **(Overdetermined Linear System)** A system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ with $\mathbf{A} \in \mathbb{R}^{m\times n}$, $m > n$, is *overdetermined*. Since the range of $\mathbf{A}$ has dimension at most $n < m$, a generic $\boldsymbol{b} \in \mathbb{R}^m$ lies outside $\mathcal{R}(\mathbf{A})$ and no exact solution exists.

Def 3.23 **(Least Squares Problem)** Given $\mathbf{A} \in \mathbb{R}^{m\times n}$ and $\boldsymbol{b} \in \mathbb{R}^m$, the *least squares problem* is

$$\hat{\boldsymbol{x}} = \arg\min_{\boldsymbol{x}\in\mathbb{R}^n} \|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2.$$

The minimizer $\hat{\boldsymbol{x}}$ makes $\mathbf{A}\hat{\boldsymbol{x}}$ the closest point in $\mathcal{R}(\mathbf{A})$ to $\boldsymbol{b}$ in the Euclidean norm.

Note 3.18 Overdetermined systems arise throughout science: fitting a model with n parameters to $m \gg n$ measurements, regression in statistics, signal denoising, and computer vision. The key insight is that the least squares solution corresponds to the orthogonal projection of $\boldsymbol{b}$ onto the column space of $\mathbf{A}$.

3.4.2 Normal Equations

Thm 3.28 **(Normal Equations)** The vector $\hat{\boldsymbol{x}}$ minimizes $\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2$ if and only if it satisfies the *normal equations*

$$\mathbf{A}^T\mathbf{A}\hat{\boldsymbol{x}} = \mathbf{A}^T\boldsymbol{b}.$$

If $\mathbf{A}$ has full column rank, the unique least squares solution is $\hat{\boldsymbol{x}} = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T\boldsymbol{b}$.

Proof *Proof.* Let $f(\boldsymbol{x}) = \|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2 = \boldsymbol{x}^T\mathbf{A}^T\mathbf{A}\boldsymbol{x} - 2\boldsymbol{b}^T\mathbf{A}\boldsymbol{x} + \boldsymbol{b}^T\boldsymbol{b}$. Since $\mathbf{A}^T\mathbf{A}$ is symmetric, the gradient is $\nabla f = 2\mathbf{A}^T\mathbf{A}\boldsymbol{x} - 2\mathbf{A}^T\boldsymbol{b}$. Setting $\nabla f = \mathbf{0}$ gives $\mathbf{A}^T\mathbf{A}\hat{\boldsymbol{x}} = \mathbf{A}^T\boldsymbol{b}$. The Hessian

is $2\mathbf{A}^T\mathbf{A} \succeq 0$, so this critical point is a global minimum; it is unique when $\mathbf{A}^T\mathbf{A} \succ 0$, i.e. when $\mathbf{A}$ has full column rank. *Geometric view:* $\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}$ must satisfy $\mathbf{A}^T\boldsymbol{r} = \mathbf{0}$ (residual is orthogonal to each column of $\mathbf{A}$), which is exactly the normal equations. □

Note 3.19 The name "normal equations" comes from the fact that the residual $\boldsymbol{r} = \boldsymbol{b} - \mathbf{A}\hat{\boldsymbol{x}}$ is *normal* (perpendicular) to the column space of $\mathbf{A}$. This orthogonality principle $\mathbf{A}^T\boldsymbol{r} = \mathbf{0}$ is fundamental to least squares theory.

Thm 3.29 **(Condition Number Squaring)** If $\mathbf{A}$ has full column rank, then $\kappa_2(\mathbf{A}^T\mathbf{A}) = \kappa_2(\mathbf{A})^2$.

Proof *Proof.* The singular values of $\mathbf{A}$ are $\sigma_1 \geq \cdots \geq \sigma_n > 0$. The matrix $\mathbf{A}^T\mathbf{A}$ is symmetric positive definite with eigenvalues equal to the squared singular values $\sigma_1^2 \geq \cdots \geq \sigma_n^2 > 0$ (since $\mathbf{A}^T\mathbf{A}\boldsymbol{v}_i = \sigma_i^2\boldsymbol{v}_i$ where $\boldsymbol{v}_i$ are right singular vectors). Therefore

$$\kappa_2(\mathbf{A}^T\mathbf{A}) = \frac{\sigma_1^2}{\sigma_n^2} = \left(\frac{\sigma_1}{\sigma_n}\right)^2 = \kappa_2(\mathbf{A})^2.$$

□

Note 3.20 Condition number squaring is a serious practical concern. If $\kappa_2(\mathbf{A}) = 10^6$ (a moderately ill-conditioned design matrix), then $\kappa_2(\mathbf{A}^T\mathbf{A}) = 10^{12}$, and we lose 12 decimal digits of accuracy in the normal equations solution. This motivates QR-based methods that avoid forming $\mathbf{A}^T\mathbf{A}$ entirely. See §3.3 for how condition number governs forward error in computed solutions generally.

Example Fitting a line $y = mx + b$ to four data points $(1, 2), (2, 3), (3, 5), (4, 4)$. The design matrix is

$$\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} 2 \\ 3 \\ 5 \\ 4 \end{pmatrix}, \quad \mathbf{A}^T\mathbf{A} = \begin{pmatrix} 4 & 10 \\ 10 & 30 \end{pmatrix}, \quad \mathbf{A}^T\boldsymbol{b} = \begin{pmatrix} 14 \\ 41 \end{pmatrix}.$$

Solving $(\mathbf{A}^T\mathbf{A})\hat{\boldsymbol{x}} = \mathbf{A}^T\boldsymbol{b}$ gives $\hat{b} = 1.3$ and $\hat{m} = 0.9$, so the best-fit line is $y = 0.9x + 1.3$.

3.4.3 QR-Based Least Squares

Thm 3.30 **(Least Squares via QR Decomposition)** Let $\mathbf{A} \in \mathbb{R}^{m\times n}$ $(m \geq n)$ have full column rank with thin QR decomposition $\mathbf{A} = \mathbf{QR}$ ($\mathbf{Q} \in \mathbb{R}^{m\times n}$ with orthonormal columns, $\mathbf{R} \in \mathbb{R}^{n\times n}$ upper triangular and invertible). Then the unique least squares solution is found by solving the triangular system

$$\mathbf{R}\hat{\boldsymbol{x}} = \mathbf{Q}^T\boldsymbol{b}.$$

Proof *Proof.* Since $\mathbf{Q}$ has orthonormal columns, $\|\mathbf{Q}\boldsymbol{v}\|_2 = \|\boldsymbol{v}\|_2$ for any $\boldsymbol{v}$. Using $\mathbf{A} = \mathbf{QR}$, write $\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2 = \|\mathbf{QR}\boldsymbol{x} - \boldsymbol{b}\|_2$. Split $\boldsymbol{b} = \mathbf{QQ}^T\boldsymbol{b} + (\mathbf{I} - \mathbf{QQ}^T)\boldsymbol{b}$ into its projection onto $\mathcal{R}(\mathbf{A})$

and orthogonal complement. Expanding and using orthogonality of the two components:

$$\|\mathbf{QR}\boldsymbol{x} - \boldsymbol{b}\|_2^2 = \|\mathbf{R}\boldsymbol{x} - \mathbf{Q}^T\boldsymbol{b}\|_2^2 + \|(\mathbf{I} - \mathbf{QQ}^T)\boldsymbol{b}\|_2^2.$$

The second term is independent of $\boldsymbol{x}$, so minimizing over $\boldsymbol{x}$ is equivalent to solving $\mathbf{R}\hat{\boldsymbol{x}} = \mathbf{Q}^T\boldsymbol{b}$, which has unique solution $\hat{\boldsymbol{x}} = \mathbf{R}^{-1}\mathbf{Q}^T\boldsymbol{b}$ since $\mathbf{R}$ is invertible. □

Note 3.21 The QR approach avoids forming $\mathbf{A}^T\mathbf{A}$ and thus does not suffer from condition number squaring. Its cost is $O(mn^2)$ for the QR factorization plus $O(n^2)$ for the triangular solve, comparable to the normal equations but numerically far superior. In NumPy, `np.linalg.lstsq(A, b)` uses this approach internally (QR or SVD depending on shape and rank).

Def 3.24 **(Weighted Least Squares)** If measurements have unequal reliability, one assigns a weight $w_i > 0$ to the i-th equation. The *weighted least squares* problem minimizes $\sum_i w_i((\mathbf{A}\boldsymbol{x})_i - b_i)^2 = \|\mathbf{W}^{1/2}(\mathbf{A}\boldsymbol{x} - \boldsymbol{b})\|_2^2$ where $\mathbf{W} = \operatorname{diag}(w_1, \ldots, w_m)$. Its normal equations are $(\mathbf{A}^T\mathbf{W}\mathbf{A})\hat{\boldsymbol{x}} = \mathbf{A}^T\mathbf{W}\boldsymbol{b}$.

Def 3.25 **(Regularized Least Squares (Ridge Regression))** When $\mathbf{A}$ is rank-deficient or nearly so, the least squares solution can have enormous norm. *Tikhonov regularization* (ridge regression) adds a penalty: $\hat{\boldsymbol{x}}_\lambda = \arg\min_{\boldsymbol{x}}(\|\mathbf{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2 + \lambda\|\boldsymbol{x}\|_2^2) = (\mathbf{A}^T\mathbf{A} + \lambda\mathbf{I})^{-1}\mathbf{A}^T\boldsymbol{b}$. The parameter $\lambda > 0$ reduces the effective condition number to $(\sigma_1^2 + \lambda)/(\sigma_n^2 + \lambda)$.

Note 3.22 The least squares framework extends to many important problems: nonlinear least squares fits nonlinear models by iteratively linearizing (Gauss-Newton/Levenberg-Marquardt); sparse regression (LASSO) replaces the ℓ^2 penalty by an ℓ^1 penalty to promote sparsity; total least squares accounts for errors in both $\mathbf{A}$ and $\boldsymbol{b}$.

Def 3.26 **(Vandermonde Matrix)** Given m sample points $t_1, \ldots, t_m$ and the monomial basis $\{1, t, t^2, \ldots, t^n\}$, the *Vandermonde matrix* is

$$\mathbf{V} = \begin{pmatrix} 1 & t_1 & t_1^2 & \cdots & t_1^n \\ 1 & t_2 & t_2^2 & \cdots & t_2^n \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & t_m & t_m^2 & \cdots & t_m^n \end{pmatrix} \in \mathbb{R}^{m\times(n+1)}.$$

The polynomial fitting problem with monomial coefficients $\boldsymbol{c} = (c_0, \ldots, c_n)^T$ is $\mathbf{V}\boldsymbol{c} \approx \mathbf{f}$, solved as a least squares problem when $m > n + 1$. For square systems ($m = n + 1$, interpolation), $\mathbf{V}$ is invertible whenever the nodes t_i are distinct.

Ex 3.27 **(Polynomial regression)** Fit a degree-2 polynomial $p(t) = c_0 + c_1 t + c_2 t^2$ to the data (t_i, y_i) for $i = 1, \ldots, 6$: $t = (0, 0.5, 1, 1.5, 2, 2.5)$, $y = (1.0, 1.8, 2.3, 2.5, 2.3, 2.0)$.

1. Write down the Vandermonde design matrix $\mathbf{A} \in \mathbb{R}^{6\times 3}$ explicitly.
2. Solve the normal equations by hand (or with NumPy). State the fitted polynomial.

3. Compute the residual $\|\mathbf{A}\hat{\boldsymbol{x}} - \boldsymbol{b}\|_2$ and the coefficient of determination $R^2 = 1 - \|\mathbf{A}\hat{\boldsymbol{x}} - \boldsymbol{b}\|_2^2 / \|\boldsymbol{b} - \bar{b}\mathbf{1}\|_2^2$.

4. Now solve via QR using `np.linalg.lstsq` and verify you get the same coefficients. Compare $\kappa_2(\mathbf{A})$ with $\kappa_2(\mathbf{A}^T\mathbf{A})$ to illustrate condition number squaring.

Ex 3.28 **(Effect of regularization)** Use the Vandermonde design matrix for fitting a degree-8 polynomial to 10 equally spaced points on $[0, 1]$ with $y_i = \sin(2\pi t_i) + 0.1\varepsilon_i$ (small noise).

1. Compute the least squares fit without regularization. Plot the fitted polynomial. Does it overfit?

2. Compute $\hat{\boldsymbol{x}}_\lambda$ for $\lambda = 10^{-4}, 10^{-2}, 1$. How does the fit change?

3. Plot $\|\hat{\boldsymbol{x}}_\lambda\|_2$ and the residual $\|\mathbf{A}\hat{\boldsymbol{x}}_\lambda - \boldsymbol{b}\|_2$ as functions of λ (the bias-variance tradeoff curve). Which λ gives the best fit?

Ex 3.29 **(Numerical stability of normal equations vs. QR)** Construct the $m \times n$ matrix $\mathbf{A}$ from `np.random.randn(m, n)` scaled so $\kappa_2(\mathbf{A}) \approx 10^6$ (use SVD to rescale singular values). Let $\boldsymbol{b} = \mathbf{A}\boldsymbol{x}_{\text{true}} + \boldsymbol{\varepsilon}$ with $\boldsymbol{x}_{\text{true}} = \mathbf{1}$ and small noise.

1. Solve via normal equations: `np.linalg.solve(A.T @ A, A.T @ b)`.

2. Solve via QR: `np.linalg.lstsq(A, b)`.

3. Compare the errors $\|\hat{\boldsymbol{x}} - \boldsymbol{x}_{\text{true}}\|_2$. Which method is more accurate? Explain in terms of condition numbers.

3.5 Chebyshev Polynomials and Polynomial Approximation

In the least squares exercises above, you built Vandermonde matrices (Def 3.26) to fit polynomials to data. That worked fine for low degrees, but something goes wrong as the degree grows: the matrix becomes catastrophically ill-conditioned, and even a carefully placed set of interpolation points leads to a polynomial that oscillates wildly. Both failures have the same root cause: the monomial basis $\{1, x, x^2, \ldots, x^n\}$ is a poor basis for polynomial approximation. Chebyshev polynomials fix both problems simultaneously. They are orthogonal, bounded, and satisfy a minimax optimality property that makes them the best possible basis for approximating smooth functions on a bounded interval. This subsection demonstrates each failure mode and shows precisely how Chebyshev polynomials resolve it.

3.5.1 Why the Monomial Basis Fails

There are two distinct ways the monomial basis breaks down. The first is numerical: the Vandermonde matrix built from $\{1, x, x^2, \ldots, x^n\}$ has a condition number that grows

exponentially with degree, regardless of the node placement. The second is analytical: even if the linear system could be solved exactly, interpolating a smooth function at equally spaced points leads to a polynomial that diverges near the endpoints as the degree grows. Understanding both failures motivates every design choice in the Chebyshev approach.

Recall from Def 3.26 that the Vandermonde matrix $\mathbf{V}$ encodes a polynomial fitting problem in the monomial basis: given nodes $x_0, \ldots, x_n$ and values $f(x_0), \ldots, f(x_n)$, the coefficients of the interpolating polynomial satisfy $\mathbf{V}\boldsymbol{c} = \mathbf{f}$. The matrix is invertible whenever the nodes are distinct (since its determinant equals $\prod_{i>j}(x_i - x_j)$). The question we focus on here is not whether a solution exists, but how reliably it can be computed.

Ex 3.30 Refer to Def 3.26.

1. Build the square Vandermonde matrix for $n+1$ equally spaced nodes on $[-1, 1]$ in NumPy for $n = 5, 10, 15, 20$. Compute $\kappa_2(\mathbf{V})$ for each. Plot $\kappa_2(\mathbf{V})$ vs. n on a log scale. How fast does the condition number grow?
2. For $n = 10$, solve $\mathbf{V}\boldsymbol{c} = \mathbf{f}$ with $f(x_i) = \cos(\pi x_i)$ using `np.linalg.solve`, then using `np.linalg.lstsq`. Do you get the same coefficients? Compute the residual $\|\mathbf{V}\boldsymbol{c} - \mathbf{f}\|_2$ for both.
3. If $\mathbf{f}$ has noise at level 10^{-12} and $\kappa(\mathbf{V}) = 10^{10}$, what is the worst-case relative error in $\boldsymbol{c}$?

Def 3.27 **(Runge's Phenomenon)** *Runge's phenomenon* is the divergence of polynomial interpolants at equally spaced nodes as the degree grows. The prototypical example is Runge's function $f(x) = \frac{1}{1+25x^2}$ on $[-1, 1]$: although f is infinitely differentiable, the degree-n interpolant at $n+1$ equally spaced nodes satisfies

$$\max_{x \in [-1,1]} |f(x) - p_n(x)| \to \infty \quad \text{as } n \to \infty.$$

The oscillations are largest near the endpoints ± 1.

Note 3.23 The interpolation error at a point x is bounded by

$$|f(x) - p_n(x)| \leq \frac{\|f^{(n+1)}\|_\infty}{(n+1)!} |\omega_n(x)|, \qquad \omega_n(x) = \prod_{k=0}^{n} (x - x_k).$$

For equally spaced nodes, $\|\omega_n\|_\infty$ grows like $(n/2e)^n$ near the endpoints (even though $(n+1)!$ also grows, the numerator wins). For Runge's function, the derivatives $\|f^{(n+1)}\|_\infty$ also blow up, compounding the problem.

Ex 3.31 **(Runge's phenomenon.)** Refer to Def 3.27.

1. In NumPy, interpolate $f(x) = 1/(1 + 25x^2)$ at $n+1$ equally spaced nodes on $[-1, 1]$ for $n = 5, 10, 15, 20$ using `np.polyfit` or by solving the Vandermonde system. For each n, plot both f and p_n on the same axes. Describe what happens near $x = \pm 1$ as n grows.

2. Compute $\max_x |f(x) - p_n(x)|$ (use 500 evaluation points) for $n = 5, 10, 15, 20$. Plot the maximum error vs. n. Does it decrease?

3. (*The node polynomial.*) For $n = 10$ equally spaced nodes on $[-1, 1]$, plot $|\omega_{10}(x)|$ vs. x. Where is $|\omega_{10}|$ largest? How does this explain the shape of the interpolation error in part 1?

3.5.2 Chebyshev Polynomials

The failures of the monomial basis come down to two causes: the monomials x^k are nearly linearly dependent on $[-1, 1]$ for large k (leading to ill-conditioning), and equispaced nodes concentrate the node polynomial near the boundary (leading to Runge's phenomenon). Chebyshev polynomials are the natural remedy: they are bounded by 1 on $[-1, 1]$, form an orthogonal basis, and their zeros (the Chebyshev nodes) distribute mass towards the endpoints in exactly the way needed to suppress boundary oscillations.

Def 3.28 **(Chebyshev Polynomials)** The *Chebyshev polynomials of the first kind* $T_k : [-1, 1] \to \mathbb{R}$ are defined by

$$T_k(x) = \cos(k \arccos x), \quad k = 0, 1, 2, \ldots$$

They satisfy the three-term recurrence

$$T_0(x) = 1, \quad T_1(x) = x, \quad T_{k+1}(x) = 2x\, T_k(x) - T_{k-1}(x),$$

and the first few are $T_0 = 1$, $T_1 = x$, $T_2 = 2x^2 - 1$, $T_3 = 4x^3 - 3x$, $T_4 = 8x^4 - 8x^2 + 1$.

Thm 3.31 **(Key Properties of Chebyshev Polynomials)** For all $k \geq 0$ and $x \in [-1, 1]$:

1. **Boundedness**: $|T_k(x)| \leq 1$, with $T_k(\cos\theta) = \cos(k\theta)$.
2. **Equioscillation**: T_k attains the values ± 1 exactly $k + 1$ times on $[-1, 1]$ (at $x_j = \cos(j\pi/k)$ for $j = 0, \ldots, k$).
3. **Zeros**: T_k has k simple roots at $\tilde{x}_j = \cos\left(\frac{(2j-1)\pi}{2k}\right)$ for $j = 1, \ldots, k$.
4. **Leading coefficient**: T_k is a polynomial of degree k with leading coefficient 2^{k-1} (for $k \geq 1$).

Proof *Proof.* All four properties follow directly from the definition $T_k(x) = \cos(k \arccos x)$.

(1) and (2): Setting $x = \cos\theta$ gives $T_k(\cos\theta) = \cos(k\theta)$. Since $|\cos(k\theta)| \leq 1$ for all θ, we have $|T_k(x)| \leq 1$. The cosine equals ± 1 at $\theta = j\pi/k$, giving the equioscillation points.
(3): The roots are where $\cos(k\theta) = 0$, i.e., $k\theta = (2j - 1)\pi/2$, giving $\theta = (2j - 1)\pi/(2k)$ and $x = \cos((2j - 1)\pi/(2k))$.
(4): The recurrence $T_{k+1} = 2x\, T_k - T_{k-1}$ starts from $T_1(x) = x$ (leading coefficient $1 = 2^0$) and $T_2(x) = 2x^2 - 1$ (leading coefficient $2 = 2^1$). By induction, the recurrence doubles the leading coefficient at each step, giving 2^{k-1} for T_k. $\square$

Ex 3.32 Refer to Def 3.28 and Thm 3.31.

1. Using the recurrence, compute $T_5(x)$ and $T_6(x)$ by hand. Verify your results with `numpy.polynomial.chebyshev.chebval`.

2. Plot $T_0, T_1, \ldots, T_5$ on $[-1, 1]$ on the same axes. Confirm that each T_k stays within $[-1, 1]$ and oscillates exactly k times.

3. Verify numerically that the roots of T_5 are $x_j = \cos((2j-1)\pi/10)$ for $j = 1, \ldots, 5$.

4. Confirm the leading coefficient of T_k is 2^{k-1} for $k = 2, 3, 4$ by expanding the recurrence and reading off the coefficient of x^k.

3.5.3 Orthogonality and Chebyshev Nodes

The cosine definition immediately reveals an orthogonality structure. Under the change of variables $x = \cos\theta$, the Chebyshev polynomials become ordinary cosines, and their orthogonality follows from the orthogonality of cosines. In the x variable, the weight function $w(x) = 1/\sqrt{1-x^2}$ appears, which concentrates mass near the endpoints ± 1. The zeros of T_n (the Chebyshev nodes) are exactly the points where this endpoint clustering is captured in a finite node set. Using them for interpolation produces a Vandermonde-type system that is dramatically better conditioned than the equispaced case.

Thm 3.32 **(Chebyshev Orthogonality)** The Chebyshev polynomials are orthogonal on $[-1, 1]$ with weight $w(x) = 1/\sqrt{1-x^2}$:

$$\int_{-1}^{1} T_j(x)\, T_k(x)\, \frac{dx}{\sqrt{1-x^2}} = \begin{cases} 0 & j \neq k, \\ \pi/2 & j = k \geq 1, \\ \pi & j = k = 0. \end{cases}$$

Proof *Proof.* Substitute $x = \cos\theta$, $dx = -\sin\theta\, d\theta$, $\sqrt{1-x^2} = \sin\theta$:

$$\int_{-1}^{1} T_j(x)\, T_k(x)\, \frac{dx}{\sqrt{1-x^2}} = \int_{\pi}^{0} \cos(j\theta)\cos(k\theta)\, \frac{-\sin\theta\, d\theta}{\sin\theta} = \int_{0}^{\pi} \cos(j\theta)\cos(k\theta)\, d\theta.$$

The last integral is the standard cosine orthogonality relation, which equals 0 for $j \neq k$, $\pi/2$ for $j = k \geq 1$, and π for $j = k = 0$. □

Def 3.29 **(Chebyshev Nodes)** The n *Chebyshev nodes of the first kind* on $[-1, 1]$ are the zeros of T_n:

$$x_k = \cos\left(\frac{(2k-1)\pi}{2n}\right), \quad k = 1, \ldots, n.$$

They cluster near the endpoints ± 1 (the density of nodes near x is proportional to $1/\sqrt{1-x^2}$, exactly matching the weight in the orthogonality relation). For nodes on a general interval $[a, b]$, shift and scale: $x_k^{[a,b]} = \frac{a+b}{2} + \frac{b-a}{2}\, x_k$.

Thm 3.33 **(The Node Polynomial at Chebyshev Nodes)** For the n Chebyshev nodes $x_1, \ldots, x_n$, the node polynomial satisfies

$$\prod_{k=1}^{n}(x - x_k) = \frac{T_n(x)}{2^{n-1}}.$$

Its supremum norm on $[-1, 1]$ is $1/2^{n-1}$.

Proof *Proof.* The zeros of T_n are exactly the Chebyshev nodes x_k. The leading coefficient of T_n is 2^{n-1} (Thm 3.31(4)), so the monic polynomial with the same roots is $T_n(x)/2^{n-1}$. Since $|T_n(x)| \leq 1$ on $[-1, 1]$, the supremum norm of the node polynomial is at most $1/2^{n-1}$, and it is attained at the equioscillation points of T_n. □

Ex 3.33 Refer to Defs 3.29 and 3.33.

1. Compute the 10 Chebyshev nodes on $[-1, 1]$ and plot them alongside 10 equally spaced nodes. Describe the qualitative difference in their distribution.

2. For $n = 10$, plot $|\omega_n(x)|$ for both equispaced nodes and Chebyshev nodes on $[-1, 1]$ using 500 evaluation points. Compare $\max_x |\omega_n(x)|$ for both. How much smaller is the Chebyshev version?

3. Now repeat the Runge function experiment (Exercise from Def 3.27) but using Chebyshev nodes instead of equispaced nodes. Plot the max interpolation error vs. n for both node sets on the same axes. Does the error decrease with Chebyshev nodes?

Ex 3.34 **(Condition numbers: monomial vs. Chebyshev basis.)** Let $\mathbf{V}_{\text{mono}}$ be the Vandermonde matrix with $n + 1$ Chebyshev nodes and the monomial basis $V_{ij} = x_i^j$. Let $\mathbf{V}_{\text{cheb}}$ be the same nodes but with the Chebyshev basis $V_{ij} = T_j(x_i)$.

1. Build both matrices in NumPy for $n = 5, 10, 15, 20$. Compute κ_2 for each. Plot both condition numbers vs. n on the same log-scale axes.

2. Explain why $\mathbf{V}_{\text{cheb}}$ has a much better condition number, using the orthogonality of $\{T_k\}$ (Thm 3.32).

3. Using both matrices, solve the interpolation problem for $f(x) = 1/(1 + 25x^2)$ at $n = 20$ Chebyshev nodes. Do both systems give the same polynomial? Which system is safer to solve numerically?

3.5.4 The Minimax Property

The node polynomial theorem says Chebyshev nodes minimize $\|\omega_n\|_\infty$ over all choices of n nodes. But Chebyshev polynomials satisfy something even stronger: $T_n/2^{n-1}$ is the unique monic polynomial of degree n that has the smallest possible sup-norm on $[-1, 1]$, among all monic polynomials of degree n. This minimax property is the algebraic reason

Chebyshev nodes and basis are optimal, and it connects directly to the equioscillation characterization of best polynomial approximations.

Thm 3.34 **(Chebyshev Minimax Property)** Among all monic polynomials of degree n, the scaled Chebyshev polynomial $\tilde{T}_n(x) = T_n(x)/2^{n-1}$ has the smallest sup-norm on $[-1, 1]$:

$$\|\tilde{T}_n\|_\infty = \frac{1}{2^{n-1}} \leq \|q\|_\infty$$

for every monic polynomial q of degree n.

Proof *Proof.* We argue by contradiction.

Step 1: Suppose a better monic polynomial exists.
Assume q is monic of degree n and $\|q\|_\infty < 1/2^{n-1}$.
Step 2: Form the difference.
Let $r = \tilde{T}_n - q$. Since both are monic of degree n, their leading coefficients cancel and r has degree at most $n - 1$.
Step 3: Count sign changes.
The polynomial $\tilde{T}_n$ equioscillates between $+1/2^{n-1}$ and $-1/2^{n-1}$ at $n+1$ points $x_0 > x_1 > \cdots > x_n$ (the extrema of T_n). At each x_j, $|\tilde{T}_n(x_j)| = 1/2^{n-1}$ and $|q(x_j)| < 1/2^{n-1}$, so $r(x_j) = \tilde{T}_n(x_j) - q(x_j)$ has the same sign as $\tilde{T}_n(x_j)$. Since $\tilde{T}_n$ alternates sign at consecutive extrema, so does r.
Step 4: Derive a contradiction.
r changes sign at least n times, so r has at least n roots. But $\deg(r) \leq n - 1$, so r can have at most $n - 1$ roots unless $r \equiv 0$. This is a contradiction, so no such q exists. □

Ex 3.35 Refer to Thm 3.34.

1. For $n = 1, 2, 3$, write out $\tilde{T}_n(x) = T_n(x)/2^{n-1}$ explicitly and verify $\|\tilde{T}_n\|_\infty = 1/2^{n-1}$ by finding its maximum on $[-1, 1]$.

2. Show that among all polynomials p of degree $\leq n-1$, the best approximation to x^n in the sup-norm on $[-1, 1]$ is $x^n - \tilde{T}_n(x)$. (*Hint: write* $x^n = \tilde{T}_n(x) + (x^n - \tilde{T}_n(x))$ *and apply the minimax theorem.*)

3. In NumPy, generate 10 random monic polynomials of degree 5 with coefficients drawn from $\mathcal{N}(0, 1)$ (for the lower-degree terms). Compare their $\|\cdot\|_\infty$ norms on $[-1, 1]$ to $1/2^4 = 1/16$. How many beat $1/16$?

3.5.5 Chebyshev Series and Approximation

Just as the Fourier series expands a periodic function in the basis $\{e^{ikx}\}$, the Chebyshev series expands a function on $[-1, 1]$ in the basis $\{T_k\}$. Under the substitution $x = \cos\theta$, a function $f(x)$ on $[-1, 1]$ becomes $g(\theta) = f(\cos\theta)$ on $[0, \pi]$, and the Chebyshev series of f is exactly the cosine series of g. This is yet another instance of the Fourier change-of-basis from Section 6.4, now applied to the interval $[-1, 1]$ via the substitution $x = \cos\theta$.

Def 3.30 **(Chebyshev Series)** For $f : [-1, 1] \to \mathbb{R}$, the *Chebyshev series* is

$$f(x) = \sum_{k=0}^{\infty}{}' c_k T_k(x),$$

where the prime on the sum means the $k = 0$ term is halved, and the coefficients are

$$c_k = \frac{2}{\pi} \int_{-1}^{1} f(x)\, T_k(x)\, \frac{dx}{\sqrt{1 - x^2}}.$$

The truncation $p_n(x) = \sum_{k=0}^{n}{}' c_k T_k(x)$ is the degree-n Chebyshev approximation to f.

Thm 3.35 **(Exponential Convergence for Analytic Functions)** If f is analytic on $[-1, 1]$ and can be analytically continued to a Bernstein ellipse with parameter $\rho > 1$ (an ellipse with foci at ± 1 and sum of semi-axes equal to ρ), then the Chebyshev coefficients satisfy

$$|c_k| \leq M\, \rho^{-k}$$

for some constant M. Consequently, the truncation error satisfies $\|f - p_n\|_\infty = O(\rho^{-n})$ as $n \to \infty$.

Note 3.24 The exponential convergence rate contrasts sharply with the polynomial convergence of finite-difference approximations ($O(h^p)$ for order-p methods). For smooth functions, increasing the polynomial degree by 1 multiplies the error by $1/\rho < 1$. In practice, just 20 to 30 Chebyshev terms often give errors near machine precision for analytic functions. This is called *spectral accuracy*.

Ex 3.36 Refer to Def 3.30 and Thm 3.35.

1. Use `numpy.polynomial.chebyshev.chebfit(x, f, deg)` to compute the degree-n Chebyshev approximation to $f(x) = e^x$ on $[-1, 1]$ for $n = 2, 4, 8, 16$. Plot $\|f - p_n\|_\infty$ vs. n on a log scale. Does the error decrease exponentially?

2. Repeat for $f(x) = |x|$ (not analytic at $x = 0$). Compare the convergence rate for e^x and $|x|$. What does Thm 3.35 predict for $|x|$, given that it cannot be analytically continued past any ellipse?

3. (*Comparison of all three methods.*) For $f(x) = 1/(1 + 25x^2)$, compute the degree-n approximation error $\|f - p_n\|_\infty$ for $n = 5, 10, 15, 20$ using three methods: (a) monomial basis with equispaced nodes (Vandermonde solve), (b) monomial basis with Chebyshev nodes, (c) Chebyshev basis with Chebyshev nodes (`chebfit`). Plot all three error curves on the same log-scale axes. Which methods converge? Which diverge? Explain each result using the theory above.

Ex 3.37 **(Chebyshev differentiation.)** Because the Chebyshev basis diagonalizes the right operations, differentiation in the Chebyshev basis is a structured recurrence rather than a full matrix-vector product.

1. Show that $T_k'(x) = kU_{k-1}(x)$ where $U_{k-1}(x) = \sin(k \arccos x)/\sin(\arccos x)$ are the Chebyshev polynomials of the second kind. (*Hint: differentiate* $T_k(x) = \cos(k \arccos x)$ *using the chain rule.*)

2. The derivative of $f(x) = \sum_k c_k T_k(x)$ has Chebyshev coefficients $c_k^{(1)}$ satisfying the recurrence $c_{k-1}^{(1)} = c_{k+1}^{(1)} + 2kc_k$ (summing from $k = n$ down to 1). Use this to differentiate $p(x) = T_3(x) + 2T_2(x)$ and verify against the exact derivative.

3. Implement spectral differentiation via `numpy.polynomial.chebyshev.chebder` and compare the accuracy of differentiating $f(x) = \sin(\pi x)$ using (a) second-order finite differences with n equispaced points and (b) the Chebyshev spectral derivative with n Chebyshev nodes. Plot the error vs. n for both. At $n = 20$, which method is more accurate?

4 Eigenvalues and the SVD

4.1 Eigenvalues and Eigenvectors

Eigenvalues and eigenvectors reveal the fundamental character of a linear transformation: the special directions along which the matrix acts by pure scaling. This section develops the theory from the characteristic polynomial through the Spectral Theorem, then turns to practical algorithms — power iteration, inverse iteration, and the QR algorithm — for computing eigenvalues numerically.

Def 4.1 **(Eigenvalues and Eigenvectors)** For a matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, a scalar $\lambda \in \mathbb{C}$ is called an *eigenvalue* of $\mathbf{A}$ if there exists a nonzero vector $\boldsymbol{v} \in \mathbb{C}^n$ such that

$$\mathbf{A}\boldsymbol{v} = \lambda \boldsymbol{v}.$$

The nonzero vector $\boldsymbol{v}$ is called an *eigenvector* of $\mathbf{A}$ corresponding to λ. The set of all eigenvalues of $\mathbf{A}$ is called the *spectrum* of $\mathbf{A}$, denoted $\sigma(\mathbf{A})$.

Intuitively, an eigenvector is a vector whose direction remains unchanged when transformed by $\mathbf{A}$ — it may be stretched, compressed, or reversed, but it stays on the same line. The eigenvalue tells us the factor by which the eigenvector is scaled. Eigenvalues and eigenvectors reveal the fundamental character of a linear transformation, showing the directions in which the transformation behaves most simply.

Ex 4.1 Refer to Def 4.1.

1. Verify that $\boldsymbol{v} = (1, 1)^T$ is an eigenvector of $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$. What is the corresponding eigenvalue?

2. If $\boldsymbol{v}$ is an eigenvector of $\mathbf{A}$ with eigenvalue λ, show that $c\boldsymbol{v}$ is also an eigenvector for any nonzero scalar c.

3. Can the zero vector be an eigenvector? Why is it explicitly excluded from the definition?

Def 4.2 **(Characteristic Polynomial)** For a matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, the *characteristic polynomial* is

$$p_{\mathbf{A}}(\lambda) = \det(\mathbf{A} - \lambda\mathbf{I}),$$

where $\mathbf{I}$ is the $n \times n$ identity matrix. The equation $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$ is the *characteristic equation* of $\mathbf{A}$. The roots of the characteristic polynomial are precisely the eigenvalues of $\mathbf{A}$.

Example Consider the matrix $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$.
To find its eigenvalues, we compute the characteristic polynomial:

$$p_{\mathbf{A}}(\lambda) = \det \begin{pmatrix} 3-\lambda & 1 \\ 1 & 3-\lambda \end{pmatrix} = (3-\lambda)^2 - 1 = \lambda^2 - 6\lambda + 8 = (\lambda - 4)(\lambda - 2). \tag{10}$$

Setting $p_{\mathbf{A}}(\lambda) = 0$ gives eigenvalues $\lambda_1 = 4$ and $\lambda_2 = 2$.
For $\lambda_1 = 4$: solving $(\mathbf{A} - 4\mathbf{I})\boldsymbol{v} = \mathbf{0}$ gives $-v_1 + v_2 = 0$, so $\boldsymbol{v}_1 = \frac{1}{\sqrt{2}}(1, 1)^T$ (normalized).
For $\lambda_2 = 2$: solving $(\mathbf{A} - 2\mathbf{I})\boldsymbol{v} = \mathbf{0}$ gives $v_1 + v_2 = 0$, so $\boldsymbol{v}_2 = \frac{1}{\sqrt{2}}(1, -1)^T$ (normalized).

Ex 4.2 Refer to Def 4.2.

1. Compute the characteristic polynomial and eigenvalues of $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$. Verify using trace: $\lambda_1 + \lambda_2 = \mathrm{tr}(\mathbf{A})$.
2. For the 2×2 rotation matrix $\mathbf{B} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$, compute the characteristic polynomial. Are the eigenvalues real? What does this say geometrically?
3. In NumPy, use `np.linalg.eigvals(A)` to find the eigenvalues of each matrix above and verify they match your hand computations.

Prop 4.1 **(Eigenvalues of Triangular Matrices)** If $\mathbf{A}$ is upper or lower triangular, its eigenvalues are precisely the diagonal entries $a_{11}, a_{22}, \ldots, a_{nn}$.

Proof *Proof.* The characteristic polynomial $\det(\mathbf{A} - \lambda\mathbf{I})$ for a triangular matrix is the product of its diagonal entries $(a_{ii} - \lambda)$, giving $p_{\mathbf{A}}(\lambda) = \prod_{i=1}^{n}(a_{ii} - \lambda)$. The roots are $\lambda_i = a_{ii}$. □

Prop 4.2 **(Similar Matrices Share Eigenvalues)** If $\mathbf{B} = \mathbf{V}^{-1}\mathbf{A}\mathbf{V}$ for some invertible $\mathbf{V}$, then $\mathbf{A}$ and $\mathbf{B}$ have the same characteristic polynomial and hence the same eigenvalues.

Proof *Proof.* $\det(\mathbf{B} - \lambda\mathbf{I}) = \det(\mathbf{V}^{-1}\mathbf{A}\mathbf{V} - \lambda\mathbf{I}) = \det(\mathbf{V}^{-1}(\mathbf{A} - \lambda\mathbf{I})\mathbf{V}) = \det(\mathbf{V}^{-1})\det(\mathbf{A} - \lambda\mathbf{I})\det(\mathbf{V}) = \det(\mathbf{A} - \lambda\mathbf{I})$. □

Thm 4.3 **(Key Eigenvalue Properties)** For $\mathbf{A} \in \mathbb{R}^{n\times n}$ with eigenvalues $\lambda_1, \ldots, \lambda_n$ (counting algebraic multiplicity):

1. $\operatorname{tr}(\mathbf{A}) = \sum_{i=1}^{n} \lambda_i$

2. $\det(\mathbf{A}) = \prod_{i=1}^{n} \lambda_i$

3. $\mathbf{A}$ is singular if and only if $0 \in \sigma(\mathbf{A})$.

4. If $\mathbf{A}$ is invertible, the eigenvalues of $\mathbf{A}^{-1}$ are $1/\lambda_1, \ldots, 1/\lambda_n$.

Proof *Proof.* (1) and (2): Both follow from comparing the characteristic polynomial

$$p_{\mathbf{A}}(\lambda) = \det(\mathbf{A} - \lambda\mathbf{I}) = (-1)^n(\lambda - \lambda_1)\cdots(\lambda - \lambda_n)$$

with the direct expansion of the determinant. Setting $\lambda = 0$ gives $\det(\mathbf{A}) = \prod_i \lambda_i$. The coefficient of λ^{n-1} gives $-\operatorname{tr}(\mathbf{A}) = -\sum_i \lambda_i$.
(3): Direct from (2): $\det(\mathbf{A}) = 0 \iff \prod_i \lambda_i = 0 \iff$ some $\lambda_i = 0$.
(4): If $\mathbf{A}\boldsymbol{v} = \lambda\boldsymbol{v}$ with $\lambda \neq 0$, multiply both sides by $\frac{1}{\lambda}\mathbf{A}^{-1}$ to get $\mathbf{A}^{-1}\boldsymbol{v} = \frac{1}{\lambda}\boldsymbol{v}$. □

Lem 4.4 **(Eigenvectors for Distinct Eigenvalues are Linearly Independent)** Let $\lambda_1, \ldots, \lambda_k$ be distinct eigenvalues of $\mathbf{A}$ with corresponding eigenvectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k$. Then $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ is linearly independent.

Proof *Proof.* By induction on k. The case $k = 1$ holds since eigenvectors are nonzero. Assume $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_{k-1}\}$ is independent. Suppose $\sum_{i=1}^{k} c_i\boldsymbol{v}_i = \mathbf{0}$. Apply $(\mathbf{A} - \lambda_k\mathbf{I})$ to both sides:

$$\sum_{i=1}^{k-1} c_i(\lambda_i - \lambda_k)\boldsymbol{v}_i = \mathbf{0}.$$

By the inductive hypothesis, $c_i(\lambda_i - \lambda_k) = 0$ for each $i < k$. Since $\lambda_i \neq \lambda_k$, each $c_i = 0$. Substituting back gives $c_k\boldsymbol{v}_k = \mathbf{0}$, so $c_k = 0$. □

Ex 4.3 Refer to Props 4.1–4.2, Thm 4.3, and Lem 4.4.

1. Read off the eigenvalues of $\mathbf{T} = \begin{pmatrix} 4 & 2 & 1 \\ 0 & -1 & 3 \\ 0 & 0 & 2 \end{pmatrix}$ without any computation. Verify using `np.linalg.eigvals(T)`.

2. For $\mathbf{A} = \begin{pmatrix} 5 & 2 \\ 2 & 2 \end{pmatrix}$, use properties (1) and (2) of Thm 4.3 to check your eigenvalue computation: verify $\lambda_1 + \lambda_2 = \operatorname{tr}(\mathbf{A})$ and $\lambda_1\lambda_2 = \det(\mathbf{A})$.

3. Let $\mathbf{A}$ have eigenvalues $2, 3, 5$. What are the eigenvalues of $\mathbf{A}^{-1}$? Of $\mathbf{A}^2$? Of $\mathbf{A} - 2\mathbf{I}$?

4. If $\mathbf{A}$ has three distinct eigenvalues, does it follow that its three eigenvectors form a basis for $\mathbb{R}^3$? Cite the relevant lemma.

Thm 4.5 **(Spectral Theorem for Real Symmetric Matrices)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be symmetric. Then:

1. All eigenvalues of $\mathbf{A}$ are real.
2. Eigenvectors corresponding to distinct eigenvalues are orthogonal.
3. $\mathbf{A}$ is orthogonally diagonalizable: $\mathbf{A} = \mathbf{Q}\mathbf{D}\mathbf{Q}^T$ where $\mathbf{Q}$ is orthogonal and $\mathbf{D} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n) \in \mathbb{R}^{n\times n}$.

Proof *Proof.* (1): Let $\lambda \in \mathbb{C}$ be an eigenvalue with eigenvector $\boldsymbol{v} \in \mathbb{C}^n$, $\mathbf{A}\boldsymbol{v} = \lambda\boldsymbol{v}$. Then $\bar{\boldsymbol{v}}^T\mathbf{A}\boldsymbol{v} = \lambda\,\bar{\boldsymbol{v}}^T\boldsymbol{v}$. But also $\bar{\boldsymbol{v}}^T\mathbf{A}\boldsymbol{v} = (\mathbf{A}\bar{\boldsymbol{v}})^T\boldsymbol{v} = \overline{(\mathbf{A}\boldsymbol{v})}^T\boldsymbol{v} = \bar{\lambda}\,\bar{\boldsymbol{v}}^T\boldsymbol{v}$. Since $\bar{\boldsymbol{v}}^T\boldsymbol{v} = \|\boldsymbol{v}\|_2^2 > 0$, we conclude $\lambda = \bar{\lambda}$, so $\lambda \in \mathbb{R}$.
(2): Let $\mathbf{A}\boldsymbol{v} = \lambda\boldsymbol{v}$ and $\mathbf{A}\boldsymbol{w} = \mu\boldsymbol{w}$ with $\lambda \neq \mu$. Then:

$$\lambda\,\boldsymbol{v}^T\boldsymbol{w} = (\mathbf{A}\boldsymbol{v})^T\boldsymbol{w} = \boldsymbol{v}^T\mathbf{A}^T\boldsymbol{w} = \boldsymbol{v}^T\mathbf{A}\boldsymbol{w} = \mu\,\boldsymbol{v}^T\boldsymbol{w}.$$

Since $\lambda \neq \mu$, we get $\boldsymbol{v}^T\boldsymbol{w} = 0$.
(3): Existence of a full orthonormal eigenbasis follows from (1), (2), and the fact that the characteristic polynomial splits over $\mathbb{R}$ for symmetric matrices (proven via induction on dimension in a full course). $\square$

Def 4.3 **(Hermitian Matrix)** A matrix $\mathbf{A} \in \mathbb{C}^{n\times n}$ is *Hermitian* if $\mathbf{A} = \mathbf{A}^*$, where $\mathbf{A}^* = \bar{\mathbf{A}}^T$ is the *conjugate transpose*: each entry a_{ij} is replaced by its complex conjugate $\overline{a_{ji}}$. For real matrices, $\mathbf{A}^* = \mathbf{A}^T$, so symmetric matrices are exactly the real Hermitian matrices.

Ex 4.4 Refer to Def 4.3.

1. Verify that $\mathbf{A} = \begin{pmatrix} 2 & 1+i \\ 1-i & 3 \end{pmatrix}$ is Hermitian by computing $\mathbf{A}^*$.
2. Show that for any $\mathbf{A} \in \mathbb{C}^{n\times n}$, the matrix $\mathbf{A} + \mathbf{A}^*$ is always Hermitian and $\mathbf{A} - \mathbf{A}^*$ is always *skew-Hermitian* ($\mathbf{M}^* = -\mathbf{M}$).
3. Show that the diagonal entries of a Hermitian matrix must be real.

Thm 4.6 **(Spectral Theorem for Hermitian Matrices)** Let $\mathbf{A} \in \mathbb{C}^{n\times n}$ be Hermitian. Then:

1. All eigenvalues of $\mathbf{A}$ are real.
2. Eigenvectors for distinct eigenvalues are orthogonal with respect to the complex inner product $\langle \boldsymbol{u}, \boldsymbol{v}\rangle = \boldsymbol{u}^*\boldsymbol{v}$.
3. $\mathbf{A}$ is unitarily diagonalizable: $\mathbf{A} = \mathbf{Q}\mathbf{D}\mathbf{Q}^*$, where $\mathbf{Q} \in \mathbb{C}^{n\times n}$ is unitary ($\mathbf{Q}^*\mathbf{Q} = \mathbf{I}$) and $\mathbf{D} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n) \in \mathbb{R}^{n\times n}$.

Proof *Proof.* The proof mirrors the real symmetric case, replacing transposes with conjugate transposes.

(1) Eigenvalues are real. Let $\mathbf{A}\boldsymbol{v} = \lambda\boldsymbol{v}$ with $\boldsymbol{v} \neq \mathbf{0}$. Compute $\boldsymbol{v}^*\mathbf{A}\boldsymbol{v}$ in two ways. On one hand, $\boldsymbol{v}^*\mathbf{A}\boldsymbol{v} = \boldsymbol{v}^*(\lambda\boldsymbol{v}) = \lambda\|\boldsymbol{v}\|_2^2$. On the other, since $\mathbf{A} = \mathbf{A}^*$:

$$\boldsymbol{v}^*\mathbf{A}\boldsymbol{v} = \boldsymbol{v}^*\mathbf{A}^*\boldsymbol{v} = (\mathbf{A}\boldsymbol{v})^*\boldsymbol{v} = (\lambda\boldsymbol{v})^*\boldsymbol{v} = \bar{\lambda}\|\boldsymbol{v}\|_2^2.$$

Since $\|\boldsymbol{v}\|_2^2 > 0$, we get $\lambda = \bar{\lambda}$, so $\lambda \in \mathbb{R}$.

(2) Eigenvectors for distinct eigenvalues are orthogonal. Let $\mathbf{A}\boldsymbol{v} = \lambda\boldsymbol{v}$ and $\mathbf{A}\boldsymbol{w} = \mu\boldsymbol{w}$ with $\lambda \neq \mu$ (both real). Then:

$$\lambda\,\boldsymbol{v}^*\boldsymbol{w} = (\mathbf{A}\boldsymbol{v})^*\boldsymbol{w} = \boldsymbol{v}^*\mathbf{A}^*\boldsymbol{w} = \boldsymbol{v}^*\mathbf{A}\boldsymbol{w} = \mu\,\boldsymbol{v}^*\boldsymbol{w}.$$

Since $\lambda \neq \mu$, we conclude $\boldsymbol{v}^*\boldsymbol{w} = 0$.

(3) Unitary diagonalizability. That a complete unitary eigenbasis always exists follows from induction on dimension (analogous to the real symmetric case) and is covered in a full linear algebra course. □

Note 4.1 **Hermitian vs. symmetric, unitary vs. orthogonal.** The complex analogs pair up cleanly: real symmetric $\leftrightarrow$ complex Hermitian (both have real eigenvalues and an orthonormal eigenbasis), and real orthogonal $\leftrightarrow$ complex unitary (both preserve the norm of every vector). The DFT matrix, which you will see in the Fourier analysis section, is unitary but not Hermitian. The discrete Laplacian matrix $\mathbf{T}$ from the operators section is real symmetric, a special case of Hermitian.

Ex 4.5 Refer to Thm 4.6.

1. Find the eigenvalues and a unitary eigenbasis for $\mathbf{A} = \begin{pmatrix} 2 & 1+i \\ 1-i & 3 \end{pmatrix}$. Verify the eigenvalues are real and the eigenvectors are orthogonal.

2. In NumPy, `np.linalg.eigh` is designed for Hermitian matrices. Apply it to the matrix above and confirm it returns real eigenvalues. What happens if you use `np.linalg.eig` instead?

3. Show that $\mathbf{A}^2$ is also Hermitian whenever $\mathbf{A}$ is Hermitian. What are the eigenvalues of $\mathbf{A}^2$ in terms of those of $\mathbf{A}$?

Def 4.4 **(Diagonalization)** A matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ is *diagonalizable* if there exists an invertible matrix $\mathbf{V}$ and a diagonal matrix $\boldsymbol{\Lambda}$ such that

$$\mathbf{A} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^{-1},$$

where the diagonal entries of $\boldsymbol{\Lambda}$ are the eigenvalues of $\mathbf{A}$ and the columns of $\mathbf{V}$ are the corresponding eigenvectors.

Note 4.2 A matrix $\mathbf{A}$ is diagonalizable if and only if there exist n linearly independent eigenvectors, which occurs if and only if the geometric multiplicity of each eigenvalue equals its algebraic multiplicity.

The eigenvalue problem is one of the most fundamental in numerical linear algebra, with applications ranging from structural engineering (vibration analysis) to quantum mechanics (energy states), machine learning (principal component analysis), and dynamical systems (stability analysis). The efficiency and accuracy of eigenvalue algorithms have direct implications for these applications, making them a central focus in scientific computing.

Ex 4.6 Refer to Thm 4.5 and Def 4.4.

1. For $\mathbf{A} = \begin{pmatrix} 4 & 1 \\ 1 & 4 \end{pmatrix}$, find eigenvalues and an orthonormal eigenbasis $\{\boldsymbol{q}_1, \boldsymbol{q}_2\}$. Write $\mathbf{A} = \mathbf{Q}\mathbf{D}\mathbf{Q}^T$ explicitly.
2. Use the decomposition to compute $\mathbf{A}^{10}$ efficiently. (*Hint:* $\mathbf{A}^k = \mathbf{Q}\mathbf{D}^k\mathbf{Q}^T$, *and* $\mathbf{D}^k$ *is easy.*)
3. In NumPy, use `np.linalg.eig(A)` and `np.linalg.eigh(A)` (the latter exploits symmetry). Compare the results and explain why `eigh` is preferred for symmetric matrices.

Ex 4.7 For each matrix, find all eigenvalues and a corresponding eigenvector for each.

1. $\mathbf{A} = \begin{pmatrix} 5 & 2 \\ 2 & 2 \end{pmatrix}$ (verify using trace and determinant)
2. $\mathbf{B} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ (rotation by 90°; are eigenvalues real?)
3. $\mathbf{C} = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}$ (use Prop 4.1)

Ex 4.8 **(Diagonalizability and the Geometric Multiplicity)** The matrix $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 0 & 2 \end{pmatrix}$ has a repeated eigenvalue $\lambda = 2$.

1. Find the eigenspace $\ker(\mathbf{A} - 2\mathbf{I})$. What is its dimension (geometric multiplicity)?
2. Is $\mathbf{A}$ diagonalizable? Why or why not?
3. Compute $\mathbf{A}^k$ for general k using the Jordan form $\mathbf{A} = 2\mathbf{I} + \mathbf{N}$ where $\mathbf{N} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ is nilpotent. (*Hint:* $\mathbf{N}^2 = \mathbf{0}$, *so* $(2\mathbf{I} + \mathbf{N})^k = \sum_{j=0}^{1} \binom{k}{j} 2^{k-j}\mathbf{N}^j$.)
4. What does `np.linalg.eig(A)` return for this matrix? Are the eigenvectors linearly independent?

4.1.1 Power Iteration and Inverse Iteration

The spectral theorem tells us eigenvalues exist and what they look like structurally. Power iteration gives a practical way to *compute* them. The fundamental idea is that repeated multiplication by $\mathbf{A}$ amplifies the dominant eigenvector component — the one corresponding to the largest eigenvalue — while the rest decay away.

Def 4.5 **(Power Iteration)** Given $\mathbf{A} \in \mathbb{R}^{n\times n}$ and a starting vector $\boldsymbol{v}^{(0)}$ with $\|\boldsymbol{v}^{(0)}\|_2 = 1$, the *power iteration* produces a sequence of unit vectors and Rayleigh quotients:

$$\boldsymbol{w}^{(k)} = \mathbf{A}\boldsymbol{v}^{(k-1)}, \quad \boldsymbol{v}^{(k)} = \frac{\boldsymbol{w}^{(k)}}{\|\boldsymbol{w}^{(k)}\|_2}, \quad \lambda^{(k)} = (\boldsymbol{v}^{(k)})^T\mathbf{A}\boldsymbol{v}^{(k)}.$$

The scalar $\lambda^{(k)}$ is the *Rayleigh quotient* at step k and serves as an estimate of the dominant eigenvalue.

Thm 4.7 **(Convergence of Power Iteration)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be diagonalizable with eigenvalues $|\lambda_1| > |\lambda_2| \geq \cdots \geq |\lambda_n|$ and corresponding eigenvectors $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$. Suppose $\boldsymbol{v}^{(0)}$ has a nonzero component in the $\boldsymbol{v}_1$ direction. Then:

$$\boldsymbol{v}^{(k)} \to \pm\boldsymbol{v}_1 \quad \text{and} \quad \lambda^{(k)} \to \lambda_1 \quad \text{as } k \to \infty,$$

with convergence rate $|\lambda_2/\lambda_1|^k$.

Proof *Proof.* Write $\boldsymbol{v}^{(0)} = c_1\boldsymbol{v}_1 + c_2\boldsymbol{v}_2 + \cdots + c_n\boldsymbol{v}_n$ with $c_1 \neq 0$. Then:

$$\mathbf{A}^k\boldsymbol{v}^{(0)} = c_1\lambda_1^k\boldsymbol{v}_1 + \cdots + c_n\lambda_n^k\boldsymbol{v}_n = \lambda_1^k\Big(c_1\boldsymbol{v}_1 + \sum_{i>1} c_i\Big(\frac{\lambda_i}{\lambda_1}\Big)^k\boldsymbol{v}_i\Big).$$

Since $|\lambda_i/\lambda_1| < 1$ for $i > 1$, the sum decays to zero and $\mathbf{A}^k\boldsymbol{v}^{(0)}/\|\mathbf{A}^k\boldsymbol{v}^{(0)}\| \to \pm\boldsymbol{v}_1$. The Rayleigh quotient satisfies $\lambda^{(k)} \to \lambda_1$ with error $O(|\lambda_2/\lambda_1|^{2k})$. □

Note 4.3 Power iteration fails when: (1) $|\lambda_1| = |\lambda_2|$ (e.g., complex conjugate pairs); (2) $\boldsymbol{v}^{(0)}$ has no component along $\boldsymbol{v}_1$ (measure zero, but can happen numerically). Normalization at each step prevents overflow/underflow. The convergence rate $|\lambda_2/\lambda_1|$ can be very slow if the two dominant eigenvalues are close in magnitude.

Def 4.6 **(Inverse Iteration)** *Inverse iteration* applies power iteration to $(\mathbf{A} - \sigma\mathbf{I})^{-1}$ for a shift σ near a target eigenvalue μ. The iteration is:

$$\text{Solve } (\mathbf{A} - \sigma\mathbf{I})\boldsymbol{w}^{(k)} = \boldsymbol{v}^{(k-1)}, \quad \boldsymbol{v}^{(k)} = \frac{\boldsymbol{w}^{(k)}}{\|\boldsymbol{w}^{(k)}\|_2}.$$

The eigenvalue estimate is updated as the Rayleigh quotient $\lambda^{(k)} = (\boldsymbol{v}^{(k)})^T\mathbf{A}\boldsymbol{v}^{(k)}$.

Note 4.4 When σ is close to μ, the matrix $\mathbf{A} - \sigma\mathbf{I}$ is nearly singular, but the solve still yields the correct eigenvector direction. In practice, $\mathbf{A} - \sigma\mathbf{I}$ is factored once (LU or Cholesky),

making each inverse iteration step $O(n^2)$ (for dense $\mathbf{A}$) instead of $O(n^3)$. The convergence rate is $|(\mu - \sigma)/(\tilde{\mu} - \sigma)|$ where $\tilde{\mu}$ is the nearest eigenvalue to σ besides μ.

Def 4.7 **(Rayleigh Quotient Iteration)** *Rayleigh Quotient Iteration* (RQI) applies inverse iteration with the shift $\sigma_k = \lambda^{(k)}$ updated at each step. Starting from a unit vector $\boldsymbol{v}^{(0)}$:

$$\sigma_k = (\boldsymbol{v}^{(k)})^T \mathbf{A} \boldsymbol{v}^{(k)}, \quad \text{solve } (\mathbf{A} - \sigma_k \mathbf{I}) \boldsymbol{w}^{(k+1)} = \boldsymbol{v}^{(k)}, \quad \boldsymbol{v}^{(k+1)} = \boldsymbol{w}^{(k+1)} / \|\boldsymbol{w}^{(k+1)}\|.$$

RQI converges *cubically* for symmetric matrices: once near an eigenvector, the number of correct digits roughly triples each step.

Example **Power iteration on a 3×3 symmetric matrix.**
Let $\mathbf{A} = \begin{pmatrix} 4 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{pmatrix}$, $\boldsymbol{v}^{(0)} = (1, 0, 0)^T$. The eigenvalues are approximately $\lambda_1 \approx 5.21$, $\lambda_2 \approx 2.79$, $\lambda_3 \approx 1.00$.
Starting power iteration:

$$\begin{aligned} \boldsymbol{w}^{(1)} &= \mathbf{A}\boldsymbol{v}^{(0)} = (4, 1, 0)^T, \quad \boldsymbol{v}^{(1)} = (4, 1, 0)^T / \sqrt{17} \approx (0.970, 0.243, 0)^T, \\ \lambda^{(1)} &= (\boldsymbol{v}^{(1)})^T \mathbf{A} \boldsymbol{v}^{(1)} \approx 4.12. \end{aligned}$$

The ratio $|\lambda_2/\lambda_1| \approx 0.535$, so convergence is moderately fast; after ~ 10 iterations the estimate $\lambda^{(k)} \approx 5.21$ to 4 digits.

Ex 4.9 Refer to Defs 4.5–4.7 and Thm 4.7.

1. Implement `power_iteration(A, v0, tol, max_iter)` that returns the dominant eigenvalue and eigenvector. Apply it to the tridiagonal 5×5 matrix with 3 on the diagonal and -1 on the off-diagonals. Plot the Rayleigh quotient error vs. k on a semi-log scale; verify the slope equals $\log |\lambda_2/\lambda_1|$.

2. Implement inverse iteration to find the *smallest* eigenvalue of the same matrix (use $\sigma = 0$). Compare the convergence rate to power iteration.

3. For a random 4×4 symmetric matrix, apply Rayleigh Quotient Iteration from a random starting vector. Plot the error in the eigenvalue estimate vs. k on a log scale. Can you observe cubic convergence?

4. What happens to power iteration when $|\lambda_1| = |\lambda_2|$? Construct an example (e.g., a rotation matrix scaled to have two equal-magnitude eigenvalues) and observe the behavior in NumPy.

4.1.2 Invariant Subspaces

Eigenvectors are special one-dimensional invariant subspaces. The notion of an invariant subspace generalizes this: a subspace that $\mathbf{A}$ maps into itself. Understanding invariant subspaces is the key to the Schur decomposition and the convergence of the QR algorithm.

Def 4.8 **(Invariant Subspace)** A subspace $\mathcal{S} \subseteq \mathbb{C}^n$ is *invariant* under $\mathbf{A} \in \mathbb{C}^{n\times n}$ if

$$\mathbf{A}\mathcal{S} \subseteq \mathcal{S}, \qquad \text{i.e., } \mathbf{A}\boldsymbol{v} \in \mathcal{S} \text{ for every } \boldsymbol{v} \in \mathcal{S}.$$

Example The canonical invariant subspaces of $\mathbf{A}$:

- $\{\mathbf{0}\}$ and $\mathbb{C}^n$ — trivially invariant for any $\mathbf{A}$.
- $\operatorname{span}\{\boldsymbol{v}\}$ for any eigenvector $\boldsymbol{v}$ — since $\mathbf{A}\boldsymbol{v} = \lambda\boldsymbol{v} \in \operatorname{span}\{\boldsymbol{v}\}$.
- $\ker(\mathbf{A} - \lambda\mathbf{I})^k$ — the *generalized eigenspace* for eigenvalue λ (Jordan block structure).
- $\mathcal{R}(\mathbf{A})$ — the column space is invariant since $\mathbf{A}^2\boldsymbol{x} = \mathbf{A}(\mathbf{A}\boldsymbol{x}) \in \mathcal{R}(\mathbf{A})$ for all $\boldsymbol{x}$.

Thm 4.8 **(Invariant Subspace and Block Triangular Form)** Let $\mathcal{S} = \operatorname{span}\{\boldsymbol{q}_1, \ldots, \boldsymbol{q}_k\}$ be a k-dimensional invariant subspace of $\mathbf{A}$, with $\mathbf{Q}_1 = [\boldsymbol{q}_1, \ldots, \boldsymbol{q}_k]$ having orthonormal columns. Extend to an orthonormal basis $\mathbf{Q} = [\mathbf{Q}_1 \mid \mathbf{Q}_2]$ of $\mathbb{C}^n$. Then:

$$\mathbf{Q}^*\mathbf{A}\mathbf{Q} = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{0} & \mathbf{A}_{22} \end{pmatrix},$$

where $\mathbf{A}_{11} \in \mathbb{C}^{k\times k}$. The eigenvalues of $\mathbf{A}_{11}$ are a subset of the eigenvalues of $\mathbf{A}$.

Proof *Proof.* Since $\mathcal{S}$ is invariant, $\mathbf{A}\mathbf{Q}_1 = \mathbf{Q}_1\mathbf{A}_{11}$ for some $\mathbf{A}_{11} \in \mathbb{C}^{k\times k}$. Multiplying on the left by $\mathbf{Q}^*$:

$$\mathbf{Q}^*\mathbf{A}\mathbf{Q} = \mathbf{Q}^*[\mathbf{A}\mathbf{Q}_1 \mid \mathbf{A}\mathbf{Q}_2] = \begin{pmatrix} \mathbf{Q}_1^*\mathbf{Q}_1\mathbf{A}_{11} & \mathbf{Q}_1^*\mathbf{A}\mathbf{Q}_2 \\ \mathbf{Q}_2^*\mathbf{Q}_1\mathbf{A}_{11} & \mathbf{Q}_2^*\mathbf{A}\mathbf{Q}_2 \end{pmatrix} = \begin{pmatrix} \mathbf{A}_{11} & \mathbf{A}_{12} \\ \mathbf{0} & \mathbf{A}_{22} \end{pmatrix},$$

using $\mathbf{Q}_1^*\mathbf{Q}_1 = \mathbf{I}$, $\mathbf{Q}_2^*\mathbf{Q}_1 = \mathbf{0}$ (orthonormality). The eigenvalues of the block upper triangular matrix are the union of eigenvalues of $\mathbf{A}_{11}$ and $\mathbf{A}_{22}$, so $\sigma(\mathbf{A}_{11}) \subseteq \sigma(\mathbf{A})$. $\square$

Note 4.5 This is why invariant subspaces are central to eigenvalue computation. If you can find a nontrivial invariant subspace $\mathcal{S}$, you *deflate* the problem: reduce finding all n eigenvalues to finding the eigenvalues of $\mathbf{A}_{11}$ (dimension k) and $\mathbf{A}_{22}$ (dimension $n-k$) independently. The QR algorithm works precisely by building up an invariant subspace one eigenvector (or complex-conjugate pair) at a time.

Def 4.9 **(Eigenspace and Generalized Eigenspace)** For $\lambda \in \sigma(\mathbf{A})$, the *eigenspace* is $\ker(\mathbf{A} - \lambda\mathbf{I})$ — the set of all eigenvectors (plus $\mathbf{0}$). Its dimension is the *geometric multiplicity* of λ. The *generalized eigenspace* is $\ker(\mathbf{A} - \lambda\mathbf{I})^n$, which captures all vectors associated with λ through the Jordan structure. Both are invariant subspaces.

Ex 4.10 Refer to Def 4.9 and Thm 4.8.

1. For $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 0 & 3 \end{pmatrix}$, find the eigenspace $\ker(\mathbf{A} - 3\mathbf{I})$ and the generalized eigenspace $\ker(\mathbf{A} - 3\mathbf{I})^2$. What is the dimension of each?

2. Identify two nontrivial invariant subspaces of $\mathbf{B} = \begin{pmatrix} 3 & 0 \\ 0 & 5 \end{pmatrix}$ and write the block triangular form $\mathbf{Q}^T\mathbf{B}\mathbf{Q}$ for each.

3. In NumPy, compute the Schur decomposition of a random 4×4 matrix and verify that $\text{span}\{\boldsymbol{q}_1, \boldsymbol{q}_2\}$ (first two Schur vectors) is invariant: check that $\mathbf{A}\mathbf{Q}[:, :2]$ lies in the column space of $\mathbf{Q}[:, :2]$.

Def 4.10 **(Stable, Unstable, and Center Subspaces)** For a matrix $\mathbf{A}$ representing a dynamical system $\boldsymbol{x}^{(k+1)} = \mathbf{A}\boldsymbol{x}^{(k)}$, the eigenvalues partition the spectrum into three sets:

- $\sigma^- = \{\lambda \in \sigma(\mathbf{A}) : |\lambda| < 1\}$ — *stable*: orbits decay to zero.
- $\sigma^+ = \{\lambda \in \sigma(\mathbf{A}) : |\lambda| > 1\}$ — *unstable*: orbits grow without bound.
- $\sigma^0 = \{\lambda \in \sigma(\mathbf{A}) : |\lambda| = 1\}$ — *center*: orbits neither grow nor decay.

The corresponding generalized eigenspaces span the *stable subspace* $\mathcal{S}^-$, *unstable subspace* $\mathcal{S}^+$, and *center subspace* $\mathcal{S}^0$ — each invariant under $\mathbf{A}$, and $\mathbb{C}^n = \mathcal{S}^- \oplus \mathcal{S}^+ \oplus \mathcal{S}^0$.

Note 4.6 In the continuous-time system $\dot{\boldsymbol{x}} = \mathbf{A}\boldsymbol{x}$, stability is governed by the *real parts* of eigenvalues instead: $\sigma^- = \{\lambda : \text{Re}(\lambda) < 0\}$ (stable), etc. The stable/unstable subspace decomposition is identical in structure — only the threshold changes from $|\lambda| = 1$ to $\text{Re}(\lambda) = 0$.

Ex 4.11 **(Identifying and using invariant subspaces)** Refer to Defs 4.8–4.10 and Thm 4.8.

1. For $\mathbf{A} = \begin{pmatrix} 3 & 1 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 5 \end{pmatrix}$, identify two nontrivial invariant subspaces. For each, write the block triangular form $\mathbf{Q}^T\mathbf{A}\mathbf{Q}$ explicitly and read off the eigenvalues.

2. Prove that if $\mathcal{S}$ and $\mathcal{T}$ are both invariant under $\mathbf{A}$, then so are $\mathcal{S} \cap \mathcal{T}$ and $\mathcal{S} + \mathcal{T} = \{\boldsymbol{u} + \boldsymbol{w} : \boldsymbol{u} \in \mathcal{S},\ \boldsymbol{w} \in \mathcal{T}\}$.

3. For a 4×4 matrix $\mathbf{A}$ with eigenvalues $0.5, 0.8, 1.2, 1.5$, identify which belong to the stable and unstable subspaces (discrete-time). What are the dimensions of each?

4. In NumPy, compute the Schur decomposition of a random 4×4 matrix. Verify that the first column of $\mathbf{Q}$ is an eigenvector and that $\text{span}\{\boldsymbol{q}_1, \boldsymbol{q}_2\}$ is invariant: check that $\mathbf{A}\mathbf{Q}[:, :2]$ lies in the column space of $\mathbf{Q}[:, :2]$.

4.1.3 Schur Decomposition

Not every matrix is diagonalizable, but every square matrix can be reduced to triangular form via a unitary similarity. The Schur decomposition is the theoretical foundation for practical eigenvalue algorithms: the QR algorithm computes the Schur form iteratively, revealing all eigenvalues on the diagonal of the triangular factor.

Thm 4.9 **(Schur Decomposition)** Every matrix $\mathbf{A} \in \mathbb{C}^{n\times n}$ has a *Schur decomposition*

$$\mathbf{A} = \mathbf{Q}\mathbf{T}\mathbf{Q}^*,$$

where $\mathbf{Q} \in \mathbb{C}^{n\times n}$ is unitary ($\mathbf{Q}^*\mathbf{Q} = \mathbf{I}$) and $\mathbf{T} \in \mathbb{C}^{n\times n}$ is upper triangular. The diagonal entries of $\mathbf{T}$ are the eigenvalues of $\mathbf{A}$ (in any order). If $\mathbf{A} \in \mathbb{R}^{n\times n}$ has all real eigenvalues, then $\mathbf{Q}$ and $\mathbf{T}$ can be taken to be real.

Proof *Proof.* By induction on n. The base case $n = 1$ is trivial. For the inductive step, let λ_1 be any eigenvalue of $\mathbf{A}$ (it exists over $\mathbb{C}$) with unit eigenvector $\boldsymbol{q}_1$. Extend $\boldsymbol{q}_1$ to an orthonormal basis $\{\boldsymbol{q}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_n\}$ and let $\mathbf{Q}_1 = [\boldsymbol{q}_1 \mid \boldsymbol{u}_2 \mid \cdots \mid \boldsymbol{u}_n]$. Then:

$$\mathbf{Q}_1^*\mathbf{A}\mathbf{Q}_1 = \begin{pmatrix} \lambda_1 & \star \\ \mathbf{0} & \mathbf{B} \end{pmatrix},$$

where $\mathbf{B} \in \mathbb{C}^{(n-1)\times(n-1)}$. By the inductive hypothesis, $\mathbf{B} = \mathbf{Q}_2\mathbf{T}'\mathbf{Q}_2^*$ for some unitary $\mathbf{Q}_2$ and upper triangular $\mathbf{T}'$. Setting $\mathbf{Q} = \mathbf{Q}_1 \operatorname{diag}(1, \mathbf{Q}_2)$ gives the Schur decomposition with $\mathbf{T} = \begin{pmatrix} \lambda_1 & \star \\ \mathbf{0} & \mathbf{T}' \end{pmatrix}$. □

Note 4.7 The Schur decomposition is a generalization of diagonalization: if $\mathbf{A}$ is normal ($\mathbf{A}^*\mathbf{A} = \mathbf{A}\mathbf{A}^*$), then $\mathbf{T}$ is diagonal (the Spectral Theorem). For non-normal matrices, off-diagonal entries of $\mathbf{T}$ are generally nonzero, but the eigenvalues are still read from the diagonal.

Def 4.11 **(Real Schur Form)** For $\mathbf{A} \in \mathbb{R}^{n\times n}$, the *real Schur decomposition* is $\mathbf{A} = \mathbf{Q}\mathbf{T}\mathbf{Q}^T$ where $\mathbf{Q} \in \mathbb{R}^{n\times n}$ is orthogonal and $\mathbf{T}$ is *quasi-upper triangular*: block upper triangular with 1×1 diagonal blocks (for real eigenvalues) and 2×2 diagonal blocks (for complex-conjugate eigenvalue pairs). This avoids complex arithmetic entirely and is the form computed by `scipy.linalg.schur(A)`.

Def 4.12 **(Schur Vectors)** The columns of $\mathbf{Q}$ in the Schur decomposition $\mathbf{A} = \mathbf{Q}\mathbf{T}\mathbf{Q}^*$ are called *Schur vectors*. Unlike eigenvectors, Schur vectors always exist and form a complete orthonormal basis, even when $\mathbf{A}$ is defective. The first k Schur vectors span an invariant subspace of $\mathbf{A}$: if $\mathbf{Q}_k = [\boldsymbol{q}_1, \ldots, \boldsymbol{q}_k]$, then $\mathbf{A}\mathbf{Q}_k = \mathbf{Q}_k\mathbf{T}_{11}$ where $\mathbf{T}_{11}$ is the leading $k \times k$ block of $\mathbf{T}$.

Ex 4.12 Refer to Defs 4.11–4.12 and Thm 4.9.

1. For $\mathbf{A} = \begin{pmatrix} 3 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{pmatrix}$, is $\mathbf{A}$ already in Schur form? What are its Schur vectors? What are the eigenvalues?

2. For a 2×2 matrix with complex-conjugate eigenvalues $a \pm bi$, write out the 2×2 real Schur form block explicitly in terms of a and b.

3. In Python, compute `T, Q = scipy.linalg.schur(A)` for a random 4×4 matrix. Verify: (a) $\mathbf{QTQ}^T = \mathbf{A}$, (b) $\mathbf{Q}$ is orthogonal, (c) the diagonal of $\mathbf{T}$ matches `np.linalg.eigvals(A)`.

Thm 4.10 **(Connection to the QR Algorithm)** The basic QR algorithm for computing eigenvalues proceeds by iteratively applying QR factorizations:

$$\mathbf{A}_0 = \mathbf{A}, \quad \mathbf{A}_k = \mathbf{R}_k\mathbf{Q}_k, \quad \mathbf{A}_{k+1} = \mathbf{Q}_k\mathbf{R}_k = \mathbf{Q}_k^T\mathbf{A}_k\mathbf{Q}_k.$$

Each $\mathbf{A}_{k+1}$ is orthogonally similar to $\mathbf{A}$, so all matrices share the same eigenvalues. Under mild conditions, the sequence $\mathbf{A}_k$ converges to the Schur form $\mathbf{T}$, with eigenvalues appearing on the diagonal.

Ex 4.13 In this exercise, we explore two practical enhancements of the basic QR algorithm (Thm 4.10): Hessenberg reduction and shifts.

1. **(Hessenberg reduction cost.)** An upper Hessenberg matrix has zeros below the first subdiagonal. A single QR factorization step on a general $n \times n$ matrix costs $O(n^3)$ flops, but on an upper Hessenberg matrix it costs only $O(n^2)$. Explain why: how many Givens rotations are needed to reduce an upper Hessenberg matrix to triangular form, and what does each rotation cost?

2. **(Shifts preserve eigenvalues.)** A shifted QR step factors $\mathbf{A}_k - \sigma_k\mathbf{I} = \mathbf{Q}_k\mathbf{R}_k$, then forms $\mathbf{A}_{k+1} = \mathbf{R}_k\mathbf{Q}_k + \sigma_k\mathbf{I}$. Show that $\mathbf{A}_{k+1}$ is orthogonally similar to $\mathbf{A}_k$, and hence has the same eigenvalues.

3. **(Convergence comparison in NumPy.)** Let $\mathbf{A} = \begin{pmatrix} 4 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{pmatrix}$.

   (a) Run 30 basic QR steps (no shifts) and record $|(\mathbf{A}_k)_{32}|$ at each step.

   (b) Repeat with the Rayleigh quotient shift $\sigma_k = (\mathbf{A}_k)_{nn}$. Compare the number of steps required to reach $|(\mathbf{A}_k)_{32}| < 10^{-10}$.

4. Verify using `np.linalg.eig(A)` that the final diagonal entries of your converged $\mathbf{A}_k$ match the computed eigenvalues. What LAPACK routine does `np.linalg.eig` call internally?

Ex 4.14 **(Schur decomposition in practice)**

1. For $\mathbf{A} = \begin{pmatrix} 3 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{pmatrix}$, compute the Schur decomposition by hand. Is $\mathbf{A}$ already in Schur form? What are its Schur vectors?

2. In Python, compute `T, Q = scipy.linalg.schur(A)` for a random 4×4 matrix. Verify: (a) $\mathbf{QTQ}^T = \mathbf{A}$, (b) $\mathbf{Q}$ is orthogonal, (c) the diagonal of $\mathbf{T}$ matches `np.linalg.eigvals(A)`.

3. For the non-diagonalizable matrix $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 0 & 2 \end{pmatrix}$, find its Schur decomposition. How does it differ from the (impossible) diagonalization?

4. Implement one step of the basic QR iteration: compute $\mathbf{A}_1 = \mathbf{R}_0\mathbf{Q}_0$ where $\mathbf{A}_0 = \mathbf{Q}_0\mathbf{R}_0$ is the QR factorization. Apply 50 steps to a random 3×3 symmetric matrix and verify that the off-diagonal entries converge to zero.

4.2 Singular Value Decomposition and the Four Fundamental Subspaces

The SVD is often regarded as the culmination of linear algebra. It reveals that any matrix — regardless of shape or rank — can be decomposed into two orthogonal transformations and a diagonal scaling. This structure simultaneously provides orthonormal bases for all four fundamental subspaces, the optimal low-rank approximation, and the numerically stable pseudoinverse.

Thm 4.11 **(Singular Value Decomposition (SVD))** For any $\mathbf{A} \in \mathbb{R}^{m\times n}$, there exist orthogonal matrices $\mathbf{U} \in \mathbb{R}^{m\times m}$ and $\mathbf{V} \in \mathbb{R}^{n\times n}$ and a rectangular diagonal matrix $\mathbf{\Sigma} \in \mathbb{R}^{m\times n}$ with diagonal entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{\min(m,n)} \geq 0$ such that

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T.$$

The values σ_i are the *singular values* of $\mathbf{A}$. The columns of $\mathbf{U}$ and $\mathbf{V}$ are the *left* and *right singular vectors*.

Proof *Proof.* We sketch the constructive proof. The matrix $\mathbf{A}^T\mathbf{A} \in \mathbb{R}^{n\times n}$ is symmetric positive semidefinite (since $\boldsymbol{x}^T\mathbf{A}^T\mathbf{A}\boldsymbol{x} = \|\mathbf{A}\boldsymbol{x}\|_2^2 \geq 0$). By the spectral theorem, there exists an orthonormal eigenbasis: $\mathbf{A}^T\mathbf{A}\boldsymbol{v}_i = \lambda_i\boldsymbol{v}_i$ with $\lambda_1 \geq \cdots \geq \lambda_n \geq 0$. Set $\sigma_i = \sqrt{\lambda_i}$ and, for $\sigma_i > 0$, define $\boldsymbol{u}_i = \mathbf{A}\boldsymbol{v}_i/\sigma_i$. Then $\{\boldsymbol{u}_i\}$ are orthonormal (since $\boldsymbol{u}_i^T\boldsymbol{u}_j = \boldsymbol{v}_i^T\mathbf{A}^T\mathbf{A}\boldsymbol{v}_j/(\sigma_i\sigma_j) = \lambda_j\delta_{ij}/(\sigma_i\sigma_j) = \delta_{ij}$). Extending $\{\boldsymbol{u}_i\}$ to an orthonormal basis of $\mathbb{R}^m$ gives $\mathbf{U}$. One verifies $\mathbf{A}\mathbf{V} = \mathbf{U}\mathbf{\Sigma}$ column-by-column, hence $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$. □

Note 4.8 Geometrically, $\mathbf{A}$ acts as: (1) rotate/reflect the input space via $\mathbf{V}^T$; (2) scale along the coordinate axes (and possibly embed into a higher or lower dimensional space) via $\mathbf{\Sigma}$; (3) rotate/reflect the output space via $\mathbf{U}$. The outer product expansion $\mathbf{A} = \sum_{i=1}^r \sigma_i\boldsymbol{u}_i\boldsymbol{v}_i^T$ expresses $\mathbf{A}$ as a sum of rank-1 matrices, each capturing one "mode" of the transformation.

4.2.1 The Four Fundamental Subspaces

Def 4.13 **(Column Space (Range))** The *column space* of $\mathbf{A} \in \mathbb{R}^{m\times n}$ is $\mathcal{R}(\mathbf{A}) = \{\mathbf{A}\boldsymbol{x} : \boldsymbol{x} \in \mathbb{R}^n\} \subseteq \mathbb{R}^m$. It is spanned by the columns of $\mathbf{A}$ and has dimension equal to the rank $r = \operatorname{rank}(\mathbf{A})$.

Def 4.14 **(Null Space (Kernel))** The *null space* of $\mathbf{A}$ is $\mathcal{N}(\mathbf{A}) = \{\boldsymbol{x} \in \mathbb{R}^n : \mathbf{A}\boldsymbol{x} = \mathbf{0}\}$. It consists of all inputs mapped to zero and has dimension $n - r$ (the *nullity* of $\mathbf{A}$).

Def 4.15 **(Row Space)** The *row space* of $\mathbf{A}$ is $\mathcal{R}(\mathbf{A}^T) = \{\mathbf{A}^T\boldsymbol{y} : \boldsymbol{y} \in \mathbb{R}^m\} \subseteq \mathbb{R}^n$, spanned by the rows of $\mathbf{A}$. It has dimension r and is the orthogonal complement of $\mathcal{N}(\mathbf{A})$ in $\mathbb{R}^n$.

Def 4.16 **(Left Null Space)** The *left null space* of $\mathbf{A}$ is $\mathcal{N}(\mathbf{A}^T) = \{\boldsymbol{y} \in \mathbb{R}^m : \mathbf{A}^T\boldsymbol{y} = \mathbf{0}\}$. It has dimension $m - r$ and is the orthogonal complement of $\mathcal{R}(\mathbf{A})$ in $\mathbb{R}^m$.

Thm 4.12 **(Rank-Nullity via SVD)** $\operatorname{rank}(\mathbf{A}) + \dim \mathcal{N}(\mathbf{A}) = n$ and $\operatorname{rank}(\mathbf{A}) + \dim \mathcal{N}(\mathbf{A}^T) = m$.

Proof *Proof.* From the SVD $\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T$, the number of positive singular values equals $r = \operatorname{rank}(\mathbf{A})$. The right singular vectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_r$ (corresponding to $\sigma_i > 0$) span $\mathcal{R}(\mathbf{A}^T)$, while $\boldsymbol{v}_{r+1}, \ldots, \boldsymbol{v}_n$ span $\mathcal{N}(\mathbf{A})$ (since $\mathbf{A}\boldsymbol{v}_i = \sigma_i \boldsymbol{u}_i = \mathbf{0}$ for $i > r$). Since $\{\boldsymbol{v}_i\}$ is an orthonormal basis of $\mathbb{R}^n$, we have $r + (n-r) = n$. The second identity follows symmetrically from the left singular vectors. □

Thm 4.13 **(SVD Provides Orthonormal Bases for All Four Subspaces)** In the SVD $\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T$ with $r = \operatorname{rank}(\mathbf{A})$: the first r columns of $\mathbf{U}$ span $\mathcal{R}(\mathbf{A})$; the last $m - r$ columns of $\mathbf{U}$ span $\mathcal{N}(\mathbf{A}^T)$; the first r columns of $\mathbf{V}$ span $\mathcal{R}(\mathbf{A}^T)$; the last $n - r$ columns of $\mathbf{V}$ span $\mathcal{N}(\mathbf{A})$.

Proof *Proof.* For any $\boldsymbol{x}$, $\mathbf{A}\boldsymbol{x} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T\boldsymbol{x} = \sum_{i=1}^{r} \sigma_i(\boldsymbol{v}_i^T\boldsymbol{x})\boldsymbol{u}_i$, which is a linear combination of $\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r$. Conversely, $\mathbf{A}\boldsymbol{v}_i = \sigma_i \boldsymbol{u}_i$ for $i \le r$, so each $\boldsymbol{u}_i$ ($i \le r$) is in $\mathcal{R}(\mathbf{A})$. Hence $\mathcal{R}(\mathbf{A}) = \operatorname{span}\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r\}$. Since $\mathcal{N}(\mathbf{A}^T)$ is the orthogonal complement of $\mathcal{R}(\mathbf{A})$ in $\mathbb{R}^m$ and $\boldsymbol{u}_{r+1}, \ldots, \boldsymbol{u}_m$ are orthogonal to $\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r$, they span $\mathcal{N}(\mathbf{A}^T)$. The arguments for $\mathcal{R}(\mathbf{A}^T)$ and $\mathcal{N}(\mathbf{A})$ are symmetric. □

Prop 4.14 **(Existence and Uniqueness of the Least Squares Solution)** The system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ has an exact solution if and only if $\boldsymbol{b} \in \mathcal{R}(\mathbf{A})$. When it does, the general solution is any particular solution $\hat{\boldsymbol{x}}$ plus an arbitrary vector from $\mathcal{N}(\mathbf{A})$. The *minimum-norm* least squares solution is $\hat{\boldsymbol{x}} = \mathbf{A}^+\boldsymbol{b} = \mathbf{V}\boldsymbol{\Sigma}^+\mathbf{U}^T\boldsymbol{b}$, where $\boldsymbol{\Sigma}^+$ has $1/\sigma_i$ for each positive singular value.

Proof *Proof.* $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is solvable iff $\boldsymbol{b} \perp \mathcal{N}(\mathbf{A}^T)$, i.e., $\boldsymbol{b} \in \mathcal{R}(\mathbf{A})$ (since the two are orthogonal complements). The general solution is $\boldsymbol{x} = \mathbf{V}\boldsymbol{\Sigma}^+\mathbf{U}^T\boldsymbol{b} + \mathbf{P}_{\mathcal{N}(\mathbf{A})}\boldsymbol{w}$ for any $\boldsymbol{w}$, where the second term lies in $\mathcal{N}(\mathbf{A})$ and contributes zero to $\mathbf{A}\boldsymbol{x}$ but increases $\|\boldsymbol{x}\|_2$. The minimum-norm solution has zero $\mathcal{N}(\mathbf{A})$ component, giving $\hat{\boldsymbol{x}} = \mathbf{A}^+\boldsymbol{b}$. □

4.2.2 Rank Truncation and the Eckart-Young-Mirsky Theorem

Def 4.17 **(Rank-r Approximation)** The *rank-r truncated SVD* of $\mathbf{A}$ is

$$\mathbf{A}_r = \sum_{i=1}^{r} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T = \mathbf{U}_r \boldsymbol{\Sigma}_r \mathbf{V}_r^T,$$

where $\mathbf{U}_r, \mathbf{V}_r$ are the first r columns of $\mathbf{U}, \mathbf{V}$ and $\boldsymbol{\Sigma}_r \in \mathbb{R}^{r \times r}$ contains the r largest singular values.

Thm 4.15 **(Eckart-Young-Mirsky Theorem)** Among all matrices of rank at most r, $\mathbf{A}_r$ is the closest to $\mathbf{A}$ in both the spectral norm and the Frobenius norm:

$$\|\mathbf{A} - \mathbf{A}_r\|_2 = \sigma_{r+1}, \qquad \|\mathbf{A} - \mathbf{A}_r\|_F = \sqrt{\sum_{i=r+1}^{p} \sigma_i^2},$$

where $p = \min(m, n)$. For any $\mathbf{B}$ with $\operatorname{rank}(\mathbf{B}) \le r$: $\|\mathbf{A} - \mathbf{B}\|_2 \ge \sigma_{r+1}$ and $\|\mathbf{A} - \mathbf{B}\|_F \ge \|\mathbf{A} - \mathbf{A}_r\|_F$.

Proof *Proof.* The error formula follows immediately: $\mathbf{A} - \mathbf{A}_r = \sum_{i=r+1}^{p} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^T$ is itself an SVD, so $\|\mathbf{A} - \mathbf{A}_r\|_2 = \sigma_{r+1}$ and $\|\mathbf{A} - \mathbf{A}_r\|_F = \sqrt{\sum_{i>r} \sigma_i^2}$.
For the lower bound: suppose $\operatorname{rank}(\mathbf{B}) \le r$, so $\dim \mathcal{N}(\mathbf{B}) \ge n - r$. The subspace $\mathcal{N}(\mathbf{B}) \cap \operatorname{span}\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_{r+1}\}$ has dimension at least $(n-r)+(r+1)-n = 1$ by dimension counting, so there exists a unit vector $\boldsymbol{z}$ in this intersection. Then

$$\|\mathbf{A} - \mathbf{B}\|_2 \ge \|(\mathbf{A} - \mathbf{B})\boldsymbol{z}\|_2 = \|\mathbf{A}\boldsymbol{z}\|_2 = \left\| \sum_{i=1}^{r+1} \sigma_i (\boldsymbol{v}_i^T \boldsymbol{z}) \boldsymbol{v}_i \right\|_2 \ge \sigma_{r+1} \|\boldsymbol{z}\|_2 = \sigma_{r+1}.$$

The Frobenius norm lower bound follows similarly using the Frobenius norm version of the argument. □

Thm 4.16 **(SVD-Eigendecomposition Relationship)** The singular values and vectors of $\mathbf{A}$ satisfy: the right singular vectors $\boldsymbol{v}_i$ are eigenvectors of $\mathbf{A}^T\mathbf{A}$ with eigenvalues σ_i^2; the left singular vectors $\boldsymbol{u}_i$ are eigenvectors of $\mathbf{A}\mathbf{A}^T$ with eigenvalues σ_i^2.

Proof *Proof.* From $\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T$: $\mathbf{A}^T\mathbf{A} = \mathbf{V}\boldsymbol{\Sigma}^T\mathbf{U}^T\mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T = \mathbf{V}\boldsymbol{\Sigma}^T\boldsymbol{\Sigma}\mathbf{V}^T$. Since $\boldsymbol{\Sigma}^T\boldsymbol{\Sigma} = \operatorname{diag}(\sigma_1^2, \ldots, \sigma_n^2)$ and $\mathbf{V}$ is orthogonal, this is the eigendecomposition $\mathbf{A}^T\mathbf{A}\boldsymbol{v}_i = \sigma_i^2 \boldsymbol{v}_i$. Symmetrically, $\mathbf{A}\mathbf{A}^T = \mathbf{U}\boldsymbol{\Sigma}\boldsymbol{\Sigma}^T\mathbf{U}^T$ gives $\mathbf{A}\mathbf{A}^T\boldsymbol{u}_i = \sigma_i^2 \boldsymbol{u}_i$. □

Def 4.18 **(Pseudoinverse via SVD)** The *Moore-Penrose pseudoinverse* of $\mathbf{A} \in \mathbb{R}^{m \times n}$ is

$$\mathbf{A}^+ = \mathbf{V}\boldsymbol{\Sigma}^+\mathbf{U}^T, \quad \text{where } (\boldsymbol{\Sigma}^+)_{ii} = \begin{cases} 1/\sigma_i & \text{if } \sigma_i > 0 \\ 0 & \text{otherwise.} \end{cases}$$

For full column rank, $\mathbf{A}^+ = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T$. For full row rank, $\mathbf{A}^+ = \mathbf{A}^T(\mathbf{A}\mathbf{A}^T)^{-1}$.

Def 4.19 **(SVD-Based Regularization)** For an ill-conditioned system, the truncated SVD solution

$$\hat{\boldsymbol{x}}_k = \sum_{i=1}^{k} \frac{\boldsymbol{u}_i^T \boldsymbol{b}}{\sigma_i} \boldsymbol{v}_i$$

ignores directions corresponding to small singular values, reducing sensitivity to perturbations. The effective condition number of the regularized system is $\sigma_1/\sigma_k \ll \sigma_1/\sigma_n$.

Ex 4.15 Let $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 2 \end{pmatrix}$.

1. Compute the SVD of $\mathbf{A}$ by hand: find the eigenvalues of $\mathbf{A}^T\mathbf{A}$, then the singular values, right singular vectors (eigenvectors of $\mathbf{A}^T\mathbf{A}$), and left singular vectors ($\boldsymbol{u}_i = \mathbf{A}\boldsymbol{v}_i/\sigma_i$).

2. Verify that $\mathbf{U}$, $\boldsymbol{\Sigma}$, $\mathbf{V}$ recover $\mathbf{A}$ via $\mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^T$.

3. What are the four fundamental subspaces of $\mathbf{A}$? (Since $\mathbf{A}$ is 2×2 and invertible, what are their dimensions?)

Ex 4.16 **(Image compression via truncated SVD)** Load a grayscale image (e.g., `from skimage import data; A = data.camera().astype(float)`) and compute its SVD using `np.linalg.svd(A, full_matrices=False)`.

1. Plot the singular values on a log scale. Where do they start to level off?
2. Compute the rank-r approximation $\mathbf{A}_r$ for $r = 5, 20, 50, 100$. Display the images.
3. For each r, compute the storage savings ($(m + n + 1) \times r$ vs. mn entries) and the relative Frobenius error $\|\mathbf{A} - \mathbf{A}_r\|_F / \|\mathbf{A}\|_F$. Plot error vs. compression ratio.

Ex 4.17 **(Four fundamental subspaces)** Let $\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$.

1. Use `np.linalg.svd` to find the singular values. How many are zero (up to numerical precision)? What is $\text{rank}(\mathbf{A})$?
2. Extract bases for all four fundamental subspaces from the SVD. Verify that $\mathcal{R}(\mathbf{A}) \perp \mathcal{N}(\mathbf{A}^T)$ and $\mathcal{R}(\mathbf{A}^T) \perp \mathcal{N}(\mathbf{A})$.
3. For $\boldsymbol{b} = (1, 1, 1)^T$, determine whether the system $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ has a solution. If not, compute the least squares solution using the pseudoinverse `np.linalg.pinv(A) @ b`.

5 Iterative Methods for Linear Systems

5.1 Iterative Methods for Linear Systems

So far we have examined direct methods for solving $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$: computing the answer in a fixed, predictable number of operations. For large sparse systems, however, direct methods become impractical — even when $\mathbf{A}$ is sparse, its LU factors may be dense (*fill-in*), destroying the memory advantage. Iterative methods offer an alternative: refine an initial guess through successive approximations, exploiting sparsity at every step.

Def 5.1 **(Direct Inverse Method)** The analytical formula $\boldsymbol{x} = \mathbf{A}^{-1}\boldsymbol{b}$ requires explicitly forming $\mathbf{A}^{-1}$, which costs $O(n^3)$ flops and can suffer from severe numerical instability when $\mathbf{A}$ is nearly singular.

Def 5.2 **(Matrix Factorization Methods)** Methods such as LU, Cholesky, and QR decompose $\mathbf{A}$ into simpler factors at a one-time cost of $O(n^3)$. Each subsequent solve with a new right-hand side then requires only $O(n^2)$ operations via triangular solves.

Note 5.1 Factorization methods are effective for small to medium-sized problems, but become impractical for very large sparse systems: even when $\mathbf{A}$ is sparse, the factors $\mathbf{L}$ and $\mathbf{U}$ may be dense (fill-in), destroying the memory advantage. For example, the $n^2 \times n^2$ matrix arising from a 2D finite difference discretization on an $n \times n$ grid has $O(n^2)$ nonzeros, but its LU factors have $O(n^3)$ nonzeros.

Def 5.3 **(Stationary Iterative Scheme)** A *stationary iterative method* for $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ takes the form

$$\boldsymbol{x}^{(k+1)} = \mathbf{T}\boldsymbol{x}^{(k)} + \boldsymbol{c}, \quad k = 0, 1, 2, \ldots$$

for a fixed *iteration matrix* $\mathbf{T} \in \mathbb{R}^{n\times n}$ and a fixed vector $\boldsymbol{c} \in \mathbb{R}^n$ derived from $\mathbf{A}$ and $\boldsymbol{b}$. Starting from an initial guess $\boldsymbol{x}^{(0)}$, the method generates a sequence $\{\boldsymbol{x}^{(k)}\}$.

Note 5.2 The iteration is consistent with $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ if every fixed point $\boldsymbol{x}^* = \mathbf{T}\boldsymbol{x}^* + \boldsymbol{c}$ satisfies $\mathbf{A}\boldsymbol{x}^* = \boldsymbol{b}$. Rearranging $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ as $\boldsymbol{x} = (\mathbf{I} - \mathbf{M}^{-1}\mathbf{A})\boldsymbol{x} + \mathbf{M}^{-1}\boldsymbol{b}$ for some invertible *splitting matrix* $\mathbf{M}$ yields $\mathbf{T} = \mathbf{I} - \mathbf{M}^{-1}\mathbf{A}$ and $\boldsymbol{c} = \mathbf{M}^{-1}\boldsymbol{b}$. The art lies in choosing $\mathbf{M}$ so that $\mathbf{T}$ has small spectral radius and $\mathbf{M}^{-1}$ is cheap to apply.

Def 5.4 **(Spectral Radius)** The *spectral radius* of a matrix $\mathbf{B} \in \mathbb{R}^{n\times n}$ is

$$\rho(\mathbf{B}) = \max\{|\lambda| : \lambda \in \sigma(\mathbf{B})\},$$

the largest absolute value among all eigenvalues of $\mathbf{B}$.

Thm 5.1 **(Gelfand's Formula)** For any matrix $\mathbf{B} \in \mathbb{R}^{n\times n}$ and any submultiplicative matrix norm $\|\cdot\|$,

$$\rho(\mathbf{B}) = \lim_{k\to\infty} \|\mathbf{B}^k\|^{1/k}.$$

In particular, for every $\epsilon > 0$ there exists $C_\epsilon > 0$ such that $\|\mathbf{B}^k\| \le C_\epsilon(\rho(\mathbf{B}) + \epsilon)^k$ for all $k \ge 0$.

Proof *Proof. Lower bound*: For any eigenvalue λ of $\mathbf{B}$ with eigenvector $\boldsymbol{v}$, $\mathbf{B}^k\boldsymbol{v} = \lambda^k\boldsymbol{v}$, so $\|\mathbf{B}^k\|\|\boldsymbol{v}\| \ge \|\mathbf{B}^k\boldsymbol{v}\| = |\lambda|^k\|\boldsymbol{v}\|$. Hence $\|\mathbf{B}^k\|^{1/k} \ge |\lambda|$ for all k, giving $\liminf_k \|\mathbf{B}^k\|^{1/k} \ge \rho(\mathbf{B})$.
Upper bound: Fix $\epsilon > 0$. The Jordan decomposition gives $\mathbf{B} = \mathbf{V}\mathbf{J}_0\mathbf{V}^{-1}$ where $\mathbf{J}_0$ is block upper triangular with the eigenvalues of $\mathbf{B}$ on the diagonal. The entries of $\mathbf{J}_0^k$ grow at most as $k^{n-1}|\lambda_{\max}|^k$ (from Jordan block structure). Since $k^{n-1} \le C_\epsilon(1+\epsilon/\rho(\mathbf{B}))^k$ for large k, we get $\|\mathbf{B}^k\| \le C_\epsilon(\rho(\mathbf{B}) + \epsilon)^k$, so $\limsup_k \|\mathbf{B}^k\|^{1/k} \le \rho(\mathbf{B}) + \epsilon$. Since ϵ was arbitrary, the limit exists and equals $\rho(\mathbf{B})$. □

Thm 5.2 **(Convergence of Stationary Iteration)** The stationary iteration $\boldsymbol{x}^{(k+1)} = \mathbf{T}\boldsymbol{x}^{(k)} + \boldsymbol{c}$ converges to the fixed point $\boldsymbol{x}^* = \mathbf{T}\boldsymbol{x}^* + \boldsymbol{c}$ for *every* initial vector $\boldsymbol{x}^{(0)}$ if and only if $\rho(\mathbf{T}) < 1$.

Proof *Proof.* Let $\boldsymbol{e}^{(k)} = \boldsymbol{x}^{(k)} - \boldsymbol{x}^*$. Subtracting the fixed-point equation from the iteration gives $\boldsymbol{e}^{(k+1)} = \mathbf{T}\boldsymbol{e}^{(k)}$, so $\boldsymbol{e}^{(k)} = \mathbf{T}^k\boldsymbol{e}^{(0)}$.

($\Leftarrow$) If $\rho(\mathbf{T}) < 1$: Choose $\epsilon > 0$ small enough that $\rho(\mathbf{T}) + \epsilon < 1$. By Gelfand's formula, $\lim_{k\to\infty} \|\mathbf{T}^k\|^{1/k} = \rho(\mathbf{T})$, so $\|\mathbf{T}^k\| \le C(\rho(\mathbf{T}) + \epsilon)^k$ for some constant $C > 0$. Hence $\|\boldsymbol{e}^{(k)}\| \le C(\rho(\mathbf{T}) + \epsilon)^k \|\boldsymbol{e}^{(0)}\| \to 0$ for all $\boldsymbol{x}^{(0)}$.
($\Rightarrow$) If $\rho(\mathbf{T}) \ge 1$: There exists an eigenvalue λ with $|\lambda| \ge 1$ and eigenvector $\boldsymbol{v} \neq \mathbf{0}$. Take $\boldsymbol{x}^{(0)} = \boldsymbol{x}^* + \boldsymbol{v}$; then $\boldsymbol{e}^{(k)} = \lambda^k \boldsymbol{v}$, so $\|\boldsymbol{e}^{(k)}\| = |\lambda|^k \|\boldsymbol{v}\| \not\to 0$. □

Cor 5.3 **(Geometric Convergence Rate)** When $\rho(\mathbf{T}) < 1$, the error satisfies $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\| = O(\rho(\mathbf{T})^k)$. To reduce the error by a factor of 10^{-p}, approximately $p/\log_{10}(1/\rho(\mathbf{T}))$ iterations are required.

Proof *Proof.* From Gelfand's formula, $\|\mathbf{T}^k\| \le C\rho(\mathbf{T})^k$ (up to a constant). Solving $C\rho(\mathbf{T})^k \le 10^{-p}$ for k and taking logarithms gives $k \ge (\log C + p)/\log_{10}(1/\rho(\mathbf{T}))$, which is $O(p/\log_{10}(1/\rho(\mathbf{T})))$. □

Ex 5.1 Let $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$, $\boldsymbol{b} = \begin{pmatrix} 4 \\ 4 \end{pmatrix}$ (exact solution $\boldsymbol{x}^* = (1,1)^T$).

1. Split $\mathbf{A} = \mathbf{D} + (\mathbf{A} - \mathbf{D})$ where $\mathbf{D} = \mathrm{diag}(3,3)$. Write the stationary iteration in the form $\boldsymbol{x}^{(k+1)} = \mathbf{T}\boldsymbol{x}^{(k)} + \boldsymbol{c}$, stating $\mathbf{T}$ and $\boldsymbol{c}$ explicitly.
2. Compute $\rho(\mathbf{T})$ and predict the number of iterations needed to reduce the initial error by 10^{-6}.
3. Starting from $\boldsymbol{x}^{(0)} = (0,0)^T$, carry out three iterations by hand and verify that the error decreases at the rate predicted by $\rho(\mathbf{T})$.

Ex 5.2 **(Fill-in phenomenon)** Let $\mathbf{A}$ be the $n \times n$ tridiagonal matrix with 2 on the main diagonal and -1 on the super- and sub-diagonals (arising from finite differences for $-u'' = f$ on $[0,1]$).

1. How many nonzeros does $\mathbf{A}$ have? Is it $O(n)$ or $O(n^2)$?
2. Use `scipy.linalg.lu` to compute the LU factorization for $n = 20$. Count the nonzeros in $\mathbf{L}$ and $\mathbf{U}$. Is there fill-in? Explain why or why not.
3. Use `scipy.sparse.linalg.spsolve` to solve $\mathbf{A}\boldsymbol{x} = \mathbf{1}$ in sparse format. Compare timing against dense LU for $n = 1000$.

5.2 Jacobi Iteration

5.2.1 Jacobi Iteration

Def 5.5 **(Splitting of A)** To derive a stationary iteration from $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$, we decompose $\mathbf{A}$ as

$$\mathbf{A} = \mathbf{D} + \mathbf{R}, \qquad \mathbf{D} = \mathrm{diag}(a_{11}, \ldots, a_{nn}), \quad \mathbf{R} = \mathbf{A} - \mathbf{D}.$$

Assuming all diagonal entries are nonzero ($a_{ii} \neq 0$ for all i), we can rearrange $(\mathbf{D}+\mathbf{R})\boldsymbol{x} = \boldsymbol{b}$ to obtain the fixed-point equation $\boldsymbol{x} = -\mathbf{D}^{-1}\mathbf{R}\boldsymbol{x} + \mathbf{D}^{-1}\boldsymbol{b}$.

Def 5.6 **(Jacobi Iteration)** The *Jacobi iteration* for $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is the stationary scheme

$$\boldsymbol{x}^{(k+1)} = \mathbf{T}_J \boldsymbol{x}^{(k)} + \mathbf{D}^{-1}\boldsymbol{b}, \qquad \mathbf{T}_J = -\mathbf{D}^{-1}\mathbf{R},$$

or in component form,

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\Big(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)}\Big), \quad i = 1, \ldots, n.$$

Each component $x_i^{(k+1)}$ depends only on values from the *previous* iterate $\boldsymbol{x}^{(k)}$, so all n components may be updated simultaneously (or in any order).

5.2.2 Convergence of Jacobi Iteration

Thm 5.4 **(Convergence of Jacobi Iteration)** The Jacobi iteration converges to the solution of $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ for every initial vector $\boldsymbol{x}^{(0)}$ if and only if $\rho(\mathbf{T}_J) < 1$.

Proof *Proof.* This is a direct application of the general convergence theorem for stationary iterations: convergence for all $\boldsymbol{x}^{(0)}$ is equivalent to $\rho(\mathbf{T}) < 1$ for the iteration matrix. Here $\mathbf{T}_J = -\mathbf{D}^{-1}\mathbf{R} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{A}$, so the fixed-point equation $\boldsymbol{x}^* = \mathbf{T}_J\boldsymbol{x}^* + \mathbf{D}^{-1}\boldsymbol{b}$ simplifies to $\mathbf{D}^{-1}\mathbf{A}\boldsymbol{x}^* = \mathbf{D}^{-1}\boldsymbol{b}$, i.e., $\mathbf{A}\boldsymbol{x}^* = \boldsymbol{b}$. □

Thm 5.5 **(Jacobi Convergence for Strictly Diagonally Dominant Matrices)** If $\mathbf{A}$ is strictly diagonally dominant, meaning

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad \text{for all } i = 1, \ldots, n,$$

then $\rho(\mathbf{T}_J) < 1$ and the Jacobi iteration converges for every $\boldsymbol{x}^{(0)}$.

Proof *Proof.* Let λ be any eigenvalue of $\mathbf{T}_J = -\mathbf{D}^{-1}\mathbf{R}$ with eigenvector $\boldsymbol{v} \neq \mathbf{0}$. Let index i^* satisfy $|v_{i^*}| = \|\boldsymbol{v}\|_\infty > 0$. The eigenvalue equation $\mathbf{T}_J\boldsymbol{v} = \lambda\boldsymbol{v}$ gives, for row i^*:

$$\lambda v_{i^*} = -\frac{1}{a_{i^*i^*}} \sum_{j \neq i^*} a_{i^*j}\, v_j.$$

Taking absolute values and dividing by $|v_{i^*}|$:

$$|\lambda| \leq \frac{1}{|a_{i^*i^*}|} \sum_{j \neq i^*} |a_{i^*j}| \cdot \frac{|v_j|}{|v_{i^*}|} \leq \frac{\sum_{j \neq i^*} |a_{i^*j}|}{|a_{i^*i^*}|} < 1,$$

where the last inequality is strict diagonal dominance. Since λ was arbitrary, $\rho(\mathbf{T}_J) < 1$. □

Example **Effect of matrix structure on convergence.**

Case 1 — Strictly diagonally dominant: $\mathbf{A}_1 = \begin{pmatrix} 5 & 1 & 1 \\ 1 & 5 & 1 \\ 1 & 1 & 5 \end{pmatrix}$. Each row satisfies $|a_{ii}| = 5 > 2 = \sum_{j \neq i} |a_{ij}|$, so the theorem guarantees convergence. One can compute $\rho(\mathbf{T}_J) = 2/5 = 0.4$.

Case 2 — Diagonal dominance violated in one row: $\mathbf{A}_2 = \begin{pmatrix} 2 & 1 & 3 \\ 1 & 3 & 1 \\ 2 & 2 & 2 \end{pmatrix}$. Row 3 violates strict diagonal dominance. The theorem gives no guarantee; $\rho(\mathbf{T}_J)$ must be checked numerically.

Case 3 — Divergent case: $\mathbf{A}_3 = \begin{pmatrix} 1 & 3 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}$. Here $\rho(\mathbf{T}_J) > 1$ and the Jacobi iteration diverges for every $\boldsymbol{x}^{(0)} \neq \boldsymbol{x}^*$.

5.2.3 Stopping Criteria

Def 5.7 **(Residual-Based Stopping Criterion)** Stop when the norm of the residual falls below a tolerance:

$$\|\mathbf{A}\boldsymbol{x}^{(k)} - \boldsymbol{b}\| < \epsilon_{\text{res}}.$$

This directly measures how well $\boldsymbol{x}^{(k)}$ satisfies the linear system.

Def 5.8 **(Increment-Based Stopping Criterion)** Stop when the change between successive iterates is small:

$$\|\boldsymbol{x}^{(k+1)} - \boldsymbol{x}^{(k)}\| < \epsilon_{\text{inc}}.$$

This is cheap (no matrix–vector product needed), but measures the rate of change of the iteration rather than the accuracy of the solution directly.

Def 5.9 **(Relative Residual Criterion)** Stop when the residual is small relative to the right-hand side:

$$\frac{\|\mathbf{A}\boldsymbol{x}^{(k)} - \boldsymbol{b}\|}{\|\boldsymbol{b}\|} < \epsilon_{\text{rel}}.$$

This is scale-invariant: multiplying $\mathbf{A}$ and $\boldsymbol{b}$ by a constant does not alter the stopping condition.

Note 5.3 A *maximum iteration count* is always imposed as a safeguard against slow convergence or divergence. Typical practice is to set $k_{\max}$ based on the problem size and report non-convergence if the tolerance is not met within the budget.

Thm 5.6 **(A Priori Error Bound for Jacobi Iteration)** Let $\rho = \rho(\mathbf{T}_J) < 1$. For all $k \geq 1$,

$$\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\| \leq \frac{\rho^k}{1-\rho}\|\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}\|.$$

Proof *Proof.* The basic contraction estimate gives $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\| \leq \rho^k\|\boldsymbol{x}^{(0)} - \boldsymbol{x}^*\|$. We bound $\|\boldsymbol{x}^{(0)} - \boldsymbol{x}^*\|$ via the telescoping sum:

$$\|\boldsymbol{x}^{(0)} - \boldsymbol{x}^*\| \leq \sum_{j=0}^{\infty}\|\boldsymbol{x}^{(j+1)} - \boldsymbol{x}^{(j)}\|.$$

Since $\|\boldsymbol{x}^{(j+1)} - \boldsymbol{x}^{(j)}\| = \|\mathbf{T}_J^j(\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)})\| \leq \rho^j\|\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}\|$, the geometric series gives $\|\boldsymbol{x}^{(0)} - \boldsymbol{x}^*\| \leq \|\boldsymbol{x}^{(1)} - \boldsymbol{x}^{(0)}\|/(1-\rho)$. Combining yields the stated bound. □

Ex 5.3 Implement the Jacobi method in Python.

1. Write `jacobi(A, b, x0, tol, max_iter)` that returns the iterate at convergence (relative residual below `tol`) and the number of iterations used.

2. Test on $\mathbf{A}_1$ from the example with $\boldsymbol{b} = (7, 7, 7)^T$. Compute the exact solution by hand. How many iterations are needed for $\epsilon_{\text{rel}} = 10^{-8}$?

3. Plot $\|\mathbf{A}\boldsymbol{x}^{(k)} - \boldsymbol{b}\|$ versus k on a semi-log scale. What is the observed slope? Compare to $\log_{10} \rho(\mathbf{T}_J)$.

Ex 5.4 **(A priori vs. observed convergence)** For the system with $\mathbf{A}_1$ and $\boldsymbol{b} = (7, 7, 7)^T$:

1. Evaluate the a priori bound for $k = 20$ iterations. How many times larger is the bound than the actual error $\|\boldsymbol{x}^{(20)} - \boldsymbol{x}^*\|$?

2. Consider the 5×5 tridiagonal matrix with 4 on the diagonal and -1 on the off-diagonals. Verify that it is strictly diagonally dominant, compute $\rho(\mathbf{T}_J)$ numerically, and determine how many iterations are needed to achieve $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\| < 10^{-10}$.

5.2.4 Gauss-Seidel Iteration

Def 5.10 **(Splitting of A for Gauss-Seidel)** Decompose $\mathbf{A} = \mathbf{L}_* + \mathbf{U}_*$, where $\mathbf{L}_*$ is the lower triangular part (including the diagonal) and $\mathbf{U}_*$ is the strictly upper triangular part:

$$\mathbf{A} = \mathbf{L}_* + \mathbf{U}_*, \qquad (\mathbf{L}_*)_{ij} = \begin{cases} a_{ij} & j \le i \\ 0 & j > i \end{cases}, \quad (\mathbf{U}_*)_{ij} = \begin{cases} a_{ij} & j > i \\ 0 & j \le i \end{cases}.$$

Rearranging $(\mathbf{L}_* + \mathbf{U}_*)\boldsymbol{x} = \boldsymbol{b}$ as $\mathbf{L}_*\boldsymbol{x} = \boldsymbol{b} - \mathbf{U}_*\boldsymbol{x}$ yields the Gauss-Seidel fixed-point equation.

Def 5.11 **(Gauss-Seidel Iteration)** The *Gauss-Seidel iteration* for $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is

$$\boldsymbol{x}^{(k+1)} = \mathbf{T}_{GS}\boldsymbol{x}^{(k)} + \mathbf{L}_*^{-1}\boldsymbol{b}, \qquad \mathbf{T}_{GS} = -\mathbf{L}_*^{-1}\mathbf{U}_*.$$

In component form, updating in order $i = 1, \ldots, n$:

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\Big(b_i - \sum_{j<i} a_{ij}\, x_j^{(k+1)} - \sum_{j>i} a_{ij}\, x_j^{(k)}\Big).$$

The key distinction from Jacobi: components $x_1^{(k+1)}, \ldots, x_{i-1}^{(k+1)}$ already computed in the current sweep are used *immediately*.

Note 5.4 Because updated values are used as soon as they are available, Gauss-Seidel typically converges roughly twice as fast as Jacobi in practice. However, the sequential dependence among components means updates cannot be parallelized directly, unlike Jacobi.

Thm 5.7 **(Gauss-Seidel Convergence for SPD Matrices)** If $\mathbf{A}$ is symmetric positive definite, then the Gauss-Seidel iteration converges for every initial vector $\boldsymbol{x}^{(0)}$.

Proof *Proof.* We show $\rho(\mathbf{T}_{GS}) < 1$. Write $\mathbf{A} = \mathbf{L}_* + \mathbf{U}_*$. Since $\mathbf{A}$ is symmetric, $\mathbf{U}_* = \mathbf{L}_*^T - \mathbf{D}$. The iteration matrix is $\mathbf{T}_{GS} = -\mathbf{L}_*^{-1}\mathbf{U}_*$. To show all eigenvalues satisfy $|\lambda| < 1$, suppose $\mathbf{T}_{GS}\boldsymbol{v} = \lambda\boldsymbol{v}$ for $\boldsymbol{v} \neq \mathbf{0}$. Then $-\mathbf{U}_*\boldsymbol{v} = \lambda\mathbf{L}_*\boldsymbol{v}$, i.e., $(\mathbf{A} - \mathbf{L}_*)\boldsymbol{v} = -\lambda\mathbf{L}_*\boldsymbol{v}$, giving $\mathbf{A}\boldsymbol{v} = (1-\lambda)\mathbf{L}_*\boldsymbol{v}$. Taking the inner product with $\boldsymbol{v}$ and using $\mathbf{A} \succ 0$:

$$(1-\lambda)\,\boldsymbol{v}^T\mathbf{L}_*\boldsymbol{v} = \boldsymbol{v}^T\mathbf{A}\boldsymbol{v} > 0.$$

Decompose $\boldsymbol{v}^T\mathbf{L}_*\boldsymbol{v} = \boldsymbol{v}^T\mathbf{D}\boldsymbol{v}/2 + \frac{1}{2}\boldsymbol{v}^T\mathbf{A}\boldsymbol{v} > 0$ (both terms positive since $\mathbf{D} \succ 0$ and $\mathbf{A} \succ 0$), so $1 - \lambda > 0$, meaning $\lambda < 1$. A symmetric argument using conjugates shows $|\lambda| < 1$, hence $\rho(\mathbf{T}_{GS}) < 1$. □

Thm 5.8 **(Gauss-Seidel Convergence for Strictly Diagonally Dominant Matrices)** If $\mathbf{A}$ is strictly diagonally dominant, the Gauss-Seidel iteration converges for every $\boldsymbol{x}^{(0)}$.

Proof *Proof.* The proof parallels the Jacobi case. Let $\mathbf{T}_{GS}\boldsymbol{v} = \lambda\boldsymbol{v}$, $\boldsymbol{v} \neq \mathbf{0}$. Pick i^* with $|v_{i^*}| = \|\boldsymbol{v}\|_\infty$. The i^*-th component of $-\mathbf{U}_*\boldsymbol{v} = \lambda\mathbf{L}_*\boldsymbol{v}$ gives

$$\lambda a_{i^*i^*}v_{i^*} = -\sum_{j<i^*} a_{i^*j}(\lambda v_j) - \sum_{j>i^*} a_{i^*j}v_j.$$

Taking absolute values and using $|v_j| \leq |v_{i^*}|$:

$$|\lambda|\,|a_{i^*i^*}| \leq |\lambda| \sum_{j<i^*} |a_{i^*j}| + \sum_{j>i^*} |a_{i^*j}|.$$

If $|\lambda| \geq 1$, then $|\lambda|\,|a_{i^*i^*}| \leq |\lambda| \sum_{j\neq i^*} |a_{i^*j}|$, contradicting strict diagonal dominance. Hence $|\lambda| < 1$. $\square$

Example **Jacobi vs. Gauss-Seidel on a tridiagonal system.**

Consider the 3×3 tridiagonal system with $\mathbf{A} = \begin{pmatrix} 4 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 4 \end{pmatrix}$, $\boldsymbol{b} = (3, 2, 3)^T$ (exact solution $\boldsymbol{x}^* = (1, 1, 1)^T$).

Jacobi step from $\boldsymbol{x}^{(0)} = \mathbf{0}$*:*

$$x_1^{(1)} = \tfrac{3}{4} = 0.75, \quad x_2^{(1)} = \tfrac{2}{4} = 0.5, \quad x_3^{(1)} = \tfrac{3}{4} = 0.75.$$

Gauss-Seidel step from $\boldsymbol{x}^{(0)} = \mathbf{0}$ *(using updated values immediately):*

$$x_1^{(1)} = \tfrac{3}{4} = 0.75, \quad x_2^{(1)} = \tfrac{2+0.75}{4} = \tfrac{2.75}{4} = 0.6875, \quad x_3^{(1)} = \tfrac{3+0.6875}{4} = \tfrac{3.6875}{4} \approx 0.922.$$

Gauss-Seidel is already significantly closer to $\boldsymbol{x}^* = (1, 1, 1)^T$ after a single sweep.

5.2.5 Successive Over-Relaxation (SOR)

Both Jacobi and Gauss-Seidel can converge slowly when $\rho(\mathbf{T})$ is close to 1. The idea behind SOR is simple: take the Gauss-Seidel update as a direction and *overshoot* by a factor $\omega > 1$ to accelerate convergence. Choosing the optimal ω can reduce the spectral radius dramatically, turning $O(n^2)$ iterations into $O(n)$ for certain model problems.

Def 5.12 **(SOR Iteration)** The *Successive Over-Relaxation (SOR)* method introduces a relaxation parameter $\omega > 0$. Let $\tilde{x}_i$ denote the Gauss-Seidel update for component i. SOR blends the new and old values:

$$x_i^{(k+1)} = (1-\omega)\, x_i^{(k)} + \omega\, \tilde{x}_i^{(k+1)},$$

where

$$\tilde{x}_i^{(k+1)} = \frac{1}{a_{ii}}\Big(b_i - \sum_{j<i} a_{ij}\, x_j^{(k+1)} - \sum_{j>i} a_{ij}\, x_j^{(k)}\Big).$$

Substituting, the full SOR update is

$$x_i^{(k+1)} = x_i^{(k)} + \frac{\omega}{a_{ii}}\Big(b_i - \sum_{j<i} a_{ij}\, x_j^{(k+1)} - a_{ii}\, x_i^{(k)} - \sum_{j>i} a_{ij}\, x_j^{(k)}\Big).$$

Special cases: $\omega = 1$ recovers Gauss-Seidel; $\omega \in (1,2)$ is over-relaxation; $\omega \in (0,1)$ is under-relaxation (useful for stabilizing slowly divergent iterations).

Thm 5.9 **(Kahan's Necessary Condition for SOR)** The SOR iteration can converge only if $0 < \omega < 2$.

Proof *Proof.* The iteration matrix for SOR is $\mathbf{T}_\omega = (\mathbf{D}+\omega\mathbf{L})^{-1}((1-\omega)\mathbf{D}-\omega\mathbf{U})$ where $\mathbf{L}, \mathbf{U}$ are the strictly lower/upper triangular parts of $\mathbf{A}$. Its determinant equals $(1-\omega)^n$ (product of the eigenvalues). Since $\rho(\mathbf{T}_\omega) \geq |\det(\mathbf{T}_\omega)|^{1/n} = |1-\omega|$, convergence requires $\rho(\mathbf{T}_\omega) < 1$, which forces $|1-\omega| < 1$, i.e., $0 < \omega < 2$. □

Thm 5.10 **(Optimal SOR for Consistently Ordered Matrices)** For a consistently ordered matrix (e.g., tridiagonal systems arising from discretizing PDEs) with Jacobi spectral radius $\rho_J = \rho(\mathbf{T}_J)$, the optimal relaxation parameter and corresponding spectral radius are

$$\omega^* = \frac{2}{1+\sqrt{1-\rho_J^2}}, \qquad \rho(\mathbf{T}_{\omega^*}) = \omega^* - 1 = \frac{1-\sqrt{1-\rho_J^2}}{1+\sqrt{1-\rho_J^2}}.$$

When $\rho_J \approx 1-\varepsilon$ (slow Jacobi convergence), $\omega^* \approx 2 - 2\sqrt{\varepsilon}$ and $\rho(\mathbf{T}_{\omega^*}) \approx 1 - 2\sqrt{\varepsilon} \ll \rho_J$.

Proof *Proof.* For consistently ordered matrices, the eigenvalues μ of $\mathbf{T}_\omega$ relate to the eigenvalues λ of $\mathbf{T}_J$ via $(\mu+\omega-1)^2 = \lambda^2\omega^2\mu$. Minimizing $\max|\mu|$ over ω gives the stated formulas. The key inequality $\rho(\mathbf{T}_{\omega^*}) \approx 1 - 2\sqrt{\varepsilon}$ versus $\rho_J \approx 1-\varepsilon$ shows SOR reduces the number of iterations by a factor of roughly $\sqrt{\varepsilon}^{-1}/2$ — a square-root speedup. □

Note 5.5 For the n-point 1D Poisson problem (tridiagonal, $h = 1/(n+1)$), Jacobi has $\rho_J = \cos(\pi h) \approx 1 - \frac{\pi^2 h^2}{2}$, requiring $O(n^2)$ iterations. With optimal SOR, $\rho(\mathbf{T}_{\omega^*}) \approx 1 - 2\pi h$,

requiring only $O(n)$ iterations — a dramatic improvement. The 2D Poisson problem shows similar gains.

Example **Effect of ω on convergence rate.**
For the $n = 9$ tridiagonal system (1D Poisson), $h = 0.1$, so $\rho_J = \cos(\pi/10) \approx 0.951$. The optimal parameter is

$$\omega^* = \frac{2}{1 + \sqrt{1 - 0.951^2}} \approx \frac{2}{1 + 0.309} \approx 1.527,$$

giving $\rho(\mathbf{T}_{\omega^*}) \approx 0.527$. With Gauss-Seidel ($\omega = 1$), $\rho \approx 0.904$. The number of iterations to reduce the error by 10^{-8} is approximately $8 \log_{10} / \log_{10}(1/\rho)$, which is about 169 for Gauss-Seidel but only about 35 for SOR — nearly a fivefold speedup.

Ex 5.5 **(Gauss-Seidel implementation and comparison)**

1. Implement `gauss_seidel(A, b, x0, tol, max_iter)` in Python. The key difference from Jacobi is to update $x[i]$ *in place* at each step.
2. Run both Jacobi and Gauss-Seidel on the 10×10 tridiagonal system with 4 on the diagonal and -1 on the off-diagonals, $\boldsymbol{b} = \mathbf{1}$. Plot the residual norm versus iteration count on a semi-log scale for both methods. Estimate the ratio of iterations needed to reach 10^{-8} residual.
3. Compute $\rho(\mathbf{T}_J)$ and $\rho(\mathbf{T}_{GS})$ numerically. Is the observed convergence ratio consistent with $\rho(\mathbf{T}_{GS}) \approx \rho(\mathbf{T}_J)^2$? (This relation holds exactly for consistently ordered matrices.)

Ex 5.6 **(SOR parameter tuning)**

1. Implement `sor(A, b, x0, omega, tol, max_iter)`.
2. For the same 10×10 tridiagonal system, compute the theoretical optimal ω^* using the formula in the Optimal SOR theorem. Run SOR with $\omega \in \{1.0, 1.2, 1.4, \omega^*, 1.8, 1.9\}$.
3. Plot iterations to convergence (residual $< 10^{-8}$) versus ω. Verify that ω^* is indeed optimal and that convergence fails for $\omega \geq 2$.
4. At the optimal ω^*, compare the convergence rate to Jacobi and Gauss-Seidel from the previous exercise.

5.3 Conjugate Gradient Method

The Conjugate Gradient (CG) method is an efficient iterative solver for symmetric positive definite linear systems. By maintaining a set of $\mathbf{A}$-conjugate search directions built from the residuals, it converges to the exact solution in at most n steps in exact arithmetic, while requiring only $O(n)$ storage and one matrix–vector product per iteration. Its convergence rate depends on $\sqrt{\kappa(\mathbf{A})}$ rather than $\kappa(\mathbf{A})$, giving a dramatic speedup over stationary methods on ill-conditioned systems.

5.3.1 From Linear Systems to Optimization

Thm 5.11 **(Equivalence of Linear System and Quadratic Minimization)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be symmetric positive definite and $\boldsymbol{b} \in \mathbb{R}^n$. Define

$$\phi(\boldsymbol{x}) = \tfrac{1}{2}\boldsymbol{x}^T\mathbf{A}\boldsymbol{x} - \boldsymbol{x}^T\boldsymbol{b}.$$

Then $\boldsymbol{x}^*$ solves $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ if and only if $\boldsymbol{x}^*$ minimizes ϕ.

Proof *Proof.* Since $\mathbf{A}$ is symmetric, $\nabla\phi(\boldsymbol{x}) = \mathbf{A}\boldsymbol{x} - \boldsymbol{b}$. A point $\boldsymbol{x}^*$ is a critical point of ϕ if and only if $\nabla\phi(\boldsymbol{x}^*) = \mathbf{0}$, i.e., $\mathbf{A}\boldsymbol{x}^* = \boldsymbol{b}$. Since $\mathbf{A}$ is positive definite, ϕ is strictly convex (its Hessian $\mathbf{A} \succ 0$), so the unique critical point is the global minimum. □

Note 5.6 When $\mathbf{A}$ is symmetric positive definite, $\phi(\boldsymbol{x})$ is a paraboloid with elliptical level sets in $\mathbb{R}^n$. The condition number $\kappa_2(\mathbf{A}) = \lambda_{\max}/\lambda_{\min}$ controls the eccentricity: high $\kappa(\mathbf{A})$ produces narrow, elongated valleys. Gradient-based methods zigzag inefficiently in these valleys, while CG exploits the geometry via $\mathbf{A}$-conjugate directions.

Def 5.13 **(Residual and Steepest Descent)** At iterate $\boldsymbol{x}^{(k)}$, the *residual* $\boldsymbol{r}^{(k)} = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(k)} = -\nabla\phi(\boldsymbol{x}^{(k)})$ points in the direction of steepest descent. The *steepest descent* update

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \alpha_k\boldsymbol{r}^{(k)}, \qquad \alpha_k = \frac{(\boldsymbol{r}^{(k)})^T\boldsymbol{r}^{(k)}}{(\boldsymbol{r}^{(k)})^T\mathbf{A}\boldsymbol{r}^{(k)}}$$

minimizes ϕ along the direction $\boldsymbol{r}^{(k)}$ but often exhibits zigzagging in ill-conditioned systems.

Def 5.14 **(Stationary Method Limitations)** A stationary iterative method $\boldsymbol{x}^{(k+1)} = \mathbf{T}\boldsymbol{x}^{(k)} + \boldsymbol{c}$ reduces the error by a constant factor $\rho(\mathbf{T})$ per step. Its convergence rate for the Jacobi splitting satisfies $\rho(\mathbf{T}_J) \le (\kappa(\mathbf{A}) - 1)/(\kappa(\mathbf{A}) + 1)$, which approaches 1 as $\kappa(\mathbf{A}) \to \infty$.

Note 5.7 For $\kappa(\mathbf{A}) = 100$, the Jacobi factor is ≈ 0.98, requiring $\sim$500 iterations to reduce the error by 10^{-4}. Steepest descent suffers the same limitation. Conjugate Gradient breaks this barrier by adapting its search directions to the problem geometry.

5.3.2 A-Conjugate Directions

Def 5.15 **(A-Conjugacy)** Two vectors $\boldsymbol{p}, \boldsymbol{q} \in \mathbb{R}^n$ are $\mathbf{A}$-*conjugate* (or $\mathbf{A}$-*orthogonal*) if $\boldsymbol{p}^T\mathbf{A}\boldsymbol{q} = 0$. A set $\{\boldsymbol{p}_0, \ldots, \boldsymbol{p}_{m-1}\}$ of nonzero vectors is $\mathbf{A}$-conjugate if $\boldsymbol{p}_i^T\mathbf{A}\boldsymbol{p}_j = 0$ for all $i \ne j$.

Prop 5.12 **(A-Conjugate Vectors are Linearly Independent)** If $\mathbf{A}$ is positive definite, every set of $\mathbf{A}$-conjugate nonzero vectors is linearly independent.

Proof *Proof.* Suppose $\sum_{i=0}^{m-1} c_i\boldsymbol{p}_i = \mathbf{0}$. Multiply on the left by $\boldsymbol{p}_j^T\mathbf{A}$:

$$0 = \boldsymbol{p}_j^T\mathbf{A}\Big(\sum_i c_i\boldsymbol{p}_i\Big) = c_j\,\boldsymbol{p}_j^T\mathbf{A}\boldsymbol{p}_j.$$

Since $\mathbf{A} \succ 0$, $\boldsymbol{p}_j^T \mathbf{A} \boldsymbol{p}_j > 0$. Hence $c_j = 0$ for each j. $\square$

Prop 5.13 **(Exact Minimization Along n Conjugate Directions)** Let $\mathbf{A}$ be symmetric positive definite and $\{\boldsymbol{p}_0, \ldots, \boldsymbol{p}_{n-1}\}$ be $\mathbf{A}$-conjugate in $\mathbb{R}^n$. Starting from $\boldsymbol{x}^{(0)}$, define

$$\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(0)} + \sum_{i=0}^{k-1} \alpha_i \boldsymbol{p}_i, \qquad \alpha_i = \frac{\boldsymbol{p}_i^T \boldsymbol{r}^{(i)}}{\boldsymbol{p}_i^T \mathbf{A} \boldsymbol{p}_i},$$

where $\boldsymbol{r}^{(i)} = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(i)}$. Then $\boldsymbol{x}^{(n)} = \boldsymbol{x}^*$ exactly.

Proof *Proof.* Since the $\boldsymbol{p}_i$ are linearly independent in $\mathbb{R}^n$, they form a basis. Write $\boldsymbol{x}^* - \boldsymbol{x}^{(0)} = \sum_i \alpha_i^* \boldsymbol{p}_i$. Multiplying by $\boldsymbol{p}_j^T \mathbf{A}$:

$$\boldsymbol{p}_j^T \mathbf{A}(\boldsymbol{x}^* - \boldsymbol{x}^{(0)}) = \alpha_j^* \boldsymbol{p}_j^T \mathbf{A} \boldsymbol{p}_j.$$

Since $\mathbf{A}(\boldsymbol{x}^* - \boldsymbol{x}^{(0)}) = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(0)} = \boldsymbol{r}^{(0)}$, we get $\alpha_j^* = \boldsymbol{p}_j^T \boldsymbol{r}^{(0)} / (\boldsymbol{p}_j^T \mathbf{A} \boldsymbol{p}_j)$. One verifies by induction that $\boldsymbol{p}_j^T \boldsymbol{r}^{(j)} = \boldsymbol{p}_j^T \boldsymbol{r}^{(0)}$ (because $\boldsymbol{r}^{(j)}$ differs from $\boldsymbol{r}^{(0)}$ only by $\mathbf{A}$-multiples of previous $\boldsymbol{p}_i$, which are $\mathbf{A}$-conjugate to $\boldsymbol{p}_j$). Hence $\alpha_j^* = \alpha_j$ and $\boldsymbol{x}^{(n)} = \boldsymbol{x}^*$. $\square$

Def 5.16 **(Iterative Generation of A-Conjugate Directions)** The CG method builds $\mathbf{A}$-conjugate directions iteratively via a three-term recurrence. Given residual $\boldsymbol{r}^{(k+1)}$, the next search direction is:

$$\boldsymbol{p}^{(k+1)} = \boldsymbol{r}^{(k+1)} + \beta_k \boldsymbol{p}^{(k)}, \qquad \beta_k = \frac{(\boldsymbol{r}^{(k+1)})^T \boldsymbol{r}^{(k+1)}}{(\boldsymbol{r}^{(k)})^T \boldsymbol{r}^{(k)}}.$$

The choice of β_k ensures $\boldsymbol{p}^{(k+1)}$ is $\mathbf{A}$-conjugate to *all* previous directions, not just $\boldsymbol{p}^{(k)}$.

5.3.3 The CG Algorithm and Convergence

Def 5.17 **(Conjugate Gradient Algorithm)** **Initialize:** $\boldsymbol{x}^{(0)}$ (arbitrary), $\boldsymbol{r}^{(0)} = \boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(0)}$, $\boldsymbol{p}^{(0)} = \boldsymbol{r}^{(0)}$.
For $k = 0, 1, 2, \ldots$ **until** $\|\boldsymbol{r}^{(k)}\| < \epsilon$**:**

$$\begin{aligned}
\alpha_k &= \frac{(\boldsymbol{r}^{(k)})^T \boldsymbol{r}^{(k)}}{(\boldsymbol{p}^{(k)})^T \mathbf{A} \boldsymbol{p}^{(k)}}, \\
\boldsymbol{x}^{(k+1)} &= \boldsymbol{x}^{(k)} + \alpha_k \boldsymbol{p}^{(k)}, \\
\boldsymbol{r}^{(k+1)} &= \boldsymbol{r}^{(k)} - \alpha_k \mathbf{A} \boldsymbol{p}^{(k)}, \\
\beta_k &= \frac{(\boldsymbol{r}^{(k+1)})^T \boldsymbol{r}^{(k+1)}}{(\boldsymbol{r}^{(k)})^T \boldsymbol{r}^{(k)}}, \\
\boldsymbol{p}^{(k+1)} &= \boldsymbol{r}^{(k+1)} + \beta_k \boldsymbol{p}^{(k)}.
\end{aligned}$$

Prop 5.14 **(CG Storage Requirement)** At each step, only $\boldsymbol{x}^{(k)}$, $\boldsymbol{r}^{(k)}$, $\boldsymbol{p}^{(k)}$, and the product $\mathbf{A}\boldsymbol{p}^{(k)}$ need to be stored simultaneously — a constant number of n-vectors, independent of the iteration count.

Prop 5.15 **(CG Cost Per Iteration)** Each CG iteration requires one matrix–vector product $\mathbf{A}\boldsymbol{p}^{(k)}$ (cost $O(n^2)$ for a dense matrix, $O(\mathrm{nnz}(\mathbf{A}))$ for a sparse matrix), two vector dot products, and three vector updates — all at $O(n)$.

Thm 5.16 **(Finite Termination of Conjugate Gradient)** Let $\mathbf{A} \in \mathbb{R}^{n\times n}$ be symmetric positive definite. The CG iterates satisfy:

1. $(\boldsymbol{r}^{(i)})^T\boldsymbol{r}^{(j)} = 0$ for $i \neq j$ (residuals are mutually orthogonal).
2. $(\boldsymbol{p}^{(i)})^T\mathbf{A}\boldsymbol{p}^{(j)} = 0$ for $i \neq j$ (search directions are $\mathbf{A}$-conjugate).

Consequently, CG terminates with $\boldsymbol{x}^{(m)} = \boldsymbol{x}^*$ for some $m \leq n$.

Proof *Proof.* We prove both properties by induction. Both hold vacuously for $k = 0$.
Assume (1) and (2) hold through step k. For the orthogonality of residuals: using $\boldsymbol{r}^{(k+1)} = \boldsymbol{r}^{(k)} - \alpha_k\mathbf{A}\boldsymbol{p}^{(k)}$,

$$(\boldsymbol{r}^{(k+1)})^T\boldsymbol{r}^{(j)} = (\boldsymbol{r}^{(k)})^T\boldsymbol{r}^{(j)} - \alpha_k(\boldsymbol{p}^{(k)})^T\mathbf{A}\boldsymbol{r}^{(j)}.$$

Since each residual $\boldsymbol{r}^{(j)}$ is a linear combination of $\boldsymbol{p}^{(0)}, \ldots, \boldsymbol{p}^{(j)}$, and the search directions satisfy $\mathbf{A}$-conjugacy, the term $(\boldsymbol{p}^{(k)})^T\mathbf{A}\boldsymbol{r}^{(j)}$ vanishes for $j < k$ and equals $(\boldsymbol{r}^{(k)})^T\boldsymbol{r}^{(k)}$ for $j = k$, which is cancelled by the choice of α_k. For the $\mathbf{A}$-conjugacy of directions: by construction of β_k,

$$(\boldsymbol{p}^{(k+1)})^T\mathbf{A}\boldsymbol{p}^{(k)} = (\boldsymbol{r}^{(k+1)})^T\mathbf{A}\boldsymbol{p}^{(k)} + \beta_k(\boldsymbol{p}^{(k)})^T\mathbf{A}\boldsymbol{p}^{(k)} = 0$$

by the choice of β_k; inner products with earlier directions vanish by the inductive hypothesis.
Since there can be at most n mutually orthogonal nonzero residuals in $\mathbb{R}^n$, the algorithm must produce $\boldsymbol{r}^{(m)} = \mathbf{0}$ for some $m \leq n$. □

Thm 5.17 **(CG Convergence Rate Bound)** Let $\mathbf{A}$ be symmetric positive definite with condition number $\kappa = \kappa_2(\mathbf{A})$. Then for all $k \geq 0$:

$$\frac{\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\|_{\mathbf{A}}}{\|\boldsymbol{x}^{(0)} - \boldsymbol{x}^*\|_{\mathbf{A}}} \leq 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k,$$

where $\|\boldsymbol{v}\|_{\mathbf{A}} = \sqrt{\boldsymbol{v}^T\mathbf{A}\boldsymbol{v}}$ is the $\mathbf{A}$-norm (energy norm).

Proof *Proof.* One can show that $\boldsymbol{x}^{(k)}$ minimizes $\|\boldsymbol{x} - \boldsymbol{x}^*\|_{\mathbf{A}}$ over all $\boldsymbol{x}^{(0)} + \mathcal{K}_k$, where $\mathcal{K}_k = \mathrm{span}\{\boldsymbol{r}^{(0)}, \mathbf{A}\boldsymbol{r}^{(0)}, \ldots, \mathbf{A}^{k-1}\boldsymbol{r}^{(0)}\}$ is the k-th Krylov subspace. Hence

$$\frac{\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\|_{\mathbf{A}}}{\|\boldsymbol{x}^{(0)} - \boldsymbol{x}^*\|_{\mathbf{A}}} \leq \min_{\substack{p\in\mathcal{P}_k\\ p(0)=1}} \max_{\lambda\in\sigma(\mathbf{A})} |p(\lambda)|,$$

where the minimum is over polynomials of degree k satisfying $p(0) = 1$. The Chebyshev polynomial T_k minimizes the maximum absolute value over $[\lambda_{\min}, \lambda_{\max}]$ among all such polynomials, and satisfies

$$\min_{\substack{p\in\mathcal{P}_k\\ p(0)=1}} \max_{\lambda\in[\lambda_{\min},\lambda_{\max}]} |p(\lambda)| = \frac{1}{T_k((\lambda_{\max}+\lambda_{\min})/(\lambda_{\max}-\lambda_{\min}))} \leq 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k,$$

where the last bound follows from the identity $T_k(\cosh\theta) = \cosh(k\theta)$ with $\cosh\theta = (\kappa + 1)/(\kappa - 1)$. $\square$

Note 5.8 The CG convergence factor $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$ is far smaller than the stationary method factor $(\kappa - 1)/(\kappa + 1)$ for large κ. For $\kappa = 100$: CG factor ≈ 0.82 (needs ≈ 22 iterations for 10^{-4} reduction), versus Jacobi factor ≈ 0.98 (needs ≈ 500 iterations). Moreover, if the eigenvalues of $\mathbf{A}$ are clustered into m distinct groups, CG converges in roughly m steps — this is *superlinear convergence*, far better than the worst-case bound.

5.3.4 Preconditioning

Def 5.18 **(Preconditioner)** A *preconditioner* for $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ is a symmetric positive definite matrix $\mathbf{M}$ such that $\mathbf{M} \approx \mathbf{A}$ and $\mathbf{M}^{-1}$ is cheap to apply. The preconditioned system $\mathbf{M}^{-1/2}\mathbf{A}\mathbf{M}^{-1/2}\boldsymbol{y} = \mathbf{M}^{-1/2}\boldsymbol{b}$ (where $\boldsymbol{y} = \mathbf{M}^{1/2}\boldsymbol{x}$) has the same solution but ideally satisfies $\kappa(\mathbf{M}^{-1/2}\mathbf{A}\mathbf{M}^{-1/2}) \ll \kappa(\mathbf{A})$.

Note 5.9 In practice, the Preconditioned Conjugate Gradient (PCG) algorithm applies $\mathbf{M}^{-1}$ at each step without forming $\mathbf{M}^{1/2}$ explicitly. Common preconditioners include: **Jacobi** ($\mathbf{M} = \mathrm{diag}(\mathbf{A})$, cheap but limited benefit), **incomplete Cholesky** ($\mathbf{M} \approx \mathbf{L}\mathbf{L}^T$ with controlled fill-in, far more effective), and **multigrid** (near-optimal $O(n)$ convergence for PDE-derived systems).

Ex 5.7 Implement the Conjugate Gradient method in Python.

1. Write `conjugate_gradient(A, b, x0, tol)` following the CG algorithm, stopping when $\|\boldsymbol{r}^{(k)}\|/\|\boldsymbol{r}^{(0)}\| <$ `tol`.
2. Test on the 100×100 SPD tridiagonal matrix $\mathbf{A} = \mathrm{tridiag}(-1, 2, -1)$. Plot the $\mathbf{A}$-norm error vs. iteration count and compare to the theoretical bound.
3. Verify your implementation against `scipy.sparse.linalg.cg`.

Ex 5.8 **(Preconditioning effect)** Using the tridiagonal $\mathbf{A}$ for $n = 200$:

1. Compute $\kappa_2(\mathbf{A})$ with `np.linalg.cond`. How many CG iterations does the bound predict for 10^{-8} accuracy?
2. Apply the Jacobi preconditioner $\mathbf{M} = \mathrm{diag}(\mathbf{A})$. What is $\kappa(\mathbf{M}^{-1}\mathbf{A})$? Why does it barely help here?
3. Implement incomplete Cholesky preconditioning via `scipy.sparse.linalg.spilu`. Compare iteration counts for the original and preconditioned CG.

Ex 5.9 **(Eigenvalue clustering and superlinear convergence)** Build a 50×50 SPD matrix with eigenvalues: 40 eigenvalues equal to 1, 5 equal to 10, and 5 equal to 100.

1. What is $\kappa(\mathbf{A})$? What does the CG bound predict for the number of iterations needed to achieve $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\|_{\mathbf{A}} / \|\boldsymbol{x}^{(0)} - \boldsymbol{x}^*\|_{\mathbf{A}} < 10^{-10}$?

2. Construct the matrix using `Q @ np.diag(eigs) @ Q.T` with a random orthogonal $\mathbf{Q}$ from `scipy.stats.ortho_group.rvs(50)`. Run CG and plot the actual error versus the bound.

3. How many iterations does CG actually need? Explain why superlinear convergence occurs when eigenvalues are clustered.

5.4 GMRES

Conjugate Gradient requires symmetry and positive definiteness. For general (possibly non-symmetric, non-definite) linear systems, the *Generalized Minimal Residual* method (GMRES) is the standard Krylov solver. It minimizes the residual norm over a growing Krylov subspace using the Arnoldi process to build an orthonormal basis, achieving the smallest possible residual at each step without any symmetry assumption.

5.4.1 Krylov Subspaces and the Arnoldi Process

Def 5.19 **(Krylov Subspace)** Given $\mathbf{A} \in \mathbb{R}^{n\times n}$ and $\boldsymbol{r} \in \mathbb{R}^n$, the *k-th Krylov subspace* is

$$\mathcal{K}_k(\mathbf{A}, \boldsymbol{r}) = \operatorname{span}\{\boldsymbol{r}, \mathbf{A}\boldsymbol{r}, \mathbf{A}^2\boldsymbol{r}, \ldots, \mathbf{A}^{k-1}\boldsymbol{r}\}.$$

As k increases, $\mathcal{K}_k$ grows (until it reaches an invariant subspace), capturing increasingly refined information about the action of $\mathbf{A}$.

Note 5.10 Krylov methods compute approximations $\boldsymbol{x}^{(k)} \in \boldsymbol{x}^{(0)} + \mathcal{K}_k(\mathbf{A}, \boldsymbol{r}^{(0)})$ by exploiting the structure of $\mathcal{K}_k$. The key insight is that matrix powers $\mathbf{A}^j\boldsymbol{r}$ can be formed iteratively with only matrix-vector products — no matrix factorization required. For sparse $\mathbf{A}$ with $\operatorname{nnz}(\mathbf{A})$ nonzeros, each step costs $O(\operatorname{nnz}(\mathbf{A}))$.

Def 5.20 **(Arnoldi Process)** The *Arnoldi process* builds an orthonormal basis $\{\boldsymbol{q}_1, \boldsymbol{q}_2, \ldots, \boldsymbol{q}_k\}$ for $\mathcal{K}_k(\mathbf{A}, \boldsymbol{r})$ via modified Gram-Schmidt. Starting with $\boldsymbol{q}_1 = \boldsymbol{r}/\|\boldsymbol{r}\|$, for $j = 1, \ldots, k$:

1. Compute $\boldsymbol{z} = \mathbf{A}\boldsymbol{q}_j$.
2. For $i = 1, \ldots, j$: set $h_{ij} = \boldsymbol{q}_i^T\boldsymbol{z}$, $\boldsymbol{z} \leftarrow \boldsymbol{z} - h_{ij}\boldsymbol{q}_i$.
3. Set $h_{j+1,j} = \|\boldsymbol{z}\|$; if $h_{j+1,j} = 0$, stop (invariant subspace found).
4. Set $\boldsymbol{q}_{j+1} = \boldsymbol{z}/h_{j+1,j}$.

The coefficients h_{ij} form an upper Hessenberg matrix $\mathbf{H}_k \in \mathbb{R}^{(k+1)\times k}$.

Thm 5.18 **(Arnoldi Relation)** Let $\mathbf{Q}_k = [\boldsymbol{q}_1, \ldots, \boldsymbol{q}_k]$ and $\mathbf{H}_k \in \mathbb{R}^{(k+1)\times k}$ be the Hessenberg matrix produced by the Arnoldi process. Then

$$\mathbf{A}\mathbf{Q}_k = \mathbf{Q}_{k+1}\mathbf{H}_k,$$

or equivalently, the leading $k \times k$ submatrix $\tilde{\mathbf{H}}_k$ of $\mathbf{H}_k$ satisfies $\mathbf{Q}_k^T\mathbf{A}\mathbf{Q}_k = \tilde{\mathbf{H}}_k$.

Proof *Proof.* By construction, each $\mathbf{A}\boldsymbol{q}_j = \sum_{i=1}^{j+1} h_{ij}\boldsymbol{q}_i$, which is precisely the j-th column of $\mathbf{Q}_{k+1}\mathbf{H}_k$. Combining all k columns yields $\mathbf{A}\mathbf{Q}_k = \mathbf{Q}_{k+1}\mathbf{H}_k$. Multiplying on the left by $\mathbf{Q}_k^T$ and using $\mathbf{Q}_{k+1}^T\mathbf{Q}_k = [\mathbf{I}_k; \mathbf{0}^T]$ gives $\mathbf{Q}_k^T\mathbf{A}\mathbf{Q}_k = \tilde{\mathbf{H}}_k$. □

5.4.2 The GMRES Algorithm

Def 5.21 **(GMRES Iterate)** GMRES seeks an approximation $\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(0)} + \mathbf{Q}_k \boldsymbol{y}_k$ that minimizes the residual norm:

$$\boldsymbol{x}^{(k)} = \arg \min_{\boldsymbol{x} \in \boldsymbol{x}^{(0)} + \mathcal{K}_k} \|\boldsymbol{b} - \mathbf{A}\boldsymbol{x}\|_2.$$

Using the Arnoldi relation, the residual becomes

$$\|\boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(k)}\|_2 = \|\beta \boldsymbol{e}_1 - \mathbf{H}_k \boldsymbol{y}_k\|_2, \qquad \beta = \|\boldsymbol{r}^{(0)}\|_2,$$

where $\boldsymbol{e}_1 = (1, 0, \ldots, 0)^T \in \mathbb{R}^{k+1}$. This is a small $(k+1) \times k$ least squares problem solved via QR factorization of $\mathbf{H}_k$.

Thm 5.19 **(GMRES Optimality)** Among all methods producing iterates in $\boldsymbol{x}^{(0)} + \mathcal{K}_k(\mathbf{A}, \boldsymbol{r}^{(0)})$, GMRES achieves the smallest possible residual norm $\|\boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(k)}\|_2$ at each step k. In exact arithmetic, GMRES terminates with $\boldsymbol{x}^{(n)} = \boldsymbol{x}^*$ (or earlier if an invariant subspace is encountered).

Proof *Proof.* The optimality is immediate from the definition: $\boldsymbol{x}^{(k)}$ is precisely the minimizer of $\|\boldsymbol{b} - \mathbf{A}\boldsymbol{x}\|_2$ over the affine subspace $\boldsymbol{x}^{(0)} + \mathcal{K}_k$. Since $\mathcal{K}_n = \mathbb{R}^n$ (for generic $\mathbf{A}$), termination within n steps follows. □

Note 5.11 The cost of GMRES grows with each iteration: at step k, the Arnoldi process requires k dot products and the Hessenberg least squares problem is $(k+1) \times k$. After m steps, the total cost is $O(mn^2 + m^2 n)$ flops and $O(mn)$ storage (to retain all m Arnoldi vectors). This motivates *restarted GMRES*, denoted GMRES(m): after m steps, restart from the current iterate.

Def 5.22 **(Restarted GMRES — GMRES(m))** *GMRES(m)* runs GMRES for at most m iterations, then restarts from the current approximate solution $\boldsymbol{x}^{(m)}$:

$$\text{Outer loop:} \quad \boldsymbol{x}_0^{\text{new}} \leftarrow \boldsymbol{x}^{(m)}, \quad \boldsymbol{r}_0^{\text{new}} \leftarrow \boldsymbol{b} - \mathbf{A}\boldsymbol{x}^{(m)}, \quad \text{repeat.}$$

Each restart cycle has fixed cost $O(m^2 n)$ and storage $O(mn)$. The tradeoff: small m reduces memory but may stall; large m recovers full GMRES but costs more. Typical default: $m = 30$ to 50.

Note 5.12 GMRES(m) is not guaranteed to converge for all m: restarting can discard important information and the method may stagnate. Full GMRES always converges (monotonically decreasing residual), but is impractical for large problems. Preconditioning is essential in practice: left preconditioning replaces $\mathbf{A}$ by $\mathbf{M}^{-1}\mathbf{A}$; right preconditioning by $\mathbf{A}\mathbf{M}^{-1}$. A good preconditioner drastically reduces the number of restarts needed.

Thm 5.20 **(GMRES Convergence Bound)** For a diagonalizable matrix $\mathbf{A} = \mathbf{V}\mathbf{D}\mathbf{V}^{-1}$ with eigenvalues $\lambda_1, \ldots, \lambda_n$,

$$\frac{\|\boldsymbol{r}^{(k)}\|_2}{\|\boldsymbol{r}^{(0)}\|_2} \leq \kappa_2(\mathbf{V}) \cdot \min_{\substack{p \in \mathcal{P}_k \\ p(0)=1}} \max_i |p(\lambda_i)|,$$

where $\kappa_2(\mathbf{V})$ measures how far $\mathbf{A}$ is from normal. If $\mathbf{A}$ is normal ($\kappa_2(\mathbf{V}) = 1$) and eigenvalues are clustered away from zero, the Chebyshev polynomial gives fast convergence.

Ex 5.10 **(GMRES vs. CG on SPD systems)**

1. For the 50×50 tridiagonal SPD matrix $\mathbf{A} = \text{tridiag}(-1, 4, -1)$, solve $\mathbf{A}\boldsymbol{x} = \boldsymbol{b}$ using both CG (`scipy.sparse.linalg.cg`) and GMRES (`scipy.sparse.linalg.gmres`). Compare residual histories.

2. Why is CG preferred over GMRES for SPD systems? (Consider both theory and cost per iteration.)

3. Now take a random non-symmetric 50×50 matrix $\mathbf{A}$ with $\kappa_2(\mathbf{A}) \approx 20$. Can CG be applied? Apply GMRES and plot the residual.

Ex 5.11 **(Effect of restart parameter m)**

1. Generate a 100×100 non-symmetric sparse matrix using `scipy.sparse.random(100, 100, density=0.1)` plus $5\mathbf{I}$ (for nonsingularity).

2. Run GMRES(m) with $m \in \{5, 10, 20, 50, 100\}$ (full GMRES is $m = 100$). For each m, record total iterations (across all restarts) and total matvecs to reach residual $< 10^{-8}$.

3. Plot total matvecs versus m. What restart parameter gives the best tradeoff between convergence speed and memory cost?

Ex 5.12 **(Arnoldi orthogonality)** Implement the Arnoldi process for $k = 10$ steps on a random 20×20 matrix.

1. Verify the Arnoldi relation $\mathbf{A}\mathbf{Q}_k = \mathbf{Q}_{k+1}\mathbf{H}_k$ by computing the Frobenius norm of the difference.

2. Check that $\mathbf{Q}_k^T\mathbf{Q}_k = \mathbf{I}_k$ (orthonormality of Arnoldi vectors). Does floating-point error accumulate over iterations?

3. Compare classical Gram-Schmidt (CGS) vs. modified Gram-Schmidt (MGS) for the Arnoldi process. Which preserves orthogonality better? Use `np.linalg.cond(`$\mathbf{Q}_k$`)` to assess.

6 Applications

6.1 Principal Component Analysis via SVD

Principal Component Analysis (PCA) is a fundamental technique for dimensionality reduction. It seeks an orthogonal change of basis that aligns the coordinate axes with the

directions of maximum variance in the data. Via the SVD, PCA is both theoretically elegant — it is the optimal linear dimensionality reduction — and computationally efficient. Understanding PCA requires understanding orthogonal projections, eigendecompositions, and the Eckart-Young theorem.

Def 6.1 **(Data Centering)** For PCA, the data matrix $\mathbf{X} \in \mathbb{R}^{n\times p}$ (n samples, p features) must be *centered*: subtract the column mean $\bar{x}_j = \frac{1}{n}\sum_i X_{ij}$ from each entry in column j. Throughout this section, $\mathbf{X}$ denotes a centered data matrix.

Def 6.2 **(Sample Covariance Matrix)** For a centered data matrix $\mathbf{X} \in \mathbb{R}^{n\times p}$, the *sample covariance matrix* is

$$\mathbf{S} = \frac{1}{n-1}\mathbf{X}^T\mathbf{X} \in \mathbb{R}^{p\times p}.$$

$\mathbf{S}$ is symmetric and positive semidefinite. Its diagonal entry S_{jj} is the sample variance of feature j, and S_{ij} is the sample covariance between features i and j.

6.1.1 Principal Components as Variance-Maximizing Directions

Thm 6.1 **(First Principal Component Maximizes Variance)** Among all unit vectors $\boldsymbol{w} \in \mathbb{R}^p$, the direction that maximizes the variance of the projected data $\mathbf{X}\boldsymbol{w}$ is the eigenvector $\boldsymbol{v}_1$ of $\mathbf{S}$ corresponding to the largest eigenvalue λ_1.

Proof *Proof.* The sample variance of $\mathbf{X}\boldsymbol{w}$ is

$$\mathrm{Var}(\mathbf{X}\boldsymbol{w}) = \frac{1}{n-1}\|\mathbf{X}\boldsymbol{w}\|_2^2 = \boldsymbol{w}^T\mathbf{S}\boldsymbol{w}.$$

We maximize $\boldsymbol{w}^T\mathbf{S}\boldsymbol{w}$ subject to $\|\boldsymbol{w}\|_2 = 1$. By the Rayleigh quotient theorem (a consequence of the spectral theorem for symmetric matrices), this maximum is λ_1, attained at $\boldsymbol{w} = \boldsymbol{v}_1$ (the eigenvector of $\mathbf{S}$ for λ_1). To verify: $\boldsymbol{v}_1^T\mathbf{S}\boldsymbol{v}_1 = \lambda_1\boldsymbol{v}_1^T\boldsymbol{v}_1 = \lambda_1$, and for any unit $\boldsymbol{w} = \sum_i c_i\boldsymbol{v}_i$, $\boldsymbol{w}^T\mathbf{S}\boldsymbol{w} = \sum_i \lambda_i c_i^2 \le \lambda_1 \sum_i c_i^2 = \lambda_1$. □

Def 6.3 **(Principal Directions (Loading Vectors))** The *principal directions* $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_p$ are the eigenvectors of $\mathbf{S}$ ordered by decreasing eigenvalue $\lambda_1 \ge \lambda_2 \ge \cdots \ge \lambda_p \ge 0$. The k-th principal direction $\boldsymbol{v}_k$ maximizes variance among all unit vectors orthogonal to $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_{k-1}$.

Def 6.4 **(Principal Component Scores)** The *principal component scores* are the projections of the data onto the principal directions:

$$\mathbf{Z} = \mathbf{X}\mathbf{V}, \qquad \mathbf{V} = [\boldsymbol{v}_1 \mid \cdots \mid \boldsymbol{v}_p].$$

The k-th column $\boldsymbol{z}_k = \mathbf{X}\boldsymbol{v}_k$ is the k-th principal component. Its sample variance is λ_k.

Thm 6.2 **(Principal Components are Uncorrelated)** The principal component scores are pairwise uncorrelated: $\mathrm{Cov}(\boldsymbol{z}_j, \boldsymbol{z}_k) = 0$ for $j \ne k$.

Proof *Proof.* The sample covariance matrix of the scores $\mathbf{Z} = \mathbf{X}\mathbf{V}$ is

$$\frac{1}{n-1}\mathbf{Z}^T\mathbf{Z} = \frac{1}{n-1}\mathbf{V}^T\mathbf{X}^T\mathbf{X}\mathbf{V} = \mathbf{V}^T\mathbf{S}\mathbf{V} = \mathbf{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_p),$$

where the last equality uses $\mathbf{S}\mathbf{V} = \mathbf{V}\mathbf{\Lambda}$ (eigendecomposition) and $\mathbf{V}^T\mathbf{V} = \mathbf{I}$ (orthogonality). The off-diagonal entries of $\mathbf{\Lambda}$ are zero, confirming the principal components are uncorrelated. □

6.1.2 The SVD-PCA Connection

Thm 6.3 **(PCA via SVD)** Let $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ be the SVD of the centered data matrix. Then: the columns of $\mathbf{V}$ are the principal directions (eigenvectors of $\mathbf{S}$); the eigenvalues of $\mathbf{S}$ satisfy $\lambda_i = \sigma_i^2/(n-1)$; the principal component scores satisfy $\mathbf{Z} = \mathbf{X}\mathbf{V} = \mathbf{U}\mathbf{\Sigma}$.

Proof *Proof.* Substitute the SVD into the covariance matrix:

$$\mathbf{S} = \frac{1}{n-1}\mathbf{X}^T\mathbf{X} = \frac{1}{n-1}(\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T)^T(\mathbf{U}\mathbf{\Sigma}\mathbf{V}^T) = \frac{1}{n-1}\mathbf{V}\mathbf{\Sigma}^T(\mathbf{U}^T\mathbf{U})\mathbf{\Sigma}\mathbf{V}^T = \frac{1}{n-1}\mathbf{V}\mathbf{\Sigma}^2\mathbf{V}^T,$$

using $\mathbf{U}^T\mathbf{U} = \mathbf{I}$ (orthogonality of $\mathbf{U}$) and $\mathbf{\Sigma}^T\mathbf{\Sigma} = \mathbf{\Sigma}^2 = \operatorname{diag}(\sigma_1^2, \ldots)$. Comparing with $\mathbf{S} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T$ identifies the columns of $\mathbf{V}$ as eigenvectors and $\lambda_i = \sigma_i^2/(n-1)$.
For the scores: $\mathbf{Z} = \mathbf{X}\mathbf{V} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T\mathbf{V} = \mathbf{U}\mathbf{\Sigma}$. □

Note 6.1 This theorem shows that PCA can be performed directly from the SVD of $\mathbf{X}$, avoiding the explicit formation of $\mathbf{S}$. Since $\mathbf{S} = \mathbf{X}^T\mathbf{X}/(n-1)$ squares the condition number, the SVD route is numerically more stable. In NumPy: `U, s, Vt = np.linalg.svd(X, full_matrices=False); Z = U * s`.

6.1.3 Dimensionality Reduction

Def 6.5 **(Rank-k Approximation via PCA)** The best rank-k approximation of $\mathbf{X}$ using PCA retains only the first k principal components:

$$\mathbf{X}_k = \mathbf{U}_k\mathbf{\Sigma}_k\mathbf{V}_k^T = \mathbf{Z}_k\mathbf{V}_k^T = \mathbf{X}\mathbf{V}_k\mathbf{V}_k^T,$$

where $\mathbf{U}_k, \mathbf{V}_k$ are the first k columns of $\mathbf{U}, \mathbf{V}$ and $\mathbf{\Sigma}_k = \operatorname{diag}(\sigma_1, \ldots, \sigma_k)$.

Thm 6.4 **(PCA is the Optimal Linear Dimensionality Reduction)** Among all rank-k approximations, $\mathbf{X}_k$ minimizes the reconstruction error:

$$\|\mathbf{X} - \mathbf{X}_k\|_F^2 = \sum_{i=k+1}^{r} \sigma_i^2 = (n-1)\sum_{i=k+1}^{r} \lambda_i,$$

and $\mathbf{X}_k$ is the best rank-k matrix approximation to $\mathbf{X}$ in the Frobenius norm.

Proof *Proof.* This is a direct consequence of the Eckart-Young-Mirsky theorem applied to $\mathbf{X}$: $\mathbf{X}_k$ is the best rank-k approximation and the error is $\|\mathbf{X} - \mathbf{X}_k\|_F^2 = \sum_{i>k} \sigma_i^2$. Converting via $\lambda_i = \sigma_i^2/(n-1)$ gives the formula in terms of eigenvalues. □

Def 6.6 **(Proportion of Variance Explained)** The *proportion of variance explained* (PVE) by the first k principal components is

$$\mathrm{PVE}_k = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^p \lambda_i} = \frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^p \sigma_i^2}.$$

A common criterion is to choose k such that $\mathrm{PVE}_k \geq 0.90$ or 0.95.

Def 6.7 **(Scree Plot)** A *scree plot* displays the eigenvalues $\lambda_1 \geq \cdots \geq \lambda_p$ (or singular values σ_i) in decreasing order. The "elbow" — the point where the values level off — suggests a natural cutoff for k. Components beyond the elbow likely represent noise rather than signal.

Note 6.2 The choice of k should also be guided by downstream tasks: visualization typically uses $k = 2$ or $k = 3$; clustering may need more components; the goal is to retain enough variance for the application while discarding noisy dimensions.

6.1.4 Statistical Interpretation

Def 6.8 **(Loadings)** The entries of the principal directions $\boldsymbol{v}_i$ are called *loadings*. The loading v_{ji} quantifies the contribution of the j-th original feature to the i-th principal component. When data are standardized (unit-variance features), v_{ji} equals the correlation between feature j and PC score $\boldsymbol{z}_i$.

Def 6.9 **(High-Dimensional PCA ($p \gg n$))** When $p \gg n$, forming the $p \times p$ covariance matrix $\mathbf{S}$ is prohibitively expensive. Instead, one works with the $n \times n$ Gram matrix $\mathbf{K} = \mathbf{X}\mathbf{X}^T$. If $\boldsymbol{u}_i$ is an eigenvector of $\mathbf{K}$ with eigenvalue σ_i^2, then $\boldsymbol{v}_i = \mathbf{X}^T\boldsymbol{u}_i/\sigma_i$ is the corresponding principal direction. This kernel trick has complexity $O(n^2p + n^3)$ vs. $O(np^2 + p^3)$ for the direct approach.

Note 6.3 **Limitations of PCA.** PCA assumes linear structure: it finds directions of high variance in the *original* feature space. If the data lies on a nonlinear manifold (e.g., a sphere), PCA may fail to capture its intrinsic geometry (Kernel PCA addresses this).
PCA is sensitive to outliers, since they strongly influence the variance and hence the principal directions.
PCA is unsupervised: it maximizes variance regardless of class labels. In classification tasks, directions that best separate classes may have low variance and be discarded by PCA (Linear Discriminant Analysis is supervised and addresses this).
The effectiveness of PCA depends on whether high-variance directions are truly informative. Rapid decay of singular values is a reliable indicator that PCA will achieve effective compression.

Example Consider a centered data matrix $\mathbf{X} \in \mathbb{R}^{100\times 3}$ with singular values $\sigma_1 = 10\sqrt{99}$, $\sigma_2 = 5\sqrt{99}$, $\sigma_3 = 0.5\sqrt{99}$. The covariance eigenvalues are $\lambda_1 = 100$, $\lambda_2 = 25$, $\lambda_3 = 0.25$.

- PVE for $k = 1$: $100/125.25 \approx 79.8\%$.
- PVE for $k = 2$: $125/125.25 \approx 99.8\%$.

The data is nearly coplanar: 99.8% of variance lies in a 2D subspace. A rank-2 approximation $\mathbf{X}_2$ has reconstruction error $\|\mathbf{X} - \mathbf{X}_2\|_F = |\sigma_3| = 0.5\sqrt{99} \approx 4.97$.

Note 6.4 The SVD $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ gives a geometric picture: $\mathbf{V}$ rotates the feature space to align the axes with principal directions; $\mathbf{\Sigma}$ reveals the variance captured by each direction; $\mathbf{U}$ contains the normalized principal component scores. The transformation $\mathbf{X} \mapsto \mathbf{X}\mathbf{V} = \mathbf{U}\mathbf{\Sigma}$ is simply a rigid rotation in feature space followed by reading off coordinates in the new basis.

Ex 6.1 **(PCA by hand)** Let $\mathbf{X} = \begin{pmatrix} 2 & 1 \\ -1 & 3 \\ -1 & -4 \end{pmatrix}$ (already centered: verify $\sum_i X_{i1} = \sum_i X_{i2} = 0$).

1. Compute $\mathbf{S} = \mathbf{X}^T\mathbf{X}/(n-1)$ with $n = 3$.
2. Find the eigenvalues and eigenvectors of $\mathbf{S}$. What are λ_1, λ_2 and the principal directions $\boldsymbol{v}_1, \boldsymbol{v}_2$?
3. Compute the principal component scores $\mathbf{Z} = \mathbf{X}\mathbf{V}$. Verify that $\mathbf{Z}^T\mathbf{Z}/(n-1)$ is diagonal.
4. Compute the SVD of $\mathbf{X}$ and verify that the right singular vectors match the principal directions from step 2.

Ex 6.2 **(PCA for dimensionality reduction)** Load the Iris dataset from `sklearn.datasets.load_iris`. The data matrix $\mathbf{X}$ has $n = 150$ samples and $p = 4$ features.

1. Center the data. Compute the SVD and plot the first two principal components as a scatter plot colored by species. Do the classes separate?
2. Plot the scree plot (singular values vs. index). How many components capture 95% of the variance?
3. Compute the loadings $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$. Which original features contribute most to each principal component? What is the biological interpretation?
4. Reconstruct the data from rank-k approximation for $k = 1, 2, 3, 4$. Plot the relative reconstruction error $\|\mathbf{X} - \mathbf{X}_k\|_F / \|\mathbf{X}\|_F$ vs. k.

Ex 6.3 **(High-dimensional PCA)** Generate a data matrix $\mathbf{X} \in \mathbb{R}^{n\times p}$ with $n = 50, p = 500$ from `np.random.randn`.

1. Compute the $p \times p$ covariance matrix $\mathbf{S} = \mathbf{X}^T\mathbf{X}/49$ and time its eigendecomposition. How long does it take?

2. Compute the $n \times n$ Gram matrix $\mathbf{K} = \mathbf{X}\mathbf{X}^T/49$ and time its eigendecomposition. Which is faster?

3. Recover the principal directions $\boldsymbol{v}_i = \mathbf{X}^T\boldsymbol{u}_i/\sigma_i$ from the Gram matrix eigenvectors. Verify these match the top eigenvectors of $\mathbf{S}$ (up to sign).

4. For $n \ll p$, how many nonzero eigenvalues does $\mathbf{S}$ have? Explain using the rank-nullity theorem.

6.2 Spectral Graph Theory and PageRank

A graph encodes pairwise relationships between objects. Spectral graph theory extracts quantitative information about connectivity, partitioning, and flow by studying the eigenvalues and eigenvectors of matrices associated with the graph. This section develops the theory of the graph Laplacian, connects its spectrum to graph structure via the Courant-Fischer theorem, applies spectral methods to graph partitioning and clustering, and culminates in the PageRank algorithm, which models the web as a Markov chain and identifies important pages as the dominant eigenvector of a specially constructed stochastic matrix.

6.2.1 Graph Matrices

Before studying eigenvalues, we need the matrices. The adjacency and degree matrices encode the raw connectivity, while the Laplacian combines them into a single operator that will carry the full spectral information about the graph.

Def 6.10 **(Graph)** A *weighted undirected graph* $G = (V, E)$ consists of a vertex set V with $n = |V|$ vertices and an edge set $E \subseteq \binom{V}{2}$ with $m = |E|$ edges. Each edge $(i, j) \in E$ carries a positive weight $w_{ij} = w_{ji} > 0$; for unweighted graphs $w_{ij} = 1$. The *degree* of vertex i is $d_i = \sum_{j:(i,j)\in E} w_{ij}$.

Ex 6.4 **(Graph degrees)** For the unweighted path graph P_5 on vertices $\{1, 2, 3, 4, 5\}$ with edges $\{1\text{–}2, 2\text{–}3, 3\text{–}4, 4\text{–}5\}$:

1. List the degree of each vertex. What is $\sum_i d_i$? Why does this always equal $2m$?

2. Repeat for the cycle C_5 (add edge 5–1). What changed?

3. For the complete graph K_n, what is d_i for all i? What is $\sum_i d_i$?

Def 6.11 **(Graph Matrices)** A graph G is associated with several matrices:

- **Adjacency matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$**: $A_{ij} = w_{ij}$ if $(i, j) \in E$, zero otherwise. Always symmetric.

- **Degree matrix** $\mathbf{D} \in \mathbb{R}^{n\times n}$: diagonal with $D_{ii} = d_i$.
- **Incidence matrix** $\mathbf{M} \in \mathbb{R}^{n\times m}$: for an arbitrary orientation of edges, $M_{ie} = +1$ if i is the head of edge e, $M_{ie} = -1$ if i is the tail, zero otherwise.
- **Graph Laplacian** $\mathbf{L} = \mathbf{D} - \mathbf{A} = \mathbf{M}\mathbf{M}^T \in \mathbb{R}^{n\times n}$.

Ex 6.5 **(Building graph matrices)** For the unweighted path P_3 on vertices $\{1, 2, 3\}$ with edges $\{1\text{–}2, 2\text{–}3\}$:

1. Write down $\mathbf{A}$, $\mathbf{D}$, and $\mathbf{L} = \mathbf{D} - \mathbf{A}$ explicitly.
2. Orient both edges from lower to higher vertex and write the incidence matrix $\mathbf{M} \in \mathbb{R}^{3\times 2}$. Compute $\mathbf{M}\mathbf{M}^T$ and verify it equals $\mathbf{L}$.
3. In NumPy, build the Laplacian for P_5 and K_4 using `numpy.diag` and `numpy.ones`. Verify $\mathbf{L}\mathbf{1} = \mathbf{0}$ for both.

6.2.2 The Graph Laplacian

The graph Laplacian is the central object of spectral graph theory. We establish two fundamental facts: it is positive semidefinite (with a proof via its quadratic form), and its spectrum directly encodes the graph's connectivity. These results together make the eigenvalues of $\mathbf{L}$ computable proxies for qualitative graph properties.

Def 6.12 **(Laplacian as a Discrete Derivative)** For any function $f : V \to \mathbb{R}$ (a vector $\boldsymbol{f} \in \mathbb{R}^n$), the Laplacian acts as

$$(\mathbf{L}\boldsymbol{f})_i = \sum_{j:(i,j)\in E} w_{ij}(f_i - f_j).$$

At each vertex i, the Laplacian measures how f_i deviates from the weighted average of its neighbors, making $\mathbf{L}$ the discrete analogue of the continuous Laplace operator $-\nabla^2$.

Ex 6.6 **(Laplacian as a smoother)** For the path graph P_4 on vertices $\{1, 2, 3, 4\}$, let $\boldsymbol{f} = (1, 2, 3, 4)^T$ (linear function) and $\boldsymbol{g} = (0, 1, 0, 1)^T$ (oscillating function).

1. Compute $\mathbf{L}\boldsymbol{f}$ and $\mathbf{L}\boldsymbol{g}$ by hand using the formula $(\mathbf{L}\boldsymbol{f})_i = \sum_{j\sim i}(f_i - f_j)$.
2. Which function has a larger $\mathbf{L}$-response? What does this say about the Laplacian as a "roughness" detector?
3. For the constant function $\boldsymbol{f} = \mathbf{1}$, compute $\mathbf{L}\mathbf{1}$ without any computation. Explain from the definition why this is always zero.

Thm 6.5 **(Graph Laplacian is Positive Semidefinite)** For any weighted graph G and any vector $\boldsymbol{x} \in \mathbb{R}^n$,

$$\boldsymbol{x}^T\mathbf{L}\boldsymbol{x} = \sum_{(i,j)\in E} w_{ij}(x_i - x_j)^2 \geq 0.$$

Therefore $\mathbf{L}$ is symmetric positive semidefinite, with $\lambda_1 = 0$ always.

Proof *Proof.* **Step 1: Expand using $\mathbf{L} = \mathbf{D} - \mathbf{A}$.**

$$\boldsymbol{x}^T \mathbf{L} \boldsymbol{x} = \boldsymbol{x}^T \mathbf{D} \boldsymbol{x} - \boldsymbol{x}^T \mathbf{A} \boldsymbol{x} = \sum_i d_i x_i^2 - \sum_{(i,j)\in E} 2 w_{ij} x_i x_j.$$

Step 2: Rewrite the degree sum. Since $d_i = \sum_{j:(i,j)\in E} w_{ij}$, we can write $\sum_i d_i x_i^2 = \sum_{(i,j)\in E} w_{ij}(x_i^2 + x_j^2)$, where each edge (i,j) contributes to both vertex i and vertex j.
Step 3: Complete the square. Combining the two terms:

$$\boldsymbol{x}^T \mathbf{L} \boldsymbol{x} = \sum_{(i,j)\in E} w_{ij}(x_i^2 + x_j^2 - 2x_i x_j) = \sum_{(i,j)\in E} w_{ij}(x_i - x_j)^2 \geq 0,$$

since each term $w_{ij} > 0$ and $(x_i - x_j)^2 \geq 0$. □

Note 6.5 The quadratic form $\boldsymbol{x}^T \mathbf{L} \boldsymbol{x} = \sum_{(i,j)\in E} w_{ij}(x_i - x_j)^2$ measures the total variation of $\boldsymbol{x}$ across the graph. A smooth signal (nearby vertices have similar values) gives a small quadratic form; a rough signal gives a large one. This is the discrete analogue of $\int |\nabla f|^2 \, d\mu$ in the continuous setting, and it connects the Laplacian to regularization (smoothing) operators.

Thm 6.6 **(Spectral Properties of the Laplacian)** Let G be a connected weighted graph with Laplacian $\mathbf{L}$ and eigenvalues $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$. Then:

1. $\lambda_1 = 0$ with eigenvector $\mathbf{1}/\sqrt{n}$, and the multiplicity of eigenvalue 0 equals the number of connected components of G.
2. $\lambda_2 > 0$ if and only if G is connected. This *Fiedler value* (algebraic connectivity) quantifies how well-connected G is.
3. $\lambda_n \leq 2 \max_i d_i$.
4. (Cheeger inequality) $\frac{\lambda_2}{2} \leq h(G) \leq \sqrt{2\lambda_2}$, where $h(G) = \min_{S \subseteq V} \frac{\mathrm{cut}(S,\bar{S})}{\min(\mathrm{vol}(S),\mathrm{vol}(\bar{S}))}$ is the Cheeger constant.

Proof *Proof.* **Step 1: 1 is in the null space.** $(\mathbf{L}\mathbf{1})_i = \sum_{j\sim i} w_{ij}(1-1) = 0$ for all i, so $\lambda_1 = 0$ with eigenvector $\mathbf{1}$.
Step 2: Null space and components. Since $\mathbf{L} \succeq 0$, $\boldsymbol{x} \in \ker(\mathbf{L})$ if and only if $\boldsymbol{x}^T \mathbf{L} \boldsymbol{x} = 0$, i.e., $\sum_{(i,j)\in E} w_{ij}(x_i - x_j)^2 = 0$. Since $w_{ij} > 0$, this forces $x_i = x_j$ for all edges (i,j). If G is connected, any such $\boldsymbol{x}$ must be constant, so $\dim \ker(\mathbf{L}) = 1$. For a graph with c components, $\boldsymbol{x}$ can take a different constant value on each component, giving $\dim \ker(\mathbf{L}) = c$.
Step 3: Fiedler value. By Step 2, G is connected $\Leftrightarrow \ker(\mathbf{L}) = \mathrm{span}(\mathbf{1}) \Leftrightarrow \lambda_2 > 0$.
Step 4: Upper bound. By Courant-Fischer (Thm 6.7), $\lambda_n = \max_{\boldsymbol{x}\neq\mathbf{0}} \boldsymbol{x}^T \mathbf{L} \boldsymbol{x} / \|\boldsymbol{x}\|^2$. For the standard basis vector $\boldsymbol{e}_i$, $\boldsymbol{e}_i^T \mathbf{L} \boldsymbol{e}_i = d_i$, so $\lambda_n \leq 2 \max_i d_i$ (a tighter bound uses the sum of degrees of adjacent pairs). The Cheeger inequality is stated without proof; it is proved in the theory of expander graphs. □

Note 6.6 The *Fiedler vector* $\boldsymbol{v}_2$ (eigenvector for λ_2) carries structural information beyond the eigenvalue itself. Its components embed the vertices on a line in a way that minimizes $\sum_{(i,j)\in E} w_{ij}(v_{2,i} - v_{2,j})^2$ subject to $\boldsymbol{v}_2 \perp \mathbf{1}$ and $\|\boldsymbol{v}_2\| = 1$. Vertices with similar Fiedler vector components tend to be well-connected to each other, making the sign of $\boldsymbol{v}_2$ a natural partition: positive components form one cluster, negative components form another.

Ex 6.7 **(Laplacian spectrum by hand and numerically)**

1. For the cycle graph C_4 on vertices $\{1, 2, 3, 4\}$: write down $\mathbf{L}$ and compute all four eigenvalues. (For C_n, $\lambda_k = 2(1 - \cos(2\pi k/n))$.) What is λ_2? How does it compare to K_4?

2. Use `scipy.linalg.eigh` to compute the full spectrum of $\mathbf{L}$ for P_5, C_5, K_5, and the path with a weak middle edge ($w_{3,4} = 0.01$, all others 1). Rank them by λ_2. Which is most and least connected?

3. Using the Matrix Tree Theorem $\tau(G) = \frac{1}{n}\prod_{i=2}^{n} \lambda_i$: compute the number of spanning trees of C_4 from its eigenvalues and verify by listing all spanning trees.

Def 6.13 **(Normalized Laplacians)** Two normalized versions of the Laplacian correct for the bias toward high-degree vertices:

- *Symmetric normalized Laplacian*: $\mathbf{L}_{\text{sym}} = \mathbf{D}^{-1/2}\mathbf{L}\mathbf{D}^{-1/2} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$.
- *Random-walk Laplacian*: $\mathbf{L}_{\text{rw}} = \mathbf{D}^{-1}\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1}\mathbf{A}$.

Both have eigenvalues in $[0, 2]$. They share the same eigenvalues: if $\boldsymbol{v}$ is an eigenvector of $\mathbf{L}_{\text{sym}}$ with eigenvalue λ, then $\mathbf{D}^{-1/2}\boldsymbol{v}$ is an eigenvector of $\mathbf{L}_{\text{rw}}$ with the same λ.

Note 6.7 The random-walk Laplacian $\mathbf{L}_{\text{rw}} = \mathbf{I} - \mathbf{P}$ where $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}$ is the transition matrix for the simple random walk (see §6.5). Its eigenvalues $\lambda_i(\mathbf{L}_{\text{rw}}) = 1 - \lambda_i(\mathbf{P})$ directly measure mixing rates: the spectral gap of $\mathbf{P}$ equals the Fiedler value of $\mathbf{L}_{\text{rw}}$. Using $\mathbf{L}_{\text{sym}}$ (which is symmetric) allows stable symmetric eigensolvers even though $\mathbf{P}$ is not symmetric for non-regular graphs.

Ex 6.8 **(Normalized Laplacians and irregular graphs)** For the star graph $K_{1,4}$ (one hub connected to four leaves):

1. Compute $\mathbf{L}$, $\mathbf{L}_{\text{sym}}$, and $\mathbf{L}_{\text{rw}}$ by hand.

2. Find the eigenvalues of $\mathbf{L}$ and $\mathbf{L}_{\text{sym}}$ using `scipy.linalg.eigh`. Do the two matrices have the same eigenvalues? The same eigenvectors?

3. The hub vertex has degree 4, the leaves degree 1. How does $\mathbf{L}_{\text{sym}}$ compensate for this imbalance relative to $\mathbf{L}$?

6.2.3 Courant-Fischer and Graph Cuts

The Fiedler value λ_2 is the key spectral quantity for graph connectivity, but to use it for graph partitioning we need a precise characterization of λ_2 as a variational problem. The Courant-Fischer theorem provides this, and the resulting variational formula reveals a deep connection between the spectral relaxation of the NP-hard graph bisection problem and the Fiedler vector.

Thm 6.7 **(Courant-Fischer Min-Max)** Let $\mathbf{B} \in \mathbb{R}^{n\times n}$ be symmetric with eigenvalues $\lambda_1 \le \lambda_2 \le \cdots \le \lambda_n$ and orthonormal eigenvectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n$. For each $k = 1, \ldots, n$:

$$\lambda_k = \min_{\substack{\mathcal{S}\subseteq\mathbb{R}^n \\ \dim(\mathcal{S})=k}} \max_{\boldsymbol{x}\in\mathcal{S},\, \boldsymbol{x}\neq\mathbf{0}} \frac{\boldsymbol{x}^T\mathbf{B}\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}}.$$

In particular, $\lambda_1 = \min_{\boldsymbol{x}\neq\mathbf{0}} \frac{\boldsymbol{x}^T\mathbf{B}\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}}$ and $\lambda_n = \max_{\boldsymbol{x}\neq\mathbf{0}} \frac{\boldsymbol{x}^T\mathbf{B}\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}}$.

Proof *Proof.* **Step 1: Lower bound.** Fix any k-dimensional subspace $\mathcal{S}$. It must intersect $\operatorname{span}\{\boldsymbol{v}_k, \boldsymbol{v}_{k+1}, \ldots, \boldsymbol{v}_n\}$ nontrivially (both spaces together span at least n dimensions, and their sum has dimension at most n, so they must share a nonzero vector). Take any $\boldsymbol{x} \neq \mathbf{0}$ in this intersection; writing $\boldsymbol{x} = \sum_{i=k}^n c_i\boldsymbol{v}_i$:

$$\frac{\boldsymbol{x}^T\mathbf{B}\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}} = \frac{\sum_{i=k}^n \lambda_i c_i^2}{\sum_{i=k}^n c_i^2} \ge \lambda_k.$$

So $\max_{\boldsymbol{x}\in\mathcal{S}} \frac{\boldsymbol{x}^T\mathbf{B}\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}} \ge \lambda_k$ for every $\mathcal{S}$, giving $\min_{\mathcal{S}} \max \ge \lambda_k$.
Step 2: Achievability. Take $\mathcal{S}^* = \operatorname{span}\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$. For any $\boldsymbol{x} \in \mathcal{S}^*$, $\boldsymbol{x} = \sum_{i=1}^k c_i\boldsymbol{v}_i$, so $\frac{\boldsymbol{x}^T\mathbf{B}\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}} = \frac{\sum_{i=1}^k \lambda_i c_i^2}{\sum c_i^2} \le \lambda_k$. The maximum over $\boldsymbol{x} \in \mathcal{S}^*$ is exactly λ_k (achieved by $\boldsymbol{x} = \boldsymbol{v}_k$). Therefore $\min_{\mathcal{S}} \max = \lambda_k$. □

Note 6.8 Applied to the graph Laplacian $\mathbf{L}$ with the constraint $\boldsymbol{x} \perp \mathbf{1}$ (since $\boldsymbol{v}_1 = \mathbf{1}/\sqrt{n}$ is the trivial eigenvector), Courant-Fischer gives:

$$\lambda_2 = \min_{\boldsymbol{x}\perp\mathbf{1},\, \boldsymbol{x}\neq\mathbf{0}} \frac{\boldsymbol{x}^T\mathbf{L}\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}} = \min_{\boldsymbol{x}\perp\mathbf{1},\, \boldsymbol{x}\neq\mathbf{0}} \frac{\displaystyle\sum_{(i,j)\in E} w_{ij}(x_i - x_j)^2}{\displaystyle\sum_i x_i^2}.$$

The Fiedler vector $\boldsymbol{v}_2$ is the smooth, zero-mean signal that maximally respects the graph structure.

Ex 6.9 **(Courant-Fischer in practice)** For the path graph P_5:

1. Use Thm 6.7 to find λ_2 by minimizing the Rayleigh quotient $\frac{\boldsymbol{x}^T\mathbf{L}\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}}$ numerically over vectors $\boldsymbol{x} \perp \mathbf{1}$. Use `scipy.optimize.minimize` with a projection step to enforce the constraint.

2. Compare to `scipy.linalg.eigh(L)`'s second eigenvalue. They should match.

3. The barbell graph consists of two copies of K_3 joined by a single edge. Build it in NumPy, compute λ_2, and plot the Fiedler vector. What does the Fiedler vector reveal about the graph structure?

Def 6.14 **(Graph Cut)** For a partition of V into disjoint sets A and $B = V \setminus A$, the *cut* is $\mathrm{cut}(A, B) = \sum_{i \in A, j \in B} w_{ij}$. Two normalized cut objectives are:

- *RatioCut*: $\mathrm{RatioCut}(A, B) = \mathrm{cut}(A, B) \left(\frac{1}{|A|} + \frac{1}{|B|} \right)$.
- *NCut (Normalized Cut)*: $\mathrm{NCut}(A, B) = \mathrm{cut}(A, B) \left(\frac{1}{\mathrm{vol}(A)} + \frac{1}{\mathrm{vol}(B)} \right)$, where $\mathrm{vol}(A) = \sum_{i \in A} d_i$.

Both objectives are NP-hard to minimize over all partitions.

Thm 6.8 **(Spectral Relaxation of RatioCut)** The RatioCut minimization over binary partitions relaxes to: find $\boldsymbol{x} \perp \mathbf{1}$, $\|\boldsymbol{x}\| = \sqrt{n}$, minimizing $\boldsymbol{x}^T \mathbf{L} \boldsymbol{x}$. The solution is the Fiedler vector $\boldsymbol{v}_2$.

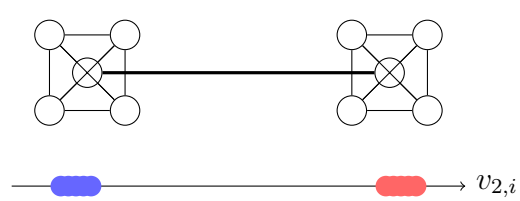
Proof *Proof.* **Step 1: Binary formulation.** For partition (A, B), define the indicator vector $\mathbf{h} \in \mathbb{R}^n$ by $h_i = \sqrt{|B|/|A|}$ if $i \in A$ and $h_i = -\sqrt{|A|/|B|}$ if $i \in B$. A direct calculation shows $\mathbf{h} \perp \mathbf{1}$, $\|\mathbf{h}\|^2 = n$, and

$$\mathbf{h}^T \mathbf{L} \mathbf{h} = \mathrm{cut}(A, B) \left(\frac{1}{|A|} + \frac{1}{|B|} \right) \cdot n = n \, \mathrm{RatioCut}(A, B).$$

Step 2: Relaxation. The NP-hard problem $\min_{\mathbf{h} \text{ binary}} \mathbf{h}^T \mathbf{L} \mathbf{h}$ relaxes to $\min_{\boldsymbol{x} \perp \mathbf{1}, \|\boldsymbol{x}\| = \sqrt{n}} \boldsymbol{x}^T \mathbf{L} \boldsymbol{x}$, allowing $\boldsymbol{x}$ to take continuous values. By Courant-Fischer (Thm 6.7), the minimizer is $\boldsymbol{x} = \sqrt{n} \, \boldsymbol{v}_2$.
Step 3: Round to partition. From the relaxed solution $\boldsymbol{v}_2$, recover a partition by thresholding: assign vertex i to A if $v_{2,i} \geq 0$ and to B otherwise. □

Note 6.9 The spectral clustering algorithm uses this approach. Build the graph, compute the k smallest eigenvectors $\boldsymbol{v}_2, \ldots, \boldsymbol{v}_{k+1}$ of $\mathbf{L}$ (or $\mathbf{L}_{\mathrm{sym}}$ for the NCut objective), form the matrix $\mathbf{Z} = [\boldsymbol{v}_2 \mid \cdots \mid \boldsymbol{v}_{k+1}] \in \mathbb{R}^{n \times k}$, and cluster the rows of $\mathbf{Z}$ using k-means. The eigenvectors map vertices to a low-dimensional space where Euclidean clustering captures graph community structure.

Example Barbell Graph and Fiedler VectorThe barbell graph consists of two copies of K_5 joined by a single bridge edge.



Fiedler vector embedding

The Fiedler value λ_2 is very small (the bridge edge is a bottleneck), and the Fiedler vector takes approximately $-1/\sqrt{5}$ on the left clique and $+1/\sqrt{5}$ on the right, cleanly exposing the natural partition.

Ex 6.10 **(Spectral clustering on circles)** Generate $n = 200$ points sampled from two concentric circles using `sklearn.datasets.make_circles(n_samples=200, noise=0.05, factor=0.4)`.

1. Apply standard k-means ($k = 2$) directly to the 2D coordinates. Does it succeed?
2. Construct a Gaussian similarity graph: $w_{ij} = \exp(-\|x_i - x_j\|^2/(2\sigma^2))$ for $\sigma = 0.3$; zero out edges with $\|x_i - x_j\| > 3\sigma$.
3. Build $\mathbf{L}_{\text{sym}}$, compute its two smallest eigenvectors, and run k-means on the rows of $[\boldsymbol{v}_1, \boldsymbol{v}_2]$. Compare to the direct k-means result.
4. Plot λ_2 vs. $\sigma \in \{0.1, 0.2, 0.3, 0.5, 1.0\}$. Which σ gives the best-separated clusters?

6.2.4 Random Walks and Spectral Gap

The matrix $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}$ defines the transition probabilities for a simple random walk on the graph: from vertex i, move to neighbor j with probability w_{ij}/d_i. This is a Markov chain (see §6.5 for the full theory), and the spectral properties of the Laplacian directly control its mixing behavior.

Def 6.15 **(Random Walk on a Graph)** The *simple random walk* on G is the Markov chain with column-stochastic transition matrix

$$\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}, \qquad P_{ij} = \frac{w_{ij}}{d_j} \quad \text{if } (i,j) \in E, \quad 0 \text{ otherwise.}$$

The matrix $\mathbf{P}$ is related to the random-walk Laplacian by $\mathbf{L}_{\text{rw}} = \mathbf{I} - \mathbf{P}$.

Note 6.10 For a connected, non-bipartite graph, the random walk chain is irreducible and aperiodic (ergodic in the sense of Def 6.33). By the Perron-Frobenius theorem (Thm 6.9), it has a unique stationary distribution. For the simple random walk, detailed balance holds with $\boldsymbol{\pi} = \boldsymbol{d}/\|\boldsymbol{d}\|_1$ where $\boldsymbol{d} = (d_1, \ldots, d_n)^T$: in this sense the walk is reversible. For a d-regular graph, $\boldsymbol{\pi} = \mathbf{1}/n$ (uniform distribution). The spectral gap of $\mathbf{P}$ equals the Fiedler value $\lambda_2(\mathbf{L}_{\text{rw}})$, so the mixing time $k_\varepsilon \approx \log(n)/\lambda_2$ is determined entirely by the graph's spectral connectivity.

Ex 6.11 **(Random walk mixing on graphs)** For the path P_{10}, cycle C_{10}, and complete graph K_{10}:

1. Build $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}$ and compute the spectral gap $\delta = 1 - |\lambda_2(\mathbf{P})|$ for each.
2. Simulate the random walk for 200 steps from $\boldsymbol{x}^{(0)} = \boldsymbol{e}_1$. Plot $\|\boldsymbol{x}^{(k)} - \boldsymbol{\pi}\|_1$ vs. k on a semi-log scale.
3. The theoretical mixing time is $k_\varepsilon \approx \log(n)/\delta$ for $\varepsilon = 0.01$. How does the empirical mixing time compare across the three graphs?

6.2.5 PageRank

PageRank, developed by Brin and Page at Google, measures the importance of web pages by modeling a random surfer navigating the web graph. Its mathematical foundation is the Perron-Frobenius theorem applied to a specially constructed column-stochastic matrix. The algorithm reduces to power iteration on the random walk chain, connecting web search to the spectral graph theory developed in this section.

Def 6.16 **(Web Graph and Random Surfer)** The web is modeled as a directed graph $G = (V, E)$ where V is the set of N pages and $(p_j, p_i) \in E$ if page p_j links to page p_i. Let $L(p_j)$ denote the number of outbound links from p_j. The *random surfer* navigates by: at each step, clicking a uniformly random outbound link from the current page (or, if the current page has no outbound links, jumping uniformly to any page).

Ex 6.12 **(Web graph modeling)** For the 4-page web graph with edges $P_1 \to P_2$, $P_1 \to P_3$, $P_2 \to P_4$, $P_3 \to P_2$, $P_3 \to P_4$, $P_4 \to P_1$:

1. Draw the directed graph. Which pages are most-linked-to?
2. Identify any dangling nodes (pages with $L(p_j) = 0$). What should the random surfer do at a dangling node?
3. Without any computation, rank the four pages by expected importance based on the graph structure. Record your guess to compare against the computed PageRank.

Def 6.17 **(Hyperlink and Stochastic Matrices)** Define the *hyperlink matrix* $\mathbf{H} \in \mathbb{R}^{N \times N}$ by $H_{ij} = 1/L(p_j)$ if p_j links to p_i and $L(p_j) > 0$, zero otherwise. Since columns for dangling nodes (where $L(p_j) = 0$) sum to zero, $\mathbf{H}$ is not stochastic. The *stochastic matrix* $\mathbf{S}$ fixes this:

$$S_{ij} = \begin{cases} 1/L(p_j) & \text{if } (p_j, p_i) \in E \text{ and } L(p_j) > 0, \\ 1/N & \text{if } L(p_j) = 0 \text{ (dangling node column)}, \\ 0 & \text{otherwise.} \end{cases}$$

Every column of $\mathbf{S}$ sums to 1, so $\mathbf{S}$ is column-stochastic.

Note 6.11 The basic PageRank equation $\mathbf{r} = \mathbf{S}\mathbf{r}$ requires $\mathbf{S}$ to be irreducible and primitive to guarantee a unique stationary distribution. However, $\mathbf{S}$ can fail both conditions: a set of pages that link only among themselves (a *rank sink*) can trap the surfer. Even with dangling node corrections, $\mathbf{S}$ may have multiple eigenvalues equal to 1, making the stationary distribution non-unique and power iteration non-convergent.

Thm 6.9 **(Perron-Frobenius Theorem)** Let $\mathbf{G} \in \mathbb{R}^{n \times n}$ be a matrix with all positive entries ($G_{ij} > 0$). Then:

1. $\mathbf{G}$ has a unique largest eigenvalue $\rho > 0$ (the *Perron root*) with $\rho > |\lambda_j|$ for all other eigenvalues λ_j.

2. The corresponding eigenvector $\boldsymbol{v}_1$ (the *Perron vector*) has all strictly positive entries and is unique up to scaling.

3. ρ is a simple eigenvalue.

If $\mathbf{G}$ is nonnegative, irreducible, and primitive, the same three conclusions hold.

Proof *Proof.* **Step 1: Existence via Brouwer.** Define the simplex $\Delta = \{\boldsymbol{x} \geq \mathbf{0} : \|\boldsymbol{x}\|_1 = 1\}$ and the normalized map $T(\boldsymbol{x}) = \mathbf{G}\boldsymbol{x}/\|\mathbf{G}\boldsymbol{x}\|_1$. Since $\mathbf{G} > 0$, T maps Δ into its interior and is continuous, so Brouwer's fixed-point theorem gives a $\boldsymbol{v} > 0$ with $T(\boldsymbol{v}) = \boldsymbol{v}$, i.e., $\mathbf{G}\boldsymbol{v} = \rho\boldsymbol{v}$ for $\rho = \|\mathbf{G}\boldsymbol{v}\|_1 > 0$.
Step 2: Dominance $\rho > |\lambda_j|$. For any eigenvalue λ with eigenvector $\boldsymbol{w}$, let i^* maximize $|w_{i^*}| = \|\boldsymbol{w}\|_\infty$. Then $|\lambda||w_{i^*}| = |(\mathbf{G}\boldsymbol{w})_{i^*}| \leq \sum_j G_{i^*j}|w_j| < |w_{i^*}|\sum_j G_{i^*j} \leq |w_{i^*}|\rho$ (using $G > 0$ and $\rho =$ spectral radius). This gives $|\lambda| < \rho$ strictly (since all $G_{ij} > 0$ prevents equality in the second step unless $\boldsymbol{w}$ is a multiple of $\boldsymbol{v}$).
Step 3: Uniqueness and simplicity. Since ρ is strictly larger than all other eigenvalues in magnitude, its eigenspace is one-dimensional and ρ is algebraically simple. □

Def 6.18 **(Google Matrix)** The *Google matrix* introduces a *damping factor* $d \in (0, 1)$ (typically $d = 0.85$): with probability d the surfer follows a random link, and with probability $1 - d$ the surfer teleports to a uniformly random page. This gives

$$\mathbf{G} = d\,\mathbf{S} + \frac{1-d}{N}\,\mathbf{J},$$

where $\mathbf{J} = \mathbf{1}\mathbf{1}^T$ is the $N \times N$ all-ones matrix. The PageRank vector $\mathbf{r}$ is the unique stationary distribution of $\mathbf{G}$: $\mathbf{G}\mathbf{r} = \mathbf{r}$.

Thm 6.10 **(PageRank Is Well-Posed)** The Google matrix $\mathbf{G} = d\mathbf{S} + \frac{1-d}{N}\mathbf{J}$ is column-stochastic, irreducible, and primitive for any $d \in (0, 1)$ and any $\mathbf{S}$. By the Perron-Frobenius theorem (Thm 6.9), $\mathbf{G}$ has a unique positive stationary distribution $\mathbf{r}$ with $\sum_i r_i = 1$.

Proof *Proof.* **Step 1: Column-stochastic.** Since $\mathbf{S}$ is column-stochastic and $\frac{1}{N}\mathbf{J}$ has all columns equal to $\frac{1}{N}\mathbf{1}$ (which sums to 1), the convex combination $\mathbf{G} = d\mathbf{S} + (1-d)\frac{\mathbf{J}}{N}$ is also column-stochastic.
Step 2: All entries positive. The term $\frac{1-d}{N}\mathbf{J}$ has all entries equal to $\frac{1-d}{N} > 0$ (since $d < 1$). Adding any nonnegative matrix $d\mathbf{S} \geq 0$ preserves positivity, so all entries of $\mathbf{G}$ are at least $\frac{1-d}{N} > 0$.
Step 3: Perron-Frobenius applies. Since $\mathbf{G}$ has all positive entries, Thm 6.9 applies: the Perron root is $\rho = 1$ (column-stochastic implies $\mathbf{1}^T\mathbf{G} = \mathbf{1}^T$, so $\rho = 1$), it is simple, and the Perron vector $\mathbf{r} > 0$ is unique up to scaling. Normalizing to $\|\mathbf{r}\|_1 = 1$ gives the unique PageRank vector. □

Note 6.12 The PageRank power iteration $\mathbf{r}^{(k+1)} = \mathbf{G}\mathbf{r}^{(k)}$ does not require forming $\mathbf{G}$ explicitly (which is dense). Using $\mathbf{J}\mathbf{r}^{(k)} = \mathbf{1}$ (since $\|\mathbf{r}^{(k)}\|_1 = 1$):

$$\mathbf{r}^{(k+1)} = d\,\mathbf{S}\mathbf{r}^{(k)} + \frac{1-d}{N}\mathbf{1}.$$

Only the sparse $\mathbf{S}$ multiply is needed. The convergence rate is $d \cdot |\lambda_2(\mathbf{S})|$, and since all eigenvalues of $\mathbf{G}$ beyond the first satisfy $|\lambda_i(\mathbf{G})| \leq d$, the spectral gap is at least $1-d = 0.15$ for $d = 0.85$. With $n \sim 10^9$ pages, about 130 iterations suffice.

Ex 6.13 **(PageRank by hand and by power iteration)** For the 4-page web graph from the first exercise in this subsubsection:

1. Construct $\mathbf{S}$ (column-stochastic). Are there dangling nodes?
2. Set $d = 0.85$ and build $\mathbf{G}$. Verify all columns sum to 1 and all entries are positive.
3. Run 10 steps of $\mathbf{r}^{(k+1)} = d\mathbf{S}\mathbf{r}^{(k)} + \frac{1-d}{N}\mathbf{1}$ from $\mathbf{r}^{(0)} = \frac{1}{4}\mathbf{1}$. Report $\mathbf{r}^{(k)}$ at steps 1, 5, and 10.
4. Solve for the exact $\mathbf{r}$ using `numpy.linalg.eig(G)` and compare to your power iteration result. Which page is most important? Does it match your initial guess?

Ex 6.14 **(Convergence rate and damping factor)** For the 4-page graph above:

1. Compute all eigenvalues of $\mathbf{G}$ for $d = 0.85$. What is the spectral gap $1 - |\lambda_2(\mathbf{G})|$?
2. Repeat for $d \in \{0.5, 0.7, 0.85, 0.95\}$. Plot the spectral gap vs. d. For which d does power iteration converge fastest?
3. For a disconnected graph with two components (no cross-links), $\mathbf{S}$ has eigenvalue 1 with multiplicity 2. Explain why the teleportation term $\frac{1-d}{N}\mathbf{J}$ is essential: what happens to the second eigenvalue of $\mathbf{G}$ as $d \to 1$?

Ex 6.15 **(Large-scale PageRank implementation)** Implement the efficient PageRank iteration using sparse matrices:

1. Generate a random directed graph on $N = 1000$ nodes using `networkx.gnm_random_graph(1000 5000, directed=True)`. Identify and handle dangling nodes.
2. Store $\mathbf{S}$ as a SciPy CSR sparse matrix. Implement the iteration $\mathbf{r}^{(k+1)} = d\mathbf{S}\mathbf{r}^{(k)} + \frac{1-d}{N}\mathbf{1}$ and run until $\|\mathbf{r}^{(k+1)} - \mathbf{r}^{(k)}\|_1 < 10^{-8}$.
3. Plot the ℓ^1 convergence gap vs. iteration on a semi-log scale. Measure the empirical convergence rate and compare to the theoretical bound $d = 0.85$.
4. Report the top-10 pages by PageRank. How does the ranking change when d is varied from 0.5 to 0.95?

6.3 Linear Operators and the Poisson Equation

This subsection is a supplement to Lab 4, which asks you to solve the Poisson heat equation numerically. The lab will have you build a large sparse matrix and solve a linear system with it. Before doing that, it is worth understanding where that matrix comes from and why it has the properties it does. We will see that it is a finite-dimensional approximation of a differential operator, and that studying its eigenvalues gives insight into both the physics of the problem and the difficulty of solving it numerically. Spectral theory, which you have already seen for matrices, extends to this setting in a way that is both natural and surprising.

6.3.1 From Differential Operators to Matrices

Def 6.19 **(Linear Operator)** Let X and Y be vector spaces of functions. A map $\mathcal{A} : X \to Y$ is a *linear operator* if for all $u, v \in X$ and scalars α:

$$\mathcal{A}(\alpha u + v) = \alpha\, \mathcal{A}u + \mathcal{A}v.$$

Every matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ is a linear operator from $\mathbb{R}^n$ to $\mathbb{R}^m$. Differential operators such as $\frac{d}{dx}$ and ∇^2 are linear operators on function spaces (e.g., on the space of twice-differentiable functions on $[0, 1]$).

The central idea of numerical methods for PDEs is *discretization*: replace an infinite-dimensional operator by a finite matrix that approximates it on a grid. The accuracy of the approximation improves as the grid is refined (grid spacing $h \to 0$), and the resulting matrices get larger ($n \to \infty$). Everything in Lab 4 is an instance of this: the $n^2 \times n^2$ matrix you will assemble is the n-point discretization of the continuous Laplacian $-\nabla^2$.

Def 6.20 **(Finite Difference Approximation)** The *centered second-difference* approximation to $u''(x)$ at a grid point x_k with spacing h is

$$u''(x_k) \approx \frac{u(x_{k-1}) - 2u(x_k) + u(x_{k+1})}{h^2}.$$

To see why, expand u at $x_k \pm h$ via Taylor series:

$$u(x_k + h) = u(x_k) + hu'(x_k) + \tfrac{h^2}{2}u''(x_k) + \tfrac{h^3}{6}u'''(x_k) + O(h^4),$$

$$u(x_k - h) = u(x_k) - hu'(x_k) + \tfrac{h^2}{2}u''(x_k) - \tfrac{h^3}{6}u'''(x_k) + O(h^4).$$

Adding these two equations cancels the odd-order terms, leaving

$$u(x_k + h) + u(x_k - h) = 2u(x_k) + h^2u''(x_k) + O(h^4).$$

Rearranging gives the formula above, with error $O(h^2)$. This is called *second-order accuracy*: halving h reduces the error by a factor of four.

6.3.2 The Discrete Laplacian

Def 6.21 **(1-D Discrete Laplacian)** On $[0,1]$ with n interior grid points $x_k = kh$, $h = 1/(n+1)$, and homogeneous Dirichlet boundary conditions $u(0) = u(1) = 0$, the operator $-d^2/dx^2$ is discretized by applying the finite difference formula at each interior point. The result is the tridiagonal linear system $\mathbf{Tu} = \mathbf{f}$, where

$$\mathbf{T} = \frac{1}{h^2}\begin{pmatrix} 2 & -1 & & \\ -1 & 2 & -1 & \\ & \ddots & \ddots & \ddots \\ & & -1 & 2 \end{pmatrix} \in \mathbb{R}^{n\times n}.$$

The -1 off-diagonals come from the neighbor terms $u(x_{k\pm1})$; the 2 on the diagonal comes from $-2u(x_k)$; and the $1/h^2$ scaling comes from the approximation formula. The boundary values $u(0) = u(1) = 0$ are incorporated by simply omitting them from the system.

Thm 6.11 **(Eigenvalues of the 1-D Discrete Laplacian)** The matrix $\mathbf{T} \in \mathbb{R}^{n\times n}$ has eigenvalues

$$\lambda_k = \frac{4}{h^2}\sin^2\left(\frac{k\pi}{2(n+1)}\right), \quad k = 1, \ldots, n,$$

with corresponding eigenvectors $(\boldsymbol{v}_k)_j = \sin(jk\pi h)$, $j = 1, \ldots, n$. As $n \to \infty$, these eigenvalues satisfy $\lambda_k \to k^2\pi^2$, which are exactly the eigenvalues of the continuous operator $-d^2/dx^2$ on $[0,1]$.

Proof *Proof.* We verify by direct substitution that $\mathbf{T}\boldsymbol{v}_k = \lambda_k \boldsymbol{v}_k$.

Step 1: Compute the j-th entry of $\mathbf{T}\boldsymbol{v}_k$.
The tridiagonal structure gives

$$(\mathbf{T}\boldsymbol{v}_k)_j = \frac{1}{h^2}\big(-\sin((j-1)k\pi h) + 2\sin(jk\pi h) - \sin((j+1)k\pi h)\big).$$

Step 2: Apply a trigonometric identity.
Using the sum-to-product identity $-\sin(\theta-\phi) + 2\sin\theta - \sin(\theta+\phi) = 2\sin\theta(1-\cos\phi)$ with $\theta = jk\pi h$ and $\phi = k\pi h$, the expression becomes

$$(\mathbf{T}\boldsymbol{v}_k)_j = \frac{2(1-\cos(k\pi h))}{h^2}\sin(jk\pi h).$$

Step 3: Simplify using the half-angle identity.
Since $1 - \cos\phi = 2\sin^2(\phi/2)$, we get

$$(\mathbf{T}\boldsymbol{v}_k)_j = \frac{4}{h^2}\sin^2\left(\frac{k\pi h}{2}\right)\cdot(\boldsymbol{v}_k)_j = \lambda_k\,(\boldsymbol{v}_k)_j.$$

This confirms $\boldsymbol{v}_k$ is an eigenvector with eigenvalue λ_k.
Step 4: The continuous limit.

As $h \to 0$, use $\sin(\theta) \approx \theta$ for small θ:

$$\lambda_k = \frac{4}{h^2}\sin^2\left(\frac{k\pi h}{2}\right) \approx \frac{4}{h^2} \cdot \frac{k^2\pi^2 h^2}{4} = k^2\pi^2.$$

The discrete eigenvalues converge to the continuous eigenvalues $k^2\pi^2$. □

Def 6.22 **(2-D Discrete Laplacian (Poisson Equation))** On the unit square $[0,1]^2$ with an $n \times n$ interior grid ($N = n^2$ unknowns), the five-point finite difference stencil approximates $-\nabla^2 u = f$ at each grid point (x_i, y_j) as

$$\frac{1}{h^2}\big(4u_{ij} - u_{i+1,j} - u_{i-1,j} - u_{i,j+1} - u_{i,j-1}\big) = f_{ij}.$$

After flattening the 2-D grid to a 1-D index $k = j + n \cdot i$, this becomes the block tridiagonal system $\mathbf{Ku} = \mathbf{f}$ where

$$\mathbf{K} = \frac{1}{h^2}\begin{pmatrix} \mathbf{C} & -\mathbf{I} & & \\ -\mathbf{I} & \mathbf{C} & -\mathbf{I} & \\ & \ddots & \ddots & \ddots \\ & & -\mathbf{I} & \mathbf{C} \end{pmatrix}, \qquad \mathbf{C} = \text{tridiag}(-1,\, 4,\, -1) \in \mathbb{R}^{n\times n}.$$

Each diagonal block $\mathbf{C}$ encodes the x-direction neighbors of a row of grid points. The off-diagonal $-\mathbf{I}$ blocks connect rows that are adjacent in y, which appear n positions apart in the flattened index.

Note 6.13 The matrix $\mathbf{K}$ is symmetric positive definite (Section 2.3), so the Poisson system has a unique solution. In practice, $\mathbf{K}$ is extremely sparse (at most 5 nonzeros per row out of $N = n^2$), so it is stored and solved using `scipy.sparse`, exactly as you will do in Lab 4. A direct sparse solve costs $O(N^{3/2})$ for 2-D problems, far better than the $O(N^3)$ dense cost.

Ex 6.16

1. Using `scipy.sparse.diags`, build the 1-D discrete Laplacian $\mathbf{T}$ for $n = 50$ interior points. Compute its eigenvalues with `np.linalg.eigvalsh` and compare them to the analytical formula $\lambda_k = \frac{4}{h^2}\sin^2(k\pi h/2)$ for $k = 1, \ldots, 50$. Plot both sets on the same axes.

2. Verify that the eigenvectors satisfy $(\boldsymbol{v}_k)_j = \sin(jk\pi h)$ by calling `np.linalg.eigh(T.toarray())` and plotting the first three eigenvectors. Do they look like $\sin(k\pi x)$?

3. Extend the Lab 4 code to impose a non-homogeneous Dirichlet boundary condition $u = g$ on the boundary of $[0,1]^2$ by modifying the right-hand side $\mathbf{f}$. (*Hint: for each boundary node k, replace row k of $\mathbf{K}$ with the corresponding identity row and set $f_k = g(x_k)$.*)

6.3.3 Operator Norms and Boundedness

For matrices, the operator norm $\|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A})$ is always finite. The same is not true for differential operators on function spaces. The distinction between *bounded* and *unbounded* operators is one of the fundamental differences between finite and infinite dimensions, and it has direct consequences for the numerical conditioning of PDE-based linear systems.

Def 6.23 **(Bounded Linear Operator)** A linear operator $\mathcal{A} : X \to Y$ between normed function spaces is *bounded* if there exists a constant $C < \infty$ such that

$$\|\mathcal{A}u\|_Y \leq C \|u\|_X \quad \text{for all } u \in X.$$

The *operator norm* is defined analogously to the matrix case:

$$\|\mathcal{A}\|_{\text{op}} = \sup_{\|u\|_X = 1} \|\mathcal{A}u\|_Y.$$

Every matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ is a bounded operator with $\|\mathbf{A}\|_{\text{op}} = \|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A})$.

Thm 6.12 **(The Differentiation Operator is Unbounded)** The operator $\mathcal{D} = -d^2/dx^2$ on $L^2[0,1]$ with domain $\{u \in L^2[0,1] : u'' \in L^2,\ u(0) = u(1) = 0\}$ is *unbounded*: no finite constant C satisfies $\|\mathcal{D}u\|_2 \leq C\|u\|_2$ for all u in its domain.

Proof *Proof.* To show $\mathcal{D}$ is unbounded, we exhibit a sequence of unit-norm functions on which $\|\mathcal{D}u_k\|_2$ grows without bound.

Step 1: Define a sequence of test functions.
Let $u_k(x) = \sqrt{2}\sin(k\pi x)$ for $k = 1, 2, 3, \ldots$. Each u_k satisfies the boundary conditions $u_k(0) = u_k(1) = 0$.
Step 2: Verify each u_k has unit norm.

$$\|u_k\|_2^2 = 2\int_0^1 \sin^2(k\pi x)\, dx = 2 \cdot \tfrac{1}{2} = 1.$$

Step 3: Compute $\mathcal{D}u_k$.
Differentiating twice: $-u_k''(x) = k^2\pi^2\sqrt{2}\sin(k\pi x) = k^2\pi^2\, u_k(x)$. So u_k is actually an eigenfunction of $\mathcal{D}$, and

$$\|\mathcal{D}u_k\|_2 = k^2\pi^2\|u_k\|_2 = k^2\pi^2.$$

Step 4: Conclude.
For any candidate bound C, choose $k > \sqrt{C}/\pi$. Then $k^2\pi^2 > C$, so $\|\mathcal{D}u_k\|_2 > C = C\|u_k\|_2$. Since no finite C works, $\mathcal{D}$ is unbounded. □

Note 6.14 Although the continuous operator is unbounded, its n-point discretization $\mathbf{T}$ has operator norm $\|\mathbf{T}\|_2 = \lambda_{\max}(\mathbf{T}) \approx 4/h^2 = 4(n+1)^2$, which is large but finite. This growing norm explains why Poisson systems become increasingly ill-conditioned as the grid is refined:

$$\kappa(\mathbf{T}) = \frac{\lambda_{\max}}{\lambda_{\min}} \approx \frac{4/h^2}{\pi^2} = O(h^{-2}).$$

Doubling the grid resolution quadruples the condition number. This is a direct numerical consequence of the operator being unbounded in the continuous limit.

Ex 6.17

1. For $n = 10, 50, 100, 500$, compute $\kappa(\mathbf{T}) = \lambda_{\max}/\lambda_{\min}$ using the analytical eigenvalue formula. Plot $\kappa(\mathbf{T})$ vs. n on a log-log scale and confirm the $O(n^2)$ growth.
2. Use `np.linalg.cond` to compute the condition number of the 2-D Laplacian $\mathbf{K}$ for $n = 5, 10, 20$. How does $\kappa(\mathbf{K})$ scale with n?

6.3.4 Point Spectrum and Continuous Spectrum

For an $n \times n$ matrix, the spectrum is a finite set of eigenvalues. For operators on infinite-dimensional spaces, the spectrum can be richer: eigenvalues (point spectrum), a continuous range of spectral values with no eigenfunctions (continuous spectrum), or both. Which type of spectrum an operator has shapes the numerical algorithms best suited to approximate it.

Def 6.24 **(Spectrum of a Linear Operator)** For an operator $\mathcal{A}$ on a function space, a scalar λ belongs to the *spectrum* $\sigma(\mathcal{A})$ if $\mathcal{A} - \lambda\mathcal{I}$ is not invertible. The spectrum decomposes as follows.

- **Point spectrum** σ_p: values λ for which $\mathcal{A}u = \lambda u$ has a nonzero solution (a genuine eigenfunction).
- **Continuous spectrum** σ_c: values λ for which no eigenfunction exists, but there are approximate eigenfunctions: a sequence u_k with $\|u_k\| = 1$ and $\|\mathcal{A}u_k - \lambda u_k\| \to 0$. Intuitively, λ is "almost" an eigenvalue but the limiting object does not live in the space.

For $n \times n$ matrices, $\sigma(\mathbf{A}) = \sigma_p(\mathbf{A})$: only point spectrum exists.

Thm 6.13 **(Spectrum of the 1-D Laplacian on $[0, 1]$)** The operator $\mathcal{D} = -d^2/dx^2$ with homogeneous Dirichlet conditions on $[0, 1]$ has purely point spectrum:

$$\sigma(\mathcal{D}) = \sigma_p(\mathcal{D}) = \{k^2\pi^2 : k = 1, 2, 3, \ldots\}.$$

The corresponding eigenfunctions $u_k(x) = \sqrt{2}\sin(k\pi x)$ form an orthonormal basis of $L^2[0, 1]$.

Proof *Proof.* We solve $-u'' = \lambda u$ subject to $u(0) = u(1) = 0$ and find all λ for which a nonzero solution exists.

Step 1: Write down the general solution.
For $\lambda > 0$, the ODE $-u'' = \lambda u$ has general solution

$$u(x) = A\sin(\sqrt{\lambda}\,x) + B\cos(\sqrt{\lambda}\,x).$$

Step 2: Apply the boundary condition at $x = 0$.
Setting $u(0) = 0$ gives $B \cdot 1 = 0$, so $B = 0$ and $u(x) = A\sin(\sqrt{\lambda}\,x)$.
Step 3: Apply the boundary condition at $x = 1$.
Setting $u(1) = 0$ requires $A\sin(\sqrt{\lambda}) = 0$. For a nonzero solution we need $A \neq 0$, so $\sin(\sqrt{\lambda}) = 0$. This forces $\sqrt{\lambda} = k\pi$ for some positive integer k.
Step 4: Conclude.
The admissible eigenvalues are $\lambda_k = k^2\pi^2$, with eigenfunctions $u_k(x) = \sqrt{2}\sin(k\pi x)$ (normalized). Completeness of $\{u_k\}$ in $L^2[0,1]$ is the Fourier sine series, a classical result from Sturm-Liouville theory. □

Thm 6.14 **(Spectrum of the Laplacian on $\mathbb{R}$)** The operator $\mathcal{D} = -d^2/dx^2$ on $L^2(\mathbb{R})$ has purely continuous spectrum:

$$\sigma(\mathcal{D}) = \sigma_c(\mathcal{D}) = [0, \infty).$$

There are no $L^2(\mathbb{R})$ eigenfunctions.

Proof *Proof.* We show two things: no eigenvalues exist, but every $\lambda \geq 0$ is in the spectrum.

Step 1: There are no L^2 eigenfunctions.
For any $\lambda = \xi^2 \geq 0$, the equation $-u'' = \xi^2 u$ has solutions $e^{i\xi x}$ and $e^{-i\xi x}$. These are bounded functions, but

$$\int_{\mathbb{R}} |e^{i\xi x}|^2\,dx = \int_{\mathbb{R}} 1\,dx = \infty,$$

so neither is in $L^2(\mathbb{R})$. Hence $\sigma_p(\mathcal{D}) = \emptyset$.
Step 2: Every $\lambda \geq 0$ is still in the spectrum via approximate eigenfunctions.
Fix $\xi \in \mathbb{R}$ and let $\lambda = \xi^2$. Choose a smooth bump function ϕ that is supported on $[-1, 1]$ and has $\|\phi\|_2 = 1$. Define the rescaled sequence

$$u_k(x) = \frac{1}{\sqrt{k}}\,\phi\Big(\frac{x}{k}\Big)\,e^{i\xi x}, \quad k = 1, 2, 3, \ldots$$

Each u_k is in $L^2(\mathbb{R})$, normalized ($\|u_k\|_2 = 1$), and one can verify that $\|(-\partial_{xx} - \xi^2)u_k\|_2 \to 0$ as $k \to \infty$. So $\lambda = \xi^2$ is in the continuous spectrum.
Step 3: The spectrum is bounded below by 0.
For any u in the domain, integration by parts gives $\langle -u'', u\rangle_{L^2} = \int_{\mathbb{R}} |u'|^2\,dx \geq 0$, so $\mathcal{D}$ has no spectrum below 0. □

Note 6.15 **Why this matters for Lab 4.** When you discretize the Laplacian on $[0, 1]$ with n points, you obtain n eigenvalues approximating $\{k^2\pi^2\}_{k=1}^n$. As n grows:

- On a bounded domain, the discrete eigenvalues converge to the countable point spectrum $\{k^2\pi^2\}$. High-frequency modes require finer grids to resolve accurately.
- On an unbounded or periodic domain, the eigenvalues of the finite discretization fill in densely, converging to the continuous spectrum $[0, \infty)$ as the domain size grows.

This is why the Fourier transform, which diagonalizes $-d^2/dx^2$ on $\mathbb{R}$ through its continuous spectrum, is the natural tool for unbounded or periodic problems, while eigenfunction expansions (Fourier sine series) are natural for bounded domains.

Ex 6.18

1. For $n = 20, 50, 100$, compute all eigenvalues of the 1-D discrete Laplacian $\mathbf{T}$ and plot them alongside the continuous values $k^2\pi^2$. At what index k do the discrete eigenvalues begin to deviate significantly from $k^2\pi^2$? (*This deviation is related to the Nyquist sampling limit.*)

2. Modify the 1-D Laplacian to use *periodic* boundary conditions by setting $T_{1n} = T_{n1} = -1/h^2$. Compute the eigenvalues and compare them to the Dirichlet case. What is the smallest eigenvalue, and why does this make physical sense for a domain with no boundaries?

3. (*Verifying second-order convergence.*) The exact solution to $-u'' = \pi^2 \sin(\pi x)$ on $[0, 1]$ with $u(0) = u(1) = 0$ is $u(x) = \sin(\pi x)$. Solve the discrete system $\mathbf{Tu} = \mathbf{f}$ for $n = 10, 20, 40, 80$ and compute the error $\|u_h - u\|_\infty$ at each resolution. Plot the error vs. h on a log-log scale. What is the slope, and does it confirm $O(h^2)$ convergence?

6.4 Fourier Analysis as Change of Basis

The Spectral Theorem says that every symmetric (or Hermitian) operator can be diagonalized in an orthonormal eigenbasis. For the Laplacian $-d^2/dx^2$, that eigenbasis is the set of sinusoids. *Fourier analysis is therefore just linear algebra in an infinite-dimensional space*: expressing a function as a Fourier series is the same operation as expanding a vector in an eigenbasis. This viewpoint makes Fourier analysis less magical and reveals exactly why it is so powerful: it diagonalizes the Laplacian, turning differential equations into simple algebra in frequency space. The Discrete Fourier Transform (DFT) is the finite-dimensional version of this, represented as a unitary matrix. Computing it efficiently via the Fast Fourier Transform (FFT) is one of the most practically impactful algorithms ever discovered.

6.4.1 The Fourier Basis

In Section 6.3, we found that the eigenfunctions of $-d^2/dx^2$ on $[0, 1]$ are $u_k(x) = \sqrt{2}\sin(k\pi x)$ with eigenvalues $k^2\pi^2$. These functions are orthonormal in $L^2[0, 1]$, and they span the whole space. Expanding any function f in this basis is the Fourier sine series. On the periodic domain $[0, 2\pi]$, the natural eigenbasis is the complex exponentials e^{ikx}, which give the full (complex) Fourier series. We work with the periodic setting here because it connects most directly to the DFT.

Def 6.25 **(Fourier Series)** For $f \in L^2([0, 2\pi])$ (periodic), the *Fourier series* of f is

$$f(x) = \sum_{k=-\infty}^{\infty} \hat{f}_k \, e^{ikx},$$

where the *Fourier coefficients* are

$$\hat{f}_k = \frac{1}{2\pi} \int_0^{2\pi} f(x) \, e^{-ikx} \, dx.$$

The coefficient $\hat{f}_k$ is the inner product of f with the basis function $e^{ikx}/(2\pi)$, exactly as in a standard change of basis.

Thm 6.15 **(Fourier Basis is Orthonormal)** The functions $\phi_k(x) = \frac{1}{\sqrt{2\pi}} e^{ikx}$, $k \in \mathbb{Z}$, form an orthonormal set in $L^2([0, 2\pi])$:

$$\langle \phi_j, \phi_k \rangle = \frac{1}{2\pi} \int_0^{2\pi} e^{ijx} \overline{e^{ikx}} \, dx = \frac{1}{2\pi} \int_0^{2\pi} e^{i(j-k)x} \, dx = \begin{cases} 1 & j = k, \\ 0 & j \neq k. \end{cases}$$

Moreover, they form a complete orthonormal basis: every $f \in L^2([0, 2\pi])$ can be recovered exactly from its Fourier coefficients via the Fourier series.

Proof *Proof.* **Orthonormality.** For $j \neq k$:

$$\frac{1}{2\pi} \int_0^{2\pi} e^{i(j-k)x} \, dx = \frac{1}{2\pi} \cdot \frac{e^{i(j-k)x}}{i(j-k)} \bigg|_0^{2\pi} = \frac{e^{2\pi i(j-k)} - 1}{2\pi i(j-k)} = 0,$$

since $e^{2\pi i m} = 1$ for any integer m. For $j = k$, the integrand is 1 and the integral equals 2π, giving norm 1 after dividing by 2π.

Completeness. Showing that every L^2 function is approximated arbitrarily well by finite Fourier sums is a deeper result (the Riesz-Fischer theorem) that goes beyond this course. It is the infinite-dimensional analog of the statement that an orthonormal set in $\mathbb{R}^n$ that spans $\mathbb{R}^n$ is a basis. $\square$

Note 6.16 **The change-of-basis picture.** In $\mathbb{R}^n$, changing from the standard basis $\{\boldsymbol{e}_j\}$ to an orthonormal basis $\{\boldsymbol{q}_j\}$ is accomplished by the unitary matrix $\mathbf{Q}$: the j-th coordinate of $\boldsymbol{x}$ in the new basis is $\boldsymbol{q}_j^T \boldsymbol{x}$. The Fourier transform does exactly the same thing in L^2: the k-th coordinate of f in the Fourier basis is $\hat{f}_k = \langle \phi_k, f \rangle$. Just as $\mathbf{Q}$ diagonalizes a symmetric matrix ($\mathbf{A} = \mathbf{Q}\mathbf{D}\mathbf{Q}^T$), the Fourier transform diagonalizes the Laplacian: $-\widehat{f''(x)}_k = k^2 \hat{f}_k$.

Ex 6.19 Refer to Def 6.25 and Thm 6.15.

1. Compute the Fourier coefficients $\hat{f}_k$ for $f(x) = x$ on $[0, 2\pi]$ by evaluating the integral directly. For which k are the coefficients nonzero?

2. Verify the diagonalization claim: if $f(x) = e^{ikx}$, compute $-f''(x)$ and show that $-\widehat{f''}_k = k^2 \hat{f}_k$ without any integration.

3. Why does the Fourier transform turn the ODE $-u'' = f$ into the algebraic equation $k^2 \hat{u}_k = \hat{f}_k$? When is this equation not solvable?

6.4.2 The Discrete Fourier Transform

The continuous Fourier series works on functions; the DFT works on finite vectors. Given a vector $\boldsymbol{x} \in \mathbb{C}^n$ (think: n samples of a signal), the DFT produces another vector $\hat{\boldsymbol{x}} \in \mathbb{C}^n$ (think: frequency amplitudes). The key fact is that this transformation is a linear map, so it can be written as a matrix. That matrix turns out to be unitary, which makes the DFT exactly a change of basis in $\mathbb{C}^n$.

Def 6.26 **(DFT Matrix)** Let $\omega_n = e^{-2\pi i/n}$ be the primitive n-th root of unity. The *DFT matrix* $\mathbf{F} \in \mathbb{C}^{n\times n}$ has entries

$$F_{jk} = \frac{1}{\sqrt{n}}\,\omega_n^{jk}, \quad j,k = 0,1,\ldots,n-1.$$

The DFT of $\boldsymbol{x} \in \mathbb{C}^n$ is $\hat{\boldsymbol{x}} = \mathbf{F}\boldsymbol{x}$, and its k-th entry

$$\hat{x}_k = \frac{1}{\sqrt{n}} \sum_{j=0}^{n-1} x_j\, e^{-2\pi ijk/n}$$

is the discrete analog of the Fourier coefficient at frequency k.

Example For $n = 4$, $\omega_4 = e^{-\pi i/2} = -i$. The DFT matrix is

$$\mathbf{F} = \frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{pmatrix}.$$

Applying $\mathbf{F}$ to $\boldsymbol{x} = (1,0,0,0)^T$ gives $\hat{\boldsymbol{x}} = \frac{1}{2}(1,1,1,1)^T$: a constant in all frequency components, just like a delta spike has equal energy at all frequencies.

Thm 6.16 **(The DFT Matrix is Unitary)** $\mathbf{F}^*\mathbf{F} = \mathbf{I}$, so $\mathbf{F}$ is a unitary matrix (Def 4.3). Equivalently, the columns of $\mathbf{F}$ form an orthonormal basis of $\mathbb{C}^n$. The inverse DFT is $\mathbf{F}^{-1} = \mathbf{F}^*$, with entries $(\mathbf{F}^*)_{jk} = \frac{1}{\sqrt{n}}\omega_n^{-jk} = \frac{1}{\sqrt{n}}e^{2\pi ijk/n}$.

Proof *Proof.* We compute the (j,k) entry of $\mathbf{F}^*\mathbf{F}$.

Step 1: Write out the entry.

$$(\mathbf{F}^*\mathbf{F})_{jk} = \sum_{m=0}^{n-1} \overline{F_{mj}}\, F_{mk} = \frac{1}{n}\sum_{m=0}^{n-1} \omega_n^{-mj}\,\omega_n^{mk} = \frac{1}{n}\sum_{m=0}^{n-1} \omega_n^{m(k-j)}.$$

Step 2: Evaluate the geometric sum.
If $j = k$, each term equals 1, so the sum is n and the entry is 1.
If $j \neq k$, let $r = \omega_n^{k-j} \neq 1$ (since $k - j \not\equiv 0 \pmod{n}$). The sum is $\sum_{m=0}^{n-1} r^m = \frac{r^n-1}{r-1}$. But $r^n = \omega_n^{n(k-j)} = e^{-2\pi i(k-j)} = 1$, so the numerator is zero and the sum is 0.
Step 3: Conclude.
Therefore $(\mathbf{F}^*\mathbf{F})_{jk} = \delta_{jk}$, confirming $\mathbf{F}^*\mathbf{F} = \mathbf{I}$. □

Thm 6.17 **(Parseval's Theorem)** For any $\boldsymbol{x} \in \mathbb{C}^n$ with DFT $\hat{\boldsymbol{x}} = \mathbf{F}\boldsymbol{x}$:

$$\|\hat{\boldsymbol{x}}\|_2 = \|\boldsymbol{x}\|_2.$$

The DFT preserves the ℓ^2 norm: total energy is the same in the time domain and the frequency domain.

Proof *Proof.* Since $\mathbf{F}$ is unitary, it is an isometry (Thm 3.1):

$$\|\hat{\boldsymbol{x}}\|_2^2 = \|\mathbf{F}\boldsymbol{x}\|_2^2 = (\mathbf{F}\boldsymbol{x})^*(\mathbf{F}\boldsymbol{x}) = \boldsymbol{x}^*\mathbf{F}^*\mathbf{F}\boldsymbol{x} = \boldsymbol{x}^*\boldsymbol{x} = \|\boldsymbol{x}\|_2^2.$$

□

Ex 6.20 Refer to Defs 6.26–6.16 and Thm 6.17.

1. For $n = 4$, write out $\mathbf{F}$ explicitly (using $\omega_4 = -i$). Verify $\mathbf{F}^*\mathbf{F} = \mathbf{I}$ by hand for the first two columns.
2. In NumPy, `np.fft.fft(x) / np.sqrt(n)` computes the normalized DFT. Verify Parseval's theorem numerically for a random $\boldsymbol{x} \in \mathbb{C}^{10}$ by comparing `np.linalg.norm(x)` and `np.linalg.norm(np.fft.fft(x) / np.sqrt(n))`.
3. Compute the DFT of the signal $x_j = \cos(2\pi j/n)$ for $n = 16$. Which frequency bins $\hat{x}_k$ are nonzero? Explain why.

6.4.3 Circulant Matrices and the Convolution Theorem

The DFT is more than just a change of basis for signals: it simultaneously diagonalizes an entire class of matrices. Circulant matrices arise naturally whenever a system has translation symmetry (same local rule everywhere), which includes the periodic discrete Laplacian and discrete convolution operators. Diagonalizing them with the DFT reduces matrix-vector multiplication from $O(n^2)$ to $O(n \log n)$.

Def 6.27 **(Circulant Matrix)** A matrix $\mathbf{C} \in \mathbb{C}^{n\times n}$ is *circulant* if each row is a cyclic right-shift of the previous row. It is completely determined by its first column $\boldsymbol{c} = (c_0, c_1, \ldots, c_{n-1})^T$:

$$\mathbf{C} = \begin{pmatrix} c_0 & c_{n-1} & c_{n-2} & \cdots & c_1 \\ c_1 & c_0 & c_{n-1} & \cdots & c_2 \\ c_2 & c_1 & c_0 & \cdots & c_3 \\ \vdots & & \ddots & & \vdots \\ c_{n-1} & c_{n-2} & \cdots & c_1 & c_0 \end{pmatrix}.$$

The product $\mathbf{C}\boldsymbol{x}$ computes the *circular convolution* $\boldsymbol{c} * \boldsymbol{x}$.

Thm 6.18 **(DFT Diagonalizes Every Circulant Matrix)** Every circulant matrix $\mathbf{C}$ with first column $\boldsymbol{c}$ satisfies

$$\mathbf{C} = \mathbf{F}^* \operatorname{diag}(\hat{\boldsymbol{c}})\, \mathbf{F},$$

where $\hat{\boldsymbol{c}} = \mathbf{F}\boldsymbol{c}$ is the DFT of $\boldsymbol{c}$. The eigenvalues of $\mathbf{C}$ are exactly the entries of $\hat{\boldsymbol{c}}$ (the DFT coefficients of the first column), and the eigenvectors are the columns of $\mathbf{F}^*$.

Cor 6.19 **(Convolution Theorem)** For any $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{C}^n$, the circular convolution satisfies

$$\mathbf{F}(\boldsymbol{x} * \boldsymbol{y}) = (\mathbf{F}\boldsymbol{x}) \odot (\mathbf{F}\boldsymbol{y}),$$

where $\odot$ denotes entry-wise (Hadamard) product. Circular convolution in the time domain is pointwise multiplication in the frequency domain.

Proof *Proof.* **Step 1: Express convolution as a matrix-vector product.** Circular convolution $\boldsymbol{z} = \boldsymbol{x} * \boldsymbol{y}$ means $z_j = \sum_{k=0}^{n-1} x_k y_{(j-k) \bmod n}$. This is exactly $\boldsymbol{z} = \mathbf{C}_{\boldsymbol{x}}\boldsymbol{y}$ where $\mathbf{C}_{\boldsymbol{x}}$ is the circulant matrix with first column $\boldsymbol{x}$.
Step 2: Diagonalize. By Thm 6.18, $\mathbf{C}_{\boldsymbol{x}} = \mathbf{F}^* \operatorname{diag}(\hat{\boldsymbol{x}})\mathbf{F}$, so:

$$\boldsymbol{z} = \mathbf{C}_{\boldsymbol{x}}\boldsymbol{y} = \mathbf{F}^* \operatorname{diag}(\hat{\boldsymbol{x}})\mathbf{F}\boldsymbol{y} = \mathbf{F}^*(\hat{\boldsymbol{x}} \odot \hat{\boldsymbol{y}}).$$

Step 3: Apply $\mathbf{F}$ to both sides.

$$\hat{\boldsymbol{z}} = \mathbf{F}\boldsymbol{z} = \mathbf{F}\mathbf{F}^*(\hat{\boldsymbol{x}} \odot \hat{\boldsymbol{y}}) = \hat{\boldsymbol{x}} \odot \hat{\boldsymbol{y}},$$

using $\mathbf{F}\mathbf{F}^* = \mathbf{I}$. □

Note 6.17 **Application to the Poisson equation.** The 1-D discrete Laplacian with *periodic* boundary conditions is circulant. Its eigenvalues are the DFT of the first column $\boldsymbol{c} = \frac{1}{h^2}(2, -1, 0, \ldots, 0, -1)^T$. This gives an $O(n \log n)$ solver for the periodic Poisson equation:

1. Compute $\hat{\mathbf{f}} = \mathbf{F}\mathbf{f}$ via FFT.
2. Divide entry-wise: $\hat{u}_k = \hat{f}_k / \hat{c}_k$.
3. Invert: $\mathbf{u} = \mathbf{F}^*\hat{\mathbf{u}}$ via inverse FFT.

Compare this to the $O(n^3)$ dense solve or $O(n^{3/2})$ sparse solve. On a 2-D grid of size $n \times n$ ($N = n^2$ unknowns), the FFT-based solver costs $O(N \log N)$, making it the method of choice for large periodic problems.

Ex 6.21 Refer to Def 6.27 and Thm 6.18.

1. Build the 5×5 circulant matrix with first column $\boldsymbol{c} = (2, -1, 0, 0, -1)^T$ (the periodic 1-D Laplacian for $n = 5$). Compute its eigenvalues directly and via $\hat{\boldsymbol{c}} = \mathbf{F}\boldsymbol{c}$ using `np.fft.fft(c)`. Verify they match.

2. Implement circular convolution two ways: (a) directly as a matrix-vector product using `scipy.linalg.circulant`, and (b) using `np.fft.fft` and pointwise multiplication. Verify they agree for random $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^{16}$.

3. Implement the FFT-based Poisson solver described in the note above for $n = 100$ with $f(x) = \sin(2\pi x)$. The exact solution is $u(x) = \frac{1}{4\pi^2}\sin(2\pi x)$. Verify your solver recovers it.

6.4.4 The Fast Fourier Transform

Computing $\hat{\boldsymbol{x}} = \mathbf{F}\boldsymbol{x}$ directly costs $O(n^2)$ flops (one row of $\mathbf{F}$ dotted with $\boldsymbol{x}$ is $O(n)$, and there are n rows). For $n = 10^6$, this is 10^{12} operations: unusable. The Fast Fourier Transform (FFT) reduces this to $O(n \log n)$ by exploiting a recursive structure in the DFT matrix. For $n = 10^6$, that is 2×10^7 operations: entirely practical. This gap is why FFT appears in audio processing, image compression (JPEG), wireless communications, and numerical PDE solvers.

Thm 6.20 **(Cooley-Tukey FFT Complexity)** When $n = 2^p$ is a power of two, the DFT $\mathbf{F}\boldsymbol{x}$ can be computed in $O(n \log n)$ flops using the Cooley-Tukey algorithm.

Proof *Proof.* The key identity splits the n-point DFT into two $(n/2)$-point DFTs.

Step 1: Split $\boldsymbol{x}$ into even and odd indexed entries.
Let $\boldsymbol{x}^{(e)} = (x_0, x_2, \ldots, x_{n-2})$ and $\boldsymbol{x}^{(o)} = (x_1, x_3, \ldots, x_{n-1})$.
Step 2: Rewrite the DFT sum.
The k-th DFT output is

$$\hat{x}_k = \sum_{j=0}^{n-1} x_j \omega_n^{jk} = \sum_{j=0}^{n/2-1} x_{2j}\, \omega_n^{2jk} + \omega_n^k \sum_{j=0}^{n/2-1} x_{2j+1}\, \omega_n^{2jk}.$$

Step 3: Recognize the sub-DFTs.
Since $\omega_n^2 = e^{-4\pi i/n} = \omega_{n/2}$, both sums are $(n/2)$-point DFTs:

$$\hat{x}_k = \hat{x}_k^{(e)} + \omega_n^k\, \hat{x}_k^{(o)}.$$

For $k \geq n/2$, a similar relation holds using $\omega_n^{k+n/2} = -\omega_n^k$.
Step 4: Count the cost.
Let $T(n)$ be the number of flops. The recursion is $T(n) = 2T(n/2) + O(n)$, which has solution $T(n) = O(n \log n)$. □

Note 6.18 **FFT in NumPy.** `np.fft.fft(x)` computes the DFT using an FFT algorithm. `np.fft.ifft(x)` computes the inverse DFT ($\mathbf{F}^*\boldsymbol{x}$). For real signals, `np.fft.rfft` exploits the symmetry $\hat{x}_{n-k} = \overline{\hat{x}_k}$ to halve the work. The 2-D FFT (`np.fft.fft2`) applies the 1-D FFT along each axis and is the key subroutine for image processing and 2-D PDE solvers.

Ex 6.22 Refer to Thm 6.20.

1. In Python, time `np.fft.fft(x)` vs. a naive DFT implemented as `F @ x` (build $\mathbf{F}$ explicitly) for $n \in \{64, 256, 1024, 4096\}$. Plot both times vs. n on a log-log scale. At what size does the FFT become noticeably faster?

2. Verify the splitting identity in the proof: for $n = 8$ and a random $\boldsymbol{x}$, check that $\hat{x}_k = \hat{x}_k^{(e)} + \omega_8^k \hat{x}_k^{(o)}$ for $k = 0, 1, 2, 3$ (where $\hat{\boldsymbol{x}}^{(e)}, \hat{\boldsymbol{x}}^{(o)}$ are 4-point DFTs of the even and odd subsequences).

3. (*Spectral differentiation.*) Given samples $x_j = f(2\pi j/n)$ for $j = 0, \ldots, n-1$ of a smooth periodic function f, one can approximate f' by: (a) FFT to get $\hat{\boldsymbol{x}}$, (b) multiply $\hat{x}_k$ by ik, (c) IFFT. Implement this for $f(x) = \sin(x) + \cos(3x)$ and verify you recover $f'(x) = \cos(x) - 3\sin(3x)$. Compare the accuracy to finite differences as n grows.

6.5 Markov Chains and Stochastic Matrices

A Markov chain models a random process that moves between states with probabilities depending only on the current state. The evolution of probability distributions is governed by repeated multiplication by a *stochastic matrix*, making Markov chain analysis a direct application of the linear algebra developed in earlier chapters. This section views stochastic matrices as linear operators (building on §6.2), characterizes their stationary distributions via the Perron-Frobenius theorem, and develops three numerically stable methods for computing these distributions. The connection to power iteration (Thm 4.7) will be central throughout.

6.5.1 Stochastic Matrices as Linear Operators

The random walk on a graph (see §6.2) is described by a transition matrix $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}$. Here we abstract that concrete example: we study the class of *column-stochastic matrices* and identify which structural properties guarantee that repeated application of $\mathbf{P}$ converges.

Def 6.28 **(Column-Stochastic Matrix)** A matrix $\mathbf{P} \in \mathbb{R}^{n \times n}$ is *column-stochastic* if

1. $P_{ij} \geq 0$ for all i, j (all entries are nonnegative), and
2. $\sum_{i=1}^{n} P_{ij} = 1$ for every column j (columns sum to 1).

The entry P_{ij} is the probability of transitioning *from* state j *to* state i.

Ex 6.23 **(Stochastic matrix basics)** Let $\mathbf{P} = \frac{1}{3}\begin{pmatrix} 1 & 2 & 0 \\ 1 & 0 & 2 \\ 1 & 1 & 1 \end{pmatrix}$.

1. Verify that $\mathbf{P}$ is column-stochastic.

2. Compute $\mathbf{1}^T\mathbf{P}$ and $\mathbf{P}\mathbf{1}$. What vectors do you get, and what do they mean probabilistically?

3. Show that $\lambda = 1$ is always an eigenvalue of a column-stochastic matrix by exhibiting a left eigenvector. (Hint: multiply $\mathbf{P}$ on the left by $\mathbf{1}^T$.)

Def 6.29 **(Markov Chain)** A *discrete-time homogeneous Markov chain* on n states is a sequence of probability distributions $\boldsymbol{x}^{(0)}, \boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots \in \mathbb{R}^n$ evolving by

$$\boldsymbol{x}^{(k+1)} = \mathbf{P}\,\boldsymbol{x}^{(k)}, \qquad k = 0, 1, 2, \ldots$$

where $\mathbf{P}$ is column-stochastic and $\boldsymbol{x}^{(0)}$ satisfies $x_i^{(0)} \geq 0$ and $\sum_i x_i^{(0)} = 1$. The i-th entry $x_i^{(k)}$ is the probability of being in state i at time k. After k steps, $\boldsymbol{x}^{(k)} = \mathbf{P}^k\,\boldsymbol{x}^{(0)}$.

Note 6.19 The Markov chain evolution $\boldsymbol{x}^{(k+1)} = \mathbf{P}\boldsymbol{x}^{(k)}$ is exactly the power iteration of Def 4.5 applied to the stochastic matrix $\mathbf{P}$. This means the long-run behavior is entirely governed by the eigenstructure of $\mathbf{P}$: the dominant eigenvalue $\lambda_1 = 1$ selects the stationary distribution, while the subdominant eigenvalue λ_2 controls how fast the chain converges to it.

Ex 6.24 **(Two-state chain)** Consider the transition matrix

$$\mathbf{P} = \begin{pmatrix} 1-\alpha & \beta \\ \alpha & 1-\beta \end{pmatrix}, \qquad \alpha, \beta \in (0, 1).$$

1. Verify that $\mathbf{P}$ is column-stochastic for any $\alpha, \beta \in (0, 1)$.

2. Compute $\mathbf{P}^2$ symbolically and identify the pattern. What do the columns of $\mathbf{P}^k$ approach as $k \to \infty$?

3. Using NumPy with $\alpha = 0.3$, $\beta = 0.2$: compute `numpy.linalg.matrix_power(P, k)` for $k = 1, 5, 20, 100$. Print the columns and observe their convergence.

Def 6.30 **(Irreducibility and Primitivity)** A column-stochastic matrix $\mathbf{P}$ is *irreducible* if for every pair of states i, j there exists $k \geq 1$ such that $(\mathbf{P}^k)_{ij} > 0$ (every state is reachable from every state). It is *primitive* (aperiodic) if in addition the greatest common divisor of all cycle lengths in the state transition graph is 1.

Ex 6.25 **(Reducibility)** Consider the block transition matrix

$$\mathbf{P} = \begin{pmatrix} 1/2 & 1/3 & 0 \\ 1/2 & 2/3 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

1. Draw the state transition graph. Which states can reach which others?

2. Starting from $\boldsymbol{x}^{(0)} = (0, 0, 1)^T$, compute $\boldsymbol{x}^{(k)}$ for $k = 1, 2, 5, 20$. What happens?

3. Modify $\mathbf{P}$ by adding a small teleportation term: $\mathbf{P}' = (1-\varepsilon)\mathbf{P} + \frac{\varepsilon}{3}\mathbf{1}\mathbf{1}^T/3$ for $\varepsilon = 0.15$. Is $\mathbf{P}'$ irreducible and primitive? (This is precisely the PageRank construction; see §6.2.5.)

6.5.2 Stationary Distributions

The central question for a Markov chain is: does the distribution $\boldsymbol{x}^{(k)}$ converge as $k \to \infty$, and if so, to what limit? The answer is provided by the Perron-Frobenius theorem, which guarantees that irreducible primitive stochastic matrices have a unique stationary distribution with all positive entries.

Def 6.31 **(Stationary Distribution)** A probability vector $\boldsymbol{\pi} \in \mathbb{R}^n$ (with $\pi_i \geq 0$ and $\sum_i \pi_i = 1$) is a *stationary distribution* of the stochastic matrix $\mathbf{P}$ if

$$\mathbf{P}\,\boldsymbol{\pi} = \boldsymbol{\pi}.$$

Equivalently, $\boldsymbol{\pi}$ is a right eigenvector of $\mathbf{P}$ with eigenvalue 1. Starting from $\boldsymbol{\pi}$, the chain does not move.

Ex 6.26 **(Stationary distribution by inspection)** For the two-state chain $\mathbf{P} = \begin{pmatrix} 1-\alpha & \beta \\ \alpha & 1-\beta \end{pmatrix}$:

1. Show by direct computation that $\boldsymbol{\pi} = \frac{1}{\alpha+\beta}\begin{pmatrix} \beta \\ \alpha \end{pmatrix}$ satisfies $\mathbf{P}\boldsymbol{\pi} = \boldsymbol{\pi}$.
2. Verify that $\pi_1 + \pi_2 = 1$.
3. For $\alpha = 0.3$, $\beta = 0.2$: compute $\boldsymbol{\pi}$ and verify numerically with `numpy.linalg.eig(P)`, extracting the eigenvector for $\lambda = 1$ and normalizing it.

Thm 6.21 **(Perron-Frobenius for Stochastic Matrices)** Let $\mathbf{P} \in \mathbb{R}^{n\times n}$ be column-stochastic, irreducible, and primitive. Then:

1. $\lambda_1 = 1$ is a simple eigenvalue (algebraic and geometric multiplicity 1).
2. All other eigenvalues satisfy $|\lambda_i| < 1$ for $i \geq 2$.
3. The eigenvector $\boldsymbol{\pi}$ for $\lambda_1 = 1$ has all strictly positive entries and is unique up to scaling.
4. For any initial probability distribution $\boldsymbol{x}^{(0)}$,

$$\lim_{k\to\infty} \mathbf{P}^k\,\boldsymbol{x}^{(0)} = \boldsymbol{\pi}.$$

Proof *Proof.* **Step 1: $\lambda = 1$ is an eigenvalue.** Since columns of $\mathbf{P}$ sum to 1, $\mathbf{1}^T\mathbf{P} = \mathbf{1}^T$, so $\mathbf{1}$ is a left eigenvector for $\lambda = 1$.
Step 2: $|\lambda| \leq 1$ for all eigenvalues. Let $\mathbf{P}\boldsymbol{v} = \lambda\boldsymbol{v}$ and let i^* achieve $|v_{i^*}| = \|\boldsymbol{v}\|_\infty$. Then

$$|\lambda|\,|v_{i^*}| = \left|\sum_j P_{i^*j}\,v_j\right| \leq \sum_j P_{i^*j}\,|v_j| \leq |v_{i^*}|\sum_j P_{i^*j} = |v_{i^*}|.$$

Step 3: Strict dominance under irreducibility. Irreducibility implies $\mathbf{P}^m > 0$ for some m. By the general Perron-Frobenius theorem (Thm 6.9), $\mathbf{P}^m$ has a unique Perron root, so the eigenspace for $\lambda_1 = 1$ is one-dimensional and $|\lambda_i| < 1$ for $i \geq 2$ (primitivity prevents $|\lambda_2| = 1$ by ruling out periodic orbits).
Step 4: Convergence. Expand $\boldsymbol{x}^{(0)} = \boldsymbol{\pi} + \sum_{i=2}^{n} c_i \boldsymbol{v}_i$. Then $\mathbf{P}^k \boldsymbol{x}^{(0)} = \boldsymbol{\pi} + \sum_{i=2}^{n} c_i \lambda_i^k \boldsymbol{v}_i \to \boldsymbol{\pi}$ since $|\lambda_i|^k \to 0$. □

Note 6.20 Thm 6.21 is the theoretical foundation for PageRank (see §6.2.5). The Google matrix $\mathbf{G}$ is made irreducible and primitive by adding the teleportation term $\frac{1-d}{N}\mathbf{J}$, which introduces a small positive probability of jumping to any page. Without this, dangling nodes and disconnected components would prevent $\lambda_1 = 1$ from being simple, and power iteration might not converge to a unique vector.

Ex 6.27 **(Failure modes without irreducibility)** Construct a 4×4 column-stochastic matrix $\mathbf{P}$ that is *not* irreducible by having states $\{1, 2\}$ and $\{3, 4\}$ form two absorbing groups.

1. How many linearly independent stationary distributions does $\mathbf{P}$ have?
2. Starting from $\boldsymbol{x}^{(0)} = (1/4, 1/4, 1/4, 1/4)^T$, run power iteration for 50 steps. What does $\boldsymbol{x}^{(k)}$ converge to?
3. Perturb $\mathbf{P}$ with teleportation ($\varepsilon = 0.05$) to restore irreducibility. How does the stationary distribution change?

6.5.3 Spectral Gap and Mixing Time

Thm 6.21 guarantees convergence, but gives no information about how fast the chain reaches stationarity. The *spectral gap*, the distance between 1 and the next-largest eigenvalue in magnitude, is the key quantity controlling convergence speed. Chains with a large spectral gap mix quickly; chains with a small spectral gap may require exponentially many steps to mix.

Def 6.32 **(Spectral Gap)** For an irreducible primitive column-stochastic matrix $\mathbf{P}$ with eigenvalues ordered as $1 = |\lambda_1| > |\lambda_2| \geq |\lambda_3| \geq \cdots \geq |\lambda_n|$, the *spectral gap* is

$$\delta = 1 - |\lambda_2|.$$

A large spectral gap (δ close to 1) indicates fast mixing; a small spectral gap (δ close to 0) indicates slow mixing.

Note 6.21 For the random walk transition matrix $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}$ on a graph, the eigenvalues of $\mathbf{P}$ relate to those of the random-walk Laplacian $\mathbf{L}_{\text{rw}} = \mathbf{I} - \mathbf{P}$ by $\lambda_i(\mathbf{P}) = 1 - \lambda_i(\mathbf{L}_{\text{rw}})$. The spectral gap of $\mathbf{P}$ therefore equals the Fiedler value (second-smallest eigenvalue) of $\mathbf{L}_{\text{rw}}$, as discussed in §6.2. Well-connected graphs have large Fiedler values and small mixing times.

Ex 6.28 **(Spectral gap and mixing speed)** Compute the spectral gap using `numpy.linalg.eig` for the following stochastic matrices, then simulate 200 steps from $\boldsymbol{x}^{(0)} = \boldsymbol{e}_1$ and plot $\|\boldsymbol{x}^{(k)} - \boldsymbol{\pi}\|_1$ versus k:

1. $\mathbf{P}_1 = \begin{pmatrix} 0.85 & 0.1 & 0.05 \\ 0.1 & 0.8 & 0.1 \\ 0.05 & 0.1 & 0.85 \end{pmatrix}$ (nearly self-absorbing states).
2. $\mathbf{P}_2 = \frac{1}{3}\mathbf{1}\mathbf{1}^T$ (fully mixed at every step).
3. Which chain mixes faster? How does the empirical mixing time compare to the theoretical bound $k \approx \log(n)/\delta$?

Thm 6.22 **(Geometric Mixing Bound)** Let $\mathbf{P}$ be column-stochastic, irreducible, and primitive with spectral gap $\delta = 1 - |\lambda_2|$. For any initial probability distribution $\boldsymbol{x}^{(0)}$ and any $k \geq 0$,

$$\|\mathbf{P}^k \boldsymbol{x}^{(0)} - \boldsymbol{\pi}\|_1 \leq C\,(1-\delta)^k,$$

where C depends on the eigenvectors of $\mathbf{P}$ and the initial condition. The number of steps to reduce the ℓ^1 error below ε is at most $k_\varepsilon \approx \frac{\log(C/\varepsilon)}{\delta}$.

Proof *Proof.* **Step 1: Eigendecomposition.** Write $\boldsymbol{x}^{(0)} = \boldsymbol{\pi} + \sum_{i=2}^n c_i \boldsymbol{v}_i$ where $\{\boldsymbol{v}_i\}$ are eigenvectors of $\mathbf{P}$. Then

$$\mathbf{P}^k \boldsymbol{x}^{(0)} - \boldsymbol{\pi} = \sum_{i=2}^n c_i\, \lambda_i^k\, \boldsymbol{v}_i.$$

Step 2: Norm bound. By the triangle inequality and $|\lambda_i| \leq |\lambda_2| = 1 - \delta$ for $i \geq 2$:

$$\|\mathbf{P}^k \boldsymbol{x}^{(0)} - \boldsymbol{\pi}\|_1 \leq \sum_{i=2}^n |c_i|\, |\lambda_i|^k\, \|\boldsymbol{v}_i\|_1 \leq (1-\delta)^k \underbrace{\sum_{i=2}^n |c_i|\, \|\boldsymbol{v}_i\|_1}_{=C}.$$

Step 3: Mixing time. Solve $(1-\delta)^k \leq \varepsilon/C$ for k: taking logarithms gives $k\log(1-\delta) \leq \log(\varepsilon/C)$. Using the bound $\log(1-\delta) \leq -\delta$ yields the stated estimate. □

Ex 6.29 **(Mixing on graphs)** For the following $n = 20$ vertex graphs, construct $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}$, compute the spectral gap δ via `numpy.linalg.eig`, and simulate the chain from $\boldsymbol{x}^{(0)} = \boldsymbol{e}_1$ to estimate the empirical mixing time (steps until $\|\boldsymbol{x}^{(k)} - \boldsymbol{\pi}\|_1 < 0.01$):

1. The path graph P_{20} (chain of 20 vertices).
2. The cycle graph C_{20} (path with endpoints connected). Is this chain aperiodic?
3. The complete graph K_{20} (all pairs connected). What is δ exactly?
4. Rank the three graphs by mixing speed. Explain the ranking using the graph structure.

6.5.4 Ergodic Theory and Time Averages

The mixing theory of the previous section describes how the *distribution* $\boldsymbol{x}^{(k)}$ converges to $\boldsymbol{\pi}$. But in practice, we never observe a distribution: we observe a single trajectory $X_0, X_1, X_2, \ldots$ of state visits. Ergodic theory asks whether time averages along a single trajectory converge to the spatial average under $\boldsymbol{\pi}$, and if so, how fast. This is the bridge between the measure-theoretic picture (probability distributions) and the empirical picture (data from a simulation), and it is the theoretical foundation for Markov Chain Monte Carlo methods.

Def 6.33 **(Ergodic Markov Chain)** A Markov chain with transition matrix $\mathbf{P}$ is *ergodic* if it is both irreducible and aperiodic (primitive, in the sense of Def 6.30). Ergodicity is the minimal condition that guarantees a unique stationary distribution and convergence of time averages.

Ex 6.30 **(Ergodicity check)** Classify each of the following chains as ergodic or not, and justify your answer by examining the structure of $\mathbf{P}$:

1. The transition matrix $\mathbf{P} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ (two-state swap).
2. The Google matrix $\mathbf{G} = d\mathbf{S} + \frac{1-d}{N}\mathbf{J}$ from §6.2.5 with $d \in (0, 1)$.
3. The random walk on the n-cycle C_n (transitions to left/right neighbor with probability 1/2 each). Does the answer depend on n?

Thm 6.23 **(Ergodic Theorem for Markov Chains)** Let $\mathbf{P}$ be an ergodic (irreducible, aperiodic) column-stochastic matrix with stationary distribution $\boldsymbol{\pi}$. For any observable $\boldsymbol{f} = (f(1), \ldots, f(n))^T \in \mathbb{R}^n$ and any initial distribution $\boldsymbol{x}^{(0)}$,

$$\frac{1}{K}\sum_{k=0}^{K-1} \boldsymbol{f}^T \boldsymbol{x}^{(k)} \xrightarrow{K\to\infty} \boldsymbol{f}^T\boldsymbol{\pi} = \sum_{i=1}^{n} \pi_i f(i),$$

i.e., the time average of f along the chain converges to its expectation under the stationary distribution.

Proof *Proof.* **Step 1: Average of iterates.** Define the Cesàro mean $\bar{\mathbf{P}}_K = \frac{1}{K}\sum_{k=0}^{K-1}\mathbf{P}^k$. Then

$$\frac{1}{K}\sum_{k=0}^{K-1} \boldsymbol{f}^T \boldsymbol{x}^{(k)} = \boldsymbol{f}^T\left(\frac{1}{K}\sum_{k=0}^{K-1}\mathbf{P}^k\boldsymbol{x}^{(0)}\right) = \boldsymbol{f}^T\bar{\mathbf{P}}_K\boldsymbol{x}^{(0)}.$$

Step 2: Convergence of $\bar{\mathbf{P}}_K$. Expand $\mathbf{P}^k = \boldsymbol{\pi}\mathbf{1}^T + \sum_{i=2}^{n}\lambda_i^k \boldsymbol{v}_i\boldsymbol{w}_i^T$ where $\{\boldsymbol{v}_i, \boldsymbol{w}_i\}$ are right and left eigenvectors. Summing:

$$\bar{\mathbf{P}}_K = \boldsymbol{\pi}\mathbf{1}^T + \sum_{i=2}^{n}\frac{1}{K}\cdot\frac{1-\lambda_i^K}{1-\lambda_i}\boldsymbol{v}_i\boldsymbol{w}_i^T.$$

Since $|\lambda_i| \leq 1-\delta < 1$ for $i \geq 2$, each term satisfies $\frac{1}{K}\left|\frac{1-\lambda_i^K}{1-\lambda_i}\right| \leq \frac{2}{K\delta} \to 0$. Therefore $\bar{\mathbf{P}}_K \to \boldsymbol{\pi}\mathbf{1}^T$.

Step 3: Conclusion. Since $\mathbf{1}^T\boldsymbol{x}^{(0)} = 1$ (probability vector),

$$\boldsymbol{f}^T\bar{\mathbf{P}}_K\boldsymbol{x}^{(0)} \;\to\; \boldsymbol{f}^T(\boldsymbol{\pi}\mathbf{1}^T)\boldsymbol{x}^{(0)} = \boldsymbol{f}^T\boldsymbol{\pi}. \qquad \square$$

$\square$

Note 6.22 The ergodic theorem is the theoretical basis for *Markov Chain Monte Carlo (MCMC)*: to compute the integral $\mathbb{E}_\pi[f] = \sum_i \pi_i f(i)$ for a complicated distribution $\boldsymbol{\pi}$, design a Markov chain with stationary distribution $\boldsymbol{\pi}$, simulate a long trajectory, and use the time average as an approximation. The error decays as $O(1/\sqrt{K})$ (by a central limit theorem for ergodic chains), at the same rate as independent Monte Carlo but without requiring independent samples from $\boldsymbol{\pi}$.

Ex 6.31 **(Ergodic theorem numerically)** For the three-state chain $\mathbf{P} = \begin{pmatrix} 0.7 & 0.2 & 0.1 \\ 0.2 & 0.5 & 0.3 \\ 0.1 & 0.3 & 0.6 \end{pmatrix}$:

1. Compute the stationary distribution $\boldsymbol{\pi}$ (use any method from §6.35).
2. Define the observable $f(i) = i$ (state index). Compute the exact time-average limit $\mathbb{E}_\pi[f] = \sum_i i\pi_i$.
3. Simulate a trajectory of length $K = 10{,}000$ starting from state 1: at each step, draw the next state from column X_k of $\mathbf{P}$ using `numpy.random.choice`. Compute the running time average $\frac{1}{k}\sum_{j=0}^{k-1} f(X_j)$ for $k = 1, \ldots, K$ and plot it. How quickly does it converge to $\mathbb{E}_\pi[f]$?
4. Repeat with $f(i) = i^2$. Compare the empirical time average to $\sum_i i^2\pi_i$.

Def 6.34 **(Detailed Balance (Reversibility))** An ergodic Markov chain with transition matrix $\mathbf{P}$ and stationary distribution $\boldsymbol{\pi}$ satisfies *detailed balance* if

$$\pi_j P_{ij} = \pi_i P_{ji} \quad \text{for all } i, j.$$

Equivalently, the matrix $\mathbf{P}\,\mathbf{D}_{\boldsymbol{\pi}}$ is symmetric, where $\mathbf{D}_{\boldsymbol{\pi}} = \operatorname{diag}(\pi_1, \ldots, \pi_n)$. A chain satisfying detailed balance is called *reversible*: the process looks statistically identical run forwards or backwards in time.

Note 6.23 Detailed balance is a *sufficient* condition for $\boldsymbol{\pi}$ to be stationary (but not necessary). Proof: $(\mathbf{P}\boldsymbol{\pi})_i = \sum_j P_{ij}\pi_j = \sum_j P_{ji}\pi_i = \pi_i \sum_j P_{ji} = \pi_i$, where the second step uses $P_{ij}\pi_j = P_{ji}\pi_i$ and the last step uses the column-stochastic property $\sum_j P_{ji} = 1$. Detailed balance is far easier to verify than the full stationarity equation, making it the standard tool for designing MCMC chains.

Ex 6.32 **(Checking and building reversible chains)**

1. For the two-state chain $\mathbf{P} = \begin{pmatrix} 1-\alpha & \beta \\ \alpha & 1-\beta \end{pmatrix}$ with stationary distribution $\boldsymbol{\pi} = \frac{1}{\alpha+\beta}(\beta, \alpha)^T$, verify detailed balance $\pi_j P_{ij} = \pi_i P_{ji}$ for all four pairs (i, j).

2. For states $\{1, 2, 3\}$ with desired stationary distribution $\boldsymbol{\pi} = (0.2, 0.5, 0.3)^T$, construct a 3×3 column-stochastic matrix $\mathbf{P}$ satisfying detailed balance. (There are many solutions; pick any valid one.) Verify that $\mathbf{P}\boldsymbol{\pi} = \boldsymbol{\pi}$.

3. For the random walk on a path graph P_5, is the transition matrix $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}$ reversible? If so, what is $\boldsymbol{\pi}$?

Thm 6.24 **(Reversible Chains Have Real Eigenvalues)** Let $\mathbf{P}$ be an ergodic column-stochastic matrix satisfying detailed balance with stationary distribution $\boldsymbol{\pi}$. Then all eigenvalues of $\mathbf{P}$ are real, and $\mathbf{P}$ has a complete set of eigenvectors orthogonal with respect to the $\boldsymbol{\pi}$-weighted inner product $\langle \boldsymbol{u}, \boldsymbol{v} \rangle_\pi = \sum_i \pi_i^{-1} u_i v_i$.

Proof *Proof.* **Step 1: Symmetrization.** Let $\mathbf{D}_{\boldsymbol{\pi}} = \operatorname{diag}(\pi_1, \ldots, \pi_n)$ and define the conjugated matrix

$$\mathbf{S} = \mathbf{D}_{\boldsymbol{\pi}}^{-1/2}\, \mathbf{P}\, \mathbf{D}_{\boldsymbol{\pi}}^{1/2}.$$

Step 2: S is symmetric. Compute the transpose using detailed balance $\mathbf{P}\mathbf{D}_{\boldsymbol{\pi}} = \mathbf{D}_{\boldsymbol{\pi}}\mathbf{P}^T$ (the matrix form of $P_{ij}\pi_j = P_{ji}\pi_i$):

$$\mathbf{S}^T = \mathbf{D}_{\boldsymbol{\pi}}^{1/2}\mathbf{P}^T\mathbf{D}_{\boldsymbol{\pi}}^{-1/2} = \mathbf{D}_{\boldsymbol{\pi}}^{1/2}(\mathbf{D}_{\boldsymbol{\pi}}^{-1}\mathbf{P}\mathbf{D}_{\boldsymbol{\pi}})\mathbf{D}_{\boldsymbol{\pi}}^{-1/2} = \mathbf{D}_{\boldsymbol{\pi}}^{-1/2}\mathbf{P}\mathbf{D}_{\boldsymbol{\pi}}^{1/2} = \mathbf{S}.$$

Step 3: Transfer eigenvalues. Since $\mathbf{S}$ is real symmetric, it has real eigenvalues and orthonormal eigenvectors $\{\boldsymbol{w}_i\}$ (Spectral Theorem). If $\mathbf{S}\boldsymbol{w}_i = \lambda_i \boldsymbol{w}_i$, then $\mathbf{P}(\mathbf{D}_{\boldsymbol{\pi}}^{1/2}\boldsymbol{w}_i) = \lambda_i(\mathbf{D}_{\boldsymbol{\pi}}^{1/2}\boldsymbol{w}_i)$, so $\boldsymbol{v}_i = \mathbf{D}_{\boldsymbol{\pi}}^{1/2}\boldsymbol{w}_i$ are eigenvectors of $\mathbf{P}$ with the same real eigenvalues. Orthogonality in the $\boldsymbol{\pi}$-weighted inner product follows from $\boldsymbol{w}_i^T\boldsymbol{w}_j = 0$. □

Note 6.24 The real-eigenvalue property is significant in three ways. First, it means we can use symmetric eigenvalue solvers (faster and more stable than the general case; see §4.2) to find the spectrum of $\mathbf{P}$. Second, the spectral gap $\delta = 1-\lambda_2$ now involves a real $\lambda_2 \in (-1, 1)$, giving a sharper mixing bound. Third, in MCMC design, one constructs a reversible chain by the *Metropolis-Hastings algorithm*: given a proposal distribution Q_{ij}, accept moves with probability $\min(1, \pi_i Q_{ji}/(\pi_j Q_{ij}))$, which enforces detailed balance by construction.

Ex 6.33 **(Reversibility and spectral structure)**

1. For the matrix $\mathbf{P} = \begin{pmatrix} 1/2 & 1/4 \\ 1/2 & 3/4 \end{pmatrix}$ with $\boldsymbol{\pi} = (1/3, 2/3)^T$: verify detailed balance, construct $\mathbf{S} = \mathbf{D}_{\boldsymbol{\pi}}^{-1/2}\mathbf{P}\mathbf{D}_{\boldsymbol{\pi}}^{1/2}$ numerically, and confirm it is symmetric. Compute the eigenvalues of $\mathbf{S}$ and $\mathbf{P}$ and verify they are the same real numbers.

2. Construct a 4×4 reversible chain on states $\{1, 2, 3, 4\}$ with $\boldsymbol{\pi} = (0.1, 0.4, 0.3, 0.2)^T$. Use `numpy.linalg.eig` and verify all eigenvalues are real.

3. Now construct a non-reversible chain with the same $\boldsymbol{\pi}$ (add a circulation: some of the probability flux goes around a cycle without satisfying detailed balance). Compare the eigenvalues. Are they still real?

Ex 6.34 **(Metropolis-Hastings MCMC)** In this exercise we implement a basic Metropolis-Hastings chain on a discrete state space $\{1, \ldots, n\}$ with $n = 10$, targeting the stationary distribution $\pi_i \propto e^{-i/3}$ (a truncated geometric distribution).

1. Normalize π_i and compute the exact distribution as a vector $\boldsymbol{\pi} \in \mathbb{R}^{10}$.
2. Implement the Metropolis-Hastings algorithm with symmetric proposal $Q_{ij} = Q_{ji} = 1/2$ for $|i - j| = 1$ (random walk proposal on a path). At each step, propose a new state $j \sim Q(i, \cdot)$ and accept with probability $\min(1, \pi_j/\pi_i)$.
3. Run the chain for $K = 100{,}000$ steps from state 1. Estimate $\boldsymbol{\pi}$ from the empirical visit frequencies and compare to the true $\boldsymbol{\pi}$ using a bar chart.
4. Construct the transition matrix $\mathbf{P}$ of the Metropolis-Hastings chain explicitly and verify that it satisfies detailed balance with $\boldsymbol{\pi}$. Compute its eigenvalues and spectral gap.

6.5.5 Computing Stationary Distributions

Given a stochastic matrix $\mathbf{P}$, three numerically distinct algorithms compute the stationary distribution $\boldsymbol{\pi}$: solving a linear system, extracting the dominant eigenvector, and running power iteration. These approaches have different costs, accuracy characteristics, and scalability. The right choice depends on the problem size and structure, and understanding the tradeoffs requires the condition number theory from §3.3.

Def 6.35 **(Null-Space Formulation)** The stationary distribution satisfies $(\mathbf{P} - \mathbf{I})\boldsymbol{\pi} = \mathbf{0}$ with the normalization constraint $\mathbf{1}^T\boldsymbol{\pi} = 1$. Since $\mathbf{P} - \mathbf{I}$ has rank $n - 1$ (by irreducibility), we append the normalization row to get the overdetermined system

$$\begin{pmatrix} \mathbf{P} - \mathbf{I} \\ \mathbf{1}^T \end{pmatrix} \boldsymbol{\pi} = \begin{pmatrix} \mathbf{0} \\ 1 \end{pmatrix} \in \mathbb{R}^{(n+1)},$$

solvable as a least squares problem via `numpy.linalg.lstsq`.

Ex 6.35 **(Null-space method)** For the 3×3 stochastic matrix

$$\mathbf{P} = \begin{pmatrix} 0.7 & 0.4 & 0.2 \\ 0.2 & 0.4 & 0.5 \\ 0.1 & 0.2 & 0.3 \end{pmatrix},$$

1. Construct the augmented matrix $\mathbf{A} = \begin{pmatrix} (\mathbf{P} - \mathbf{I})^T & \mathbf{1} \end{pmatrix}^T$ and the right-hand side $\mathbf{b} = (0, \ldots, 0, 1)^T$.

2. Solve using `numpy.linalg.lstsq(A, b)`. Report $\boldsymbol{\pi}$ and verify $\mathbf{P}\boldsymbol{\pi} = \boldsymbol{\pi}$.

3. Compute $\kappa_2(\mathbf{A})$. Is the system well-conditioned?

Thm 6.25 **(Power Iteration Convergence for Markov Chains)** Let $\mathbf{P}$ be column-stochastic, irreducible, and primitive with spectral gap δ. For any initial probability distribution $\boldsymbol{x}^{(0)}$, the iterates $\boldsymbol{x}^{(k+1)} = \mathbf{P}\boldsymbol{x}^{(k)}$ satisfy

$$\|\boldsymbol{x}^{(k)} - \boldsymbol{\pi}\|_1 \leq C\,(1-\delta)^k,$$

i.e., the Markov chain simulation is exactly power iteration (Thm 4.7) applied to $\mathbf{P}$, with convergence rate $|\lambda_2| = 1 - \delta$.

Note 6.25 For large sparse systems such as the web graph used in PageRank, factoring $\mathbf{P} - \mathbf{I}$ is prohibitive (the matrix is $n \times n$ with $n \sim 10^9$). Power iteration is preferred because each step requires only a sparse matrix-vector product. The price is slower convergence: with the Google damping factor $d = 0.85$, the spectral gap is $\delta \geq 1 - d = 0.15$, so about $k \approx \log(n)/0.15 \approx 130$ iterations suffice for $n = 10^9$ (see §6.2.5).

Ex 6.36 **(Three methods comparison)** For the same 3×3 matrix from the null-space exercise:

1. **Eigenvector:** Use `numpy.linalg.eig(P)`, identify $\lambda = 1$, extract and normalize the eigenvector.

2. **Power iteration:** Iterate $\boldsymbol{x}^{(k+1)} = \mathbf{P}\boldsymbol{x}^{(k)}$ from $\boldsymbol{x}^{(0)} = (1/3, 1/3, 1/3)^T$. Count iterations until $\|\boldsymbol{x}^{(k)} - \boldsymbol{\pi}\|_1 < 10^{-10}$.

3. **Null-space:** Use the approach from Def 6.35.

4. Compare the three results and time each method using `time.perf_counter`. Which is fastest? Which is most accurate (smallest residual $\|\mathbf{P}\boldsymbol{\pi} - \boldsymbol{\pi}\|$)?

Ex 6.37 **(In this exercise, we will prove a property of doubly-stochastic matrices.)** A matrix is *doubly-stochastic* if both its rows and columns sum to 1.

1. Show that the uniform distribution $\boldsymbol{\pi} = \frac{1}{n}\mathbf{1}$ is always stationary for any doubly-stochastic matrix $\mathbf{P}$.

2. For an unweighted d-regular graph (every vertex has degree d), show that $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A} = \frac{1}{d}\mathbf{A}$ is doubly-stochastic and that $\boldsymbol{\pi} = \frac{1}{n}\mathbf{1}$ without computing any eigenvalues. (Use only the column-sum and row-sum properties of $\mathbf{A}$.)

3. Construct the transition matrix for the 5-cycle C_5 and verify the uniform stationary distribution numerically.

6.6 Koopman Operator Theory, DMD, and SINDy

Most real-world dynamical systems are nonlinear: a pendulum, a turbulent flow, an epidemic. The spectral methods developed in this course apply cleanly only to *linear* operators. Koopman operator theory resolves this tension by a remarkable lifting: every nonlinear dynamical system induces a *linear* operator acting on the space of *observable functions* of the state. This section develops Koopman theory from the operator perspective of §6.3, then introduces two data-driven algorithms for approximating the Koopman operator from measurement data: Dynamic Mode Decomposition (DMD), which uses the SVD (see §4.2) to extract dominant dynamical modes, and SINDy, which identifies governing equations via sparse regression (see §3.4).

6.6.1 The Koopman Operator

Recall from §6.3 that a linear operator maps functions to functions, and that even simple operators such as differentiation require careful treatment of domains and ranges. The Koopman operator lives in this world: it acts on the space of scalar-valued functions of the state, and it is linear regardless of whether the underlying dynamics are nonlinear. The remarkable consequence is that nonlinear dynamics can always be analyzed with linear spectral theory, at the cost of working in a higher (often infinite) dimensional function space.

Def 6.36 **(Dynamical System and Observable)** A *discrete-time dynamical system* on state space $\mathcal{M} \subseteq \mathbb{R}^n$ is defined by a (possibly nonlinear) map $\mathbf{F} : \mathcal{M} \to \mathcal{M}$, with trajectories $\boldsymbol{x}^{(k+1)} = \mathbf{F}(\boldsymbol{x}^{(k)})$. An *observable* is any function $g : \mathcal{M} \to \mathbb{R}$ (or $\mathbb{C}$) that measures some feature of the state, such as $g(\boldsymbol{x}) = x_1$, $g(\boldsymbol{x}) = \|\boldsymbol{x}\|^2$, or $g(\boldsymbol{x}) = \sin(x_1 + x_2)$.

Ex 6.38 **(Observables and measurements)** For the logistic map $F(x) = rx(1-x)$ on $[0, 1]$ with $r = 3.8$:

1. Write a Python function that evolves the map for $N = 300$ steps from $x_0 = 0.4$.
2. Consider the three observables $g_1(x) = x$, $g_2(x) = x^2$, and $g_3(x) = x(1-x)$. Plot the time series $g_i(x^{(k)})$ for each. Do any of the observable sequences look more regular or predictable than the state $x^{(k)}$ itself?
3. This map is nonlinear. Does it have a fixed point? If so, find it and classify its stability.

Def 6.37 **(Koopman Operator)** Given a dynamical system $\boldsymbol{x}^{(k+1)} = \mathbf{F}(\boldsymbol{x}^{(k)})$, the *Koopman operator* $\mathcal{K}$ acts on observables by composition with $\mathbf{F}$:

$$(\mathcal{K}\, g)(\boldsymbol{x}) \;=\; g\big(\mathbf{F}(\boldsymbol{x})\big).$$

The operator $\mathcal{K}$ is *linear* on the space of observables even when $\mathbf{F}$ is nonlinear: for any observables g, h and scalars α, β, $\mathcal{K}(\alpha g + \beta h) = \alpha \mathcal{K} g + \beta \mathcal{K} h$.

Proof *Proof.* **(Linearity of $\mathcal{K}$)** For any $\boldsymbol{x} \in \mathcal{M}$:

$$\big[\mathcal{K}(\alpha g + \beta h)\big](\boldsymbol{x}) = (\alpha g + \beta h)\big(\mathbf{F}(\boldsymbol{x})\big) = \alpha\, g\big(\mathbf{F}(\boldsymbol{x})\big) + \beta\, h\big(\mathbf{F}(\boldsymbol{x})\big) = (\alpha \mathcal{K} g + \beta \mathcal{K} h)(\boldsymbol{x}). \qquad \square$$

$\square$

Note 6.26 The tradeoff in passing from $\mathbf{F}$ to $\mathcal{K}$ is dimensionality: $\mathbf{F}$ acts on $\mathbb{R}^n$ (finite-dimensional) while $\mathcal{K}$ acts on an infinite-dimensional function space (analogous to the infinite-dimensional operators discussed in §6.3). For linear systems $\mathbf{F}(\boldsymbol{x}) = \mathbf{A}\boldsymbol{x}$, the Koopman operator restricted to linear observables $g(\boldsymbol{x}) = \boldsymbol{c}^T\boldsymbol{x}$ is represented exactly by $\mathbf{A}^T$, recovering the finite matrix picture. For nonlinear systems, the goal is to find a finite-dimensional *approximation* to $\mathcal{K}$ from data.

Ex 6.39 **(Koopman eigenfunctions for linear systems)** For the linear system $\boldsymbol{x}^{(k+1)} = \mathbf{A}\boldsymbol{x}^{(k)}$ with $\mathbf{A} \in \mathbb{R}^{n\times n}$:

1. Show that if $\mathbf{A}^T\boldsymbol{w} = \lambda\boldsymbol{w}$ (i.e., $\boldsymbol{w}$ is a left eigenvector of $\mathbf{A}$), then $\varphi(\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x}$ is a Koopman eigenfunction with eigenvalue λ: $\varphi(\mathbf{A}\boldsymbol{x}) = \lambda\varphi(\boldsymbol{x})$.
2. For $\mathbf{A} = \begin{pmatrix} 0.9 & 0 \\ 0 & 0.5 \end{pmatrix}$, write down two explicit Koopman eigenfunctions and verify them by computing $\varphi(\mathbf{A}\boldsymbol{x})$ directly.
3. Explain why $g(\boldsymbol{x}) = x_1 x_2$ is *not* a Koopman eigenfunction for this $\mathbf{A}$.

Def 6.38 **(Koopman Eigenfunction)** A nonzero observable $\varphi : \mathcal{M} \to \mathbb{C}$ is a *Koopman eigenfunction* with eigenvalue $\mu \in \mathbb{C}$ if

$$\mathcal{K}\,\varphi = \mu\,\varphi, \qquad \text{i.e.,} \qquad \varphi\big(\mathbf{F}(\boldsymbol{x})\big) = \mu\,\varphi(\boldsymbol{x}) \quad \text{for all } \boldsymbol{x} \in \mathcal{M}.$$

Along any trajectory, $\varphi(\boldsymbol{x}^{(k)}) = \mu^k\,\varphi(\boldsymbol{x}^{(0)})$: the eigenfunction evolves as a pure geometric sequence, regardless of the nonlinearity of $\mathbf{F}$.

Ex 6.40 **(Eigenfunction time series)** For the logistic map $F(x) = 3.2\,x(1-x)$ (period-2 regime):

1. Find the two fixed points of $F^2 = F \circ F$ (the period-2 orbit). Call them p_1 and p_2.
2. Consider $\varphi(x) = (x - p_1)(x - p_2)$. Check numerically whether $\varphi(F(x)) = \mu\,\varphi(x)$ for some constant μ by computing $\varphi(F(x))/\varphi(x)$ at several x values. Is φ an eigenfunction?
3. Plot the time series $\varphi(x^{(k)})$ for 50 steps. Does it evolve as a pure geometric sequence?

Def 6.39 **(EDMD Finite-Dimensional Approximation)** To approximate $\mathcal{K}$ numerically, choose a dictionary of p observables $\boldsymbol{\psi}(\boldsymbol{x}) = (\psi_1(\boldsymbol{x}), \ldots, \psi_p(\boldsymbol{x}))^T$. Given m pairs of consecutive snapshots $\{(\boldsymbol{x}^{(k)}, \boldsymbol{x}^{(k+1)})\}_{k=0}^{m-1}$, form the dictionary matrices

$$\boldsymbol{\Psi}_X = \big[\boldsymbol{\psi}(\boldsymbol{x}^{(0)}) \;\; \cdots \;\; \boldsymbol{\psi}(\boldsymbol{x}^{(m-1)})\big] \in \mathbb{R}^{p\times m}, \qquad \boldsymbol{\Psi}_Y = \big[\boldsymbol{\psi}(\boldsymbol{x}^{(1)}) \;\; \cdots \;\; \boldsymbol{\psi}(\boldsymbol{x}^{(m)})\big] \in \mathbb{R}^{p\times m}.$$

The *Extended Dynamic Mode Decomposition (EDMD)* matrix is the least squares solution

$$\mathbf{K} = \underset{\mathbf{K}\in\mathbb{R}^{p\times p}}{\arg\min} \|\mathbf{\Psi}_Y - \mathbf{K}\,\mathbf{\Psi}_X\|_F^2 = \mathbf{\Psi}_Y\,\mathbf{\Psi}_X^+.$$

Note 6.27 EDMD reduces to standard DMD when $\boldsymbol{\psi}(\boldsymbol{x}) = \boldsymbol{x}$ (the identity dictionary). The quality of the approximation depends on the dictionary: polynomial dictionaries work well for polynomial dynamics, but may fail for systems with complex invariant sets. This is a bias-variance tradeoff: a richer dictionary reduces bias but increases variance and cost.

6.6.2 Dynamic Mode Decomposition

DMD is the most widely used data-driven spectral method in computational science. Given a sequence of state snapshots, DMD computes the best-fit linear operator $\mathbf{A}$ satisfying $\boldsymbol{x}^{(k+1)} \approx \mathbf{A}\boldsymbol{x}^{(k)}$ and decomposes the data into spatial modes oscillating at identified frequencies. The algorithm is built entirely on the truncated SVD (see §4.2), making it both numerically stable and scalable to high-dimensional systems.

Def 6.40 **(DMD Data Matrices)** Given $m+1$ state snapshots $\boldsymbol{x}^{(0)}, \boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)} \in \mathbb{R}^n$, form the two snapshot matrices

$$\mathbf{X} = \begin{bmatrix}\boldsymbol{x}^{(0)} & \boldsymbol{x}^{(1)} & \cdots & \boldsymbol{x}^{(m-1)}\end{bmatrix} \in \mathbb{R}^{n\times m}, \qquad \mathbf{Y} = \begin{bmatrix}\boldsymbol{x}^{(1)} & \boldsymbol{x}^{(2)} & \cdots & \boldsymbol{x}^{(m)}\end{bmatrix} \in \mathbb{R}^{n\times m}.$$

DMD seeks the best-fit matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ minimizing $\|\mathbf{Y} - \mathbf{A}\mathbf{X}\|_F$.

Thm 6.26 **(DMD via Truncated SVD)** Let $\mathbf{X} \approx \hat{\mathbf{U}}\hat{\mathbf{S}}\hat{\mathbf{V}}^T$ be the rank-r truncated SVD of $\mathbf{X}$. Define

$$\tilde{\mathbf{A}} = \hat{\mathbf{U}}^T\mathbf{Y}\hat{\mathbf{V}}\hat{\mathbf{S}}^{-1} \in \mathbb{R}^{r\times r}.$$

The DMD eigenvalues $\{\lambda_i\}$ are the eigenvalues of $\tilde{\mathbf{A}}$. The corresponding full-space DMD modes are the columns of $\mathbf{\Phi} = \mathbf{Y}\hat{\mathbf{V}}\hat{\mathbf{S}}^{-1}\mathbf{W}$, where $\mathbf{W}$ contains the eigenvectors of $\tilde{\mathbf{A}}$.

Proof *Proof.* **Step 1: Least squares for A.** The best-fit operator is $\mathbf{A} = \mathbf{Y}\mathbf{X}^+$ where $\mathbf{X}^+ = \hat{\mathbf{V}}\hat{\mathbf{S}}^{-1}\hat{\mathbf{U}}^T$ is the pseudoinverse of $\mathbf{X}$ (from the SVD-based least squares formula of §3.4). This follows from the normal equations $\mathbf{A}\mathbf{X}\mathbf{X}^T = \mathbf{Y}\mathbf{X}^T$, whose minimum-norm solution uses the pseudoinverse.
Step 2: Projection onto dominant subspace. When n is large (e.g., $n = 10^6$ spatial degrees of freedom from a fluid simulation), forming and diagonalizing the $n \times n$ matrix $\mathbf{A}$ is infeasible. Instead, we project onto the r-dimensional column space of $\hat{\mathbf{U}}$:

$$\tilde{\mathbf{A}} = \hat{\mathbf{U}}^T\mathbf{A}\hat{\mathbf{U}} = \hat{\mathbf{U}}^T(\mathbf{Y}\mathbf{X}^+)\hat{\mathbf{U}} = \hat{\mathbf{U}}^T\mathbf{Y}\hat{\mathbf{V}}\hat{\mathbf{S}}^{-1}.$$

The $r \times r$ matrix $\tilde{\mathbf{A}}$ captures the dominant dynamics at cost $O(nmr + r^3)$ rather than $O(n^3)$.
Step 3: Mode reconstruction. If $\tilde{\mathbf{A}}\mathbf{w}_i = \lambda_i\mathbf{w}_i$, the full-space DMD mode is $\boldsymbol{\phi}_i = \hat{\mathbf{U}}\mathbf{w}_i$ in the projected formulation, or more accurately $\boldsymbol{\phi}_i = \mathbf{Y}\hat{\mathbf{V}}\hat{\mathbf{S}}^{-1}\mathbf{w}_i/\lambda_i$ (the "exact DMD" formula), which lies in the range of $\mathbf{Y}$ rather than $\mathbf{X}$. □

Note 6.28 The DMD expansion $\boldsymbol{x}^{(k)} \approx \sum_{i=1}^{r} b_i \lambda_i^k \boldsymbol{\phi}_i$ decomposes the data into r spatiotemporal modes. Each mode $\boldsymbol{\phi}_i$ is a spatial pattern oscillating at frequency $\arg(\lambda_i)/(2\pi)$ and growing or decaying at rate $|\lambda_i|$ per time step. Stable modes lie on the unit circle ($|\lambda_i| = 1$). The amplitudes b_i satisfy a small least squares problem $\mathbf{\Phi b} \approx \boldsymbol{x}^{(0)}$.

Ex 6.41 **(DMD on a coupled oscillator)** Generate data from the linear system $\dot{\boldsymbol{x}} = \mathbf{A}_c \boldsymbol{x}$ with

$$\mathbf{A}_c = \begin{pmatrix} -0.1 & 2 \\ -2 & -0.1 \end{pmatrix}$$

using `scipy.integrate.solve_ivp` for $t \in [0, 10]$ with 201 time steps and $\boldsymbol{x}^{(0)} = (1, 0)^T$.

1. Form $\mathbf{X}$ and $\mathbf{Y}$ from consecutive snapshots. Compute the rank-2 truncated SVD of $\mathbf{X}$.
2. Compute $\tilde{\mathbf{A}}$ and its eigenvalues λ_i. The continuous-time eigenvalues of $\mathbf{A}_c$ are $-0.1 \pm 2i$. Convert DMD eigenvalues to continuous time via $\omega_i = \log(\lambda_i)/\Delta t$ and compare.
3. Reconstruct the time series $\hat{\boldsymbol{x}}^{(k)} = \sum_i b_i \lambda_i^k \boldsymbol{\phi}_i$ and plot against the true solution. What is the relative reconstruction error $\|\mathbf{X} - \hat{\mathbf{X}}\|_F / \|\mathbf{X}\|_F$?

Ex 6.42 **(Effect of noise and truncation rank)** Using the same oscillator system, add measurement noise: $\tilde{\boldsymbol{x}}^{(k)} = \boldsymbol{x}^{(k)} + \sigma \boldsymbol{\varepsilon}^{(k)}$ where $\boldsymbol{\varepsilon}^{(k)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

1. For noise levels $\sigma \in \{0, 0.01, 0.1, 0.5\}$, plot the singular value decay of $\mathbf{X}$ in each case. Where does the numerical rank appear to be?
2. Run DMD with truncation ranks $r \in \{1, 2, 3, 5\}$ on the $\sigma = 0.1$ case. Compare the identified eigenvalues and reconstruction errors.
3. Which r gives the best tradeoff between model accuracy and robustness to noise? How does this relate to the singular value spectrum?

6.6.3 SINDy: Sparse Identification of Nonlinear Dynamics

DMD finds the best-fit *linear* model $\boldsymbol{x}^{(k+1)} = \mathbf{A}\boldsymbol{x}^{(k)}$. But many systems of scientific interest are nonlinear in a structured, interpretable way: the Lorenz equations are quadratic, the logistic map is a degree-2 polynomial. SINDy (Sparse Identification of Nonlinear Dynamics) exploits this by positing that the governing equations involve only a few terms from a large library of candidate nonlinear functions. The identification problem then reduces to sparse regression: a natural extension of the regularized least squares of §3.4.

Def 6.41 **(SINDy Library Matrix)** Given m state snapshots forming the rows of $\mathbf{X} \in \mathbb{R}^{m \times n}$, define a library of p candidate functions $\psi_1, \ldots, \psi_p$ (e.g., monomials up to degree d, trigonometric functions). Evaluating each library function on each snapshot yields the *library matrix*

$$\mathbf{\Theta}(\mathbf{X}) = \begin{bmatrix} \boldsymbol{\psi}_1(\mathbf{X}) & \boldsymbol{\psi}_2(\mathbf{X}) & \cdots & \boldsymbol{\psi}_p(\mathbf{X}) \end{bmatrix} \in \mathbb{R}^{m \times p},$$

where $\boldsymbol{\psi}_j(\mathbf{X}) \in \mathbb{R}^m$ evaluates ψ_j on each row of $\mathbf{X}$. For an n-dimensional system with degree-2 polynomial library, $p = 1 + n + \binom{n+2}{2} - 1$.

Note 6.29 Each column $\boldsymbol{\xi}_i$ of the coefficient matrix $\boldsymbol{\Xi} \in \mathbb{R}^{p\times n}$ gives the right-hand side equation for $\dot{x}_i$ as a linear combination of library terms. For the Lorenz system, the $\dot{x}$ equation is $\dot{x} = \sigma(y - x) = -\sigma x + \sigma y$, so $\boldsymbol{\xi}_1$ has nonzeros only in the x and y positions with values $-\sigma$ and $+\sigma$. Identifying which entries are nonzero, and their values, from data is the core SINDy problem.

Ex 6.43 **(Library construction)** For states in $\mathbb{R}^2$ with $\boldsymbol{x} = (x_1, x_2)^T$:

1. Write out all degree-2 polynomial library functions $\{1, x_1, x_2, x_1^2, x_1x_2, x_2^2\}$ and construct $\boldsymbol{\Theta}(\mathbf{X})$ for $m = 5$ random snapshots in NumPy.
2. If the true dynamics are $\dot{x}_1 = -2x_1 + x_1x_2$, $\dot{x}_2 = x_2 - x_1x_2$, write down the true coefficient matrix $\boldsymbol{\Xi}$.
3. What is the sparsity (fraction of zeros) of $\boldsymbol{\Xi}$ for this example? How does it change as n or the polynomial degree increases?

Def 6.42 **(Sparse Thresholded Least Squares (STLS))** Given the library regression problem $\dot{\mathbf{X}} \approx \boldsymbol{\Theta}(\mathbf{X})\,\boldsymbol{\Xi}$, the *STLS algorithm* finds a sparse $\boldsymbol{\Xi}$ by iterating between least squares regression on the active terms and hard thresholding to eliminate small coefficients:

1. **Initialize:** $\boldsymbol{\Xi}^{(0)} = \boldsymbol{\Theta}(\mathbf{X})^+\,\dot{\mathbf{X}}$ (unconstrained least squares solution via `numpy.linalg.lstsq`).
2. **Threshold:** Set $\xi_{ij}^{(j)} \leftarrow 0$ whenever $|\xi_{ij}^{(j)}| < \lambda$.
3. **Regress:** For each state variable i, let $\mathcal{A}_i^{(j)}$ be the set of active (nonzero) library indices. Solve
$$\boldsymbol{\xi}_i^{(j+1)} = \arg\min_{\boldsymbol{\xi}} \|\boldsymbol{\Theta}_{\mathcal{A}_i^{(j)}}\,\boldsymbol{\xi} - \dot{\mathbf{X}}_{:,i}\|_2^2,$$
where $\boldsymbol{\Theta}_{\mathcal{A}}$ denotes the columns of $\boldsymbol{\Theta}$ indexed by $\mathcal{A}$.
4. Repeat steps 2 and 3 until $\boldsymbol{\Xi}$ does not change.

Note 6.30 STLS approximates the LASSO problem $\min_{\boldsymbol{\Xi}} \|\dot{\mathbf{X}} - \boldsymbol{\Theta}\boldsymbol{\Xi}\|_F^2 + \lambda\|\boldsymbol{\Xi}\|_1$ using hard rather than soft thresholding, which tends to produce sparser solutions. Step 3 is a standard (underdetermined or overdetermined) least squares solve on a reduced column set, directly reusing the QR-based `lstsq` routine. The threshold λ plays the same role as the regularization parameter in ridge regression: too small yields a dense, overfit $\boldsymbol{\Xi}$; too large prunes true terms and underspecifies the model.

Ex 6.44 **(SINDy on the Lorenz system)** The Lorenz system is
$$\dot{x} = \sigma(y - x), \quad \dot{y} = x(\rho - z) - y, \quad \dot{z} = xy - \beta z,$$
with parameters $\sigma = 10$, $\rho = 28$, $\beta = 8/3$.

1. Simulate with `scipy.integrate.solve_ivp` for $t \in [0, 10]$, 2001 time steps, initial condition $(1, 1, 1)$. Store $\mathbf{X} \in \mathbb{R}^{2001\times 3}$.

2. Build the degree-2 polynomial library $\mathbf{\Theta}(\mathbf{X}) \in \mathbb{R}^{2001\times 10}$ with columns $\{1, x, y, z, x^2, xy, xz, y^2, yz,$

3. Estimate $\dot{\mathbf{X}}$ by central finite differences: $\dot{\boldsymbol{x}}^{(k)} \approx (\boldsymbol{x}^{(k+1)} - \boldsymbol{x}^{(k-1)})/(2\Delta t)$ for interior points.

4. Implement STLS with $\lambda = 0.5$. Print the identified $\mathbf{\Xi}$ and compare to the true equations. Do you recover σ, ρ, and β?

5. Repeat with $\lambda = 0.05$ and $\lambda = 2.0$. How does the sparsity of $\mathbf{\Xi}$ change? Which λ gives the most physically interpretable result?

Ex 6.45 **(Model selection and cross-validation)** Using the Lorenz simulation from the previous exercise:

1. Split the trajectory: use the first 70% for training and the last 30% for testing.

2. Run STLS on the training data for $\lambda \in \{0.02, 0.05, 0.1, 0.5, 1.5\}$. For each λ, integrate the identified model forward from the test initial condition using `solve_ivp` and compute the prediction error $\|\boldsymbol{x}_{\text{pred}}(T) - \boldsymbol{x}_{\text{true}}(T)\|_2$ at the final test time T.

3. Plot two curves against λ: (a) the number of nonzero terms in $\mathbf{\Xi}$, and (b) the test prediction error. Identify the best λ.

4. Instead of STLS, use ridge regression (replace step 3 with $\mathbf{\Xi} = (\mathbf{\Theta}^T\mathbf{\Theta} + \mu\mathbf{I})^{-1}\mathbf{\Theta}^T\dot{\mathbf{X}}$ from Def 3.4) for the same μ values. Compare the sparsity and interpretability of the two approaches.